Getting Started

Thank you for choosing Freenove products!

Get Support and Offer Input

Freenove provides free, responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down! When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

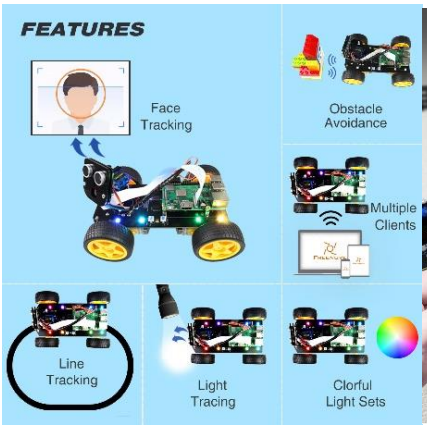
Car and Robot for Raspberry Pi

We also have cars and robot kit for Raspberry Pi. If you are interested in them, please visit our website for details.

http://www.freenove.com/store.html

FNK0043 Freenove 4WD Smart Car Kit for Raspberry Pi



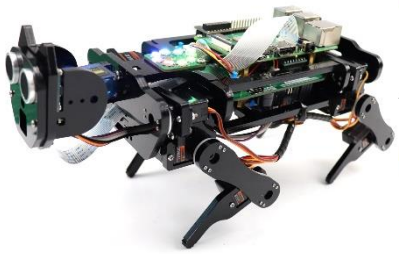


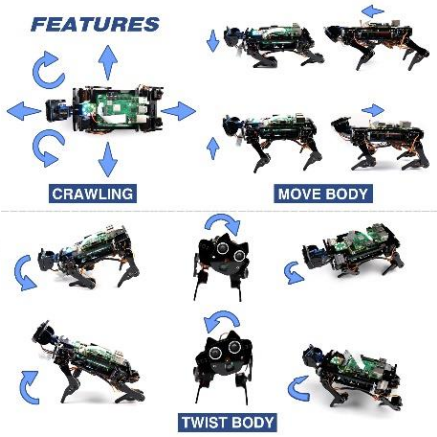


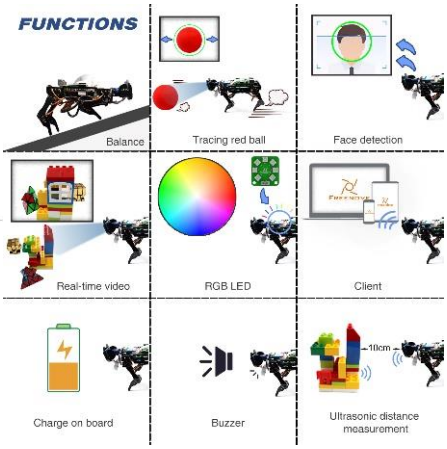
https://www.youtube.com/watch?v=4Zv0GZUQjZc

Freenove Robot Dog Kit for Raspberry Pi

FNK0050







https://www.youtube.com/watch?v=7BmIZ8_R9d4&t=35s

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

Copyright

All the files, materials and instructional guides provided are released under Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License. A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.

This means you can use these resource in your own derived works, in part or completely, but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.

FREENOVE®
FREE YOUR INNOVATION

Raspberry Pi® is a trademark of Raspberry Pi Foundation (https://www.raspberrypi.org/).

Contents

Getting Started I
Safety and Precautions I
Car and Robot for Raspberry Pi II
About Freenove III
Copyright III
Contents IV
Preface 1
Raspberry Pi 2
Chapter 0 Preparation 9
Linux Command 9
Install WiringPi 12
Obtain the Project Code 14
Python2 & Python3 16
Projects Board for Raspberry Pi 18
Assembly 19
Chapter 1 LED 22
Project 1.1 Blink 22
Chapter 2 FlowingLight 35
Project 2.1 Flowing Water Light 35
Chapter 3 Buttons & LEDs 40
Project 3.1 Push Button Switch & LED 40
Chapter 4 Analog & PWM 47
Project 4.1 Breathing LED 47
Chapter 5 RGB LED 54
Project 5.1 RainbowLED 55
Chapter 6 Buzzer 61
Project 6.1 Doorbell 61
Project 6.2 Alertor 68
(Important) Chapter 7 ADC 74
Project 7.1 Read the Voltage of Potentiometer 74
Project 7.2 Soft Light 86
Project 7.3 Colorful Light 91
Chapter 8 Photoresistor & LED 97
Project 8.1 NightLamp 97
Chapter 9 Thermistor 103
Project 9.1 Thermometer 103
Chapter 10 Joystick 111
Project 10.1 Joystick 111
Chapter 11 Motor & Driver 118
Project 11.1 Control a DC Motor with a Potentiometer 118
Chapter 12 Relay & LED 129

Project 12.1 Relay & LED 129
Chapter 13 Servo 137
Project 13.1 Sweep 137
Project 13.2 Knob 145
Chapter 14 Stepper Motor 151
Project 14.1 Stepper Motor 151
Chapter 15 LEDpixel 162
Project 15.1 LEDpixel 162
Project 15.2 Rainbow Light 170
Project 15.3 SpiLEDpixel 178
Chapter 16 74HC595 & Bar Graph LED 188
Project 16.1 Flowing Water Light 188
Chapter 17 74HC595 & 4-Digit 7-Segment Display 198
Project 17.1 4-Digit 7-Segment Display 198
Project 17.2 4-Digit 7-Segment Display 205
Chapter 18 74HC595 & LED Matrix 215
Project 18.1 LED Matrix 215
Chapter 19 LCD1602 226
Project 19.1 I2C LCD1602 226
Chapter 20 Hygrothermograph DHT11 237
Project 20.1 Hygrothermograph 237
Chapter 21 Matrix Keypad 246
Project 21 Matrix Keypad 246
Chapter 22 Infrared Motion Sensor 254
Project 22.1 PIR Infrared Motion Detector with LED Indicator 254
Chapter 23 Ultrasonic Ranging 261
Project 23.1 Ultrasonic Ranging 261
Chapter 24 Attitude Sensor MPU6050 270
Project 24.1 Read an MPU6050 Sensor Module 270
Chapter 25 RFID 278
Project 25.1 RFID 278
What's Next? 298

Preface

Raspberry Pi is a low cost, **credit card sized computer** that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is an incredibly capable little device that enables people of all ages to explore computing, and to learn how to program in a variety of computer languages like Scratch and Python. It is capable of doing everything you would expect from a desktop computer, such as browsing the internet, playing high-definition video content, creating spreadsheets, performing word-processing, and playing video games. For more information, you can refer to Raspberry Pi official website. For clarification, this tutorial will also reference Raspberry Pi as RPi, RPI and RasPi.

In this tutorial, most chapters consist of **Components List**, **Component Knowledge**, **Circuit**, and **Code** (**C** code and **Python** code). We provide both C and Python code for each project in this tutorial. After completing this tutorial, you can learn Java by reading Processing.pdf.

This kit contains all the accessory electronic components and modules needed to complete the projects described in the index. You can also use these components and modules to create projects of your own design.

Additionally, if you encounter any issues or have questions about this tutorial or the contents of kit, you can always contact us for free technical support at:

support@freenove.com

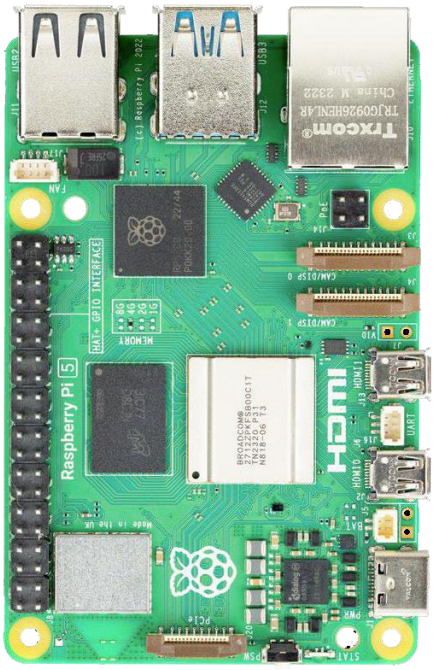
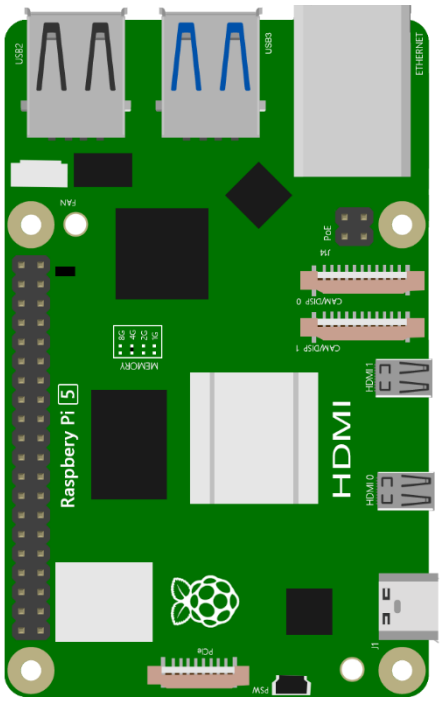
Raspberry Pi

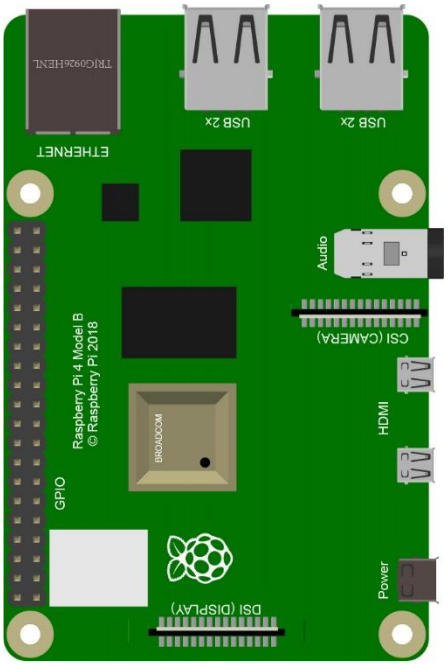
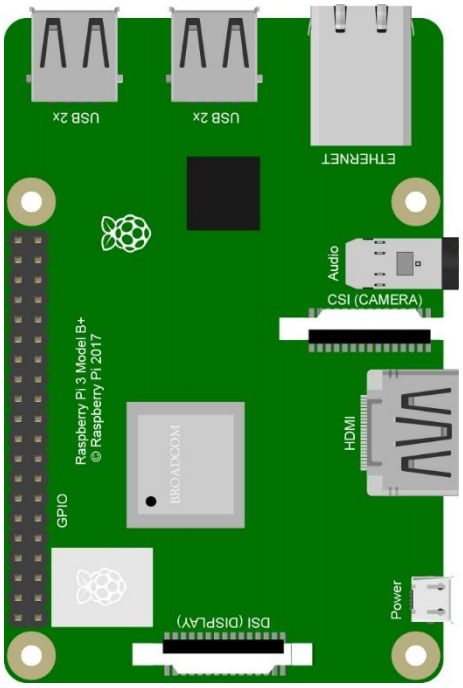
Raspberry Pi (called RPi, RPI, RasPi, the text these words will be used alternately later), a micro-computer with size of a card, quickly swept the world since its debut. It is widely used in desktop workstation, media center, smart home, robots, and even the servers, etc. It can do almost anything, which continues to attract fans to explore it. Raspberry Pi used to be running with Linux system and along with the release of windows 10 IoT. We can also run it with Windows. Raspberry Pi (with interfaces USB, network, HDMI, camera, audio, display and GPIO), as a microcomputer, can be running in command line mode and desktop system mode. Additionally, it is easy to operate just like Arduino, and you can even directly operate the GPIO of CPU.

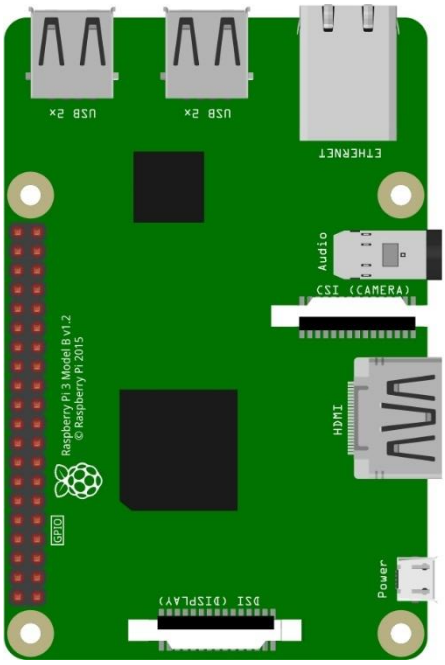
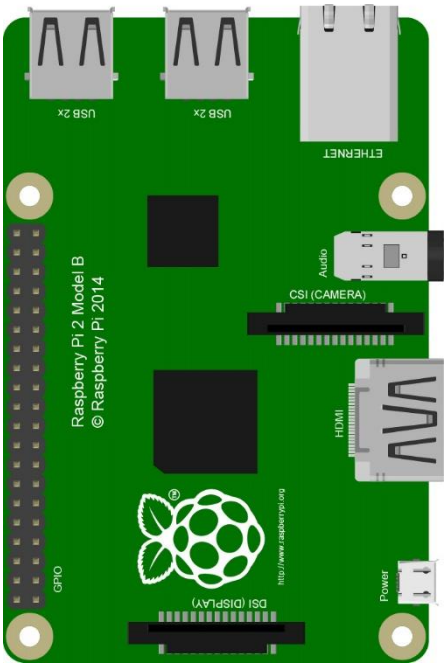
So far, at this writing, Raspberry Pi has advanced to its fifth generation product offering. Version changes are accompanied by increases in upgrades in hardware and capabilities.

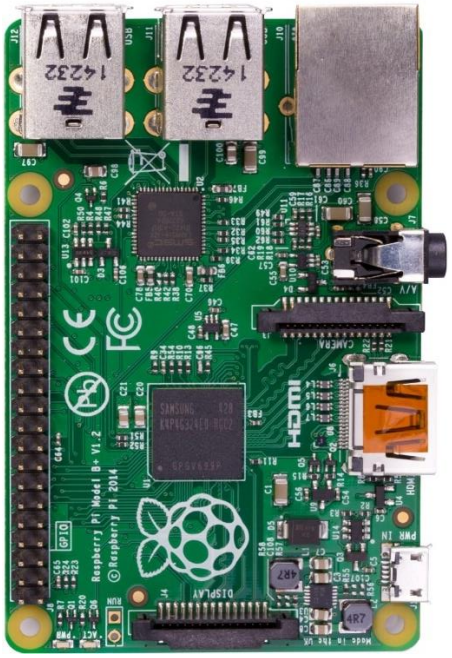
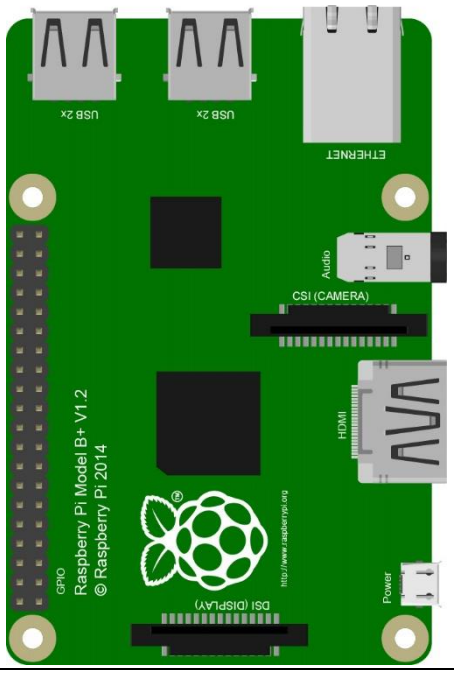
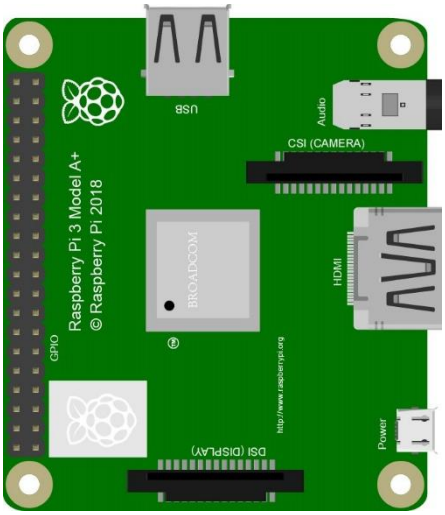
The A type and B type versions of the first generation products have been discontinued due to various reasons. What is most important is that other popular and currently available versions are consistent in the order and number of pins and their assigned designation of function, making compatibility of peripheral devices greatly enhanced between versions.

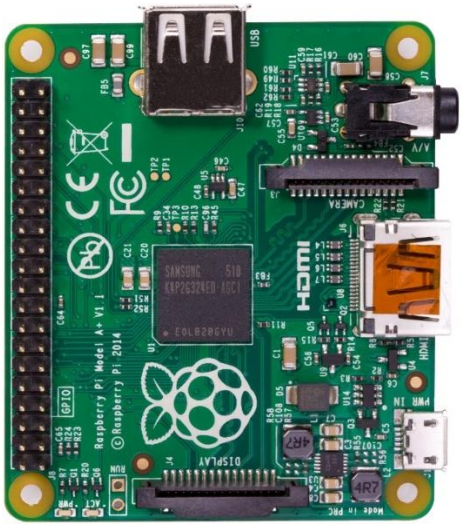
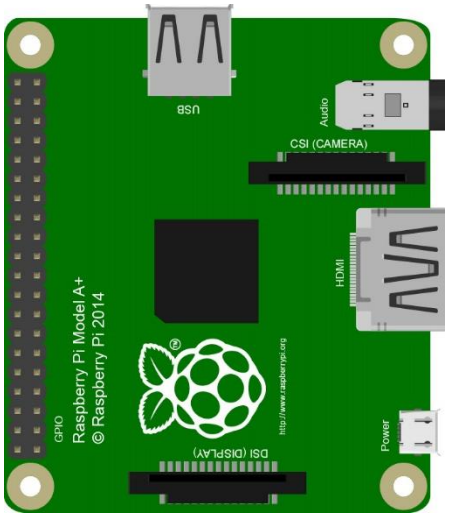
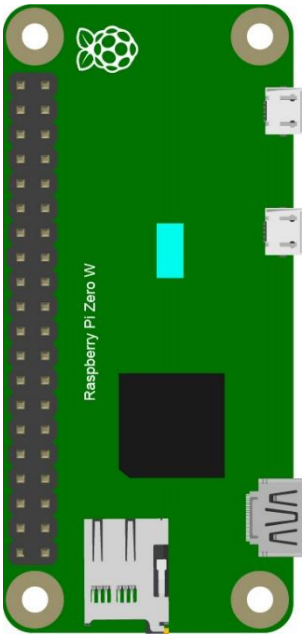
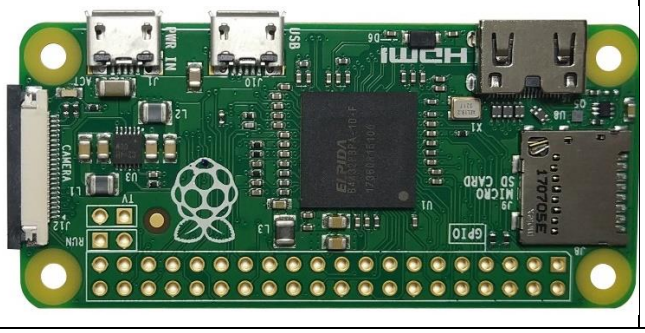
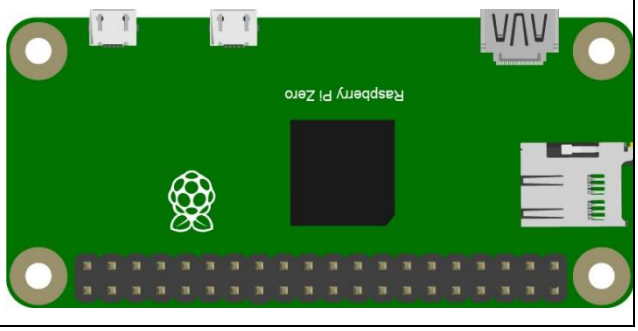
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.

Practicality picture of Raspberry Pi 5:	Model diagram of Raspberry Pi 5:
	

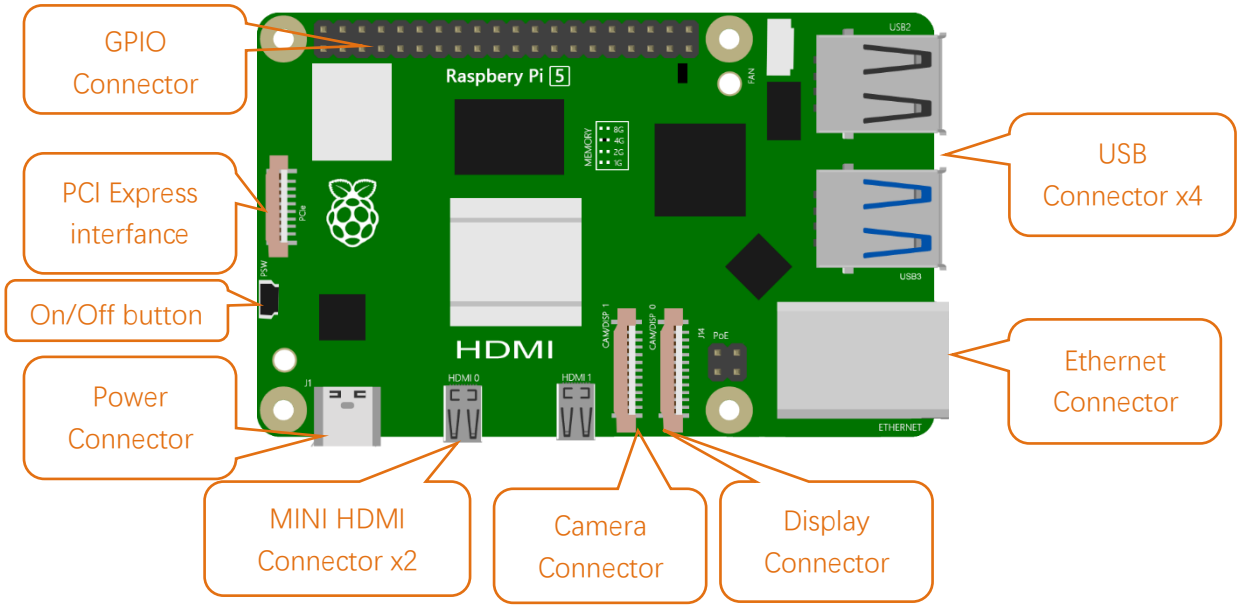
Actual image of Raspberry Pi 4 Model B:	CAD image of Raspberry Pi 4 Model B:
	
Actual image of Raspberry Pi 3 Model B+:	CAD image of Raspberry Pi 3 Model B+:
	

Actual image of Raspberry Pi 3 Model B: 	CAD image of Raspberry Pi 3 Model B: 
Actual image of Raspberry Pi 2 Model B: 	CAD image of Raspberry Pi 2 Model B: 

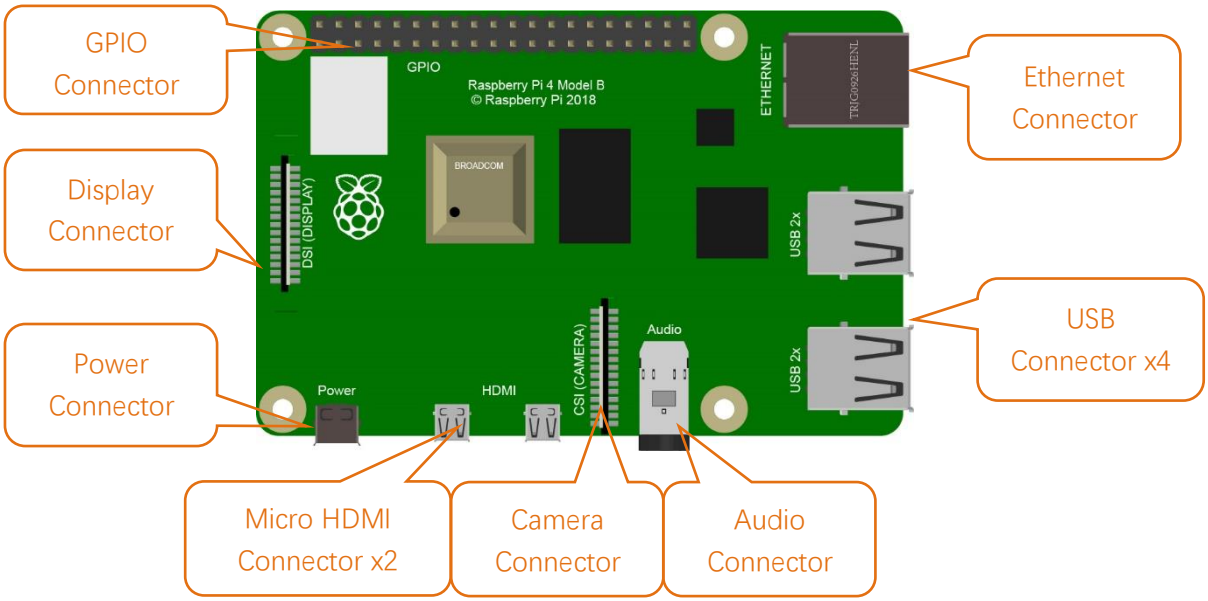
Actual image of Raspberry Pi 1 Model B+:	CAD image of Raspberry Pi 1 Model B+:
	
Actual image of Raspberry Pi 3 Model A+:	CAD image of Raspberry Pi 3 Model A+:
	

Actual image of Raspberry Pi 1 Model A+:	CAD image of Raspberry Pi 1 Model A+:
	
Actual image of Raspberry Pi Zero W:	CAD image of Raspberry Pi Zero W:
	
Actual image of Raspberry Pi Zero:	CAD image of Raspberry Pi Zero:
	

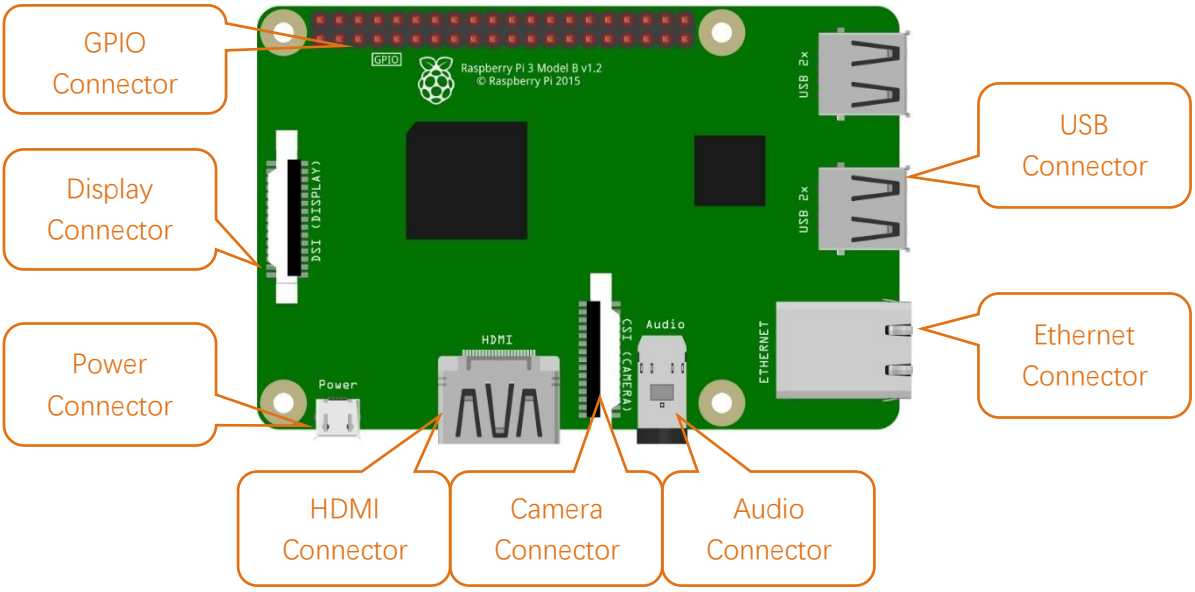
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.
Hardware interface diagram of RPi 5 is shown below:



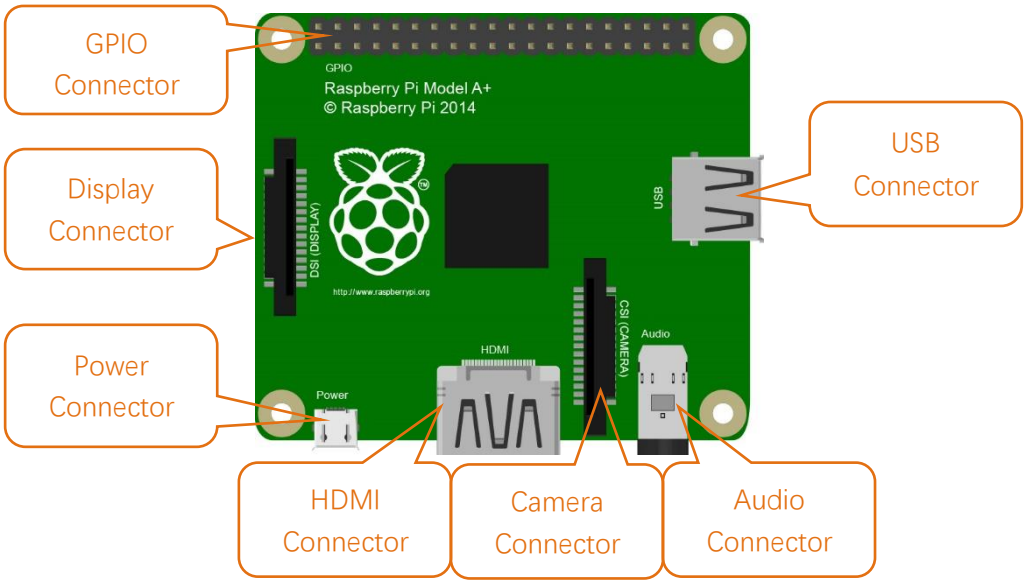
Hardware interface diagram of RPi 4B is shown below:



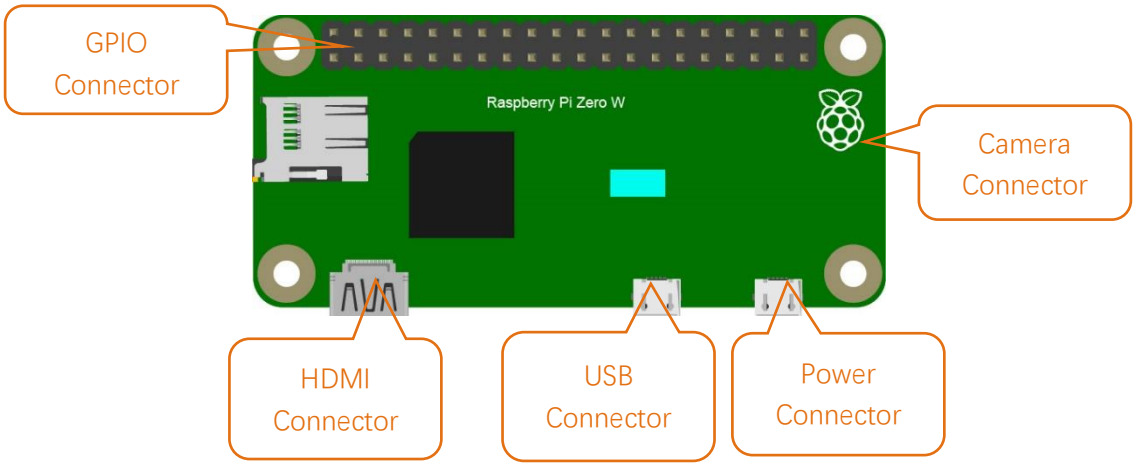
Hardware interface diagram of RPi 3B+/3B/2B/1B+:



Hardware interface diagram of RPi 3A+/A+:



Hardware interface diagram of RPi Zero/Zero W:

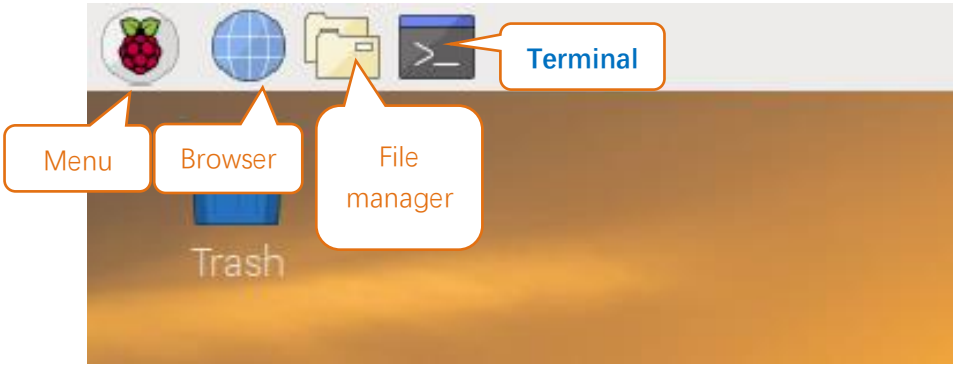


Chapter 0 Preparation

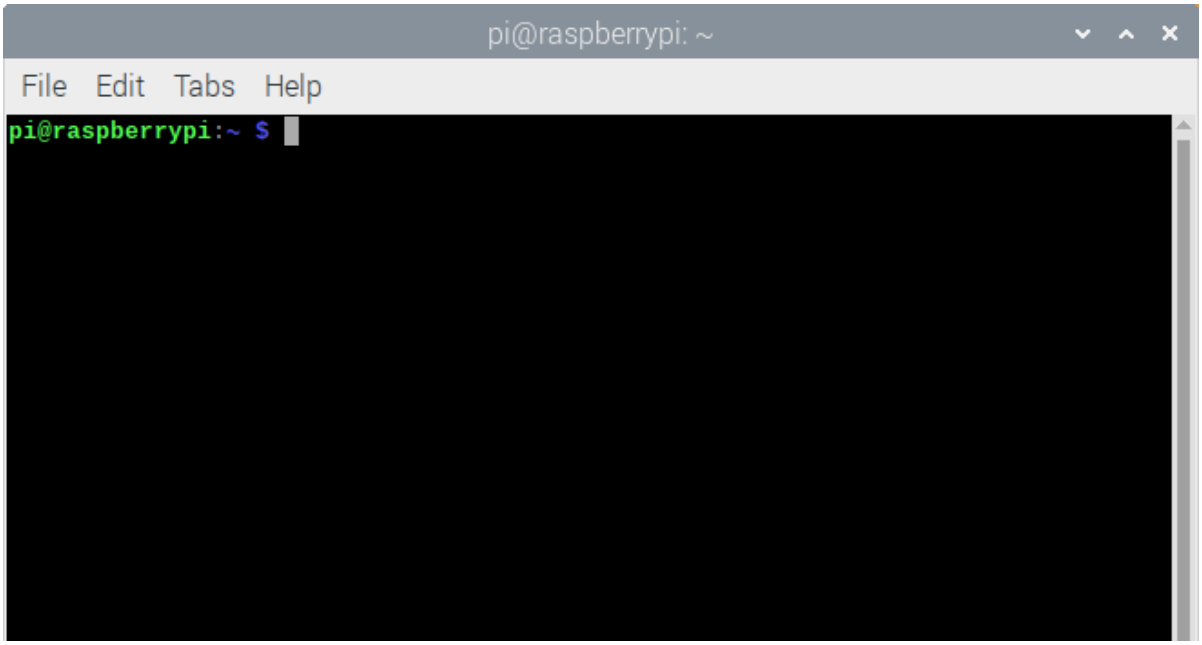
Why “Chapter 0”? Because in program code the first number is 0. We choose to follow this rule. In this chapter, we will do some necessary foundational preparation work: Start your Raspberry Pi and install some necessary libraries.

Linux Command

Raspberry Pi OS is based on the Linux Operation System. Now we will introduce you to some frequently used Linux commands and rules.
Raspberry Pi OS is based on the Linux Operation System. Now we will introduce you to some frequently used Linux commands and rules.
First, open the Terminal. All commands are executed in Terminal.

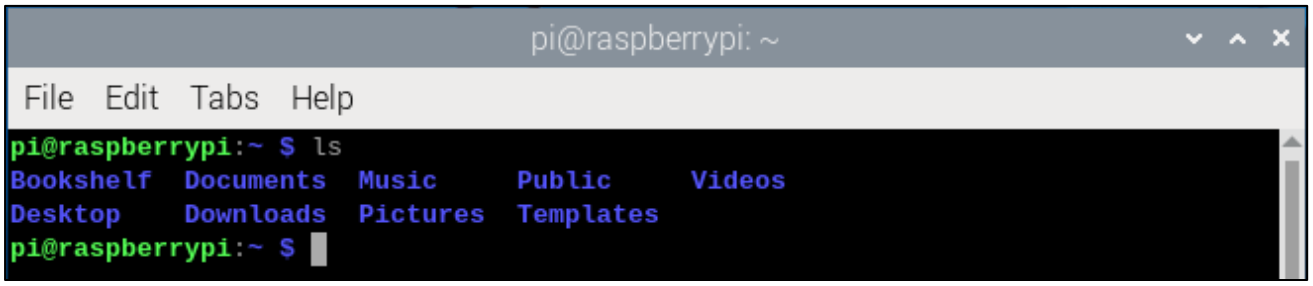


When you click the Terminal icon, following interface appears.



Note: The Linux is case sensitive.

First, type "ls" into the Terminal and press the "Enter" key. The result is shown below:



The "ls" command lists information about the files (the current directory by default).

Content between "$" and "pi@raspberrypi:" is the current working path. "~" represents the user directory, which refers to "/home/pi" here.

```
pi@raspberrypi:~ $ pwd
/home/pi
```

"cd" is used to change directory. "/" represents the root directory.

```
pi@raspberrypi:~ $ cd /usr
pi@raspberrypi:/usr $ ls
bin  games  include  lib  local  man  sbin  share  src
pi@raspberrypi:/usr $ cd ~
pi@raspberrypi:~ $
```

Later in this Tutorial, we will often change the working path. Typing commands under the wrong directory may cause errors and break the execution of further commands.

Many frequently used commands and instructions can be found in the following reference table.

Command	instruction
ls	Lists information about the FILEs (the current directory by default) and entries alphabetically.
cd	Changes directory
sudo + cmd	Executes cmd under root authority
./	Under current directory
gcc	GNU Compiler Collection
git clone URL	Use git tool to clone the contents of specified repository, and URL in the repository address.

There are many commands, which will come later. For more details about commands. You can refer to:
http://www.linux-commands-examples.com

Shortcut Key

Now, we will introduce several commonly used shortcuts that are very useful in Terminal.

1. **Up and Down Arrow Keys**: Pressing "↑" (the Up key) will go backwards through the command history and pressing "↓" (the Down Key) will go forwards through the command history.

2. **Tab Key**: The Tab key can automatically complete the command/path you want to type. When there is only one eligible option, the command/path will be completely typed as soon as you press the Tab key even you only type one character of the command/path.

As shown below, under the '~' directory, you enter the Documents directory with the "cd" command. After typing "cd D", pressing the Tab key (there is no response), pressing the Tab key again then all the files/folders that begin with "D" will be listed. Continue to type the letters "oc" and then pressing the Tab key, the "Documents" is typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

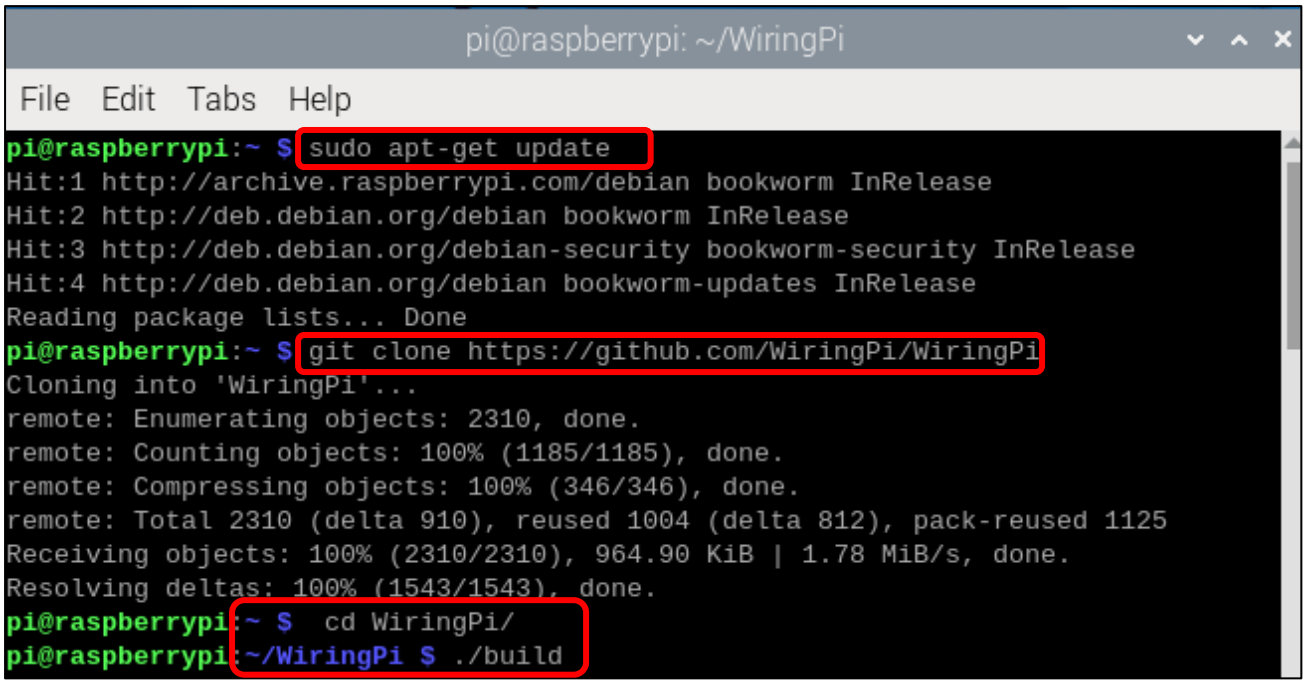
Install WiringPi

WiringPi is a GPIO access library written in C language for the used in the Raspberry Pi.

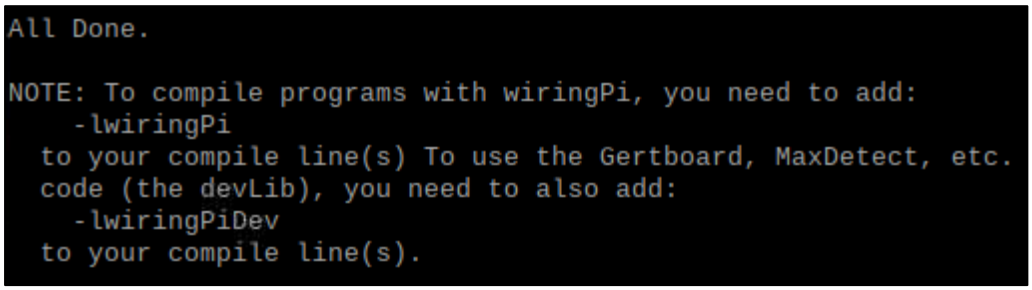
WiringPi Installation Steps

To install the WiringPi library, please open the Terminal and then follow the steps and commands below.
Note: For a command containing many lines, execute them one line at a time.
Enter the following commands **one by one** in the **terminal** to install WiringPi:

```
sudo apt-get update
git clone https://github.com/WiringPi/WiringPi
cd WiringPi
./build
```



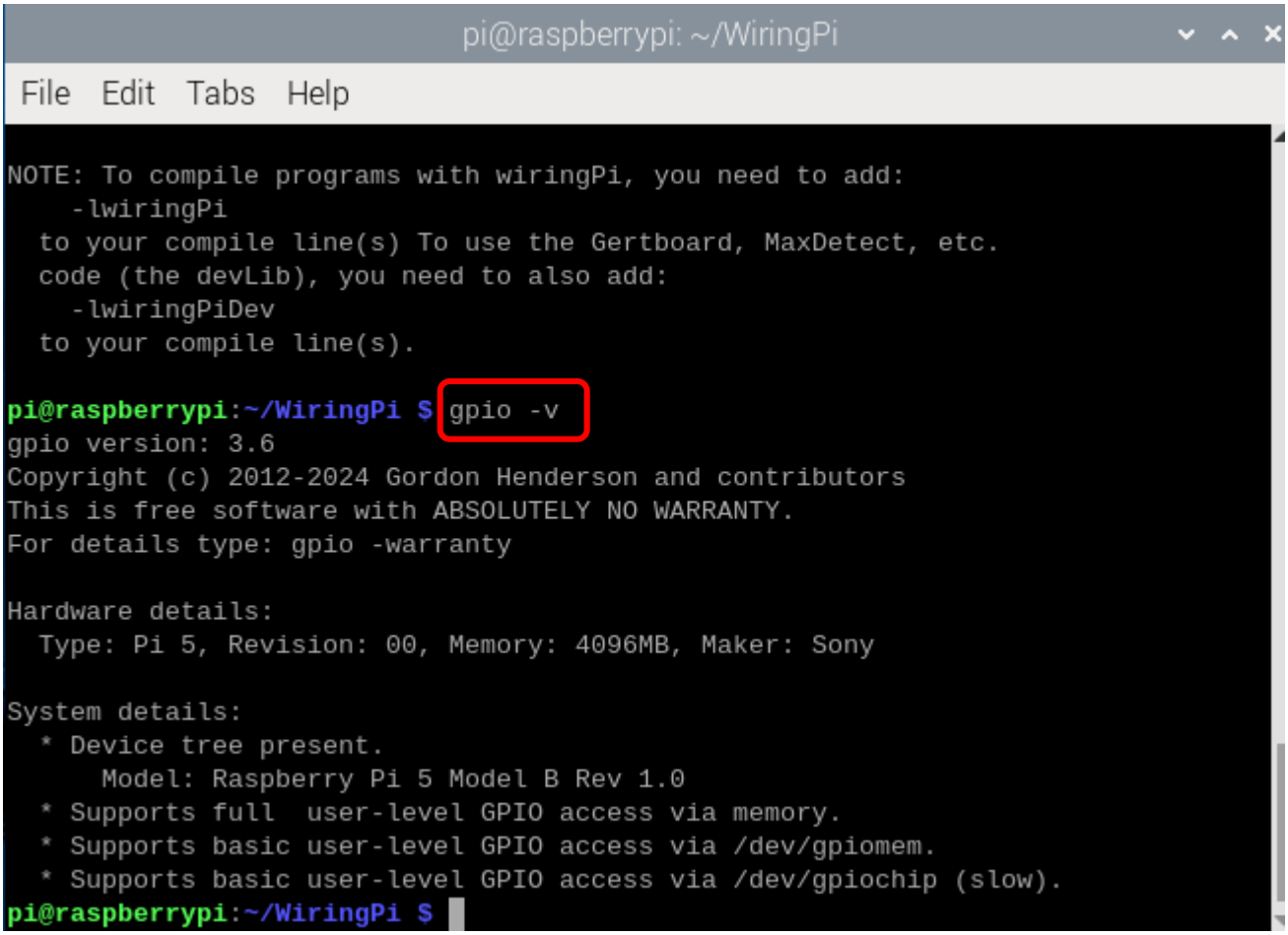
The following figure shows the successful installation.



Run the gpio command to check the installation:

```
gpio -v
```

That should give you some confidence that the installation was a success.



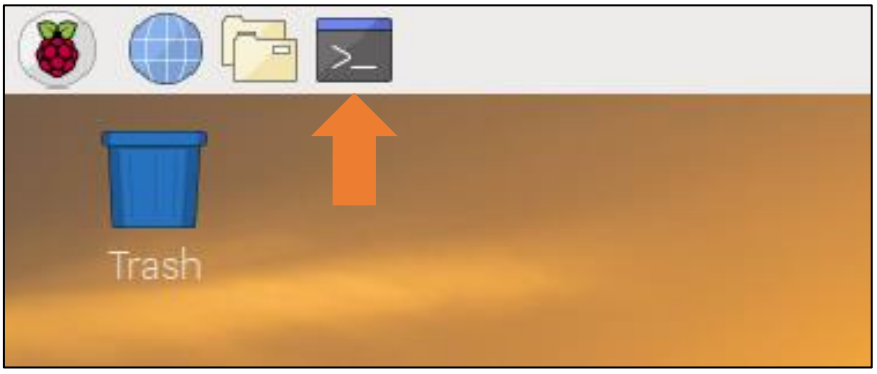
Obtain the Project Code

After the above installation is completed, you can visit our official website (http://www.freenove.com) or our GitHub resources at (https://github.com/freenove) to download the latest available project code. We provide both **C** language and **Python** language code for each project to allow ease of use for those who are skilled in either language.

This is the method for obtaining the code:
In the pi directory of the RPi terminal, enter the following command.

```
cd ~
git clone --depth 1 https://github.com/Freenove/Freenove_Projects_Kit_for_Raspberry_Pi.git
```

(There is no need for a password. If you get some errors, please check your commands.)

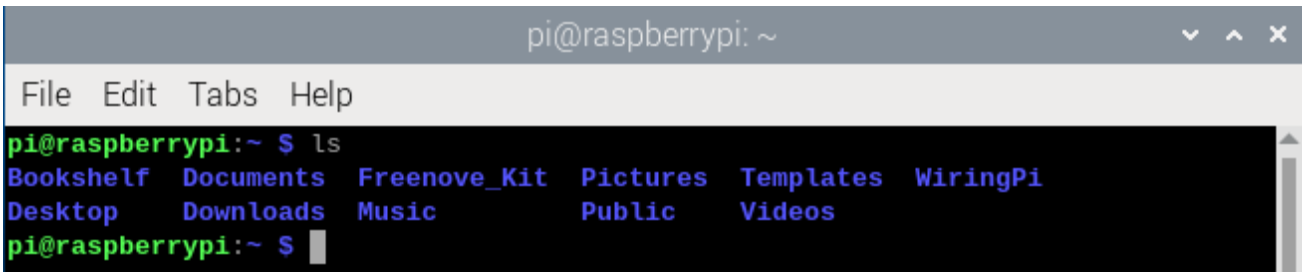


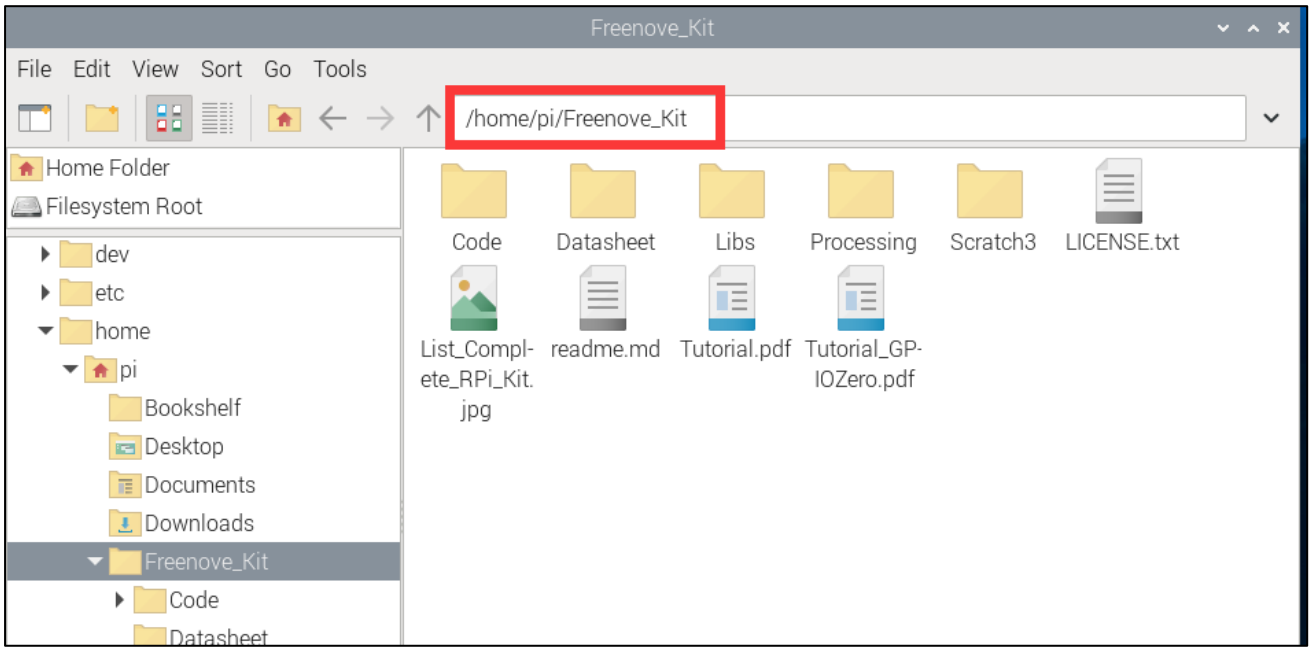

```
File  Edit  Tabs  Help
pi@raspberrypi:~ $ git clone https://github.com/Freenove/Freenove_Projects_Kit_for_Raspberry_Pi.git
Cloning into 'Freenove_Projects_Kit_for_Raspberry_Pi'...
remote: Enumerating objects: 439, done.
remote: Counting objects: 100% (94/94), done.
remote: Compressing objects: 100% (59/59), done.
remote: Total 439 (delta 34), reused 90 (delta 32), pack-reused 345
Receiving objects: 100% (439/439), 147.96 MiB | 1.13 MiB/s, done.
Resolving deltas: 100% (75/75), done.
pi@raspberrypi:~ $
```

After the download is completed, a new folder "Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi" is generated, which contains all of the tutorials and required code.

This folder name looks a little too long. We can simply rename it by using the following command.

```
mv Freenove_Projects_Kit_for_Raspberry_Pi/ Freenove_Kit/
```





If you have no experience with Python, we suggest that you refer to this website for basic information and knowledge.

https://python.swaroopch.com/basics.html

Python2 & Python3

If you only use C/C++, you can skip this section.

Python code, used in our kits, can now run on Python2 and Python3. **Python3 is recommended**. If you want to use Python2, please make sure your Python version is 2.7 or above. Python2 and Python3 are not fully compatible. However, Python2.6 and Python2.7 are transitional versions to python3, therefore you can also use Python2.6 and 2.7 to execute some Python3 code.

You can type "python2" or "python3" respectively into Terminal to check if python has been installed. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python2
Python 2.7.13 (default, Nov 24 2017, 17:33:09)
[GCC 6.3.0 20170516] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
[2]+  Stopped                 python2
pi@raspberrypi:~ $ python3
Python 3.5.3 (default, Jan 19 2017, 14:11:04)
[GCC 6.3.0 20170124] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Type "python", and Terminal shows that it links to python2.

```
pi@raspberrypi:~ $ python
Python 2.7.13 (default, Nov 24 2017, 17:33:09)
[GCC 6.3.0 20170516] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If you want to use Python3 in Raspberry Pi, it is recommended to set python3 as default Python by following the steps below.

1. Enter directory /usr/bin

```
cd /usr/bin
```

2. Delete the old python link.

```
sudo rm python
```

3. Create new python links to python3.

```
sudo ln -s python3 python
```

4. Execute python to check whether the link succeeds.

```
python
```

```
pi@raspberrypi:/usr/bin $ sudo rm python
pi@raspberrypi:/usr/bin $ sudo ln -s python3 python
pi@raspberrypi:/usr/bin $ python
Python 3.5.3 (default, Jan 19 2017, 14:11:04)
[GCC 6.3.0 20170124] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

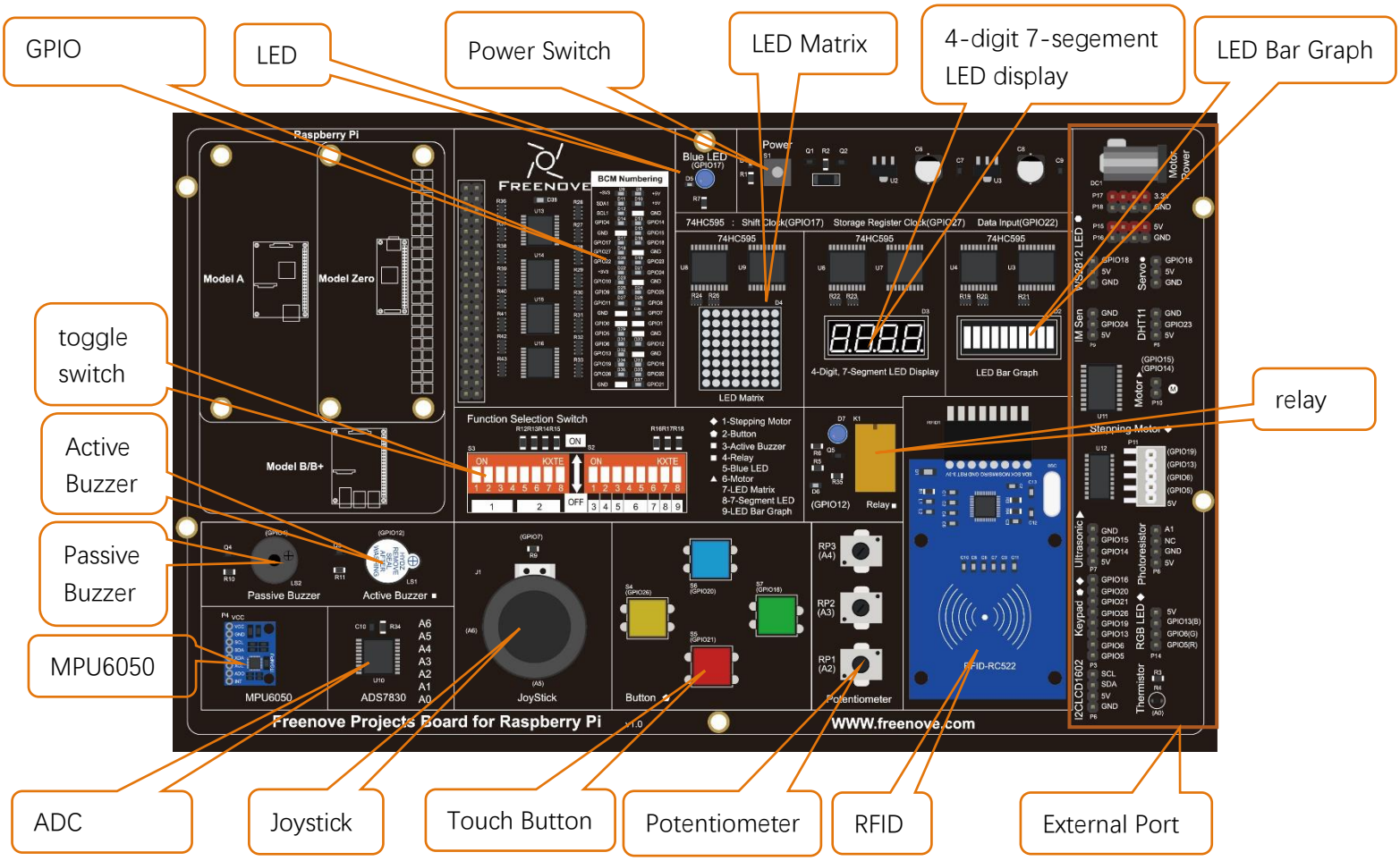
If you want to use Python2, repeat the steps above and just change the third command to the following:

```
sudo ln -s python2 python
```

```
pi@raspberrypi:/usr/bin $ sudo rm python
pi@raspberrypi:/usr/bin $ sudo ln -s python2 python
pi@raspberrypi:/usr/bin $ python
Python 2.7.13 (default, Nov 24 2017, 17:33:09)
[GCC 6.3.0 20170516] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

We will only use the term “Python” without reference to Python2 or Python3. You can choose to use either. Finally, all the necessary preparations have been completed! Next, we will combine the RPi and electronic components to build a series of projects from easy to the more challenging and difficult as we focus on learning the associated knowledge of each electronic circuit.

Projects Board for Raspberry Pi

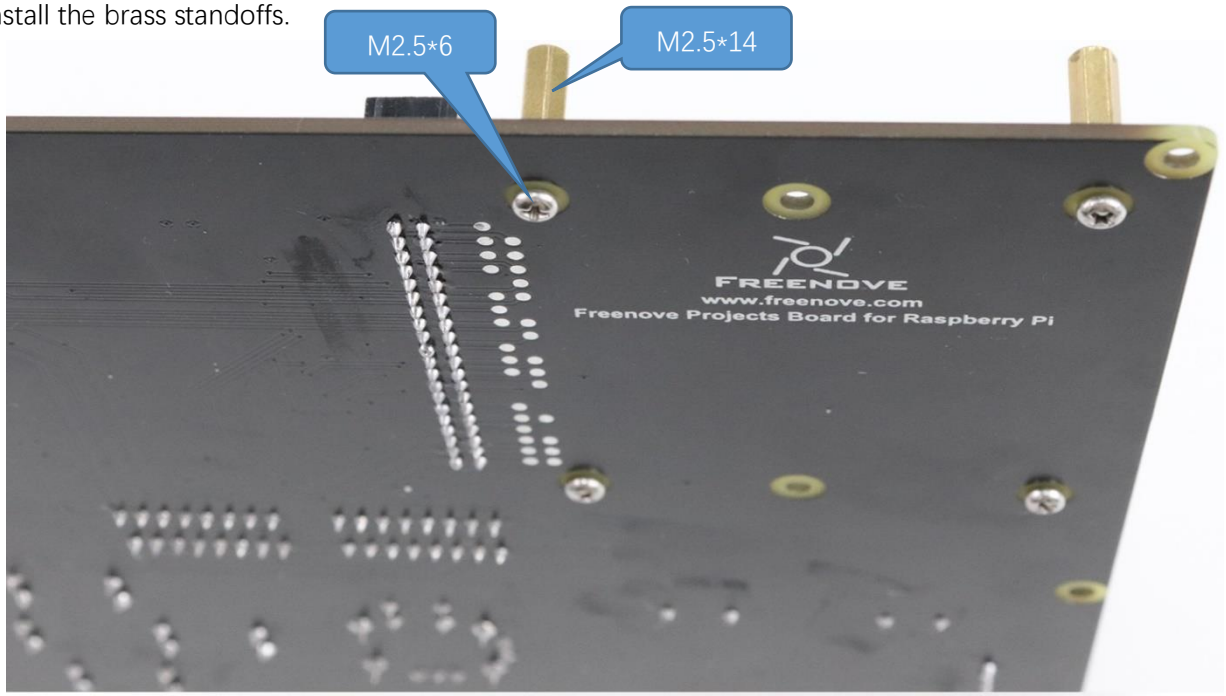


Note:

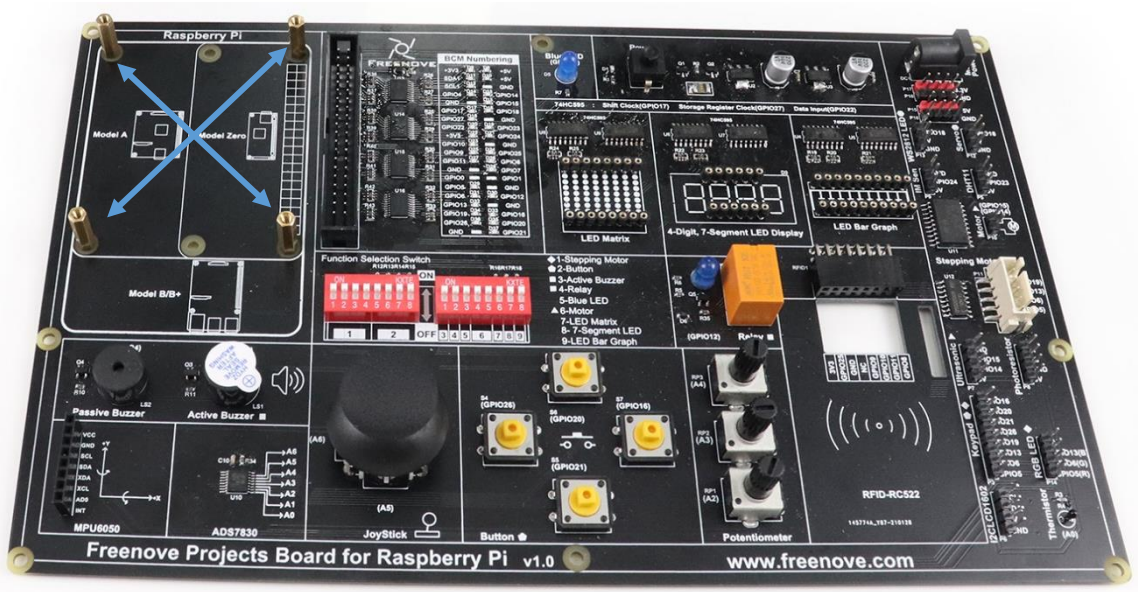
1. Stepper motor, keypad and RGBLED must NOT be used at the same time.
2. Touch button and keypad must NOT be used at the same time.
3. Active buzzer and relay must NOT be used at the same time.
4. Motor and ultrasonic module must NOT be used at the same time.
5. Servo and WS2812LED must NOT be used at the same time.
6. Batteries need to be plugged in when using the motor.

Assembly

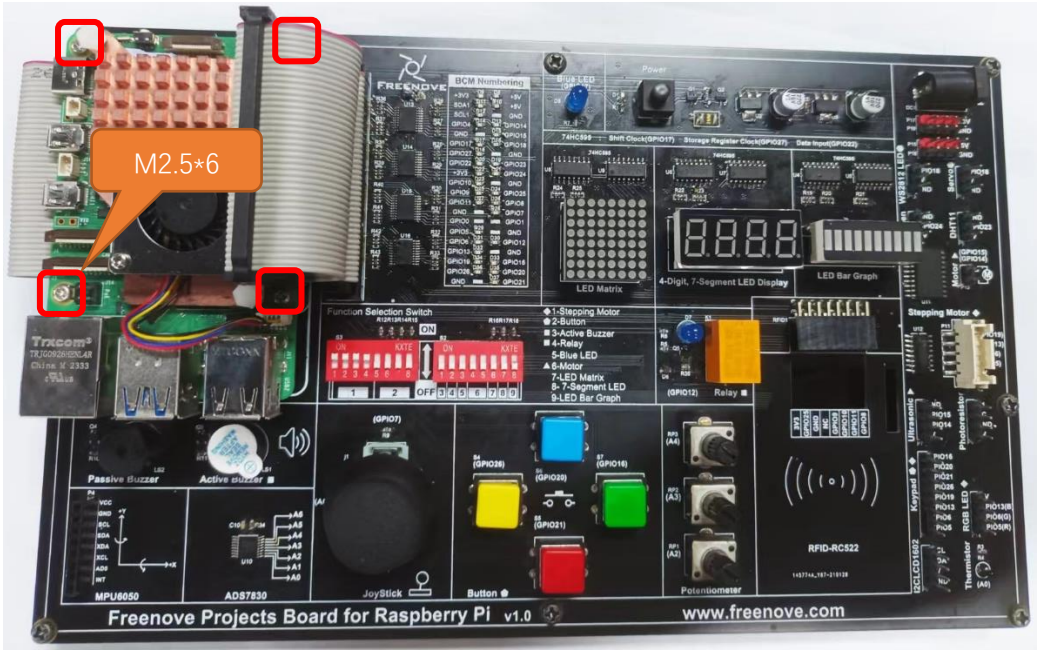
Install the brass standoffs.



Finish

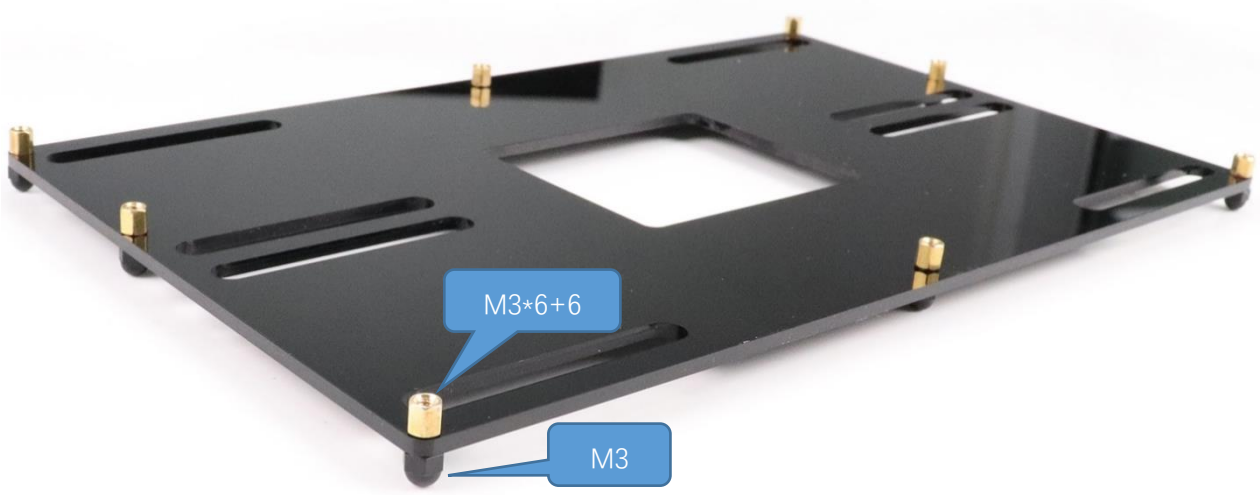


Install the Raspberry Pi.

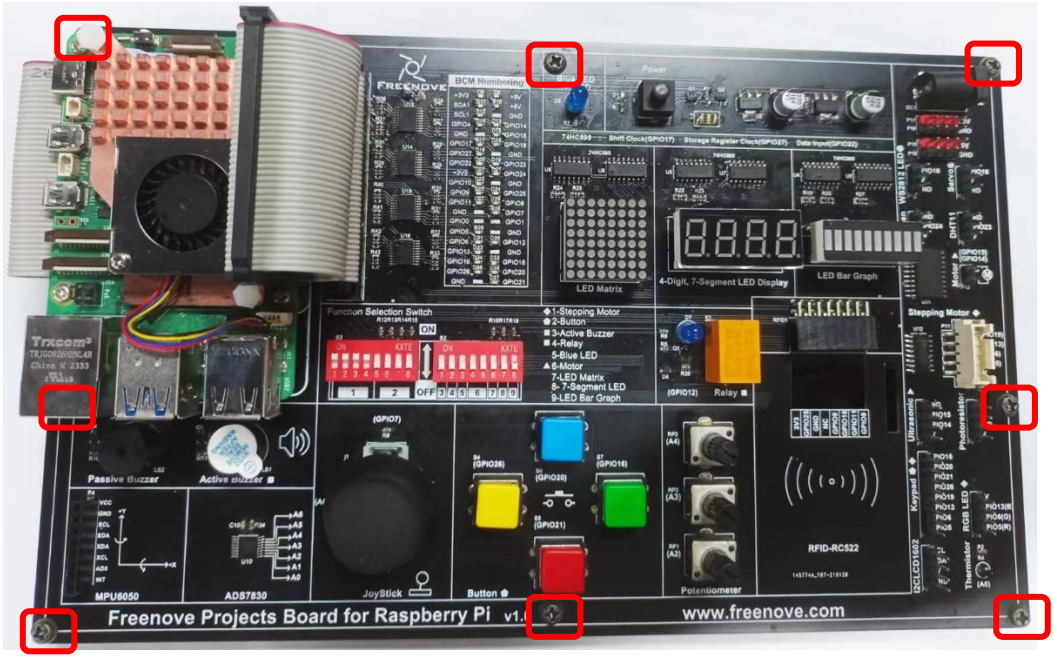


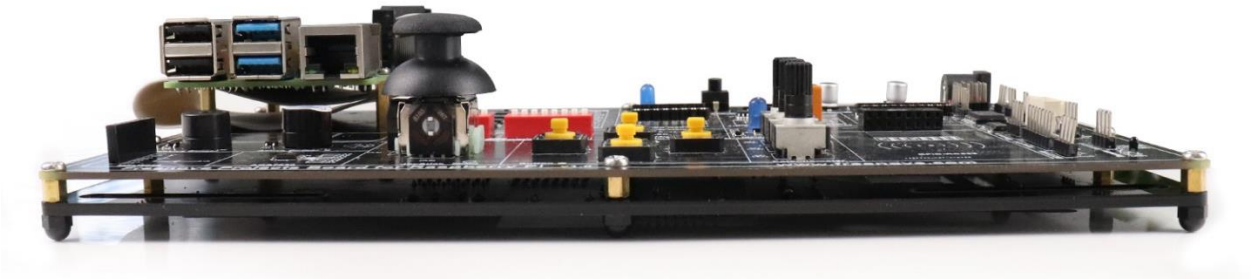


Install the acrylic part



Finish


Freenove Projects Board for Raspberry Pi
www.freenove.com
Passive Buzzer
Active Buzzer
MPU6050
ADS7830
JoyStick
Button
Potentiometer
RFID-RC522
LED Matrix
4-Digit, 7-Segment LED Display
LED Bar Graph
Stepping Motor
Relay



Chapter 1 LED

This chapter is the Start Point in the journey to build and explore RPi electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

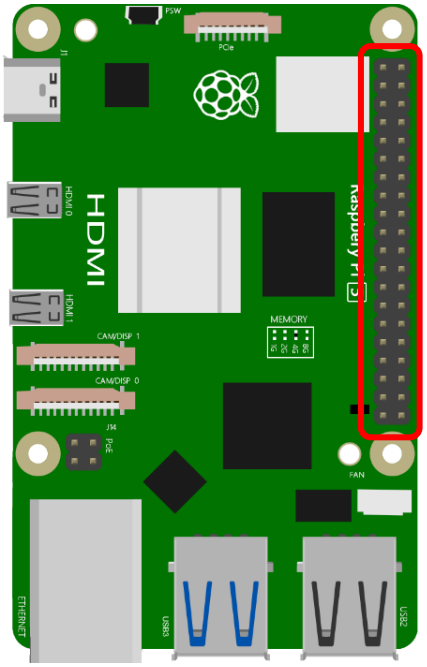
In this project, we will use RPi to control blinking a common LED.

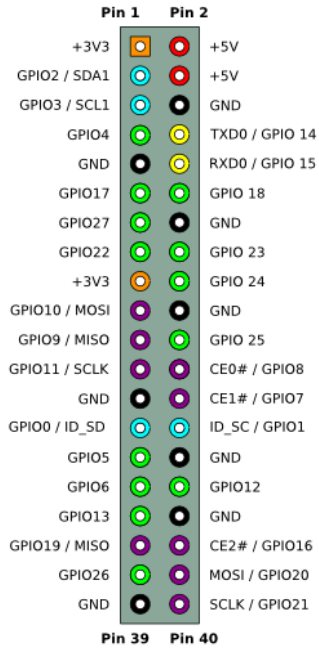
GPIO

GPIO: General Purpose Input/Output. Here we will introduce the specific function of the pins on the Raspberry Pi and how you can utilize them in all sorts of ways in your projects. Most RPi Module pins can be used as either an input or output, depending on your program and its functions.
When programming GPIO pins, there are 3 different ways to reference them: GPIO Numbering, Physical Numbering and WiringPi GPIO Numbering.

BCM GPIO Numbering

The Raspberry Pi CPU uses Broadcom (BCM) processing chips BCM2835, BCM2836 or BCM2837. GPIO pin numbers are assigned by the processing chip manufacturer and are how the computer recognizes each pin. The pin numbers themselves do not make sense or have meaning as they are only a form of identification. Since their numeric values and physical locations have no specific order, there is no way to remember them, so you will need to have a printed reference or a reference board that fits over the pins.
Each pin’s functional assignment is defined in the image below:

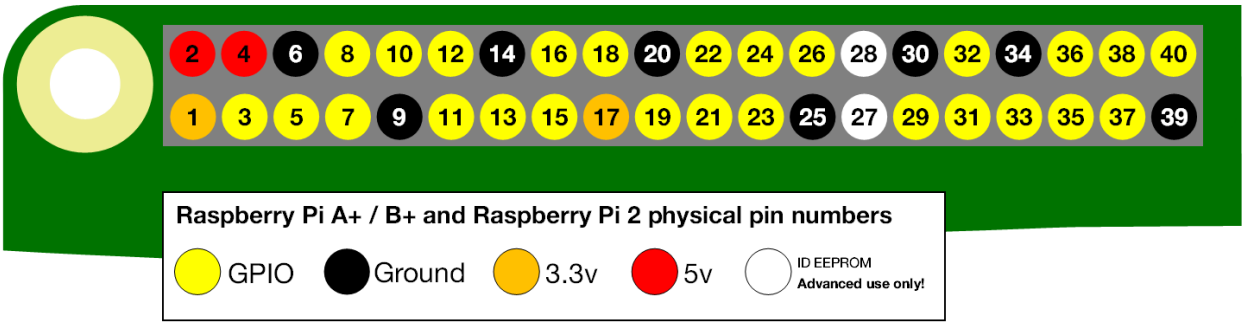




For more details about pin definition of GPIO, please refer to http://pinout.xyz/

PHYSICAL Numbering

Another way to refer to the pins is by simply counting across and down from pin 1 at the top left (nearest to the SD card). This is 'Physical Numbering', as shown below:



WiringPi GPIO Numbering

Different from the previous two types of GPIO serial numbers, RPi GPIO serial number of the WiringPi are numbered according to the BCM chip used in RPi.

wiringPi Pin	BCM GPIO	Name	Header	Name	BCM GPIO	wiringPi Pin	
—	—	3.3v	1 | 2	5v	—	—	For A+, B+, 2B, 3B, 3B+, 4B, Zero
8	R1:0/R2:2	SDA	3 | 4	5v	—	—	
9	R1:1/R2:3	SCL	5 | 6	0v	—	—	
7	4	GPIO7	7 | 8	TxD	14	15	
—	—	0v	9 | 10	RxD	15	16	
0	17	GPIO0	11 | 12	GPIO1	18	1	
2	R1:21/R2:27	GPIO2	13 | 14	0v	—	—	
3	22	GPIO3	15 | 16	GPIO4	23	4	
—	—	3.3v	17 | 18	GPIO5	24	5	
12	10	MOSI	19 | 20	0v	—	—	
13	9	MISO	21 | 22	GPIO6	25	6	
14	11	SCLK	23 | 24	CE0	8	10	
—	—	0v	25 | 26	CE1	7	11	
30	0	SDA.0	27 | 28	SCL.0	1	31	
21	5	GPIO.21	29 | 30	0V			
22	6	GPIO.22	31 | 32	GPIO.26	12	26	
23	13	GPIO.23	33 | 34	0V			
24	19	GPIO.24	35 | 36	GPIO.27	16	27	
25	26	GPIO.25	37 | 38	GPIO.28	20	28	
		0V	39 | 40	GPIO.29	21	29	
wiringPi Pin	BCM GPIO	Name	Header	Name	BCM GPIO	wiringPi Pin	

For Pi B

(For more details, please refer to https://projects.drogon.net/raspberry-pi/wiringpi/pins/)

You can also use the following command to view their correlation.

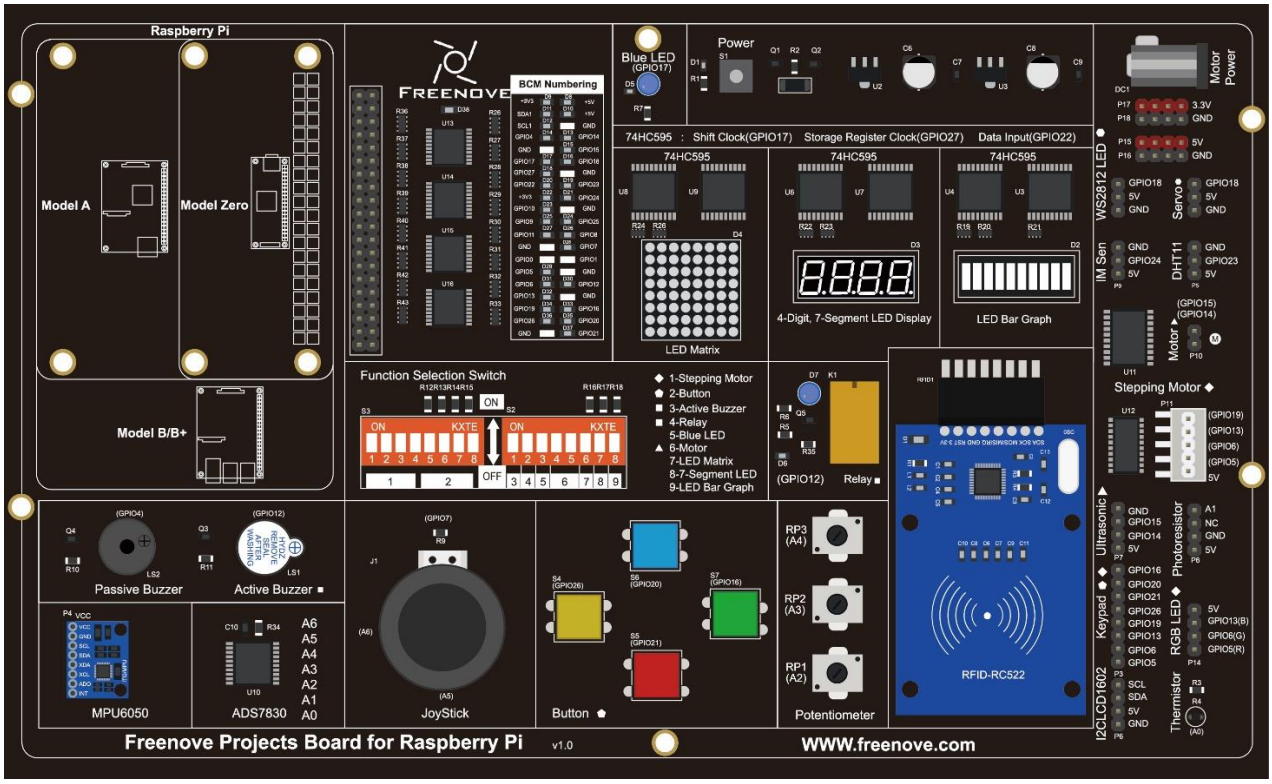
```
gpio readall
```

```
pi@raspberrypi:~ $ gpio readall
 +-----+-----+---------+------+---+---Pi 5---+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 |     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
 |   2 |   8 |   SDA.1 | ALT3 | 1 |  3 || 4  |   |      | 5v      |     |     |
 |   3 |   9 |   SCL.1 | ALT3 | 1 |  5 || 6  |   |      | 0v      |     |     |
 |   4 |   7 | GPIO. 7 |   - | 0 |  7 || 8  | 0 |   -  | TxD     | 15  | 14  |
 |     |     |      0v |      |   |  9 || 10 | 0 |   -  | RxD     | 16  | 15  |
 |  17 |   0 | GPIO. 0 |   - | 0 | 11 || 12 | 0 |   -  | GPIO. 1 | 1   | 18  |
 |  27 |   2 | GPIO. 2 |   - | 0 | 13 || 14 |   |      | 0v      |     |     |
 |  22 |   3 | GPIO. 3 |   - | 0 | 15 || 16 | 1 | OUT  | GPIO. 4 | 4   | 23  |
 |     |     |    3.3v |      |   | 17 || 18 | 0 |   -  | GPIO. 5 | 5   | 24  |
 |  10 |  12 |    MOSI | ALT0 | 0 | 19 || 20 |   |      | 0v      |     |     |
 |   9 |  13 |    MISO | ALT0 | 0 | 21 || 22 | 0 |   -  | GPIO. 6 | 6   | 25  |
 |  11 |  14 |    SCLK | ALT0 | 0 | 23 || 24 | 1 | OUT  | CE0     | 10  | 8   |
 |     |     |      0v |      |   | 25 || 26 | 1 | OUT  | CE1     | 11  | 7   |
 |   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
 |   5 |  21 | GPIO.21 |   - | 0 | 29 || 30 |   |      | 0v      |     |     |
 |   6 |  22 | GPIO.22 |   - | 0 | 31 || 32 | 0 |   -  | GPIO.26 | 26  | 12  |
 |  13 |  23 | GPIO.23 |   IN | 0 | 33 || 34 |   |      | 0v      |     |     |
 |  19 |  24 | GPIO.24 |   - | 0 | 35 || 36 | 0 |   -  | GPIO.27 | 27  | 16  |
 |  26 |  25 | GPIO.25 |   - | 0 | 37 || 38 | 0 |   -  | GPIO.28 | 28  | 20  |
 |     |     |      0v |      |   | 39 || 40 | 0 |   -  | GPIO.29 | 29  | 21  |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+---Pi 5---+---+------+---------+-----+-----+
```

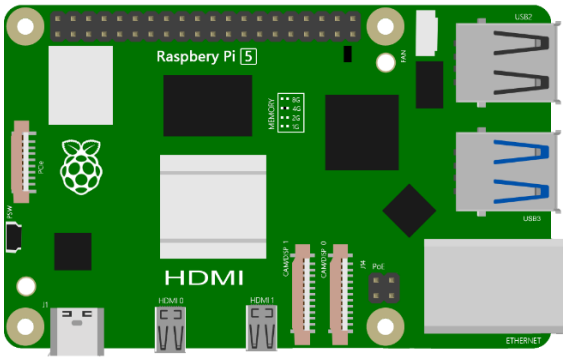
For more details about wiringPi, please refer to https://github.com/WiringPi/WiringPi .

Component List

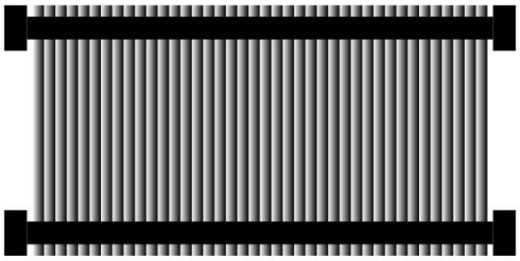
Freenove Projects Board for Raspberry Pi



Raspberry Pi

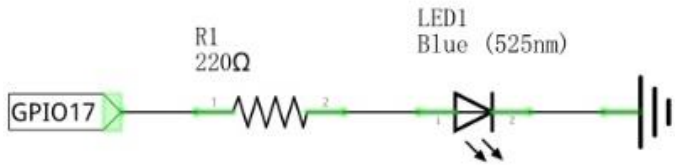


GPIO Ribbon Cable



Circuit

Schematic diagram

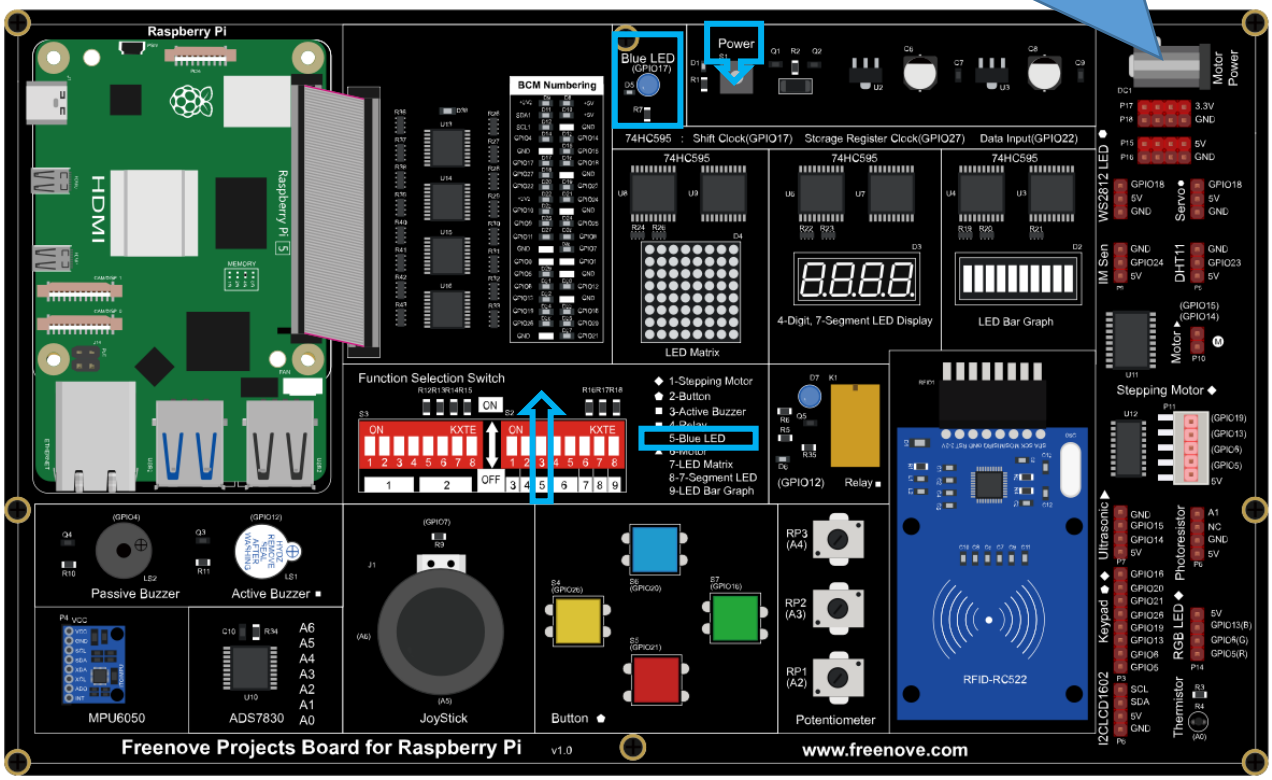


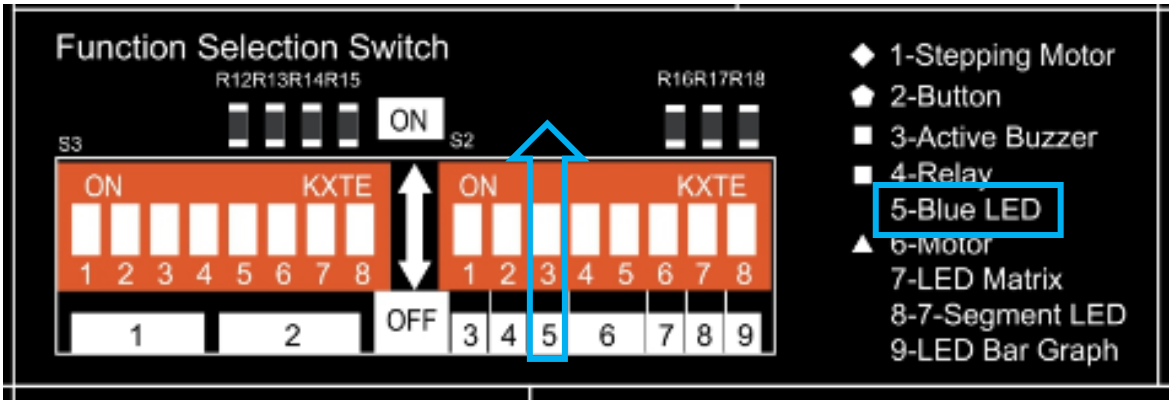
Hardware connection:

Turn ON the power switch and NO.5 toggle switch.

Power switch should be turned ON in all the projects.

It will be introducd when it is needed.





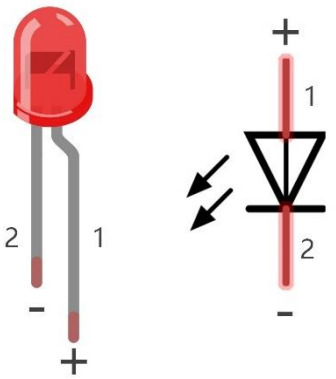
If you have any concerns, please send an email to: support@freenove.com

Component knowledge

LED

An LED is a type of diode. All diodes have two Poles and only work if current is flowing in the correct direction. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-) output, which is also referred to as Ground (GND). This type of component is known as “Polar” (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burnt out.



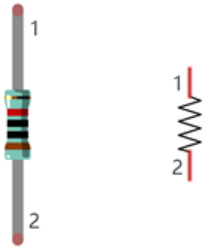
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). 1MΩ=1000kΩ, 1kΩ=1000Ω.

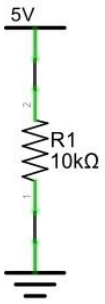
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the card in the kit package.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: I=V/R known as Ohm’s Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: I=U/R=5V/10kΩ=0.0005A=0.5mA.



WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire). This is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and re non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

Code

According to the circuit, when the GPIO17 of RPi output level is high, the LED turns ON. Conversely, when the GPIO17 RPi output level is low, the LED turns OFF. Therefore, we can let GPIO17 cycle output high and output low level to make the LED blink. We will use both C code and Python code to achieve the target.

C Code 1.1 Blink

First, enter this command into the Terminal one line at a time. Then observe the results it brings on your project, and learn about the code in detail.

If you want to execute it with editor, please refer to section Code Editor to configure.

If you have any concerns, please send an email to: support@freenove.com

It is recommended to execute the code via command line.

1. If you did not update wiring pi, please execute following commands **one by one**.

```
sudo apt-get update
git clone https://github.com/WiringPi/WiringPi
cd WiringPi
./build
```

2. Use cd command to enter 1_Blink directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/1_Blink
```

3. Use the following command to compile the code “Blink.c” and generate executable file “Blink”.

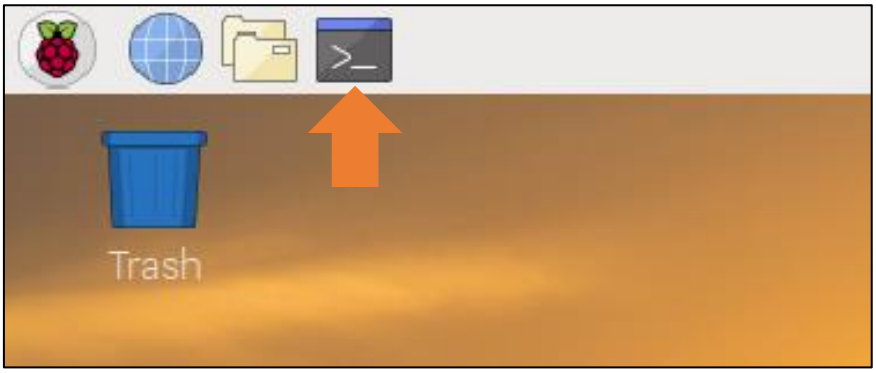
“l” of “lwiringPi” is low case of “L”.

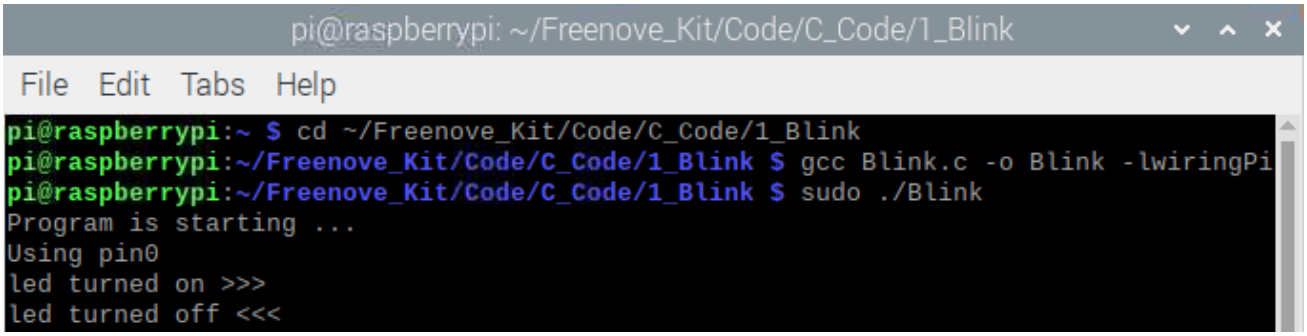
```
gcc Blink.c -o Blink -lwiringPi
```

4. Then run the generated file “blink”.

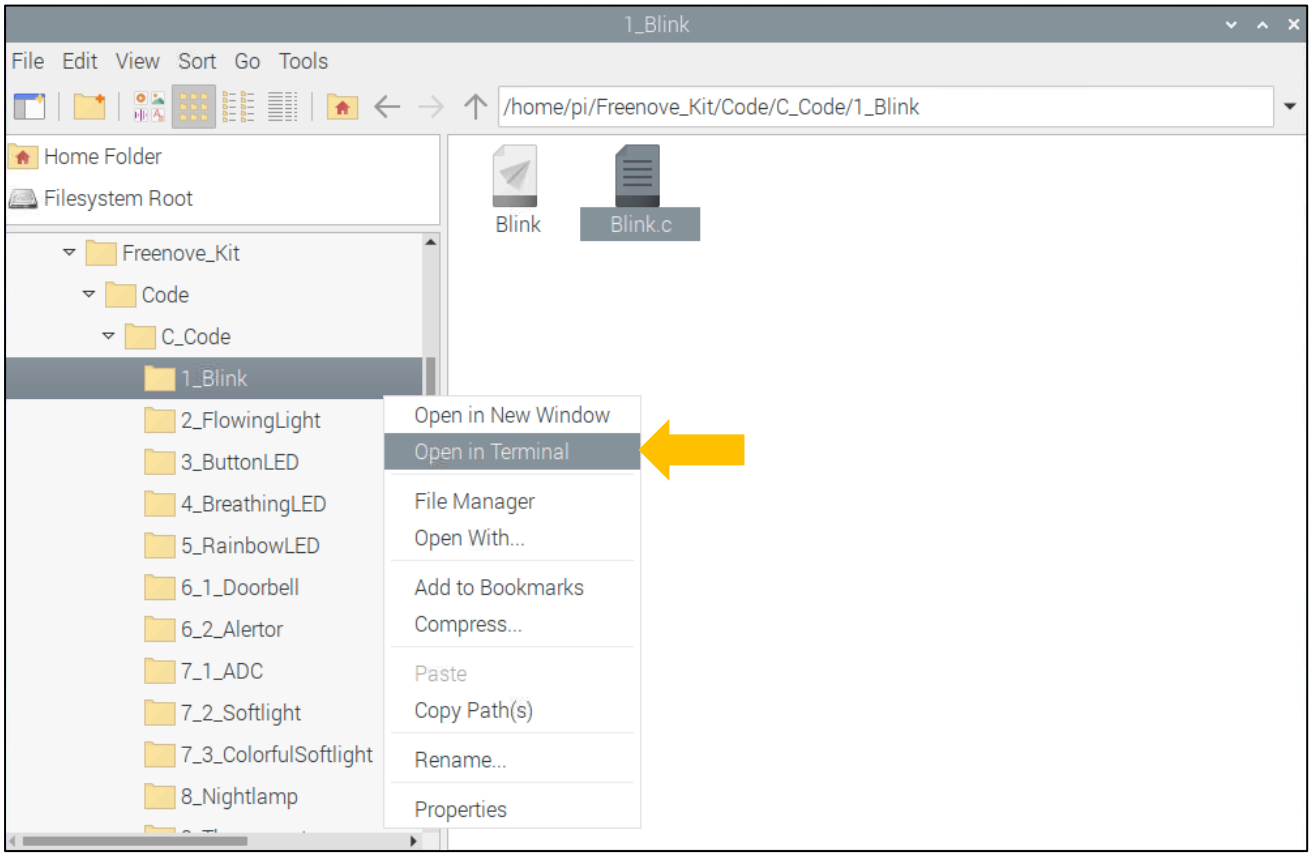
```
./Blink
```

Now your LED should start blinking! CONGRATUALTIONS! You have successfully completed your first RPi circuit!





You can also use the file browser. On the left of folder tree, right-click the folder you want to enter, and click "Open in Terminal".



You can press "Ctrl+C" to end the program. The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#define  ledPin    17 //define the led pin number
void main(void)
{
    printf("Program is starting ... \n");
    wiringPiSetupGpio();//Initialize wiringPi. Use BCM Number.
    pinMode(ledPin, OUTPUT);//Set the pin mode
    printf("Using pin%d\n",ledPin);     //Output information on terminal
    while(1){
        digitalWrite(ledPin, HIGH);  //Make GPIO output HIGH level
        printf("led turned on >>>\n");      //Output information on terminal
        delay(1000);                        //Wait for 1 second
        digitalWrite(ledPin, LOW);  //Make GPIO output LOW level
        printf("led turned off <<<\n");         //Output information on terminal
        delay(1000);                        //Wait for 1 second
    }
}
```

In the code above, the configuration function for GPIO is shown below as:

void pinMode(int pin, int mode);

This sets the mode of a pin to either INPUT, OUTPUT, PWM_OUTPUT or GPIO_CLOCK. Note that only wiringPi pin 1 (BCM_GPIO 18) supports PWM output and only wiringPi pin 7 (BCM_GPIO 4) supports CLOCK output modes.

This function has no effect when in Sys mode. If you need to change the pin mode, then you can do it with the gpio program in a script before you start your program

void digitalWrite (int pin, int value);

Writes the value HIGH or LOW (1 or 0) to the given pin, which must have been previously set as an output.

For more related wiringpi functions, please refer to https://github.com/WiringPi/WiringPi

GPIO connected to ledPin in the circuit is GPIO17.

So ledPin should be defined as 17 pin. You can refer to the **GPIO Numbering Relationship**.

```
#define  ledPin    17 //define the led pin number
```

GPIO Numbering Relationship

WiringPi	BCM(Extension)	Physical		BCM(Extension)	WiringPi
3.3V	3.3V	1	2	5V	5V
8	SDA1	3	4	5V	5V
9	SCL1	5	6	GND	GND
7	GPIO4	7	8	GPIO14/TXD0	15
GND	GND	9	10	GPIO15/RXD0	16
0	GPIO17	11	12	GPIO18	1
2	GPIO27	13	14	GND	GND
3	GPIO22	15	16	GPIO23	4
3.3V	3.3V	17	18	GPIO24	5
12	GPIO10/MOSI)	19	20	GND	GND
13	GPIO9/MOIS	21	22	GPIO25	6
14	GPIO11/SCLK	23	24	GPIO8 /CE0	10
GND	GND	25	26	GPIO7 CE1	11
30	GPIO0/SDA0	27	28	GPIO1 /SCL0	31
21	GPIO5	29	30	GND	GND
22	GPIO6	31	32	GPIO12	26
23	GPIO13	33	34	GND	GND
24	GPIO19	35	36	GPIO16	27
25	GPIO26	37	38	GPIO20	28
GND	GND	39	40	GPIO21	29

In the main function main(), initialize GPIO first.

```
    //wiringPiSetup();  //Initialize wiringPi. Use WiringPi Number.
    wiringPiSetupGpio();//Initialize wiringPi. Use BCM Number.
```

After the wiringPi is initialized successfully, you can set the ledPin to output mode and then enter the while loop, which is an endless loop (a while loop). That is, the program will always be executed in this cycle, unless it is ended because of external factors. In this loop, use digitalWrite (ledPin, HIGH) to make ledPin output high level, then LED turns ON. After a period of time delay, use digitalWrite(ledPin, LOW) to make ledPin output low level, then LED turns OFF, which is followed by a delay. Repeat the loop, then LED will start blinking.

```
    pinMode(ledPin, OUTPUT);              //Set the pin mode
    printf("Using pin%d\n",%ledPin);      //Output information on terminal
    while(1){
        digitalWrite(ledPin, HIGH);       //Make GPIO output HIGH level
        printf("led turned on >>>\n");    //Output information on terminal
        delay(1000);                      //Wait for 1 second
        digitalWrite(ledPin, LOW);        //Make GPIO output LOW level
        printf("led turned off <<<\n");   //Output information on terminal
        delay(1000);                        //Wait for 1 second
    }
```

Python Code 1.1 Blink

Now, we will use Python language to make a LED blink.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

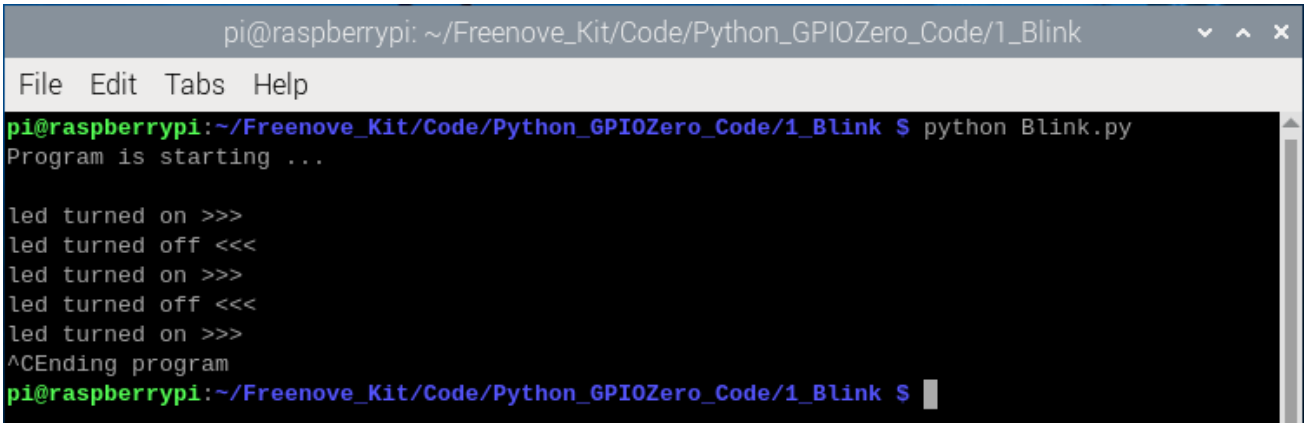
1. Use cd command to enter 1_Blink directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/1_Blink
```

2. Use python command to execute python code blink.py.

```
python Blink.py
```

The LED starts blinking.



You can press "**Ctrl+C**" to end the program. The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3
4  led = LED(17)           # define LED pin according to BCM Numbering
5  '''
6  # pins numbering, the following lines are all equivalent
7  led = LED(17)           # BCM
8  led = LED("GPIO17")     # BCM
9  led = LED("BCM17")      # BCM
10 led = LED("BOARD11")    # BOARD
11 led = LED("WPIO")       # WiringPi
12 led = LED("J8:11")      # BOARD
13 '''
14
15 def loop():
16     while True:
17         led.on()     # turn on LED
18         print ('led turned on >>>')  # print message on terminal
19         sleep(1)     # wait 1 second
20         led.off()    # turn off LED
21         print ('led turned off <<<') # print message on terminal
22         sleep(1)     # wait 1 second
23
```

```
24  if __name__ == '__main__':     # Program entrance
25      print ('Program is starting ... \n')
26      try:
27          loop()
28      except KeyboardInterrupt:  # Press ctrl-c to end the program.
29          print("Ending program")
30      finally:
31          led.close()
```

About **gpiozero.LED**:

gpiozero.LED
The gpiozero library is an object-oriented python library where the LED module is used to control leds.
LED(pin)

LED module, used to apply for a control LED object, please fill in the gpio pin correctly. Please refer to the following writing method.
led = LED(17) # BCM
led = LED("GPIO17") # BCM
led = LED("BCM17") # BCM
led = LED("BOARD11") # BOARD
led = LED("WPI0") # WiringPi
led = LED("J8:11") # BOARD

led.on()
Control GPIO output high level.
led.off()
Control GPIO output low level.
led.close()
Release the GPIO resource.

For more functions related to GPIOZERO, please refer to:
https://github.com/gpiozero/gpiozero
"import time" time is a module of python.
https://docs.python.org/2/library/time.html?highlight=time%20time#module-time

The Blue LED is controlled using GPIO17. GPIO17 is based on the BCM Number, so just fill in 17 in the code.

```
    led = LED(17)           # define LED pin according to BCM Numbering
```

GPIO Numbering Relationship

WingPi	BCM(Extension)	Physical		BCM(Extension)	WingPi
3.3V	3.3V	1	2	5V	5V
8	SDA1	3	4	5V	5V
9	SCL1	5	6	GND	GND
7	GPIO4	7	8	GPIO14/TXD0	15
GND	GND	9	10	GPIO15/RXD0	16
0	GPIO17	11	12	GPIO18	1
2	GPIO27	13	14	GND	GND
3	GPIO22	15	16	GPIO23	4
3.3V	3.3V	17	18	GPIO24	5
12	GPIO10/MOSI)	19	20	GND	GND
13	GPIO9/MOIS	21	22	GPIO25	6
14	GPIO11/SCLK	23	24	GPIO8 /CE0	10
GND	GND	25	26	GPIO7 CE1	11
30	GPIO0/SDA0	27	28	GPIO1 /SCL0	31
21	GPIO5	29	30	GND	GND
22	GPIO6	31	32	GPIO12	26
23	GPIO13	33	34	GND	GND
24	GPIO19	35	36	GPIO16	27
25	GPIO26	37	38	GPIO20	28
GND	GND	39	40	GPIO21	29

In loop(), there is a while loop, which is an endless loop (a while loop). That is, the program will always be executed in this loop, unless it is ended because of external factors. In this loop, set led pin output high level, then the LED turns ON. After a period of time delay, set led pin output low level, then the LED turns OFF, which is followed by a delay. Repeat the loop, then LED will start blinking.

```
def loop():
    while True:
        led.on()     # turn on LED
        print ('led turned on >>>')  # print message on terminal
        sleep(1)     # wait 1 second
        led.off()    # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)     # wait 1 second
```

Each time the program terminates, the led module needs to release the GPIO resource. If you do not release the GPIO pin, the GPIO pin cannot be used the next time, and an error message is displayed in Terminal. Like "GPIO Busy."

```
    led.close()
```

Each time you want to exit the program, use Ctrl+C to exit the program. Do not use Ctrl+Z or close Terminal to exit the code directly.

This may cause the GPIO resource to be occupied, making it impossible to continue calling the GPIO. If you accidentally use this action. You can use this by enter "**fg**" in Terminal, then pressing Enter. It will resume suspended tasks that occupy GPIO resources. Then you can use Ctrl+C to exit the code normally. Of course, you can also choose to restart the RPI directly. It can also be used to free up GPIO resources.

Chapter 2 FlowingLight

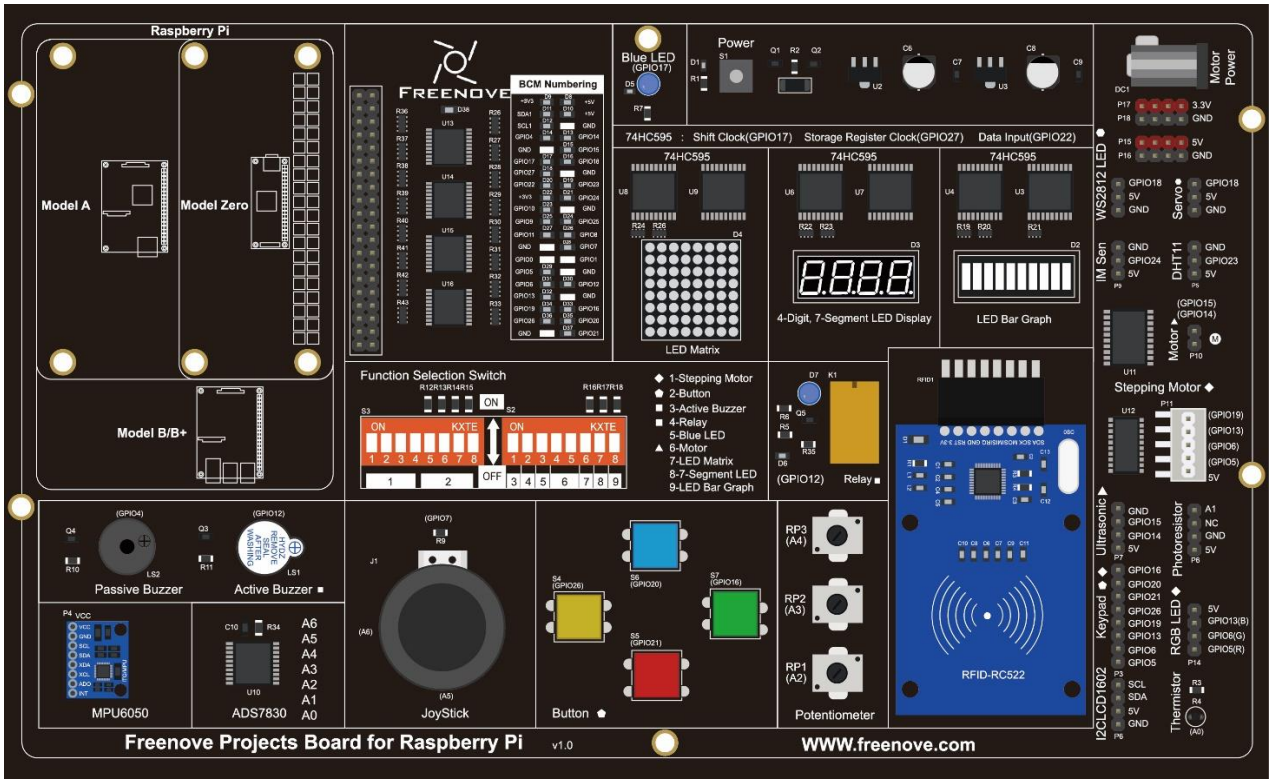
We have learned how to control one LED to blink. Next, we will learn how to control a number of LEDs.

Project 2.1 Flowing Water Light

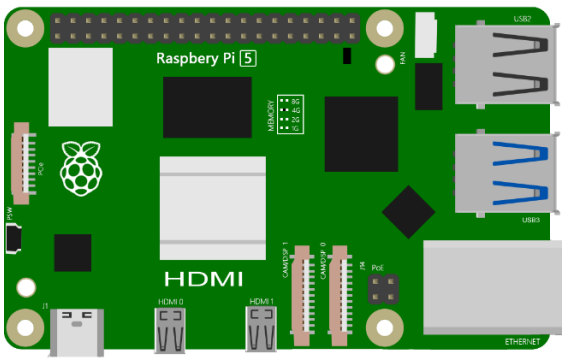
In this project, we use a number of LEDs to make a flowing water light.

Component List

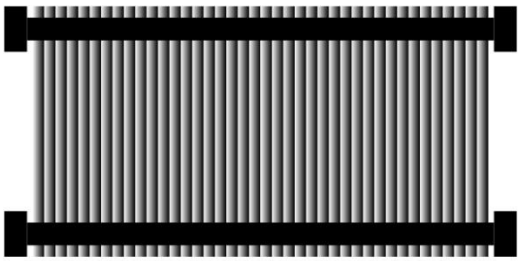
Freenove Projects Board for Raspberry Pi



Raspberry Pi

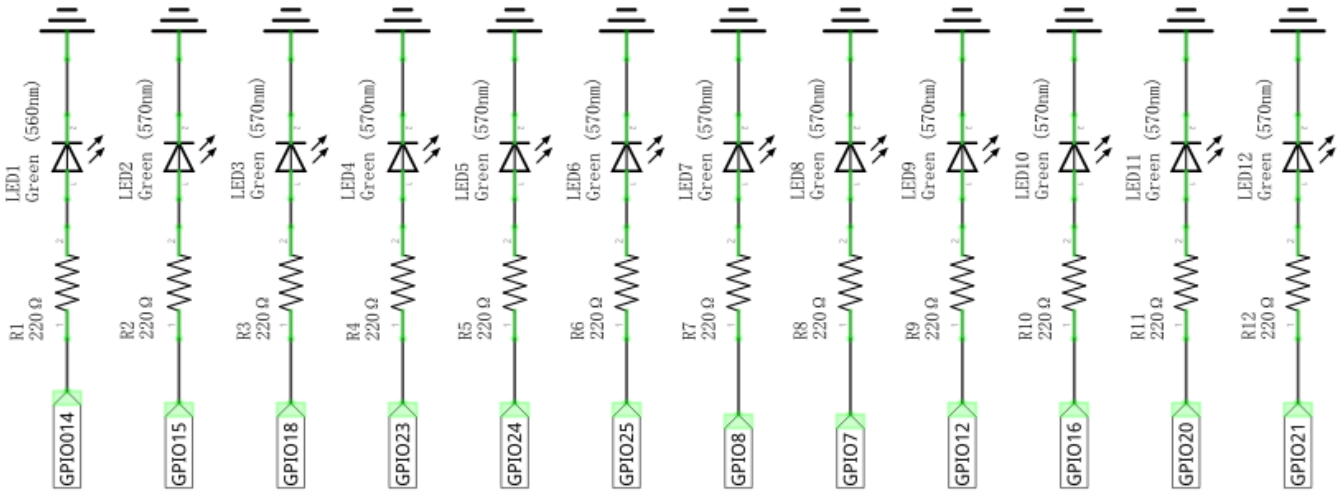


GPIO Ribbon Cable

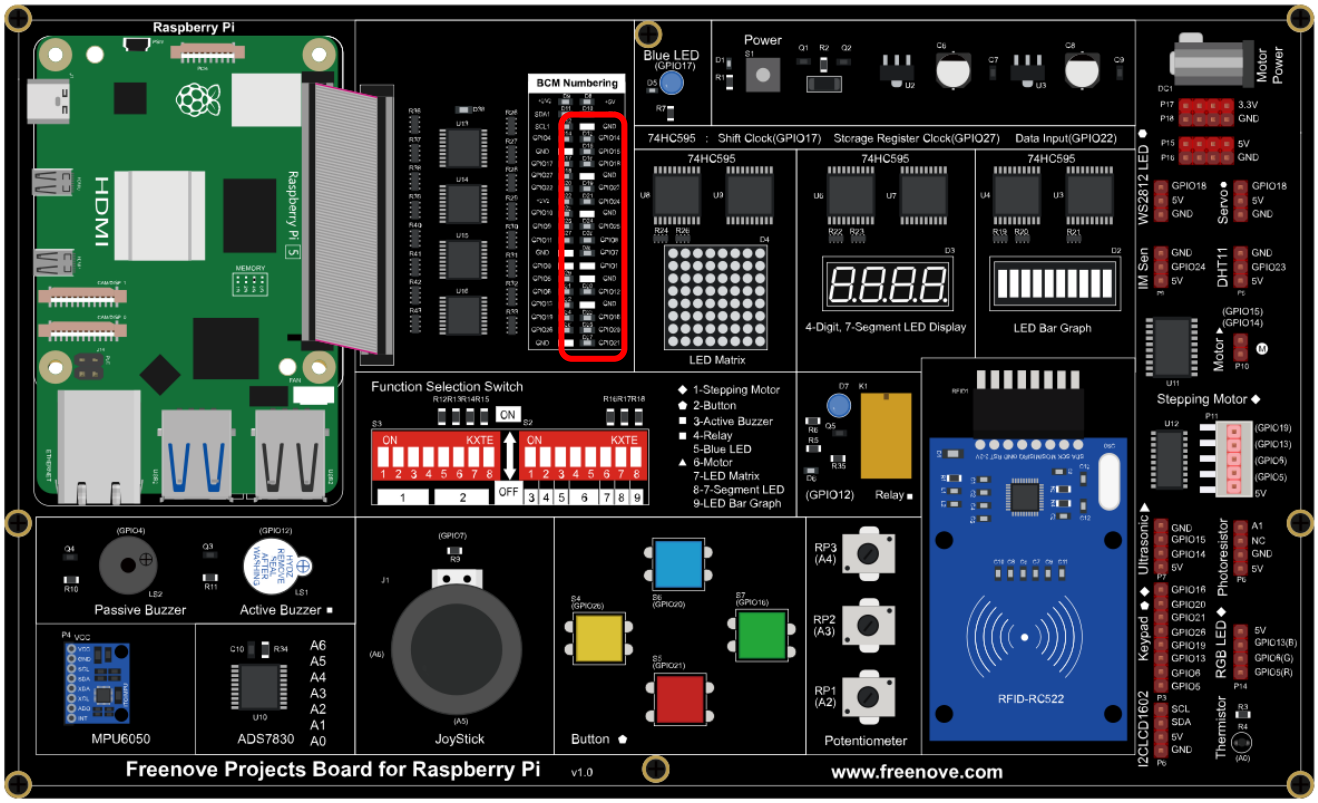


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 2.1 LightWater

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 2_FlowingLight directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/2_FlowingLight
```

2. Use the following command to compile "LightWater.c" and generate executable file "LightWater".

```
gcc FlowingLight.c -o FlowingLight -lwiringPi
```

3. Then run the generated file "LightWater".

```
./FlowingLight
```

You can see the LEDs lighting from top to bottom and then back from bottom to top.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>

#define ledCounts 12
int pins[ledCounts] = {14, 15, 18, 23, 24, 25, 8, 7, 12, 16, 20, 21};

void main(void)
{
    int i;
    printf("Program is starting ... \n");

    wiringPiSetupGpio();             //Initialize wiringPi. Use BCM Number.

    for(i=0;i<ledCounts;i++){        //Set pinMode for all led pins to output
        pinMode(pins[i], OUTPUT);
    }
    while(1){
        for(i=0;i<ledCounts;i++){   // move led(on) from top to bottom
            digitalWrite(pins[i],LOW);
            delay(100);
            digitalWrite(pins[i],HIGH);
        }
        for(i=ledCounts-1;i>-1;i--){   // move led(on) from bottom to top
            digitalWrite(pins[i],LOW);
            delay(100);
            digitalWrite(pins[i],HIGH);
        }
    }
}
```

In the "while" loop, apply two "for" loop to achieve the flowing water light lighting from top to bottom and then back from bottom to top.

```
while(1){
        for(i=0;i<ledCounts;i++){   // move led(on) from top to bottom
            digitalWrite(pins[i],LOW);
            delay(100);
            digitalWrite(pins[i],HIGH);
        }
        for(i=ledCounts-1;i>-1;i--){   // move led(on) from bottom to top
            digitalWrite(pins[i],LOW);
            delay(100);
            digitalWrite(pins[i],HIGH);
        }
    }
```

Python Code 2.1 LightWater

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 2_FlowingLight directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/2_FlowingLight
```

2. Use Python command to execute Python code "LightWater.py".

```
python LightWater.py
```

You can see the LEDs lighting from top to bottom and then back from bottom to top.

The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3
4  ledPins = [14, 15, 18, 23, 24, 25, 8, 7, 12, 16, 20, 21]
5  leds = [LED(pin=pin) for pin in ledPins]
6
7  def loop():
8      while True:
9          for index in range(0,len(ledPins),1):      # make led(on) move from left to right
10             leds[index].on()
11             sleep(0.1)
12             leds[index].off()
13         for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
14             leds[index].on()
15             sleep(0.1)
16             leds[index].off()
17
18 if __name__ == '__main__':     # Program entrance
19     print ('Program is starting...')
20     try:
```

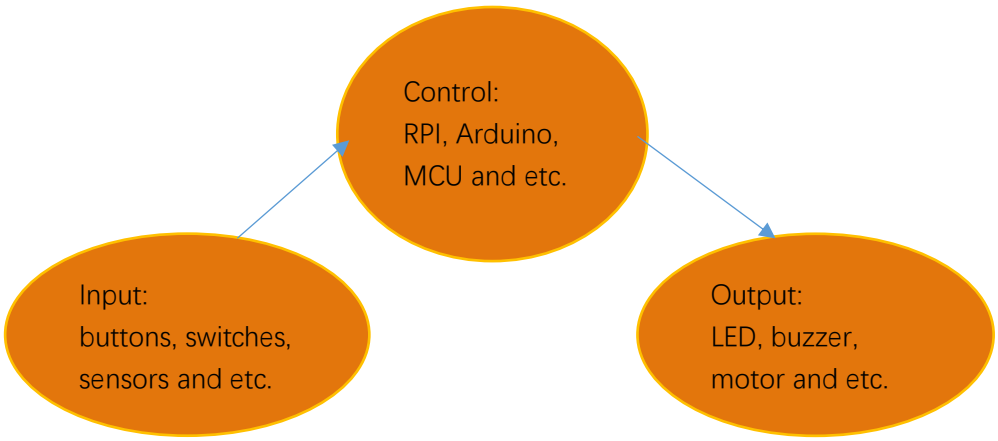
```
21          loop()
22      except KeyboardInterrupt:  # Press ctrl-c to end the program.
23          print("Ending program")
24      finally:
25          for index in range(0,len(ledPins),1):
26              leds[index].close()
```

In the "while" loop, apply two "for" loop to achieve the flowing water light lighting from top to bottom and then back from bottom to top.

```
def loop():
    while True:
        for index in range(0,len(ledPins),1):      # make led(on) move from left to right
            leds[index].on()
            sleep(0.1)
            leds[index].off()
        for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
            leds[index].on()
            sleep(0.1)
            leds[index].off()
```

Chapter 3 Buttons & LEDs

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and RPI was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as turn on LEDs, make a buzzer beep and so on.



Next, we will build a simple control system to control an LED through a push button switch.

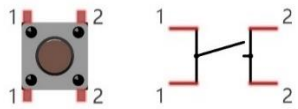
Project 3.1 Push Button Switch & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

Component knowledge

Push Button Switch

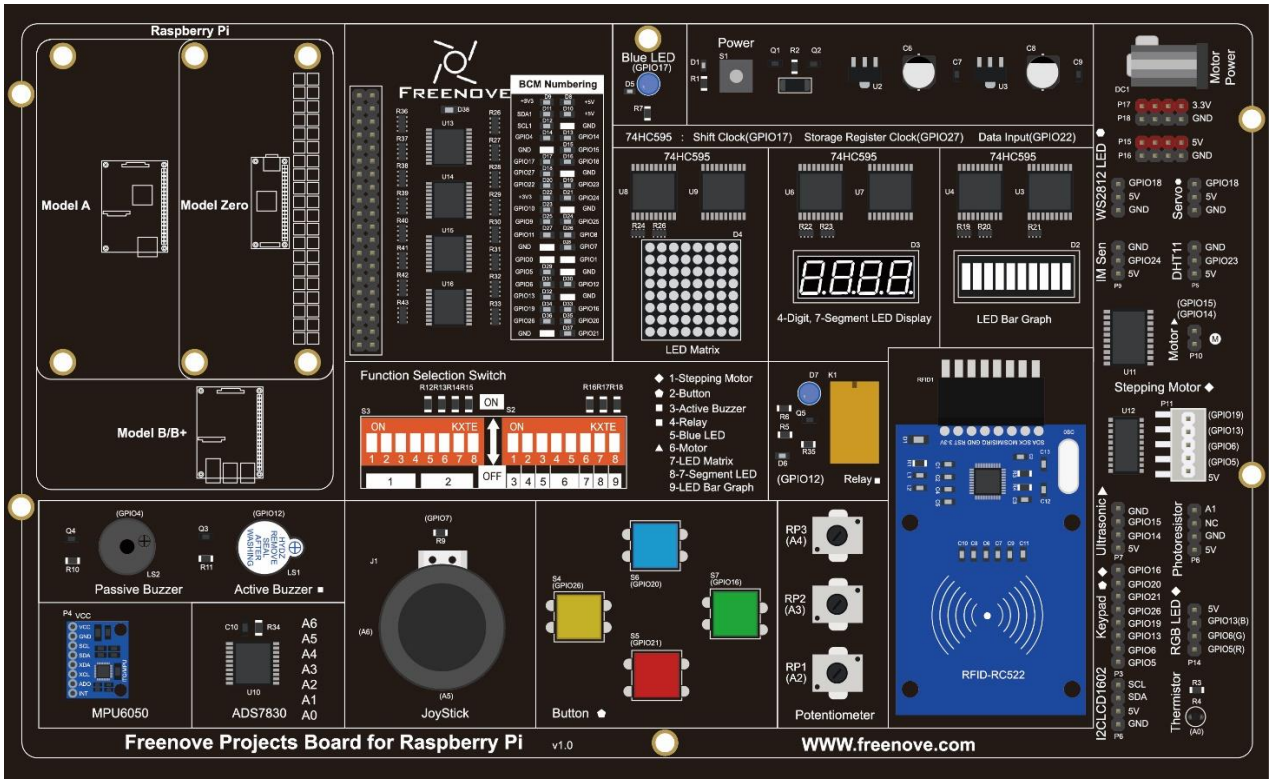
This type of Push Button Switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same as per the illustration:



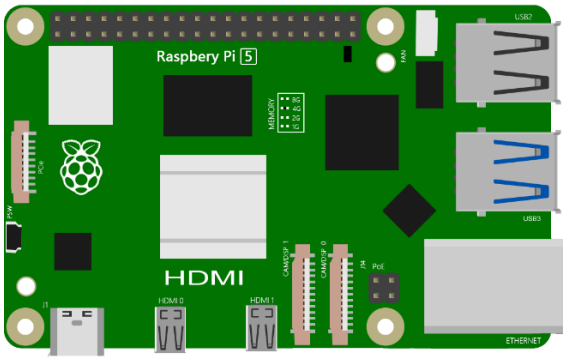
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Component List

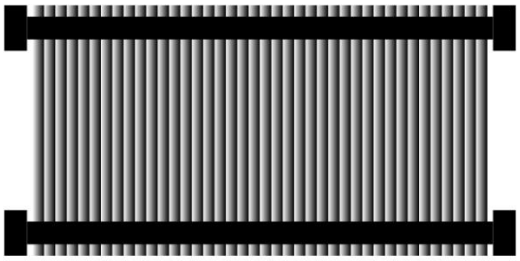
Freenove Projects Board for Raspberry Pi



Raspberry Pi

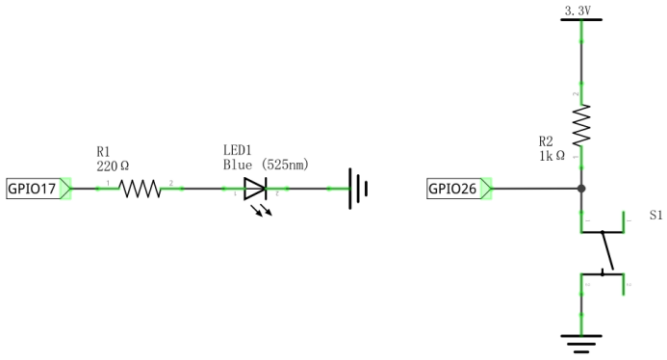


GPIO Ribbon Cable



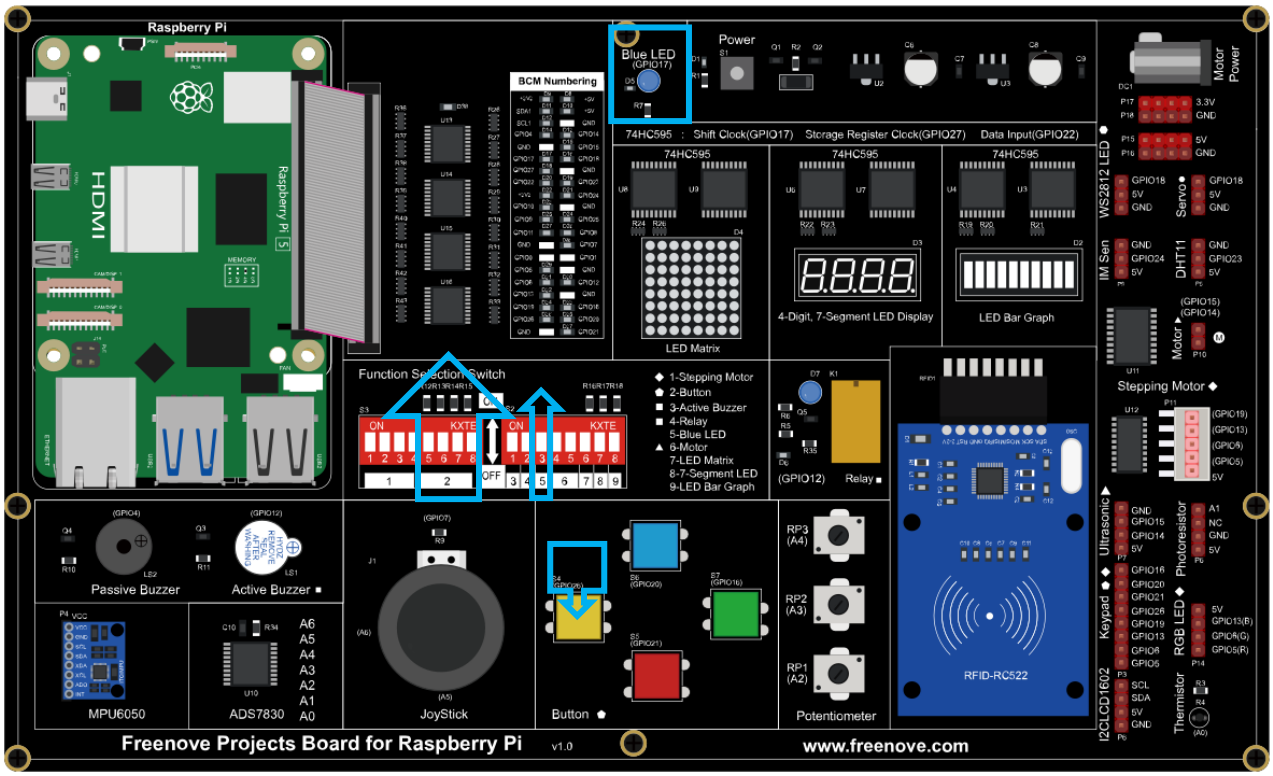
Circuit

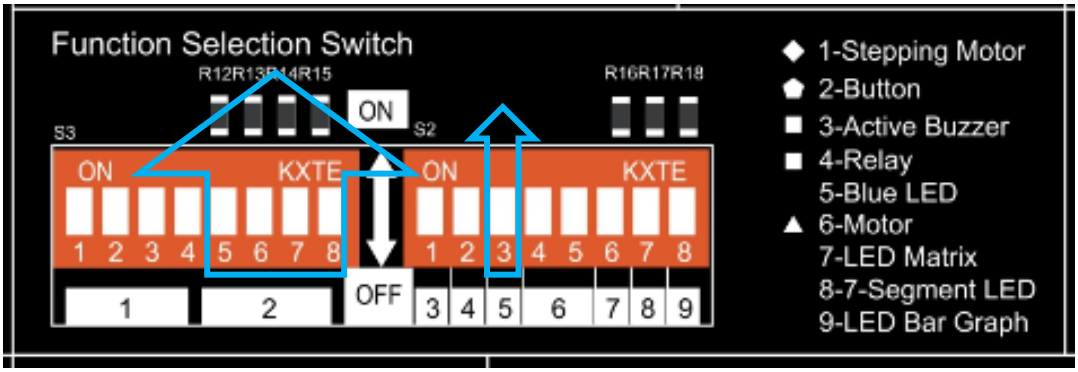
Schematic diagram



Hardware connection.

Switch ON NO.5 switch and the four switches of NO.2.





If you have any concerns, please send an email to: support@freenove.com

Code

This project is designed for learning how to use Push Button Switch to control an LED. We first need to read the state of switch, and then determine whether to turn the LED ON in accordance to the state of the switch.

C Code 3.1 ButtonLED

First, observe the project result, then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 3_ButtonLED directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/3_ButtonLED
```

2. Use the following command to compile the code "ButtonLED.c" and generate executable file "ButtonLED"

```
gcc ButtonLED.c -o ButtonLED -lwiringPi
```

3. Then run the generated file "ButtonLED".

```
./ButtonLED
```

Later, the terminal window continues to print out the characters "led off…". Press the **S4** button, then LED is turned on and then terminal window prints out the "led on…". Release the button, then LED is turned off and then terminal window prints out the "led off…". You can press "Ctrl+C" to terminate the program.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>

#define ledPin    17  //define the ledPin
#define buttonPin 26  //define the buttonPin

void  main(void)
{
    printf("Program is starting ... \n");

    wiringPiSetupGpio();//Initialize wiringPi. Use BCM Number.

    pinMode(ledPin, OUTPUT); //Set ledPin to output
    pinMode(buttonPin, INPUT);//Set buttonPin to input

    pullUpDnControl(buttonPin, PUD_UP);  //pull up to HIGH level
    while(1){
        if(digitalRead(buttonPin) == LOW){ //button is pressed
            digitalWrite(ledPin, HIGH);  //Make GPIO output HIGH level
            printf("Button is pressed, led turned on >>>\n");        //Output information on terminal
        }
        else {                              //button is released
            digitalWrite(ledPin, LOW);  //Make GPIO output LOW level
```

```
25              printf("Button is released, led turned off <<<\n");       //Output information on
26  terminal
27          }
28      }
29  }
```

Define ledPin and buttonPin as 17 and 26 respectively.

```
#define ledPin    17  //define the ledPin
#define buttonPin 26  //define the buttonPin
```

In the while loop of main function, use digitalRead(buttonPin) to determine the state of Button. When the button is pressed, the function returns low level, the result of "if" is true, and then turn on LED. Or, turn off LED.

```
        if(digitalRead(buttonPin) == LOW){ //button is pressed
            digitalWrite(ledPin, HIGH);  //Make GPIO output HIGH level
            printf("Button is pressed, led turned on >>>\n");        //Output information on
terminal
        }
        else {                              //button is released
            digitalWrite(ledPin, LOW);  //Make GPIO output LOW level
            printf("Button is released, led turned off <<<\n");      //Output information on
terminal
        }
```

Reference:

int digitalRead (int pin);

This function returns the value read at the given pin. It will be "**HIGH**" or "**LOW**"(1 or 0) depending on the logic level at the pin.

Python Code 3.1 ButtonLED

First, observe the project result, then learn about the code in detail. Remember in code “button” = switch function

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 3_ButtonLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/3_ButtonLED
```

2. Use Python command to execute btnLED.py.

```
python ButtonLED.py
```

Then the Terminal window continues to show the characters “led off…”, press the switch button and the LED turns ON and then Terminal window shows "led on…". Release the button, then LED turns OFF and then the terminal window text "led off…" appears. You can press "Ctrl+C" at any time to terminate the program.

The following is the program code:

```
1  from gpiozero import LED, Button
2
3  led = LED(17)        # define LED pin according to BCM Numbering
4  button = Button(26) # define Button pin according to BCM Numbering
5
6  def loop():
7      key_state = 0
8      while True:
9          if button.is_pressed==True and key_state == 0:  # if button is pressed
10             led.on()         # turn on led
11             key_state = 1
12             print("Button is pressed, led turned on >>>") # print information on terminal
13         elif button.is_pressed==False and key_state == 1:
14             led.off() # turn off led
15             key_state = 0
16             print("Button is released, led turned off <<<")
17
18 if __name__ == '__main__':     # Program entrance
19     print ('Program is starting...')
20     try:
21         loop()
22     except KeyboardInterrupt:  # Press ctrl-c to end the program.
23         print("Ending program")
24     finally:
25         led.close()
26         button.close()
```

Apply for an led module to control the LED, apply for a Button module to read the status of the button.

```
led = LED(17)        # define LED pin according to BCM Numbering
button = Button(26) # define Button pin according to BCM Numbering
```

The loop continues endlessly to judge whether the key is pressed. When the button is detected to be pressed, let the LED light up and set the key status value to 1 to ensure that it does not trigger again. When the button release is detected, let the LED go off and set the key status value to 0 to ensure that it does not trigger again.

```
def loop():
    key_state = 0
    while True:
        if button.is_pressed==True and key_state == 0:  # if button is pressed
            led.on()         # turn on led
            key_state = 1
            print("Button is pressed, led turned on >>>") # print information on terminal
        elif button.is_pressed==False and key_state == 1:
            led.off() # turn off led
            key_state = 0
            print("Button is released, led turned off <<<")
```

Release the GPIO resource each time you use Ctrl+C to exit the code.

```
    finally:
        led.close()
        button.close()
```

Chapter 4 Analog & PWM

In previous chapters, we learned that a Push Button Switch has two states: Pressed (ON) and Released (OFF), and an LED has a Light ON and OFF state. Is there a middle or intermediated state? We will next learn how to create an intermediate output state to achieve a partially bright (dim) LED.
First, let us learn how to control the brightness of an LED.

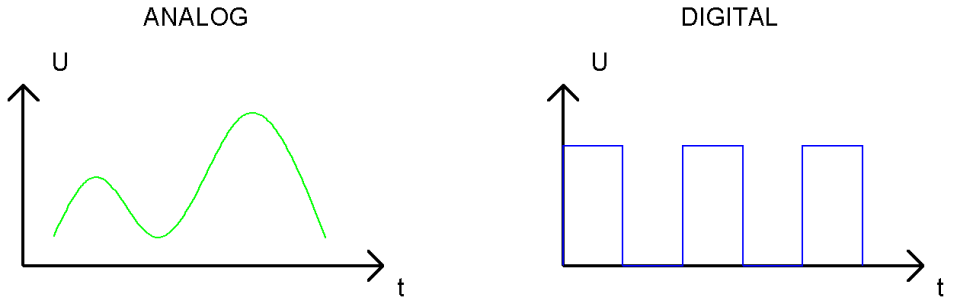
Project 4.1 Breathing LED

We describe this project as a Breathing Light. This means that an LED that is OFF will then turn ON gradually and then gradually turn OFF like "breathing". Okay, so how do we control the brightness of an LED to create a Breathing Light? We will use PWM to achieve this goal.

Component Knowledge

Analog & Digital

An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal **or discrete-time signal is a time series consisting of a sequence of quantities**. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0℃ to 10℃. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.

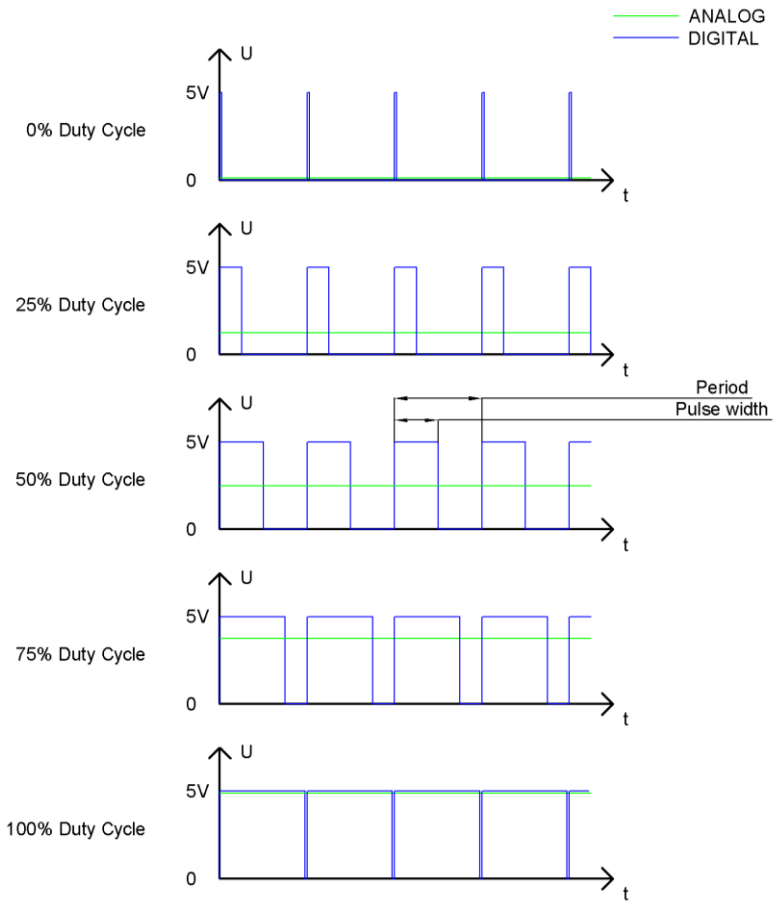


Note that the Analog signals are curved waves and the Digital signals are “Square Waves”.
In practical applications, we often use binary as the digital signal, that is a series of 0’s and 1’s. Since a binary signal only has two values (0 or 1) it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Digital processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals).
PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low

levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called “pulse width”, and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform. The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:

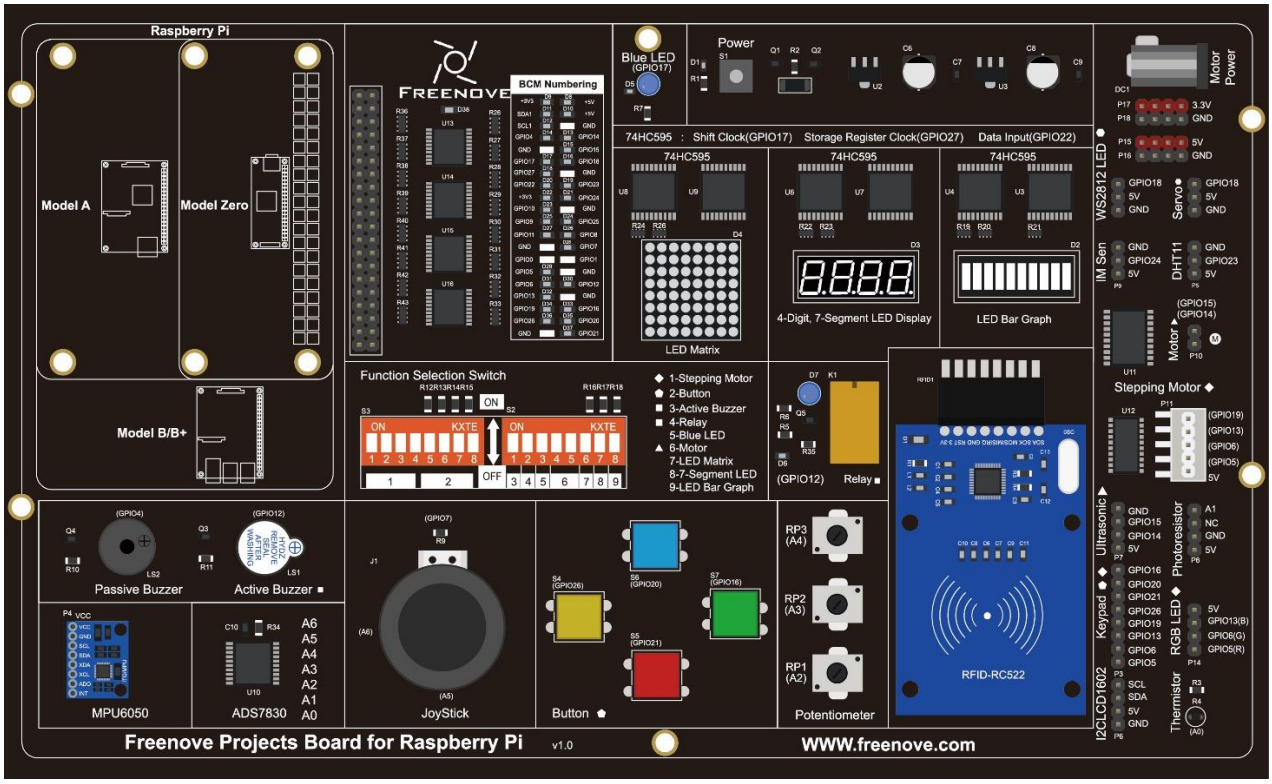


The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.
It is evident, from the above, that PWM is not actually analog but the effective value of voltage is equivalent to the corresponding analog value. Therefore, by using PWM, we can control the output power of to an LED and control other devices and modules to achieve multiple effects and actions.
In RPi, GPIO18 pin has the ability to output to hardware via PWM with a 10-bit accuracy. This means that 100% of the pulse width can be divided into 2^{10}=1024 equal parts.
The wiringPi library of C provides both a hardware PWM and a software PWM method, while the wiringPi library of Python does not provide a hardware PWM method. There is only a software PWM option for Python.
The hardware PWM only needs to be configured, does not require CPU resources and is more precise in time control. The software PWM requires the CPU to work continuously by using code to output high level and low level. This part of the code is carried out by multi-threading, and the accuracy is relatively not high enough.

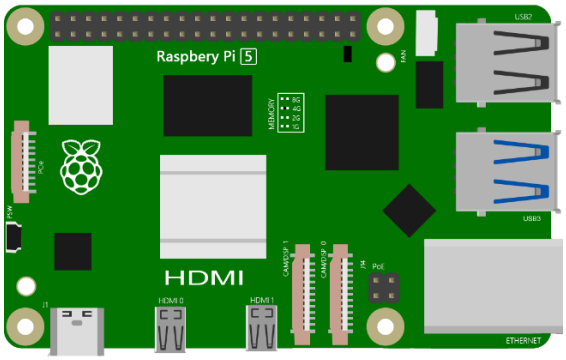
In order to keep the results running consistently, we will use PWM.

Component List

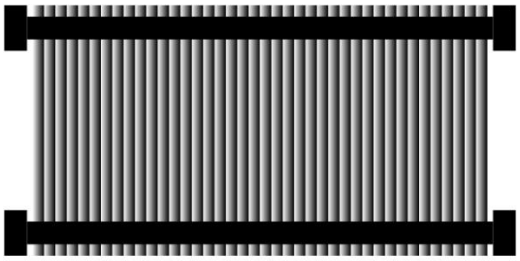
Freenove Projects Board for Raspberry Pi



Raspberry Pi

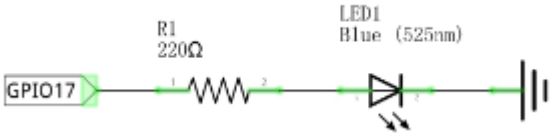


GPIO Ribbon Cable

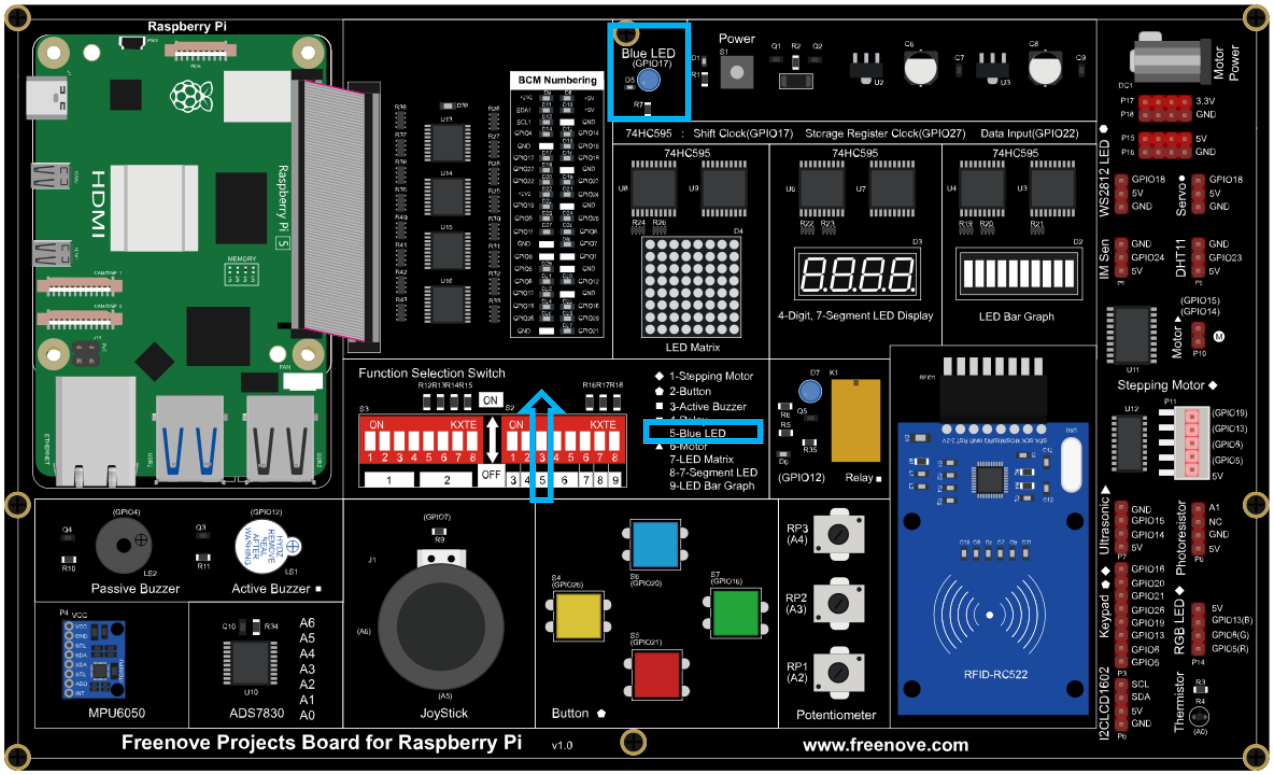


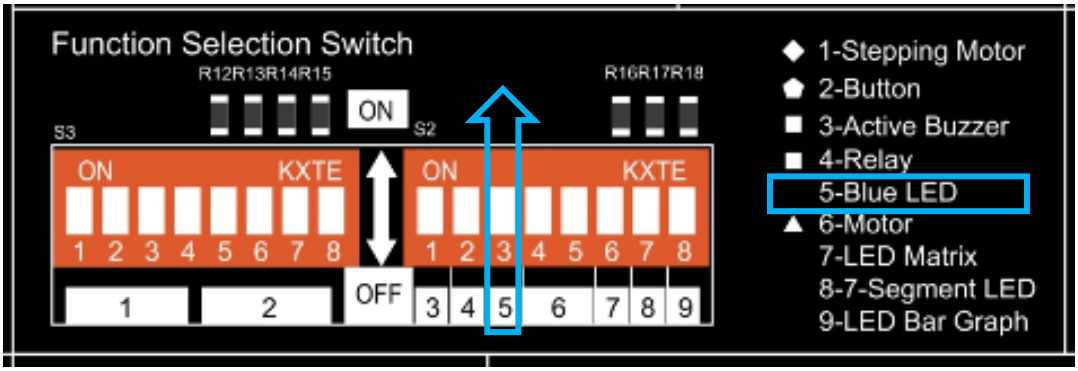
Circuit

Schematic diagram



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

C Code 4.1 BreathingLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 4_BreathingLED directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/4_BreathingLED
```

2. Use following command to compile "BreathingLED.c" and generate executable file "BreathingLED".

```
gcc BreathingLED.c -o BreathingLED -lwiringPi
```

3. Then run the generated file "BreathingLED"

```
./BreathingLED
```

After the program is executed, you'll see that LED is turned from on to off and then from off to on gradually like breathing.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softPwm.h>

#define ledPin    17

void main(void)
{
    int i;
    printf("Program is starting ... \n");
    wiringPiSetupGpio();           //Initialize wiringPi. Use BCM Number.
    softPwmCreate(ledPin,  0, 100);//Creat SoftPWM pin

    while(1){
        for(i=0;i<100;i++){  //make the led brighter
            softPwmWrite(ledPin, i);
            delay(20);
        }
        delay(300);
        for(i=100;i>=0;i--){  //make the led darker
            softPwmWrite(ledPin, i);
            delay(20);
        }
        delay(300);
    }
}
```

First, create a software PWM pin.

```
softPwmCreate(ledPin,  0, 100);//Creat SoftPWM pin
```

There are two “for” loops in the next endless “while” loop. The first loop outputs a power signal to the ledPin PWM from 0% to 100% and the second loop outputs a power signal to the ledPin PWM from 100% to 0%.

```
    while(1){
        for(i=0;i<100;i++){  //make the led brighter
            softPwmWrite(ledPin, i);
            delay(20);
        }
        delay(300);
        for(i=100;i>=0;i--){  //make the led darker
            softPwmWrite(ledPin, i);
            delay(20);
        }
        delay(300);
    }
```

You can also adjust the rate of the state change of LED by changing the parameter of the delay() function in the “for” loop.

int softPwmCreate (int pin, int initialValue, int pwmRange) ;
This creates a software controlled PWM pin.
void softPwmWrite (int pin, int value) ;
This updates the PWM value on the given pin.

For more details, please refer https://github.com/WiringPi/WiringPi

Python Code 4.1 BreathingLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 4_BreathingLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/4_BreathingLED
```

2. Use the Python command to execute Python code “BreathingLED.py”.

```
python BreathingLED.py
```

After the program is executed, you will see that the LED gradually turns ON and then gradually turns OFF similar to “breathing”.

The following is the program code:

```
1  from gpiozero import PWMLED
2  import time
3
4  led = PWMLED(17 ,initial_value=0 ,frequency=1000)
5  def loop():
6      while True:
7          for b in range(0, 101, 1):     # make the led brighter
8              led.value = b / 100.0      # set dc value as the duty cycle
```

```
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)

if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        led.close()
```

The LED is connected to the IO port called GPIO17. The LedPin is defined as pin 17 and set to output mode according to the corresponding chart for pin designations. Then create a PWM instance and set the PWM frequency to 1000HZ and the initial value to 0.

```
led = PWMLED(17 ,initial_value=0 ,frequency=1000)
```

There are two “for” loops used to control the breathing LED in the next endless “while” loop. The first loop outputs a power signal to the ledPin PWM from 0% to 100% and the second loop outputs a power signal to the ledPin PWM from 100% to 0%.

```
def loop():
    while True:
        for b in range(0, 101, 1):    # make the led brighter
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
```

The related functions of PWM are described as follows:

gpiozeri.PWMLED(pin, initial_value, frequency)

To create a PWM instance:

pin: This is the number of the GPIO pin on the Raspberry PI, used to connect the LED.

initial_value: Initial brightness value, a floating point number ranging from 0 to 1, 0 being fully off and 1 being fully on.

frequency: This is the frequency of the PWM signal in Hz.

led.value

Value Indicates the duty cycle of pwm. Floating-point type ranging from 0 to 1.

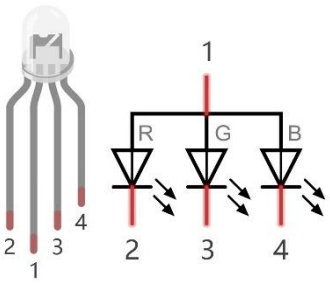
led.close()

Release the GPIO resource.

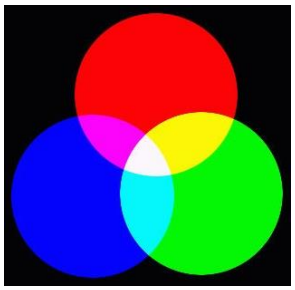
Chapter 5 RGB LED

In this chapter, we will learn how to control an RGB LED.

An RGB LED has 3 LEDs integrated into one LED component. It can respectively emit Red, Green and Blue light. In order to do this, it requires 4 pins (this is also how you identify it). The long pin (1) is the common which is the Anode (+) or positive lead, the other 3 are the Cathodes (-) or negative leads. A rendering of an RGB LED and its electronic symbol are shown below. We can make RGB LED emit various colors of light and brightness by controlling the 3 Cathodes (2, 3 & 4) of the RGB LED



Red, Green, and Blue light are called 3 Primary Colors when discussing light (Note: for pigments such as paints, the 3 Primary Colors are Red, Blue and Yellow). When you combine these three Primary Colors of light with varied brightness, they can produce almost any color of visible light. Computer screens, single pixels of cell phone screens, neon lamps, etc. can all produce millions of colors due to phenomenon.



RGB

If we use a three 8 bit PWM to control the RGB LED, in theory, we can create $2^8*2^8*2^8$=16777216 (16 million) colors through different combinations of RGB light brightness.

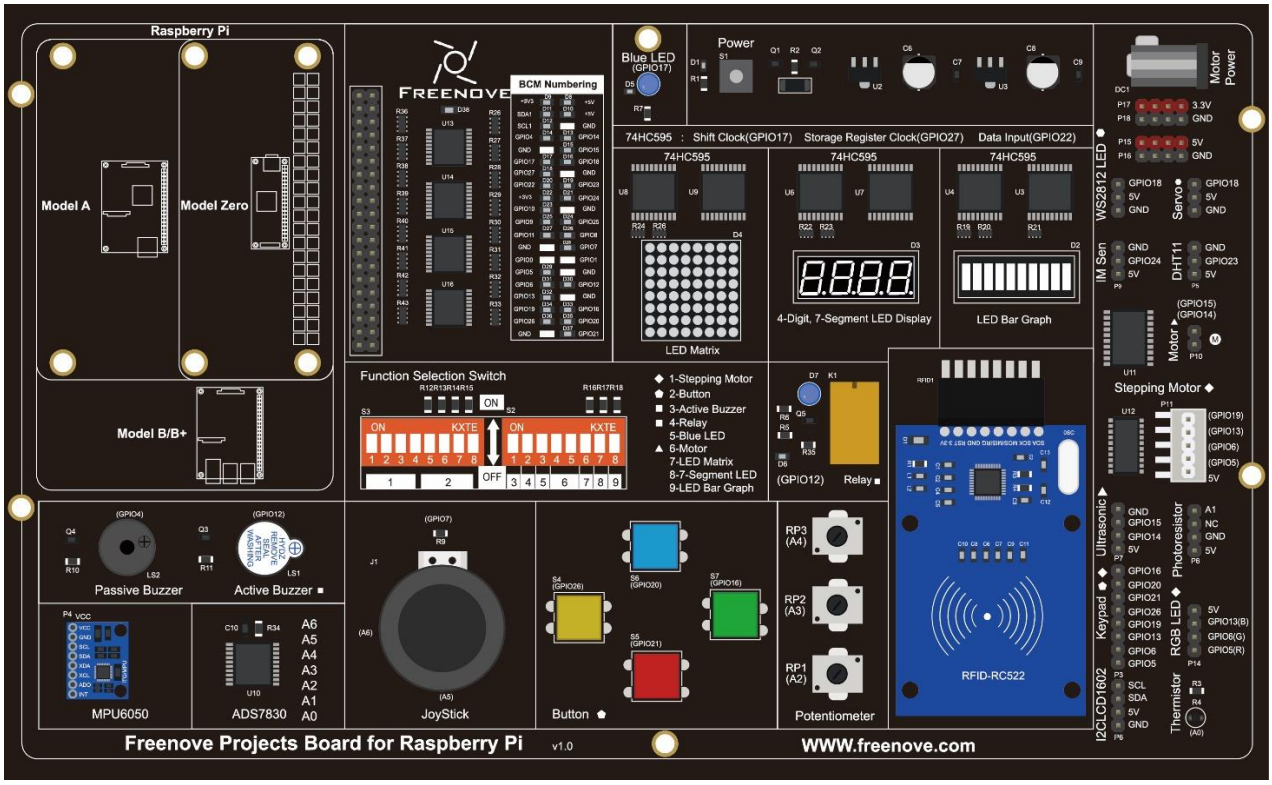
Next, we will use RGB LED to make a multicolored LED.

Project 5.1 RainbowLED

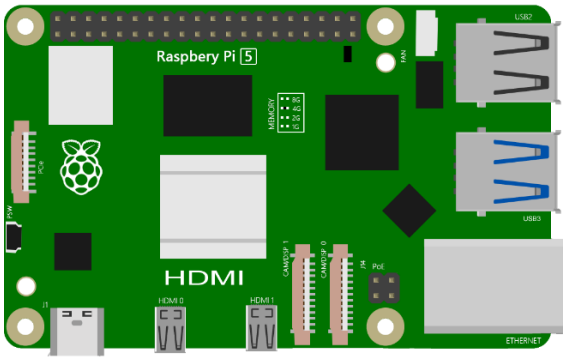
In this project, we will make a multicolored LED, which we can program the RGB LED to automatically change colors.

Component List

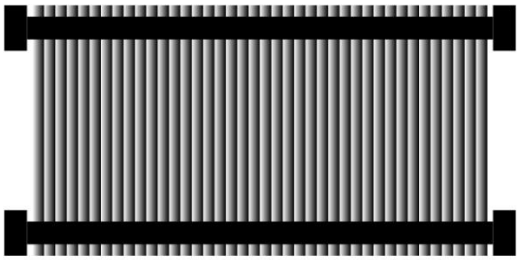
Freenove Projects Board for Raspberry Pi



Raspberry Pi

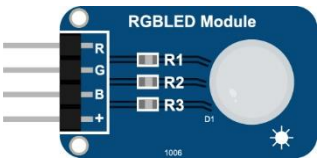


GPIO Ribbon Cable



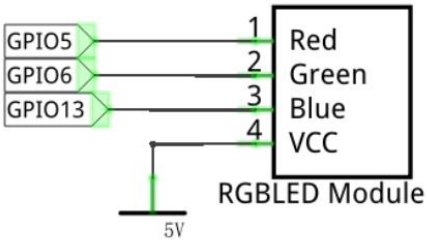
Jumper Wire

RGBLED Module

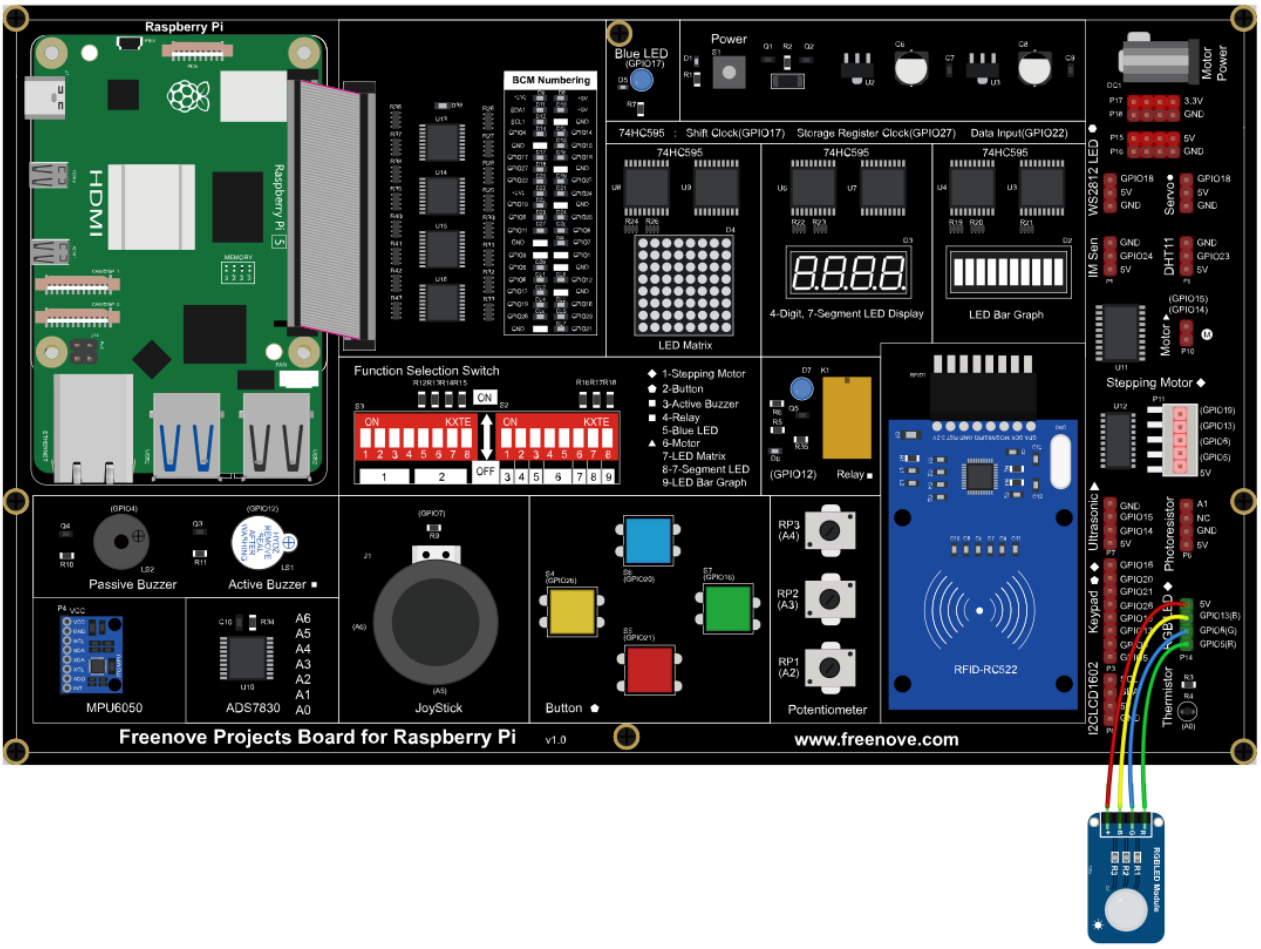


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

We need to use the software to make the ordinary GPIO output PWM, since this project requires 3 PWM and in RPi only one GPIO has the hardware capability to output PWM,

C Code 5.1 RainbowLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 5_ RainbowLED directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/5_RainbowLED
```

2. Use following command to compile "RainbowLED.c" and generate executable file "RainbowLED".
   Note: in this project, the software PWM uses a multi-threading mechanism. So "-lpthread" option need to be add to the compiler.

```
gcc RainbowLED.c -o RainbowLED -lwiringPi -lpthread
```

3. And then run the generated file "ColorfulLED".

```
./RainbowLED
```

After the program is executed, you will see that the RGB LED shows lights of different colors randomly.

The following is the program code:

```
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#include <stdlib.h>

#define ledPinRed    5
#define ledPinGreen  6
#define ledPinBlue   13

void setupLedPin(void){
    softPwmCreate(ledPinRed,   0, 100); //Creat SoftPWM pin for red
    softPwmCreate(ledPinGreen,0, 100);  //Creat SoftPWM pin for green
    softPwmCreate(ledPinBlue, 0, 100);  //Creat SoftPWM pin for blue
}

void setLedColor(int r, int g, int b){
    softPwmWrite(ledPinRed,   r);  //Set the duty cycle
    softPwmWrite(ledPinGreen, g);   //Set the duty cycle
    softPwmWrite(ledPinBlue,  b);   //Set the duty cycle
}

int main(void){
    int r,g,b;

    printf("Program is starting ...\n");

```

```
27      wiringPiSetup();  //Initialize wiringPi.
28
29      setupLedPin();
30      while(1){
31          r=random()%100;  //get a random in (0,100)
32          g=random()%100;  //get a random in (0,100)
33          b=random()%100;  //get a random in (0,100)
34          setLedColor(r,g,b);//set random as the duty cycle value
35          printf("r=%d,   g=%d,   b=%d \n",r,g,b);
36          delay(1000);
37      }
38      return 0;
39  }
```

First, in subfunction of ledInit(), create the software PWM control pins used to control the R, G, B pin respectively.

```
void setupLedPin(void)
{
    softPwmCreate(ledPinRed,   0,  100);  //Creat SoftPWM pin for red
    softPwmCreate(ledPinGreen,0,  100);   //Creat SoftPWM pin for green
    softPwmCreate(ledPinBlue,  0,  100);   //Creat SoftPWM pin for blue
}
```

Then create subfunction, and set the PWM of three pins.

```
void setLedColor(int r,  int g,  int b)
{
    softPwmWrite(ledPinRed,    r);  //Set the duty cycle
    softPwmWrite(ledPinGreen,  g);    //Set the duty cycle
    softPwmWrite(ledPinBlue,   b);    //Set the duty cycle
}
```

Finally, in the “while” loop of main function, get three random numbers and specify them as the PWM duty cycle, which will be assigned to the corresponding pins. So RGB LED can switch the color randomly all the time.

```
    while(1){
        r=random()%100;  //get a random in (0,100)
        g=random()%100;  //get a random in (0,100)
        b=random()%100;  //get a random in (0,100)
        setLedColor(r,g,b);//set random as the duty cycle value
        printf("r=%d,   g=%d,   b=%d \n",r,g,b);
        delay(1000);
    }
```

The related function of PWM Software can be described as follows:

long random();

This function will return a random number.

For more details about Software PWM, please refer to: https://github.com/WiringPi/WiringPi

Python Code 5.1 RainbowLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 5_RainbowLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/5_RainbowLED
```

2. Use python command to execute python code "ColorfulLED.py".

```
python RainbowLED.py
```

After the program is executed, you will see that the RGB LED randomly lights up different colors.

The following is the program code:

```
from gpiozero import RGBLED
import time
import random

led = RGBLED(red=5, green=6, blue=13, active_high=False)

def setColor(r_val,g_val,b_val):
    led.red = r_val/100          # change pwmRed duty cycle to r_val
    led.green = g_val/100        # change pwmRed duty cycle to r_val
    led.blue = b_val/100         # change pwmRed duty cycle to r_val

def loop():
    while True :
        r=random.randint(0,100)  #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)          #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)

if __name__ == '__main__':       # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:    # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        led.close()
```

In last chapter, we learned how to use Python language to make a pin output PWM. In this project, we output to three pins via PWM and the method is exactly the same as we used in the last chapter. In the "while" loop of "loop" function, we first generate three random numbers, and then specify these three random numbers as the PWM values for the three pins, which will make the RGB LED produce multiple colors randomly.

```
def loop():
    while True :
        r=random.randint(0,100)   #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)           #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)
```

About the randint() function :

random.randint(a, b)

This function can return a random integer (a whole number value) within the specified range (a, b).

Chapter 6 Buzzer

In this chapter, we will learn about buzzers and the sounds they make. And in our next project, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

Project 6.1 Doorbell

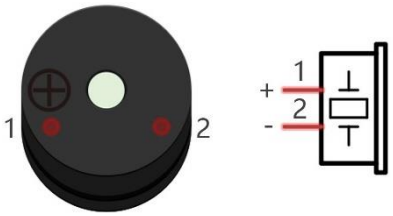
We will make a doorbell with this functionality: when the Push Button Switch is pressed the buzzer sounds and when the button is released, the buzzer stops. This is a momentary switch function.

Component knowledge

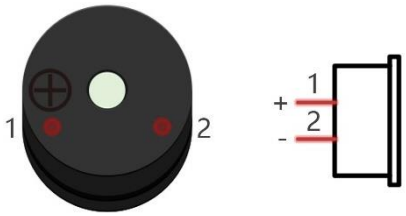
Buzzer

A buzzer is an audio component. They are widely used in electronic devices such as calculators, electronic alarm clocks, automobile fault indicators, etc. There are both active and passive types of buzzers. Active buzzers have oscillator inside, these will sound as long as power is supplied. Passive buzzers require an external oscillator signal (generally using PWM with different frequencies) to make a sound.

Active buzzer



Passive buzzer



Active buzzers are easier to use. Generally, they only make a specific sound frequency. Passive buzzers require an external circuit to make sounds, but passive buzzers can be controlled to make sounds of various frequencies. The resonant frequency of the passive buzzer in this Kit is 2kHz, which means the passive buzzer is the loudest when its resonant frequency is 2kHz.

How to identify active and passive buzzer?

1. As a rule, there is a label on an active buzzer covering the hole where sound is emitted, but there are exceptions to this rule.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes, view of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model.



Active buzzer bottom



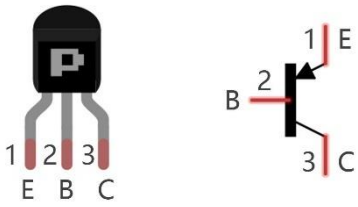
Passive buzzer bottom

Transistors

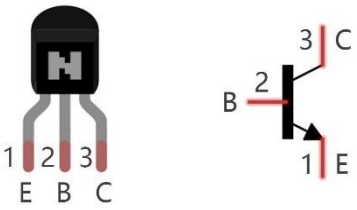
A transistor is required in this project due to the buzzer's current being so great that GPIO of RPi's output capability cannot meet the power requirement necessary for operation. A NPN transistor is needed here to amplify the current.

Transistors, full name: semiconductor transistor, is a semiconductor device that controls current (think of a transistor as an electronic "amplifying or switching device". Transistors can be used to amplify weak signals, or to work as a switch. Transistors have three electrodes (PINs): base (b), collector (c) and emitter (e). When there is current passing between "be" then "ce" will have a several-fold current increase (transistor magnification), in this configuration the transistor acts as an amplifier. When current produced by "be" exceeds a certain value, "ce" will limit the current output. at this point the transistor is working in its saturation region and acts like a switch. Transistors are available as two types as shown below: PNP and NPN,

PNP transistor



NPN transistor

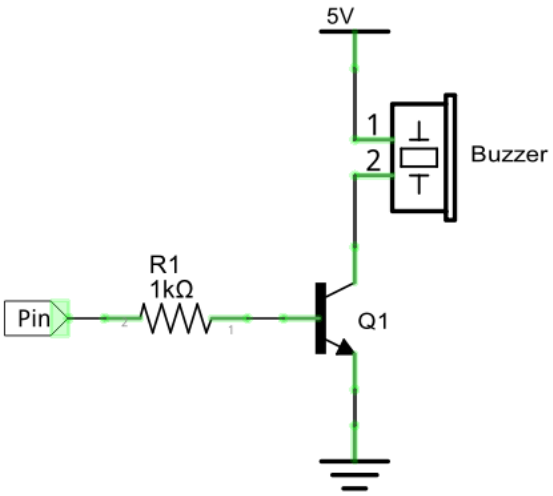
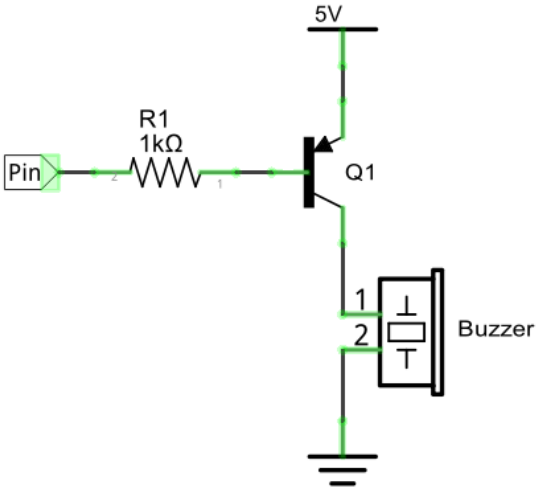


In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Thanks to the transistor's characteristics, they are often used as switches in digital circuits. As micro-controllers output current capacity is very weak, we will use a transistor to amplify its current in order to drive components requiring higher current.

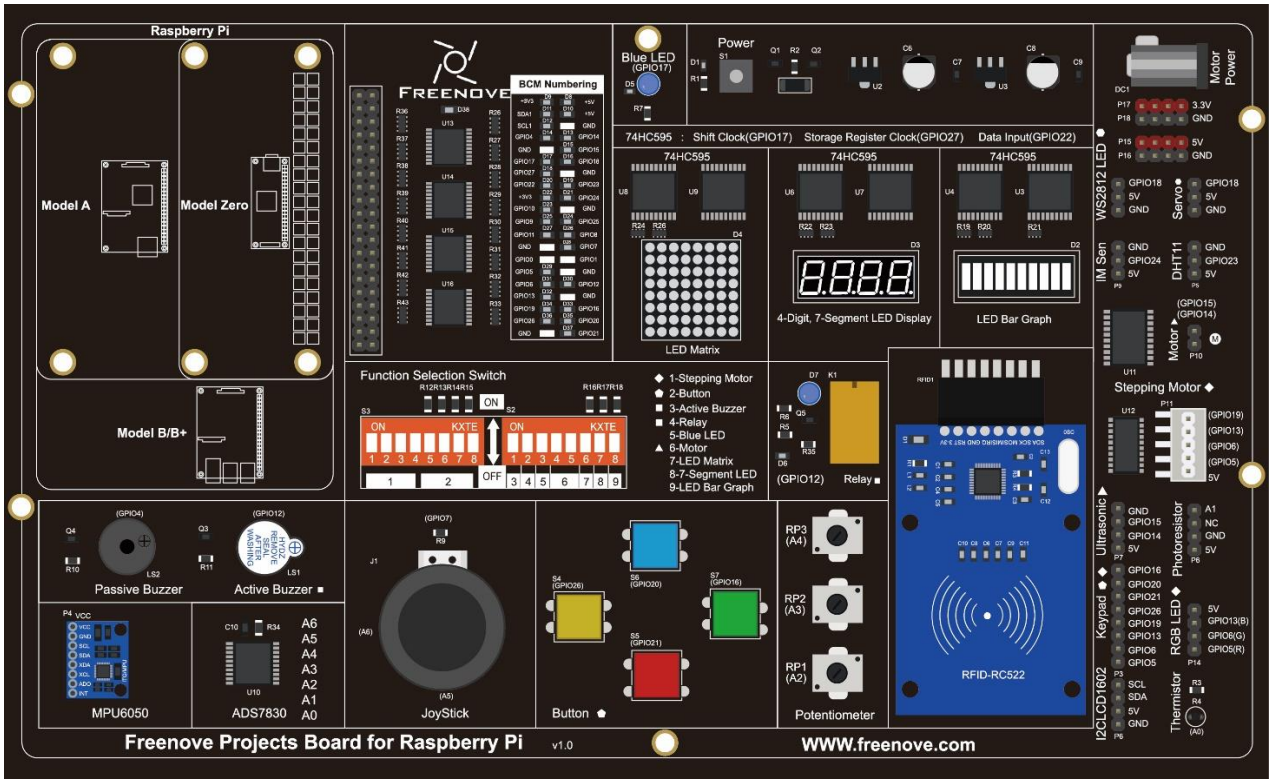
When we use a NPN transistor to drive a buzzer, we often use the following method. If GPIO outputs high level, current will flow through R1 (Resistor 1), the transistor conducts current and the buzzer will make sounds. If GPIO outputs low level, no current will flow through R1, the transistor will not conduct currentand buzzer will remain silent (no sounds).

When we use a PNP transistor to drive a buzzer, we often use the following method. If GPIO outputs low level, current will flow through R1. The transistor conducts current and the buzzer will make sounds. If GPIO outputs high level, no current flows through R1, the transistor will not conduct current and buzzer will remain silent (no sounds). Below are the circuit schematics for both a NPN and PNP transistor to power a buzzer.

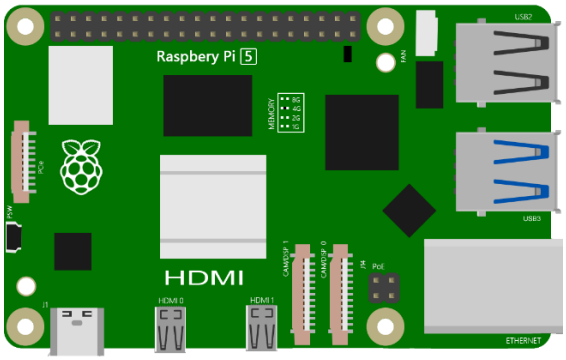
NPN transistor to drive buzzer	PNP transistor to drive buzzer
	

Component List

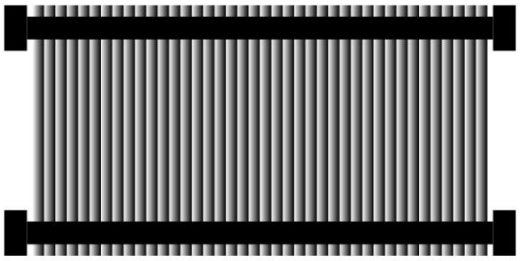
Freenove Projects Board for Raspberry Pi



Raspberry Pi

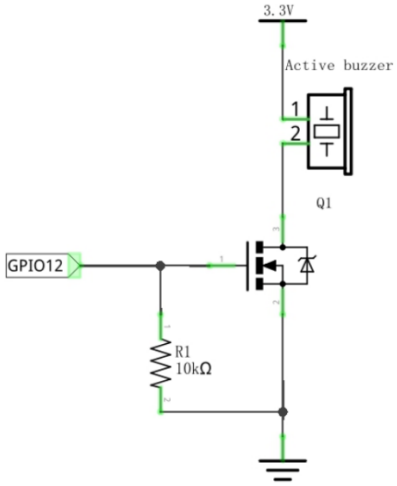


GPIO Ribbon Cable

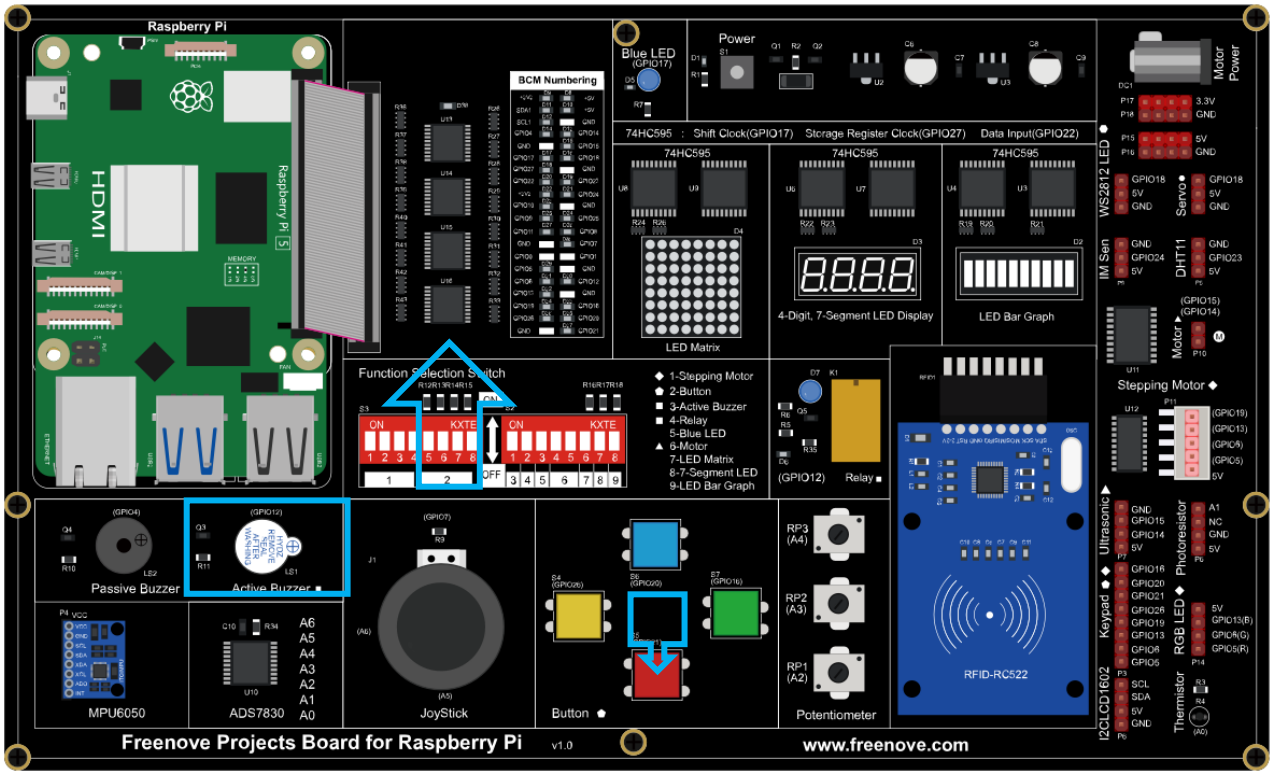


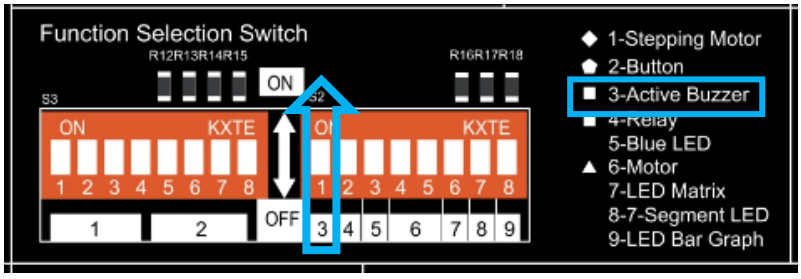
Circuit

Schematic diagram with RPi GPIO Extension Shield



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

C Code 6.1 Doorbell

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 6_1_Doorbell directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/6_1_Doorbell
```

2. Use following command to compile "Doorbell.c" and generate executable file "Doorbell.c".

```
gcc Doorbell.c -o Doorbell -lwiringPi
```

3. Then run the generated file "Doorbell".

```
./Doorbell
```

After the program is executed, press the push button switch and the will buzzer sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>

#define buzzerPin 12  //define the buzzerPin
#define buttonPin 21  //define the buttonPin

void main(void){
    printf("Program is starting ... \n");
    wiringPiSetupGpio();  //Initialize wiringPi. Use BCM Number.
    pinMode(buzzerPin, OUTPUT);
    pinMode(buttonPin, INPUT);
    pullUpDnControl(buttonPin, PUD_UP);      //pull up to HIGH level

    while(1){
        if(digitalRead(buttonPin) == LOW){  //button is pressed
            digitalWrite(buzzerPin, HIGH);  //Turn on buzzer
            printf("buzzer turned on >>> \n");
        }
        else {                              //button is released
            digitalWrite(buzzerPin, LOW);   //Turn off buzzer
            printf("buzzer turned off <<< \n");
        }
    }
}
```

Python Code 6.1 Doorbell

First, observe the project result, then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 6_1_Doorbell directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/6_1_Doorbell
```

2. Use python command to execute python code "Doorbell.py".

```
python Doorbell.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
from gpiozero import Buzzer, Button

buzzer = Buzzer(12)
button = Button(21)

def loop():
    key_state = 0
    while True :
        if button.is_pressed==True and key_state == 0:
            buzzer.on()
            print("Button is pressed, buzzer turned on >>>")
            key_state = 1
        elif button.is_pressed==False and key_state == 1:
            buzzer.off()
            print("Button is released, buzzer turned on <<<")
            key_state = 0

def destroy():
    buzzer.close()
    button.close()

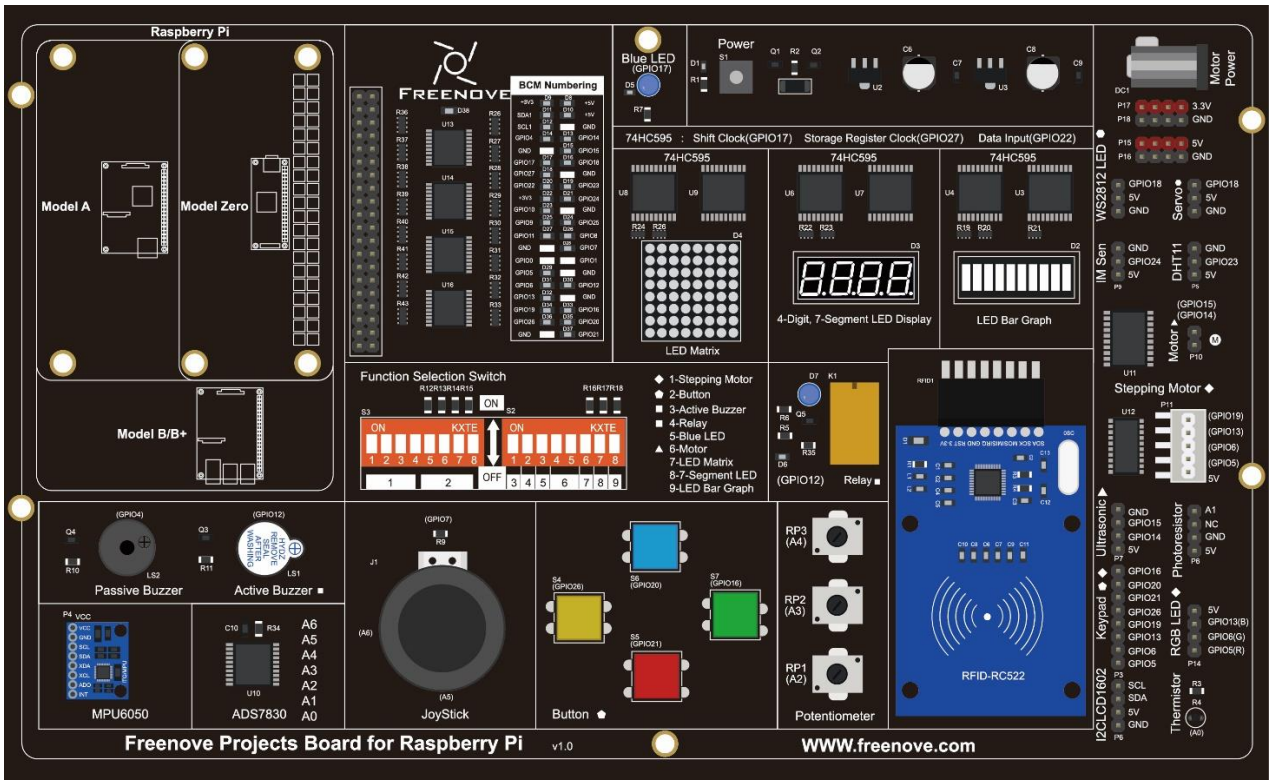
if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

Project 6.2 Alertor

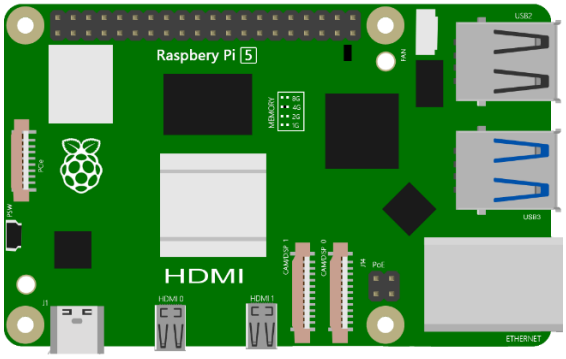
Next, we will use a passive buzzer to make an alarm.

Component List

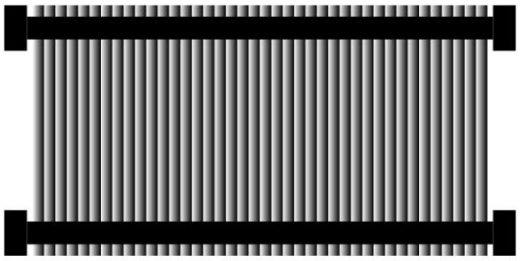
Freenove Projects Board for Raspberry Pi



Raspberry Pi

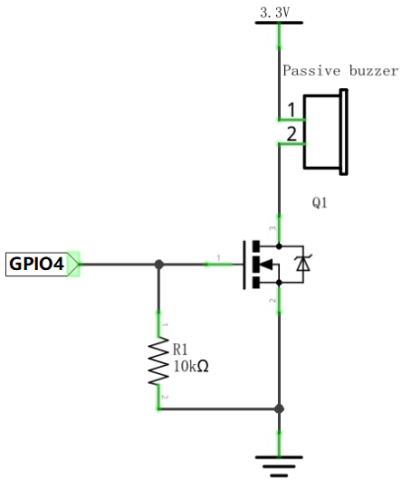


GPIO Ribbon Cable

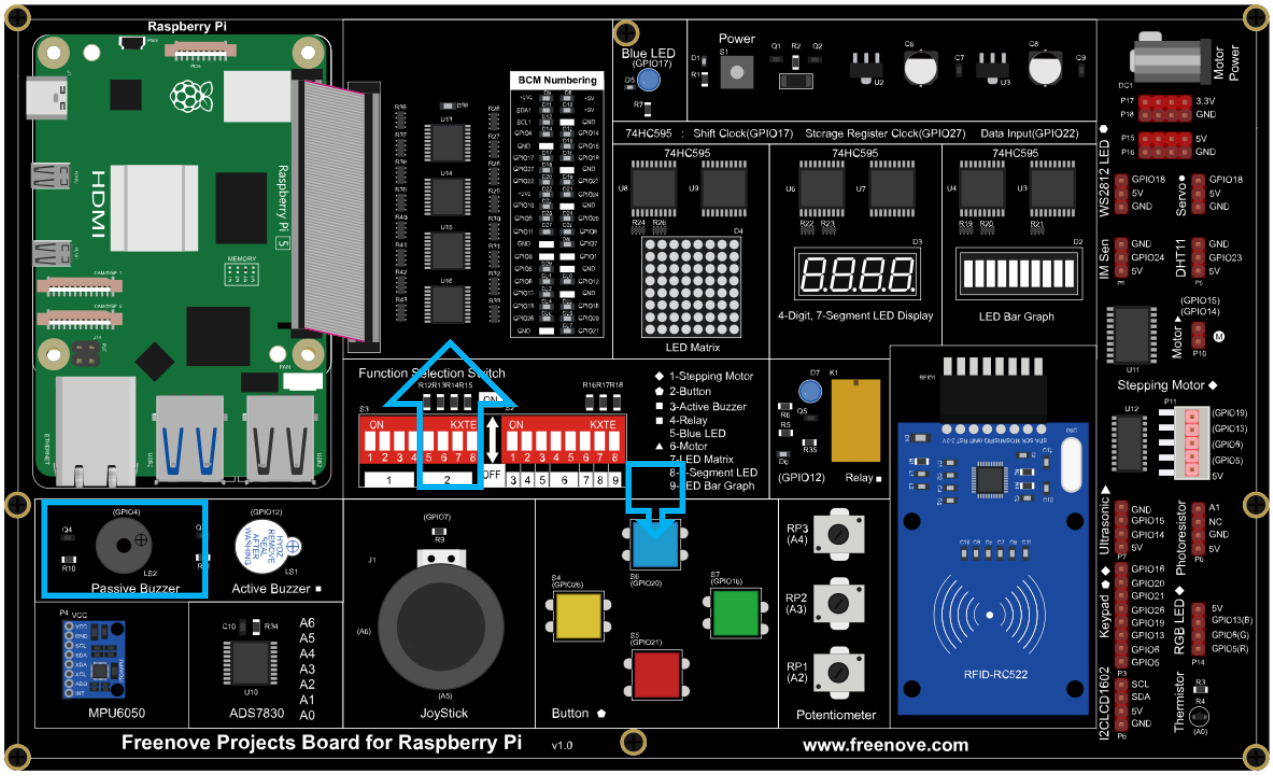


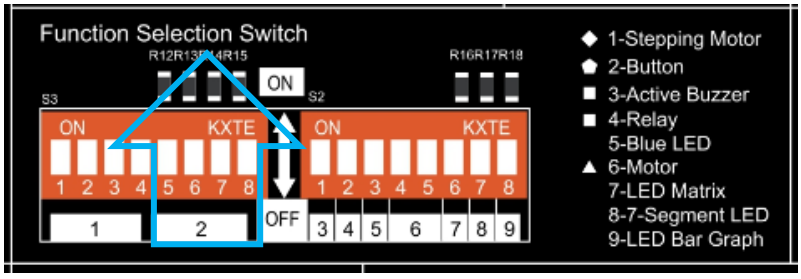
Circuit

Schematic diagram with RPi GPIO Extension Shield



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

In this project, our buzzer alarm is controlled by the push button switch. Press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
As stated before, it is analogous to our earlier project that controlled an LED ON and OFF.
To control a passive buzzer requires PWM of certain sound frequency.

C Code 6.2 Alertor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 6_2_Alertor directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/6_2_Alertor
```

2. Use following command to compile "Alertor.c" and generate executable file "Alertor". "-lm" and "-lpthread" compiler options need to added here.

```
gcc Alertor.c -o Alertor -lwiringPi -lm -lpthread
```

3. Then run the generated file "Alertor".

```
./Alertor
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softTone.h>
#include <math.h>

#define buzzerPin    4    //define the buzzerPin
#define buttonPin   20    //define the buttonPin

void alertor(int pin){
    int x;
    double sinVal, toneVal;
    for(x=0;x<360;x++){ // frequency of the alertor is consistent with the sine wave
        sinVal = sin(x * (M_PI / 180)); //Calculate the sine value
        toneVal = 2000 + sinVal * 500;  //Add the resonant frequency and weighted sine value
        softToneWrite(pin,toneVal);     //output corresponding PWM
        delay(1);
    }
}
void stopAlertor(int pin){
    softToneWrite(pin,0);
}
int main(void)
```

```
{
    printf("Program is starting ... \n");
    wiringPiSetupGpio();//Initialize wiringPi. Use BCM Number.
    pinMode(buzzerPin, OUTPUT);
    pinMode(buttonPin, INPUT);
    softToneCreate(buzzerPin); //set buzzerPin
    pullUpDnControl(buttonPin, PUD_UP);  //pull up to HIGH level

    while(1){
        if(digitalRead(buttonPin) == LOW){ //button is pressed
            alertor(buzzerPin);   // turn on buzzer
            printf("alertor turned on >>> \n");
        }
        else {                  //button is released
            stopAlertor(buzzerPin);   // turn off buzzer
            printf("alertor turned off <<< \n");
        }
    }
    return 0;
}
```

The code is the same to the active buzzer but the method is different. A passive buzzer requires PWM of a certain frequency, so you need to create a software PWM pin though softToneCreate (buzzeRPin). Here softTone is designed to generate square waves with variable frequency and a duty cycle fixed to 50%, which is a better choice for controlling the buzzer.

```
softToneCreate(buzzeRPin);
```

In the while loop of the main function, when the push button switch is pressed the subfunction alertor() will be called and the alarm will issue a warning sound. The frequency curve of the alarm is based on a sine curve. We need to calculate the sine value from 0 to 360 degrees and multiplied by a certain value (here this value is 500) plus the resonant frequency of buzzer. We can set the PWM frequency through softToneWrite (pin, toneVal).

```
void alertor(int pin){
    int x;
    double sinVal, toneVal;
    for(x=0;x<360;x++){ //The frequency is based on the sine curve.
        sinVal = sin(x * (M_PI / 180));
        toneVal = 2000 + sinVal * 500;
        softToneWrite(pin,toneVal);
        delay(1);
    }
}
```

If you want to stop the buzzer, just set PWM frequency of the buzzer pin to 0.

```
void stopAlertor(int pin){
    softToneWrite(pin,0);
}
```

The related functions of softTone are described as follows:

int softToneCreate (int pin) ;
This creates a software controlled tone pin.
void softToneWrite (int pin, int freq) ;
This updates the tone frequency value on the given pin.

For more details about softTone, please refer to : https://github.com/WiringPi/WiringPi

Python Code 6.2 Alertor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 6_2_Alertor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/6_2_Alertor
```

2. Use the python command to execute the Python code "Alertor.py".

```
python Alertor.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
1  from gpiozero import TonalBuzzer,Button
2  from gpiozero.tones import Tone
3  import time
4  import math
5
6  buzzer = TonalBuzzer(4)
7  button = Button(20) # define Button pin according to BCM Numbering
8
9  def loop():
10     while True:
11         if button.is_pressed:  # if button is pressed
12             alertor()
13             print ('alertor turned on >>> ')
14         else :
15             stopAlertor()
16             print ('alertor turned off <<<')
17
18 def alertor():
19     buzzer.play(Tone(220.0))
20     time.sleep(0.1)
21
22 def stopAlertor():
23     buzzer.stop()
24
25 def destroy():
```

```
26      buzzer.close()
27      button.close()
28
29  if __name__ == '__main__':     # Program entrance
30      print ('Program is starting...')
31      try:
32          loop()
33      except KeyboardInterrupt:  # Press ctrl-c to end the program.
34          print("Ending program")
35      finally:
36          destroy()
```

The code is the same as the active buzzer, but the method is different. Passive buzzers require PWM of a certain frequency, so you need to create a buzzer object through TonalBuzzer.

```
buzzer = TonalBuzzer(4)
button = Button(20) # define Button pin according to BCM Numbering
```

In the while loop of the main function, when the button switch is pressed, the child function alertor() will be called and the alarm will sound a warning. The buzzer will sound at 220Hz.

```
def alertor():
    buzzer.play(Tone(220.0))
    time.sleep(0.1)
```

When the push button switch is released, the buzzer (in this case our Alarm) will stop.

```
def stopAlertor():
    buzzer.stop()
```

(Important) Chapter 7 ADC

We have learned how to control the brightness of an LED through PWM and that PWM is not a real analog signal. In this chapter, we will learn how to read analog values via an ADC Module and convert these analog values into digital.

Project 7.1 Read the Voltage of Potentiometer

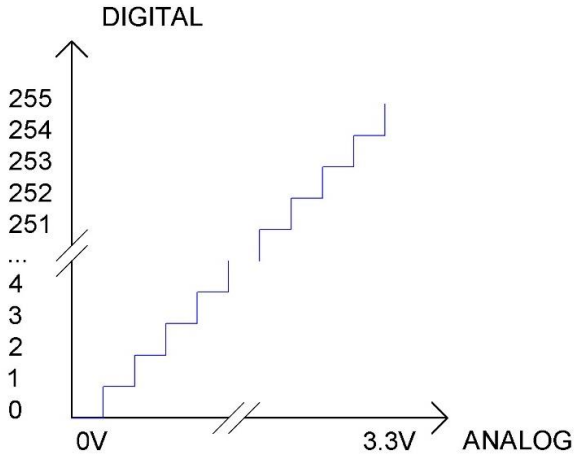
In this project, we will use the ADC function of an ADC Module to read the voltage value of a potentiometer.

Circuit knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC module is 8 bits, that means the resolution is 2^8=256, so that its range (at 3.3V) will be divided equally to 256 parts.

Any analog value can be mapped to one digital value using the resolution of the converter. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of 0V-3.3/256 V corresponds to digital 0;

Subsection 2: the analog in range of 3.3 /256 V-2*3.3 /256V corresponds to digital 1;

The resultant analog signal will be divided accordingly.

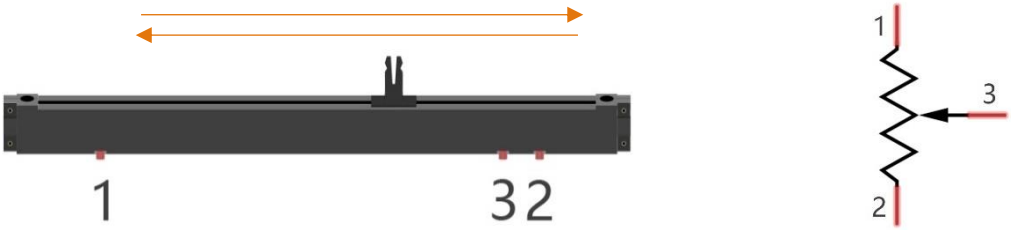
DAC

The reversing this process requires a DAC, Digital-to-Analog Converter. The digital I/O port can output high level and low level (0 or 1), but cannot output an intermediate voltage value. This is where a DAC is useful. The DAC module PCF8591 has a DAC output pin with 8-bit accuracy, which can divide VDD (here is 3.3V) into 2^8=256 parts. For example, when the digital quantity is 1, the output voltage value is 3.3/256 *1 V, and when the digital quantity is 128, the output voltage value is 3.3/256 *128=1.65V, the higher the accuracy of DAC, the higher the accuracy of output voltage value will be.

Component knowledge

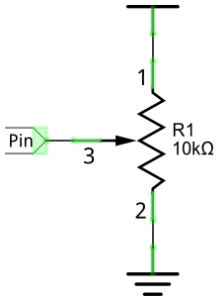
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



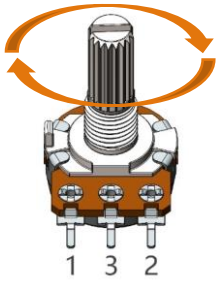
Between potentiometer pin 1 and pin 2 is the resistive element (a resistance wire or carbon) and pin 3 is connected to the brush that makes contact with the resistive element. In our illustration, when the brush moves from pin 1 to pin 2, the resistance value between pin 1 and pin 3 will increase linearly (until it reaches the highest value of the resistive element) and at the same time the resistance between pin 2 and pin 3 will decrease linearly and conversely down to zero. At the midpoint of the slider the measured resistance values between pin 1 and 3 and between pin 2 and 3 will be the same.

In a circuit, both sides of resistive element are often connected to the positive and negative electrodes of power. When you slide the brush "pin 3", you can get variable voltage within the range of the power supply.



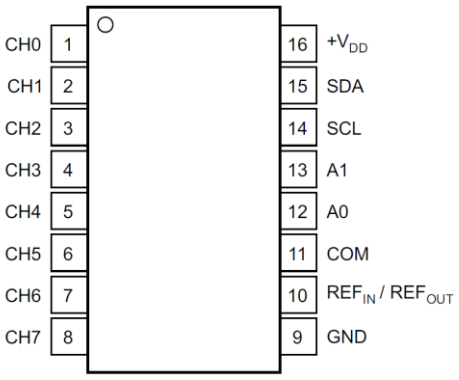
Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.



ADS7830

The ADS7830 is a single-supply, low-power, 8-bit data acquisition device that features a serial I2C interface and an 8-channel multiplexer. The following table is the pin definition diagram of ADS7830.

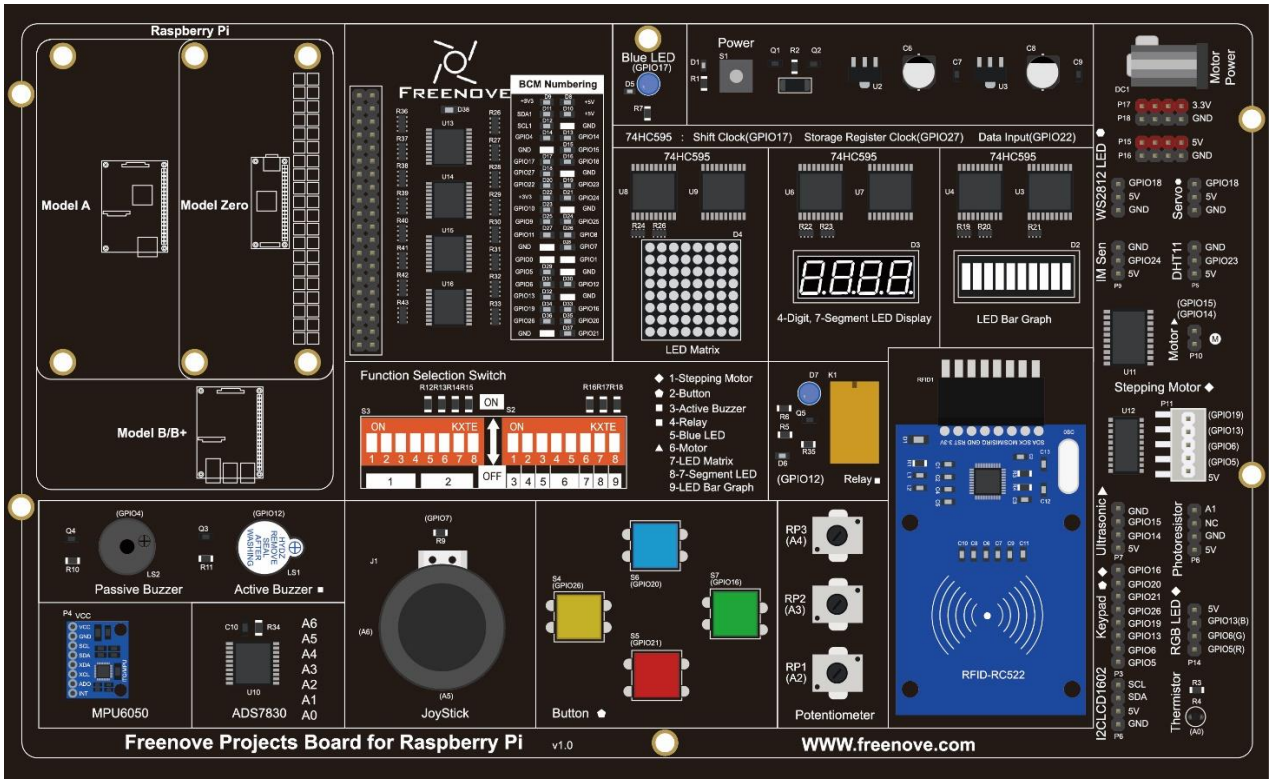
SYMBOL	PIN	DESCRIPTION	TOP VIEW
CH0	1	Analog input channels (A/D converter)	
CH1	2		
CH2	3		
CH3	4		
CH4	5		
CH5	6		
CH6	7		
CH7	8		
GND	9	Ground	
REF in/out	10	Internal +2.5V Reference, External Reference Input	
COM	11	Common to Analog Input Channel	
A0	12	Hardware address	
A1	13		
SCL	14	Serial Clock	
SDA	15	Serial Sata	
+VDD	16	Power Supply, 3.3V Nominal	

I2C communication

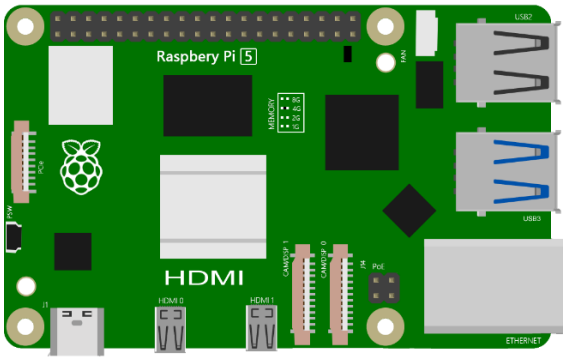
I2C (Inter-Integrated Circuit) has a two-wire serial communication mode, which can be used to connect a micro-controller and its peripheral equipment. Devices using I2C communications must be connected to the serial data line (SDA), and serial clock line (SCL) (called I2C bus). Each device has a unique address which can be used as a transmitter or receiver to communicate with devices connected via the bus.

Component List

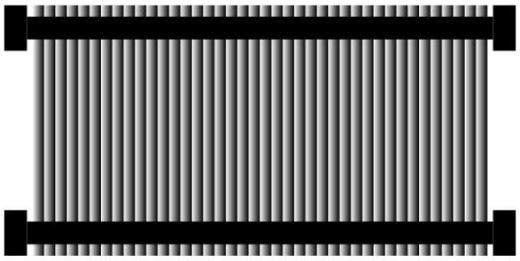
Freenove Projects Board for Raspberry Pi



Raspberry Pi

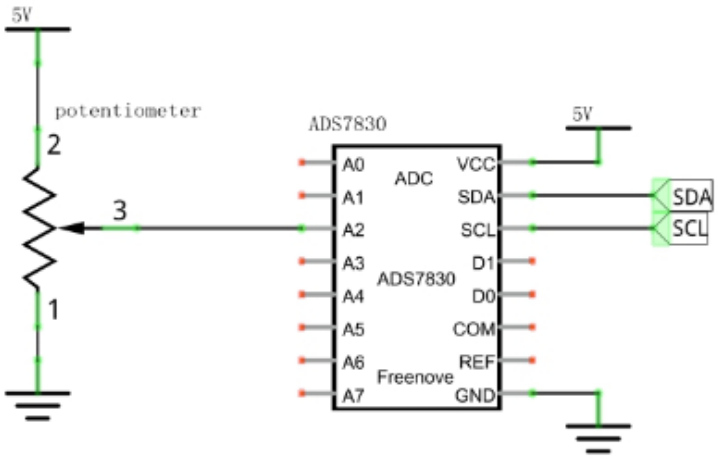


GPIO Ribbon Cable

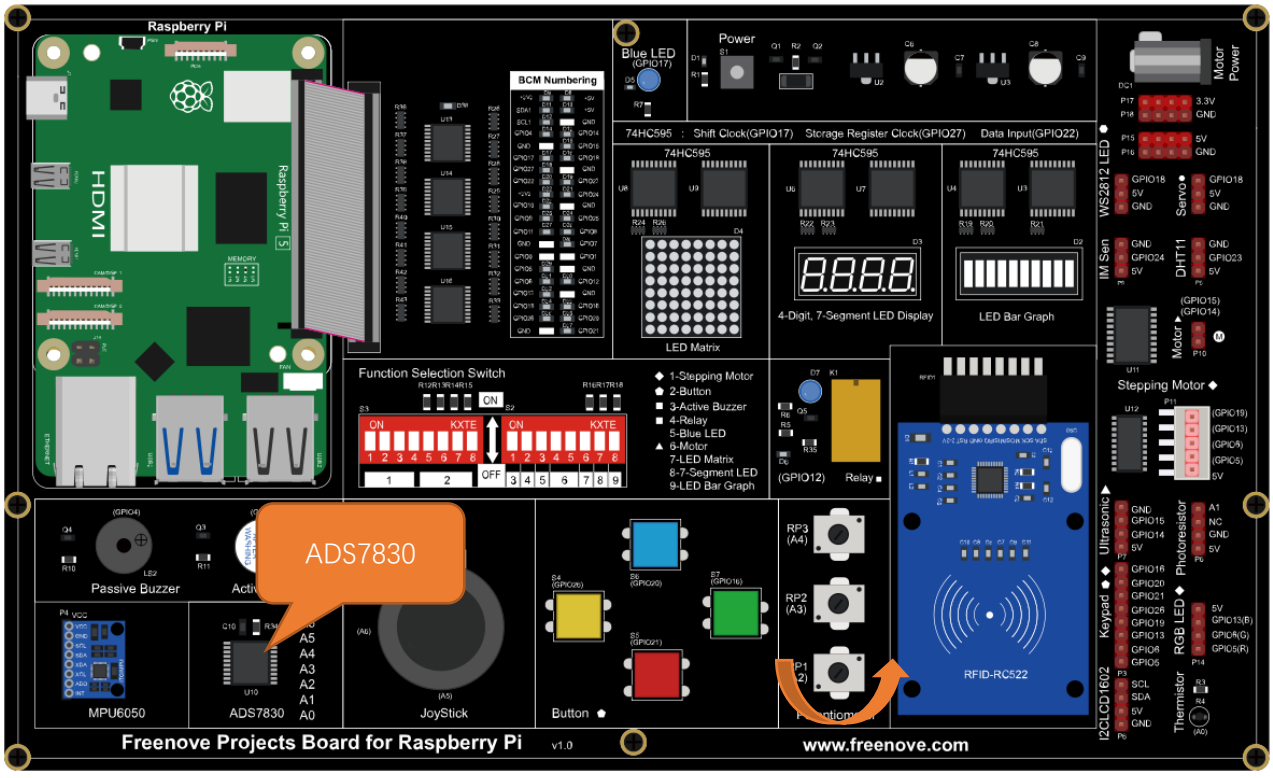


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Configure I2C and Install Smbus

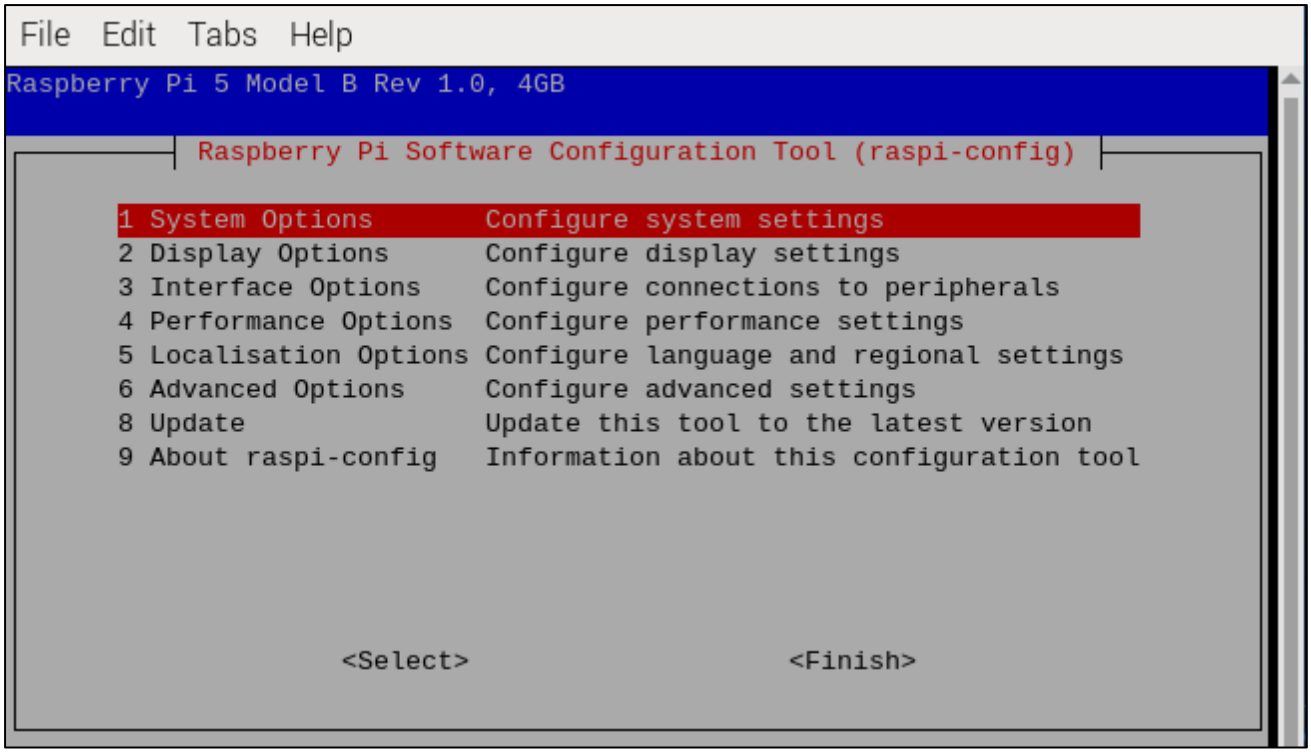
Enable I2C

The I2C interface in Raspberry Pi is disabled by default. You will need to open it manually and enable the I2C interface as follows:

Type command in the Terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose "**Interfacing Options**" then "I5 I2C" then "Yes" and then "Finish" in this order and restart your RPi.

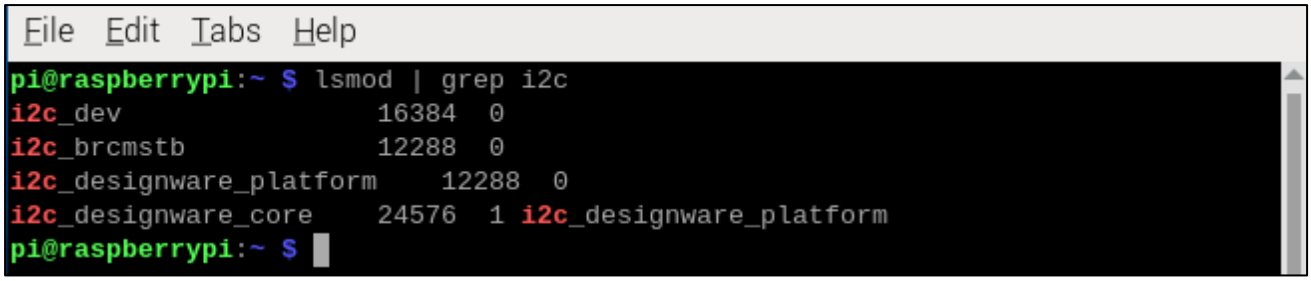
The I2C module will then be started.

Type a command to check whether the I2C module is started:

```
lsmod | grep i2c
```

If the I2C module has been started, the following content will be shown.

Different models of Raspberry Pi display different contents depending on the CPU installed:



Install I2C-Tools

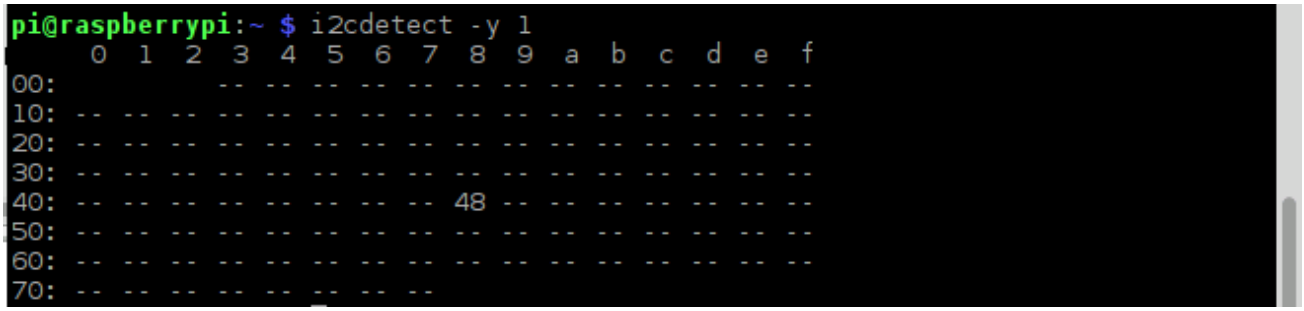
Next, type the command to install I2C-Tools. It is available with the Raspberry Pi OS by default.

```
sudo apt-get install i2c-tools
sudo apt-get install python3-smbus
```

I2C device address detection:

```
i2cdetect -y 1
```

When you are using the ADS7830 Module, the result should look like this:



Here, 48 (HEX) is the I2C address of ADC Module (ADS7830).

Code

C Code 7.1 ADC

For C code for the ADC Device, a custom library needs to be installed.

If you have any concerns, please send an email to: support@freenove.com

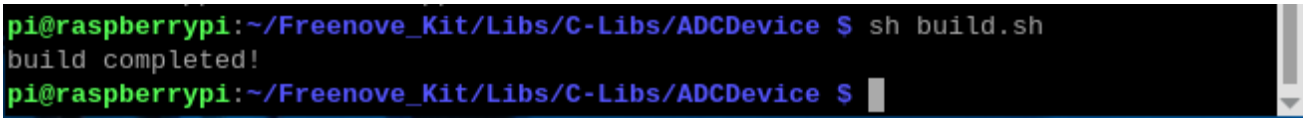
1. Use cd command to enter folder of the ADC Device library.

```
cd ~/Freenove_Kit/Libs/C-Libs/ADCDevice
```

2. Execute command below to install the library.

```
sh ./build.sh
```

A successful installation, without error prompts, is shown below:



Next, we will execute the code for this project.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 7_1_ADC directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/7_1_ADC
```

2. Use following command to compile "ADC.cpp" and generate the executable file "ADC".

```
g++ ADC.cpp -o ADC -lwiringPi -lADCDevice
```

3. Then run the generated file "ADC".

```
./ADC
```

After the program is executed, adjusting the potentiometer will produce a readout display of the potentiometer voltage values in the Terminal and the converted digital content.

```
ADC value : 135  ,      Voltage : 1.75V
ADC value : 135  ,      Voltage : 1.75V
ADC value : 136  ,      Voltage : 1.76V
ADC value : 141  ,      Voltage : 1.82V
ADC value : 144  ,      Voltage : 1.86V
ADC value : 146  ,      Voltage : 1.89V
ADC value : 148  ,      Voltage : 1.92V
ADC value : 149  ,      Voltage : 1.93V
ADC value : 149  ,      Voltage : 1.93V
ADC value : 144  ,      Voltage : 1.86V
ADC value : 143  ,      Voltage : 1.85V
ADC value : 143  ,      Voltage : 1.85V
ADC value : 142  ,      Voltage : 1.84V
ADC value : 141  ,      Voltage : 1.82V
```

The following is the code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <ADCDevice.hpp>

ADCDevice *adc;  // Define an ADC Device class object

int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");
    if(adc->detectI2C(0x48)){   // Detect the ads7830
        delete adc;               // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }

    while(1){
        int adcValue = adc->analogRead(2);    //read analog value of A0 pin
        float voltage = (float)adcValue / 255.0 * 5.0;  // Calculate voltage
        printf("ADC value : %d  ,\tVoltage : %.2fV\n",adcValue,voltage);
        delay(100);
    }
    return 0;
}
```

In this code, a custom class library "ADCDevice" is used. It contains the method of utilizing the ADC Module in this project, through which the ADC Module can easily and quickly be used. In the code, you need to first create a class pointer adc, and then point to an instantiated object. (Note: **An instantiated object is given a name and created in memory or on disk using the structure described within a class declaration.**)

```
ADCDevice *adc;  // Define an ADC Device class object
.....
adc = new ADCDevice();
```

Then use the member function detectIC(addr) in the class to detect the I2C module in the circuit. Different modules have different I2C addresses. The default address of ADC module ADS7830 is 0x48.

```
    if(adc->detectI2C(0x48)){   // Detect the ads7830
        delete adc;               // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }
```

When you have a class object pointed to a specific device, you can get the ADC value of the specific channel by calling the member function analogRead (chn) in this class

```
int adcValue = adc->analogRead(2);    //read analog value of A2 pin
```

Then according to the formula, the voltage value is calculated and displayed on the Terminal.

```
        float voltage = (float)adcValue / 255.0 * 5.0;  // Calculate voltage
        printf("ADC value : %d  ,\tVoltage : %.2fV\n",adcValue,voltage);
```

Reference

class ADCDevice

This is a base class. All ADC module classes are its derived classes. It has a real function and a virtual function.

```
int detectI2C(int addr);
```

This is a real function, which is used to detect whether the device with given I2C address exists. If it exists, return 1, otherwise return 0.

```
virtual int analogRead(int chn);
```

This is a virtual function that reads the ADC value of the specified channel. It is implemented in a derived class.

class ADS7830:public ADCDevice

These classes are derived from the ADCDevice class and mainly implement the function analogRead(chn).

```
int analogRead(int chn);
```

This returns the value read on the supplied analog input pin.
Parameter ADS7830, the range of is 0, 1, 2, 3, 4, 5, 6, 7.

Python Code 7.1 ADC

For Python code, ADCDevice requires a custom module which needs to be installed.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter folder of ADCDevice.

```
cd ~/Freenove_Kit/Libs/Python-Libs
```

2. Open the unzipped folder.

```
cd ADCDevice-1.0.4
```

3. Install library for python2 and python3.

```
sudo python3 setup.py install
```

A successful installation, without error prompts, is shown below:

```
Installed /usr/local/lib/python3.7/dist-packages/ADCDevice-1.0.4-py3.7.egg
Processing dependencies for ADCDevice==1.0.4
Finished processing dependencies for ADCDevice==1.0.4
```

Execute the following command. Observe the project result and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 7_1_ADC directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/7_1_ADC
```

2. Use the Python command to execute the Python code "ADC.py".

```
python ADC.py
```

After the program is executed, adjusting the potentiometer will produce a readout display of the potentiometer voltage values in the Terminal and the converted digital content.

```
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 169, Voltage : 2.19
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
```

The following is the code:

```
1  import time
2  from ADCDevice import *
3
4  adc = ADCDevice(0x48) # Define an ADCDevice class object
5
6  def setup():
7      global adc
8      if(adc.detectI2C(0x48)):
9          adc = ADS7830(0x48)
```

```
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(2)     # read the ADC value of channel 2
        voltage = value / 255.0 * 5.0  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)

def destroy():
    adc.close()

if __name__ == '__main__':   # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt: # Press ctrl-c to end the program.
        destroy()
```

In this code, a custom Python module "ADCDevice" is used. It contains the method of utilizing the ADC Module in this project, through which the ADC Module can easily and quickly be used. In the code, you need to first create an ADCDevice object adc.

```
adc = ADCDevice(0x48) # Define an ADCDevice class object
```

Then in setup(), use detecticIC(addr), the member function of ADCDevice, to detect the I2C module in the circuit. The default address of ADS7830 is 0x48.

```
def setup():
    global adc
    if(adc.detectI2C(0x48)):
        adc = ADS7830(0x48)
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)
```

When you have a class object of a specific device, you can get the ADC value of the specified channel by calling the member function of this class, analogRead(chn). In loop(), get the ADC value of potentiometer.

```
        value = adc.analogRead(2)     # read the ADC value of channel 2
```

Then according to the formula, the voltage value is calculated and displayed on the terminal monitor.

```
        voltage = value / 255.0 * 5.0  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)
```

Reference

About smbus Module:

smbus Module

The System Management Bus Module defines an object type that allows SMBus transactions on hosts running the Linux kernel. The host kernel must support I2C, I2C device interface support, and a bus adapter driver. All of these can be either built-in to the kernel, or loaded from modules.

In Python, you can use help(smbus) to view the relevant functions and their descriptions.

bus=smbus.SMBus(1): Create an SMBus class object.

bus.read_byte_data(address,cmd+chn): Read a byte of data from an address and return it.

bus.write_byte_data(address,cmd,value): Write a byte of data to an address.

class ADCDevice(object)

This is a base class.

```
int detectI2C(int addr);
```

This is a member function, which is used to detect whether the device with the given I2C address exists. If it exists, it returns true. Otherwise, it returns false.

class ADS7830(ADCDevice)

These classes are derived from the ADCDevice class and mainly implement the function analogRead(chn).

```
int analogRead(int chn);
```

This returns the value read on the supplied analog input pin.

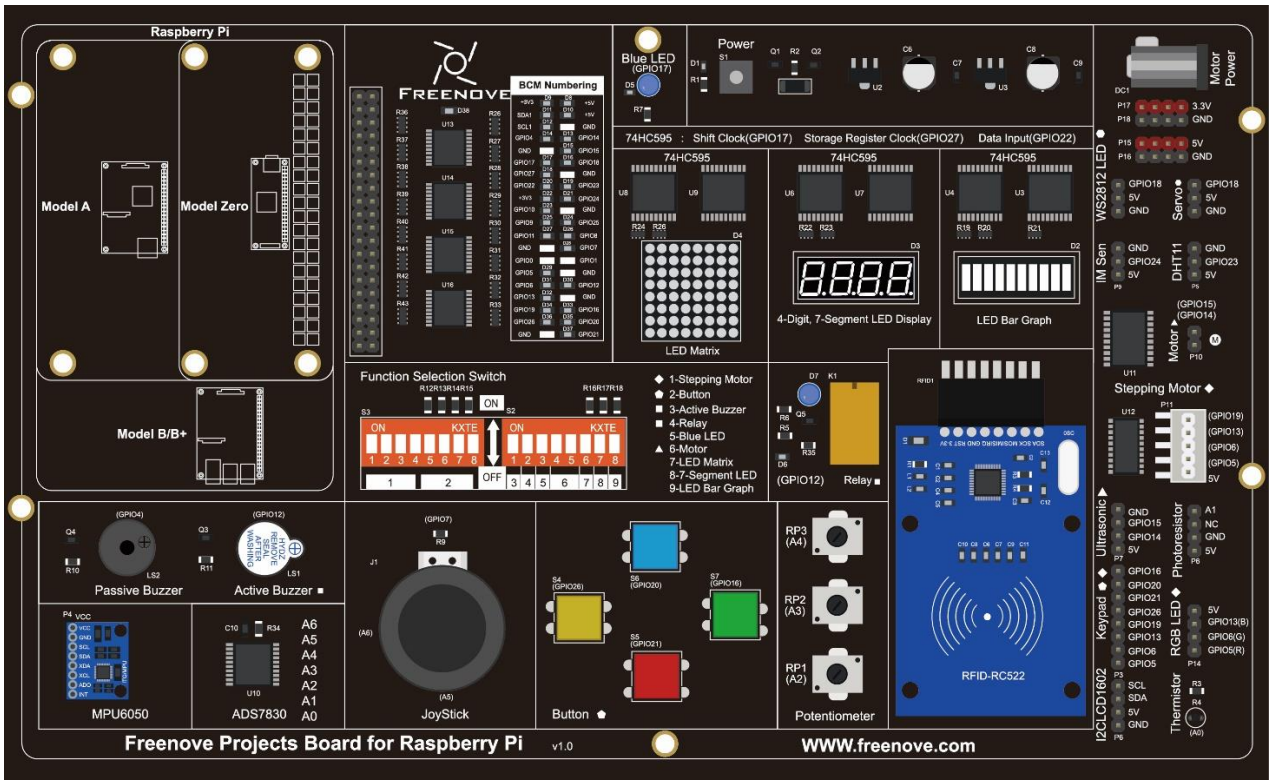
Parameter chn: For ADS7830, the range is 0, 1, 2, 3, 4, 5, 6, 7.

Project 7.2 Soft Light

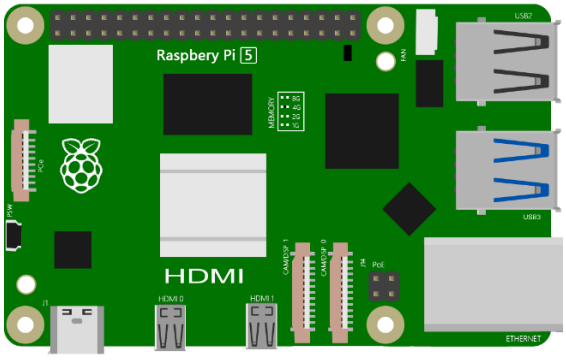
In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle ratio of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

Component List

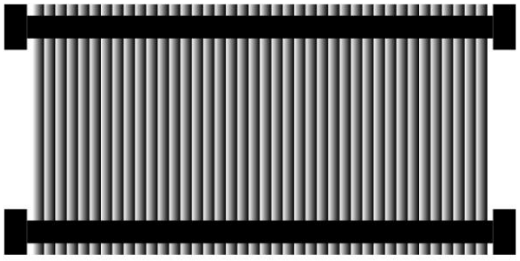
Freenove Projects Board for Raspberry Pi



Raspberry Pi

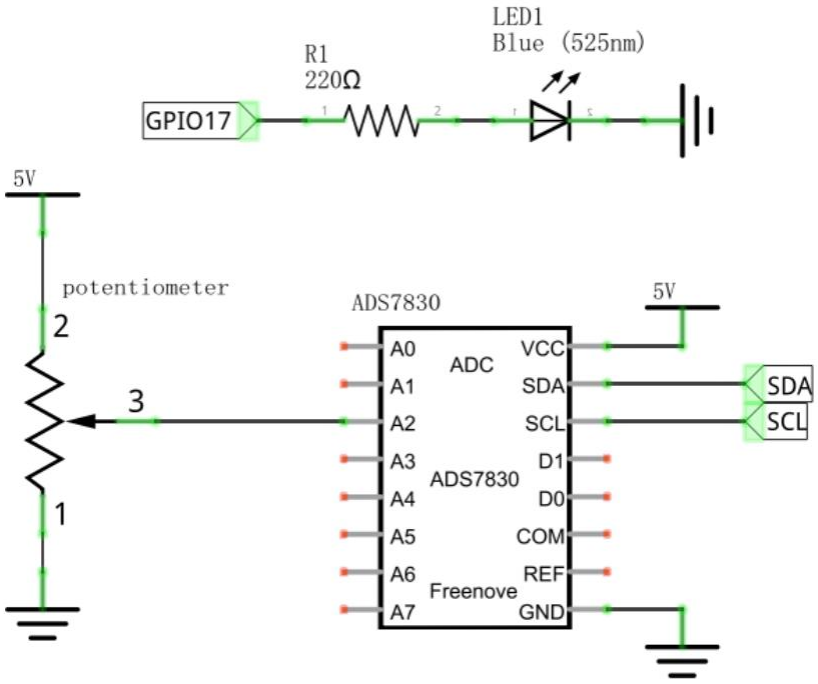


GPIO Ribbon Cable

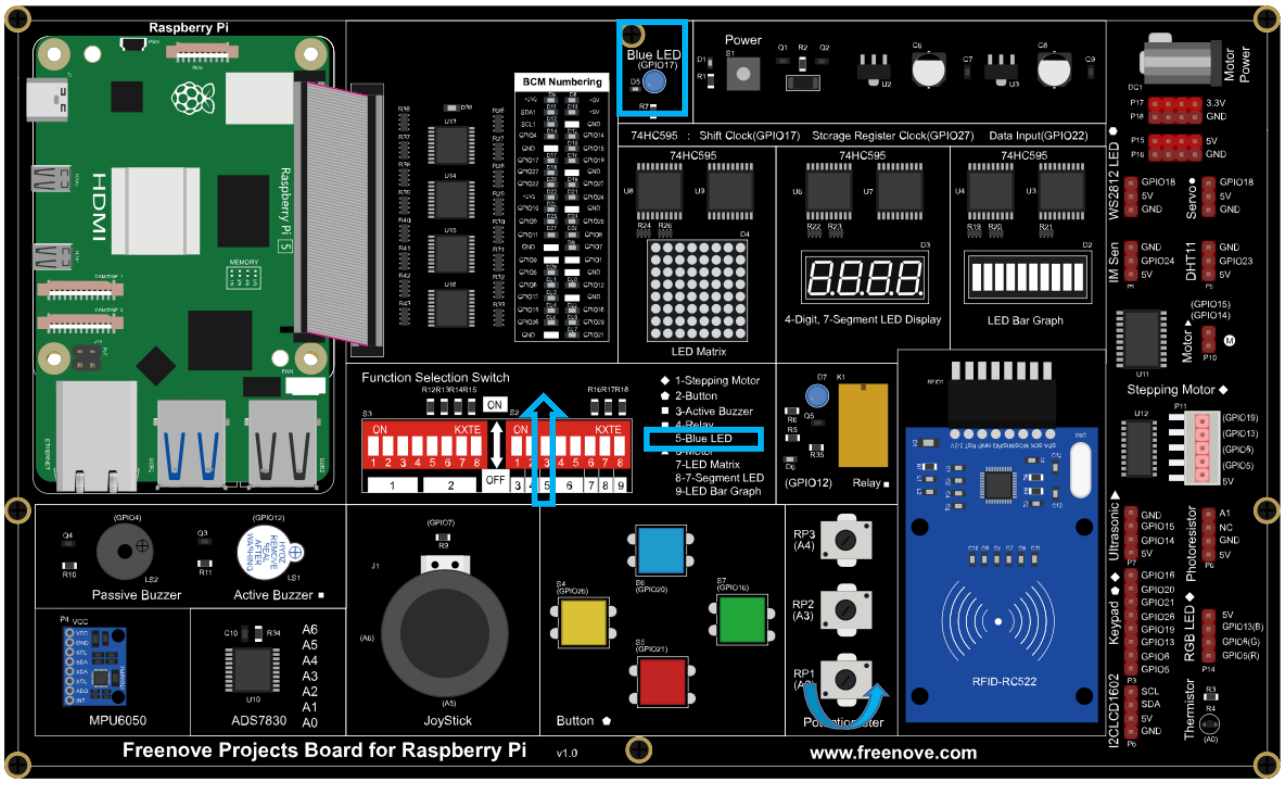


Circuit

Schematic diagram



Hardware connection



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 7.2 Softlight

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please move on.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 7_2_Softlight directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/7_2_Softlight
```

2. Use following command to compile "Softlight.cpp" and generate executable file "Softlight".

```
g++ Softlight.cpp -o Softlight -lwiringPi -lADCDevice
```

3. Then run the generated file "Softlight".

```
./Softlight
```

After the program is executed, adjusting the potentiometer will display the voltage values of the potentiometer in the Terminal window and the converted digital quantity. As a consequence, the brightness of LED will be changed.

The following is the code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softPwm.h>
#include <ADCDevice.hpp>

#define ledPin 17
ADCDevice *adc;  // Define an ADC Device class object

int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");

    if(adc->detectI2C(0x48)){    // Detect the ads7830
        delete adc;              // Free previously pointed memory
        adc = new ADS7830(0x48);     // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }
    wiringPiSetupGpio();//Initialize wiringPi. Use BCM Number.
    softPwmCreate(ledPin,0,100);
```

```
25
26      while(1){
27          int adcValue = adc->analogRead(2);                //read analog value of A2 pin
28          softPwmWrite(ledPin,adcValue*100/255);            // Mapping to PWM duty cycle
29          float voltage = (float)adcValue / 255.0 * 5.0;    // Calculate voltage
30          printf("ADC value : %d  ,\tVoltage : %.2fV\n",adcValue,voltage);
31          delay(30);
32      }
33      return 0;
34  }
```

In the code, read the ADC value of potentiometer and map it to the duty cycle of PWM to control LED brightness.

```
            int adcValue = adc->analogRead(2);          //read analog value of A2 pin
            softPwmWrite(ledPin,adcValue*100/255);      // Mapping to PWM duty cycle
```

Python Code 7.2 Softlight

If you haven't configured I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.
If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 7_2_Softlight directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/7_2_Softlight
```

2. Use the python command to execute the Python code "Softlight.py".

```
python Softlight.py
```

After the program is executed, adjusting the potentiometer will display the voltage values of the potentiometer in the Terminal window and the converted digital quantity. As a consequence, the brightness of LED will be changed.

The following is the code:

```
1   from gpiozero import PWMLED
2   import time
3   from ADCDevice import *
4
5   led = PWMLED(17,frequency=1000) # define LED pin according to BCM Numbering
6   adc = ADCDevice(0x48)           # Define an ADCDevice class object
7
8   def setup():
9       global adc
10      if(adc.detectI2C(0x48)):
11          adc = ADS7830(0x48)
12      else:
13          print("No correct I2C address found, \n"
```

```
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(2)      # read the ADC value of channel 0
        led.value = value / 255.0      # Mapping to PWM duty cycle
        voltage = value / 255.0 * 5.0  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.03)

def destroy():
    led.close()
    adc.close()

if __name__ == '__main__':   # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

In the code, read ADC value of potentiometers and map it to the duty cycle of the PWM to control LED brightness.

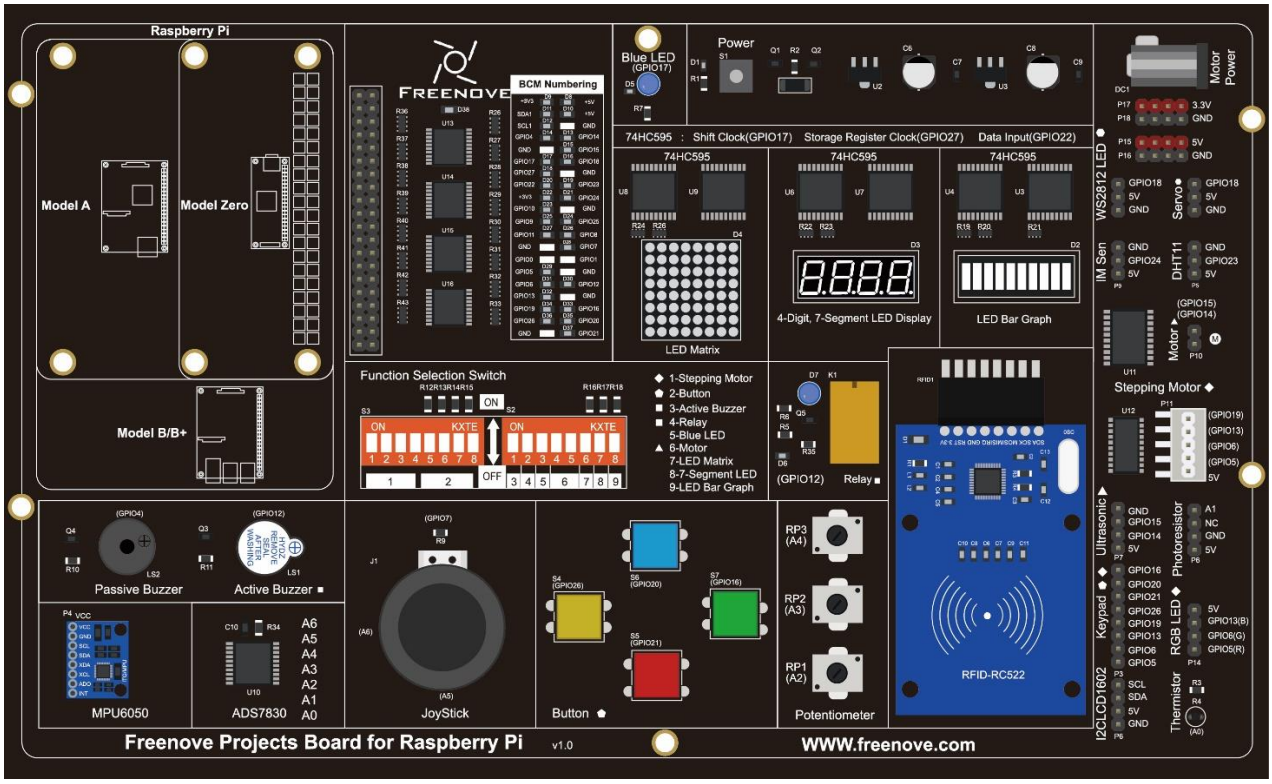
```
        value = adc.analogRead(2)      # read the ADC value of channel 0
        led.value = value / 255.0      # Mapping to PWM duty cycle
```

Project 7.3 Colorful Light

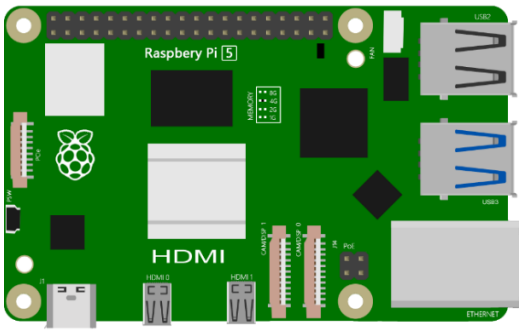
In this project, 3 potentiometers are used to control the RGB LED and in principle it is the same as with the Soft Light. project. Namely, read the voltage value of the potentiometer and then convert it to PWM used to control LED brightness. Difference is that the previous soft light project needed only one LED while this one required (3) RGB LEDs.

Component List

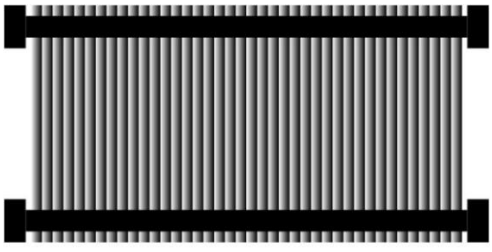
Freenove Projects Board for Raspberry Pi



Raspberry Pi



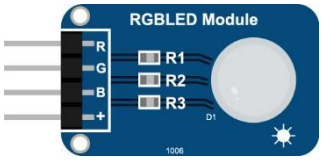
GPIO Ribbon Cable



Jumper Wire

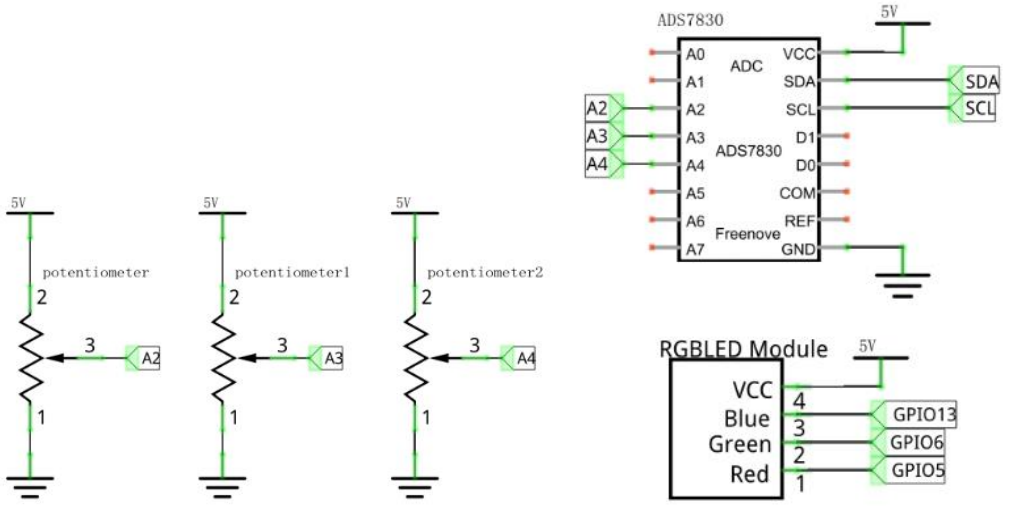


RGBLED Module

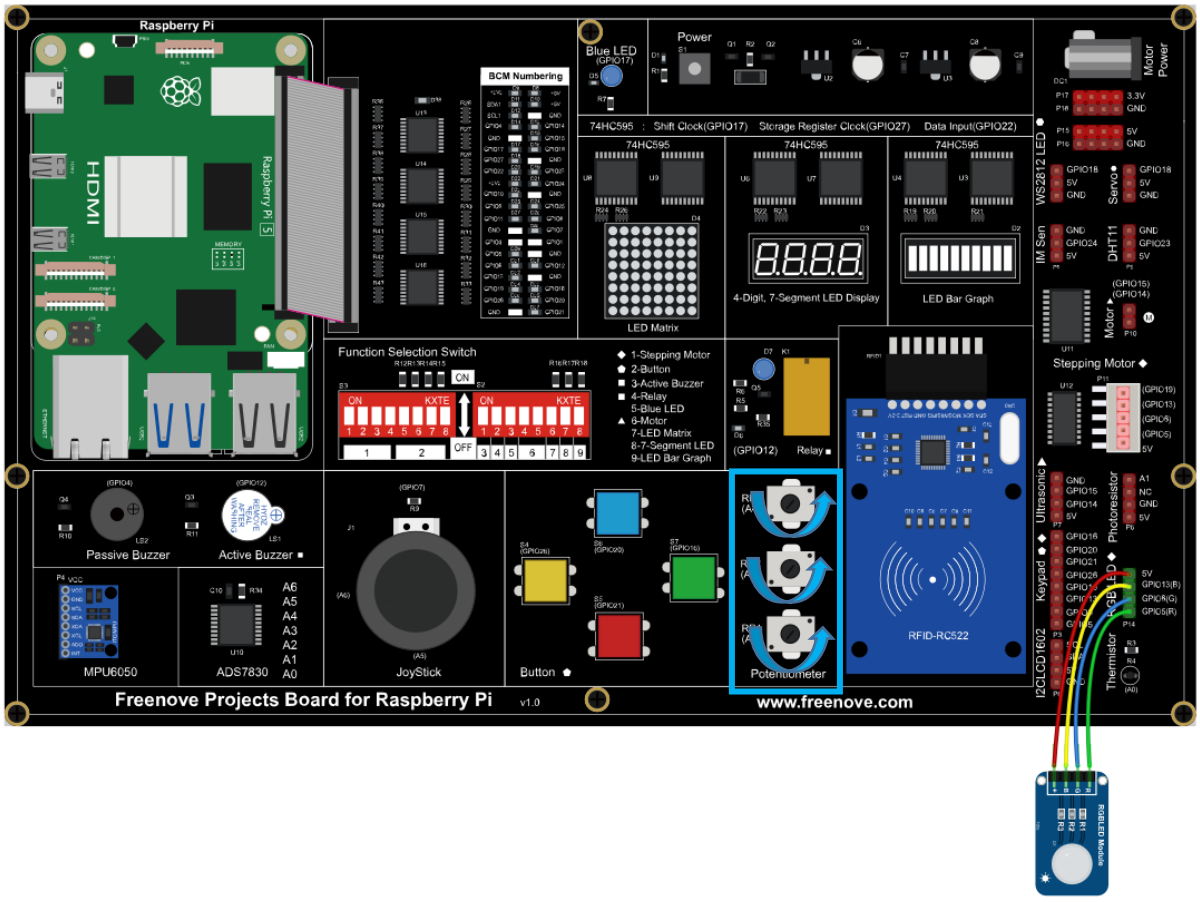


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 7.3 Colorful Softlight

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 7_3_ColorfulSoftlight directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/7_3_ColorfulSoftlight
```

2. Use following command to compile "ColorfulSoftlight.cpp" and generate executable file "ColorfulSoftlight".

```
g++ ColorfulSoftlight.cpp -o ColorfulSoftlight -lwiringPi -lADCDevice
```

3. Then run the generated file "ColorfulSoftlight".

```
./ColorfulSoftlight
```

After the program is executed, rotate one of the potentiometers, and the color of RGB LED will change. The Terminal window will display the ADC value of each potentiometer.

```
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 238
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 206
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 174
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 152
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 139
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 138
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 138
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 138
ADC value val_Red: 147  ,       val_Green: 192  ,       val_Blue: 138
```

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softPwm.h>
#include <ADCDevice.hpp>

#define ledRedPin    5    //define 3 pins for RGBLED
#define ledGreenPin  6
#define ledBluePin   13

ADCDevice *adc;  // Define an ADC Device class object

int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");

    if(adc->detectI2C(0x48)){     // Detect the ads7830
        delete adc;               // Free previously pointed memory
```

```
18          adc = new ADS7830(0x48);       // If detected, create an instance of ADS7830.
19      }
20      else{
21          printf("No correct I2C address found, \n"
22          "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
23          "Program Exit. \n");
24          return -1;
25      }
26      wiringPiSetupGpio();
27      softPwmCreate(ledRedPin,0,100);         //creat 3 PMW output pins for RGBLED
28      softPwmCreate(ledGreenPin,0,100);
29      softPwmCreate(ledBluePin,0,100);
30      while(1){
31          int val_Red = adc->analogRead(2);  //read analog value of 3 potentiometers
32          int val_Green = adc->analogRead(3);
33          int val_Blue = adc->analogRead(4);
34          softPwmWrite(ledRedPin,100-val_Red*100/255);
35          softPwmWrite(ledGreenPin,100-val_Green*100/255);
36          softPwmWrite(ledBluePin,100-val_Blue*100/255);
37
38          //print out the read ADC value
39          printf("ADC value val_Red: %d  ,\tval_Green: %d  ,\tval_Blue: %d
    \n",val_Red,val_Green,val_Blue);
40          delay(100);
41      }
42      return 0;
43  }
```

In the code you can read the ADC values of the 3 potentiometers and map it into a PWM duty cycle to control the 3 LED elements to vary the color of their respective RGB LED.

Python Code 7.3 ColorfulSoftlight

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 7_3_ColorfulSoftlight directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/7_3_ColorfulSoftlight
```

2. Use python command to execute python code "ColorfulSoftlight.py".

```
python ColorfulSoftlight.py
```

After the program is executed, rotate one of the potentiometers, and the color of RGB LED will change. The Terminal window will display the ADC value of each potentiometer.

The following is the program code:

```
1  from gpiozero import RGBLED
2  import time
3  from ADCDevice import *
4
5  led = RGBLED(red=5, green=6, blue=13, active_high=False)
6  adc = ADCDevice(0x48) # Define an ADCDevice class object
7
8  def setup():
9      global adc
10     if(adc.detectI2C(0x48)): # Detect the pcf8591.
11         adc = ADS7830(0x48)
12     else:
13         print("No correct I2C address found, \n"
14         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
15         "Program Exit. \n");
16         exit(-1)
17
18 def loop():
19     while True:
20         value_Red = adc.analogRead(2)        # read ADC value of 3 potentiometers
21         value_Green = adc.analogRead(3)
22         value_Blue = adc.analogRead(4)
23         led.red = value_Red/255  # map the value of potentiometers into PWM value.
24         led.green = value_Green/255
25         led.blue  = value_Blue/255
26         # print read ADC value
27         print ('ADC Value value_Red: %d ,\tvlue_Green: %d ,\tvalue_Blue: %d'
   %(value_Red,value_Green,value_Blue))
28         time.sleep(0.01)
29
```

```
30  def destroy():
31      adc.close()
32      led.close()
33
34  if __name__ == '__main__': # Program entrance
35      print ('Program is starting ... ')
36      try:
37          setup()
38          loop()
39      except KeyboardInterrupt:  # Press ctrl-c to end the program.
40          print("Ending program")
41      finally:
42          destroy()
```

In the code you can read the ADC values of the 3 potentiometers and map it into a PWM duty cycle to control the 3 LED elements to vary the color of their respective RGB LED.

Chapter 8 Photoresistor & LED

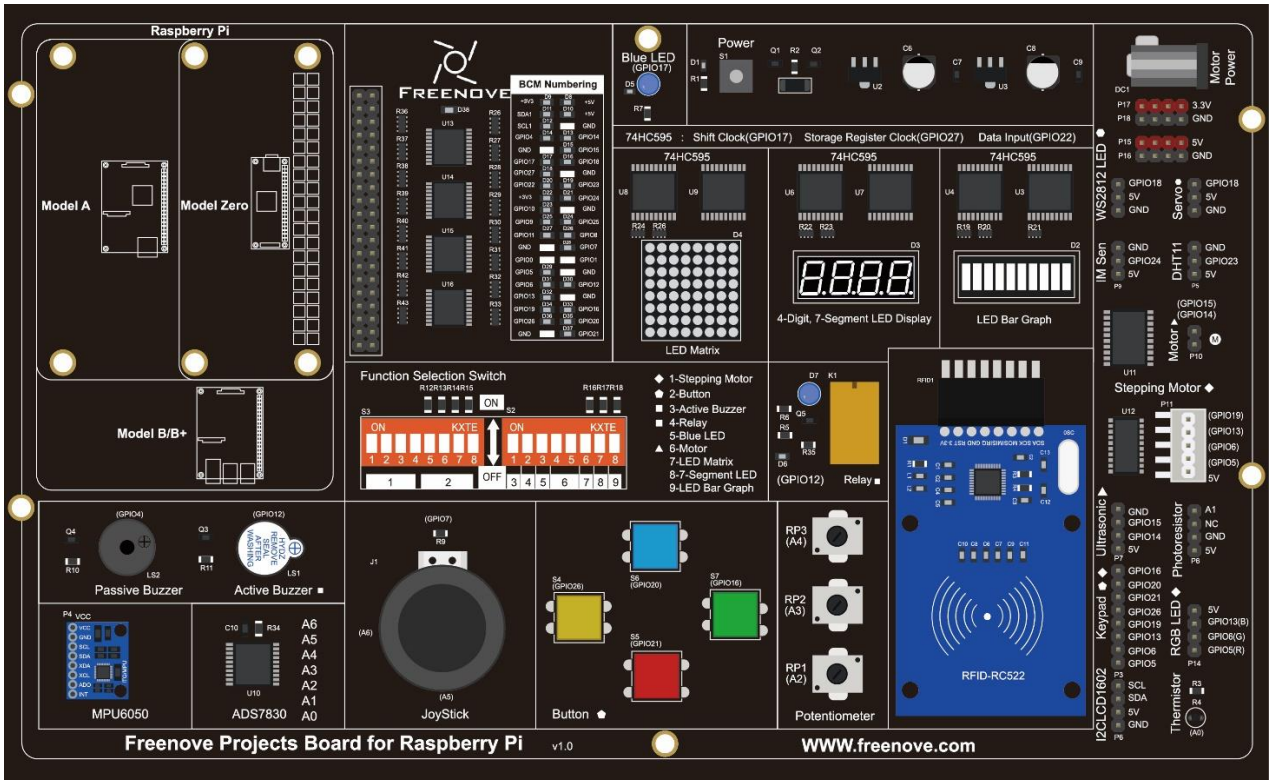
In this chapter, we will learn how to use a photoresistor to make an automatic dimming nightlight.

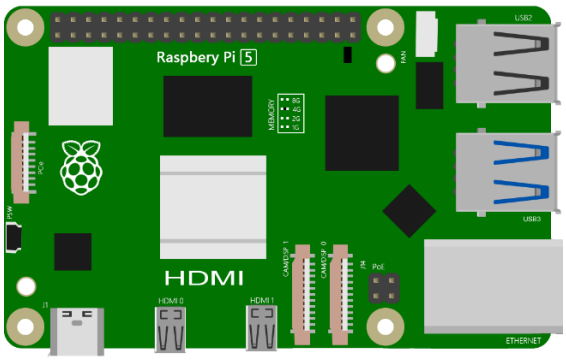
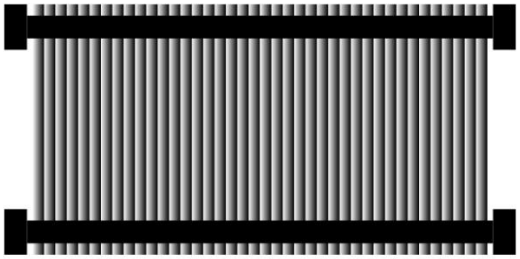
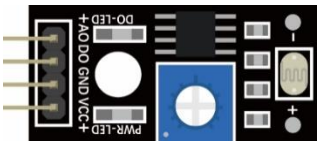
Project 8.1 NightLamp

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function. When the ambient light is less (darker environment), the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

Component List

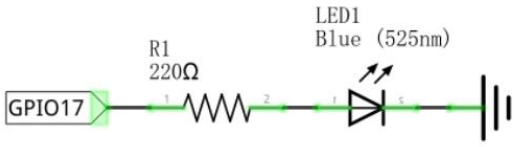
Freenove Projects Board for Raspberry Pi

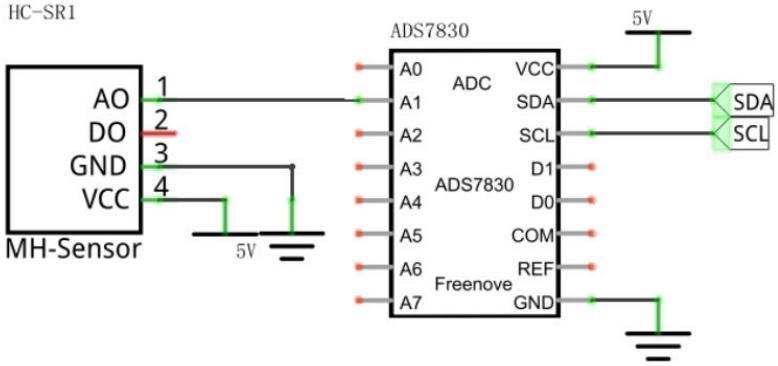


Raspberry Pi	GPIO Ribbon Cable
	
Jumper Wire	**Photoresistor**
	

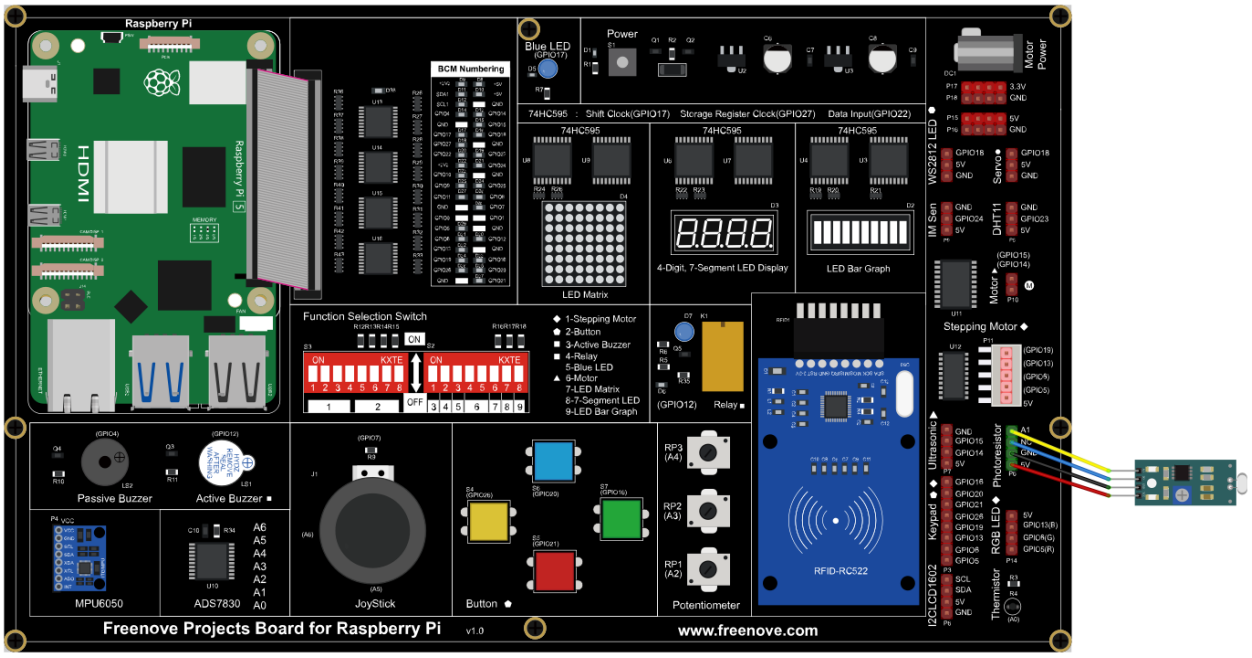
Circuit

Schematic diagram





Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

The code used in this project is identical with what was used in the last chapter.

C Code 8.1 Nightlamp

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 8_Nightlamp directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/8_Nightlamp
```

2. Use following command to compile "Nightlamp.cpp" and generate executable file "Nightlamp".

```
g++ Nightlamp.cpp -o Nightlamp -lwiringPi -lADCDevice
```

3. Then run the generated file "Nightlamp".

```
./Nightlamp
```

After the program is executed, if you cover the Photoresistor or increase the light shining on it, the brightness of the LED changes accordingly. As in previous projects the Terminal window will display the current input voltage value of ADC module A1 pin and the converted digital quantity.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softPwm.h>
#include <ADCDevice.hpp>

#define ledPin 17

ADCDevice *adc;  // Define an ADC Device class object

int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");

    if(adc->detectI2C(0x48)){    // Detect the ads7830
        delete adc;               // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }
    wiringPiSetupGpio();
    softPwmCreate(ledPin,0,100);
    while(1){
        int value = adc->analogRead(1);  //read analog value of A1 pin
        softPwmWrite(ledPin,value*100/255);
        float voltage = (float)value / 255.0 * 5.0;  // calculate voltage
        printf("ADC value : %d  ,\tVoltage : %.2fV\n",value,voltage);
        delay(100);
    }
    return 0;
}
```

Python Code 8.1 Nightlamp

If you haven't configure I2C, please refer to Chapter 7. If you have done it, please continue.
First, observe the project result, and then learn about the code in detail.
If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 8_Nightlamp directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/8_Nightlamp
```

2. Use the python command to execute the Python code "Nightlamp.py".

```
python Nightlamp.py
```

After the program is executed, if you cover the Photoresistor or increase the light shining on it, the brightness of the LED changes accordingly. As in previous projects the Terminal window will display the current input voltage value of ADC module A1 pin and the converted digital quantity.

The following is the program code:

```
1  from gpiozero import PWMLED
2  import time
3  from ADCDevice import *
4
5  ledPin = 17 # define ledPin
6  led = PWMLED(ledPin)
7  adc = ADCDevice(0x48) # Define an ADCDevice class object
8
9  def setup():
10     global adc
11     if(adc.detectI2C(0x48)): # Detect the pcf8591.
12         adc = ADS7830(0x48)
13     else:
14         print("No correct I2C address found, \n"
15         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
16         "Program Exit. \n");
17         exit(-1)
18
19 def loop():
20     while True:
21         value = adc.analogRead(1)     # read the ADC value of channel 1
22         led.value = value / 255.0     # Mapping to PWM duty cycle
23         voltage = value / 255.0 * 3.3
24         print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
25         time.sleep(0.01)
26
27 def destroy():
28     led.close()
29     adc.close()
```

```
30
31  if __name__ == '__main__':   # Program entrance
32      print ('Program is starting ... ')
33      try:
34          setup()
35          loop()
36      except KeyboardInterrupt:  # Press ctrl-c to end the program.
37          print("Ending program")
38      finally:
39          destroy()
```

Chapter 9 Thermistor

In this chapter, we will learn about Thermistors which are another kind of Resistor.

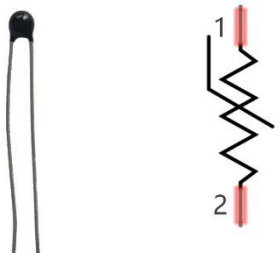
Project 9.1 Thermometer

A Thermistor is a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

Component knowledge

Thermistor

Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

Rt=R*EXP [B*(1/T2-1/T1)]

Where:

Rt is the thermistor resistance under T2 temperature;

R is in the nominal resistance of thermistor under T1 temperature;

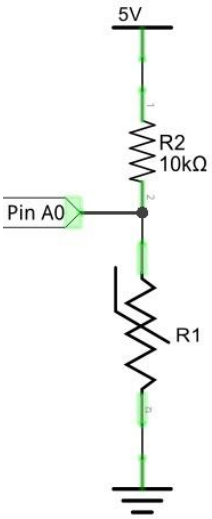
EXP[n] is nth power of e;

B is for thermal index;

T1, T2 is Kelvin temperature (absolute temperature). Kelvin temperature=273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: B=3950, R=10k, T1=25.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



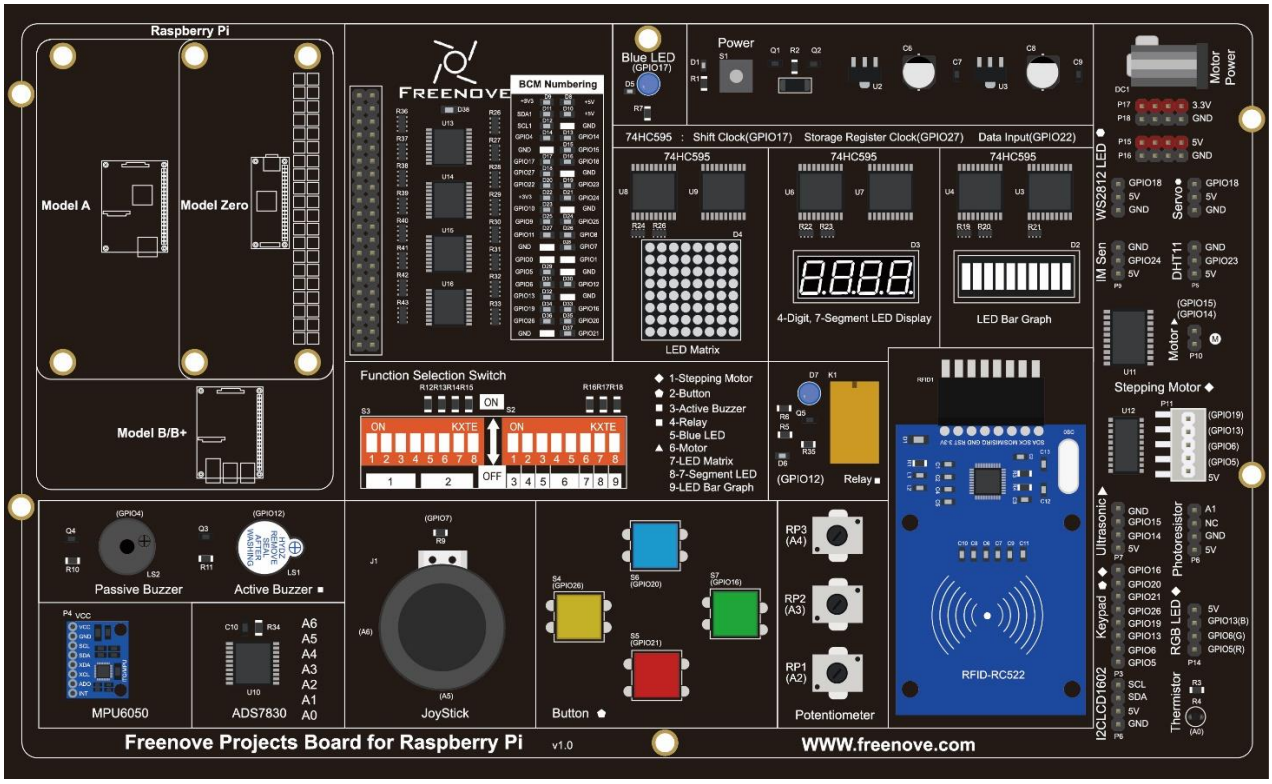
We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

Therefore, the temperature formula can be derived as:

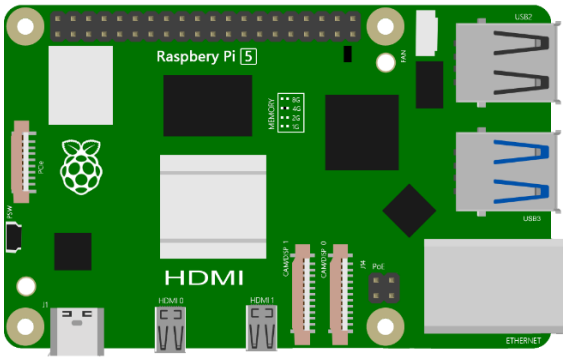
$$T2 = 1/(1/T1 + \ln(Rt/R)/B)$$

Component List

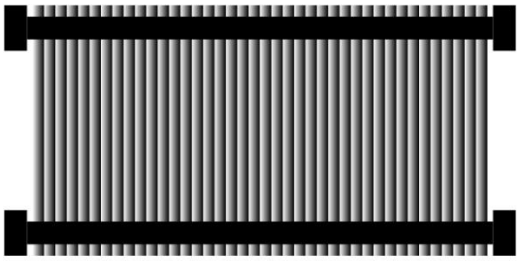
Freenove Projects Board for Raspberry Pi



Raspberry Pi

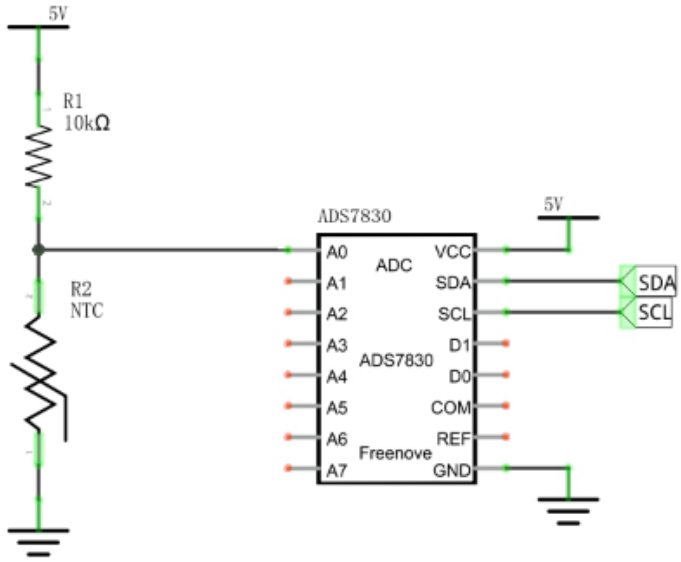


GPIO Ribbon Cable



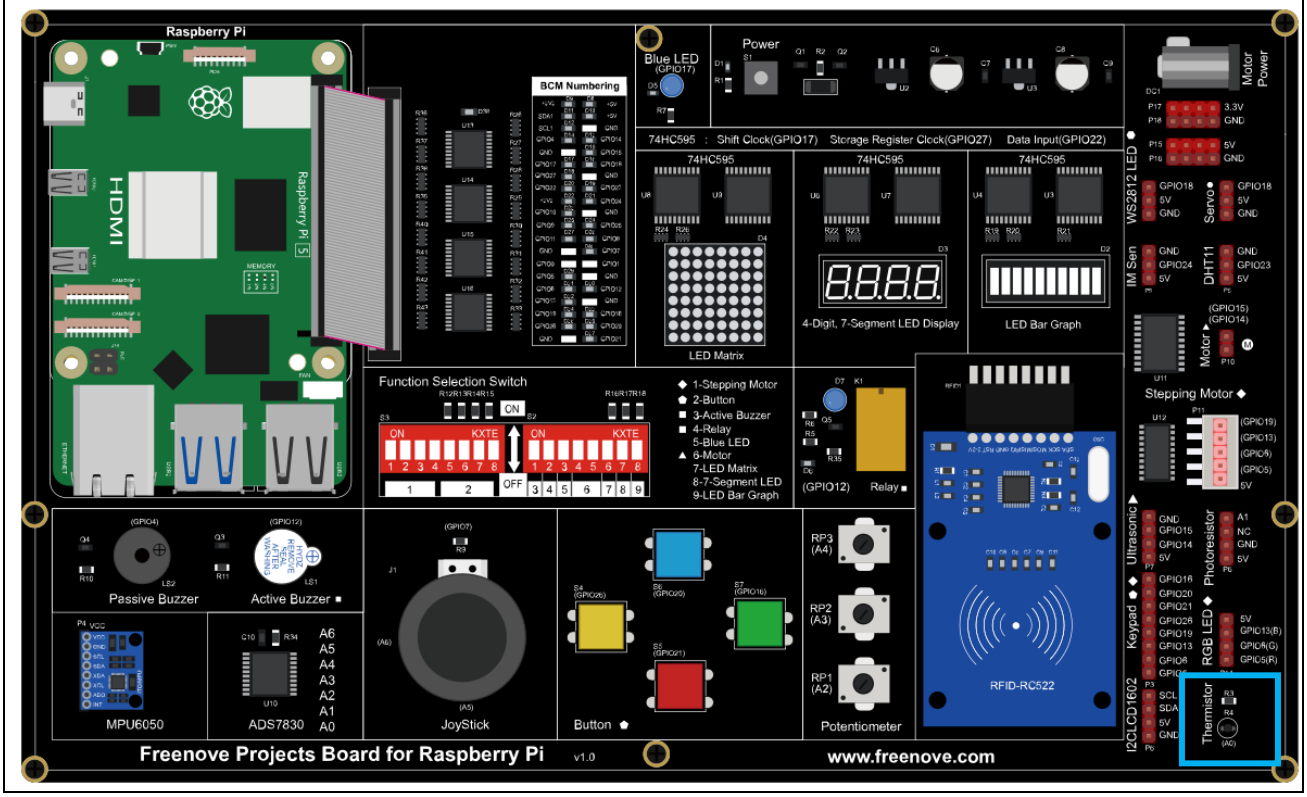
Circuit

Schematic diagram



Hardware connection.

After running the program, hold your finger against the sensor to observe the change.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project code, the ADC value still needs to be read, but the difference here is that a specific formula is used to calculate the temperature value.

C Code 9.1 Thermometer

If you haven't configure I2C, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 9_Thermometer directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/9_Thermometer
```

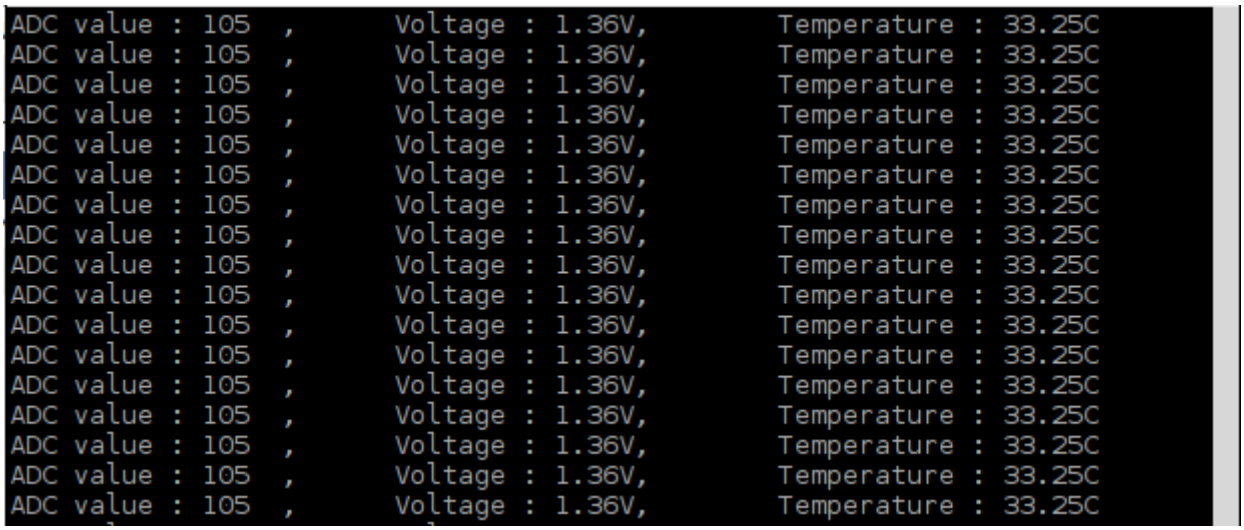
2 Use following command to compile "Thermometer.cpp" and generate executable file "Thermometer".

```
g++ Thermometer.cpp -o Thermometer -lwiringPi -lADCDevice
```

3 Then run the generated file "Thermometer".

```
./Thermometer
```

After the program is executed, the Terminal window will display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor with your index finger and thumb for a brief time, you should see that the temperature value increases.



The following is the code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <math.h>
#include <ADCDevice.hpp>

ADCDevice *adc;  // Define an ADC Device class object

int main(void){
```

```
    adc = new ADCDevice();
    printf("Program is starting ... \n");
    if(adc->detectI2C(0x48)){    // Detect the ads7830
        delete adc;               // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }
    printf("Program is starting ... \n");
    while(1){
        int adcValue = adc->analogRead(0);  //read analog value A0 pin
        float voltage = (float)adcValue / 255.0 * 5.0;    // calculate voltage
        float Rt = 10 * voltage / (5.0 - voltage);        //calculate resistance value of
thermistor
        float tempK = 1/(1/(273.15 + 25) + log(Rt/10)/3950.0); //calculate temperature
(Kelvin)
        float tempC = tempK -273.15;        //calculate temperature (Celsius)
        printf("ADC value : %d  ,\tVoltage : %.2fV,
\tTemperature : %.2fC\n",adcValue,voltage,tempC);
        delay(100);
    }
    return 0;
}
```

In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

Python Code 9.1 Thermometer

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 9_Thermometer directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/9_Thermometer
```

2. Use python command to execute Python code "Thermometer.py".

```
python Thermometer.py
```

After the program is executed, the Terminal window will display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor with your index finger and thumb for a brief time, you should see that the temperature value increases.

```
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
```

The following is the code:

```
1  import time
2  import math
3  from ADCDevice import *
4
5  adc = ADCDevice(0x48) # Define an ADCDevice class object
6
7  def setup():
8      global adc
9      if(adc.detectI2C(0x48)): # Detect the pcf8591.
10         adc = ADS7830(0x48)
11     else:
12         print("No correct I2C address found, \n"
13         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
14         "Program Exit. \n");
15         exit(-1)
16
17 def loop():
```

```
    while True:
        value = adc.analogRead(0)        # read ADC value A0 pin
        voltage = value / 255.0 * 3.3        # calculate voltage
        Rt = 10 * voltage / (3.3 - voltage)    # calculate resistance value of thermistor
        tempK = 1/(1/(273.15 + 25) + math.log(Rt/10)/3950.0) # calculate temperature (Kelvin)
        tempC = tempK -273.15        # calculate temperature (Celsius)
        print ('ADC Value : %d, Voltage : %.2f, Temperature : %.2f'%(value,voltage,tempC))
        time.sleep(0.01)

def destroy():
    adc.close()

if __name__ == '__main__':  # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

Chapter 10 Joystick

In an earlier chapter, we learned how to use Rotary Potentiometer. We will now learn about joysticks, which are electronic modules that work on the same principle as the Rotary Potentiometer.

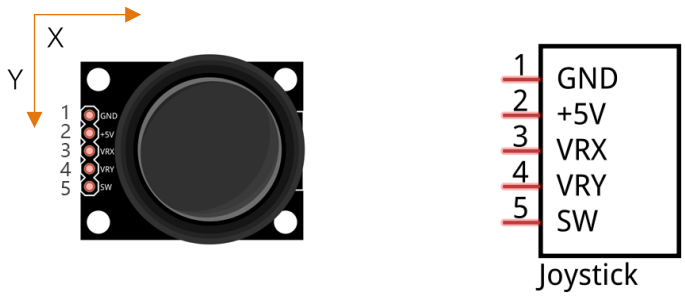
Project 10.1 Joystick

In this project, we will read the output data of a joystick and display it to the Terminal screen.

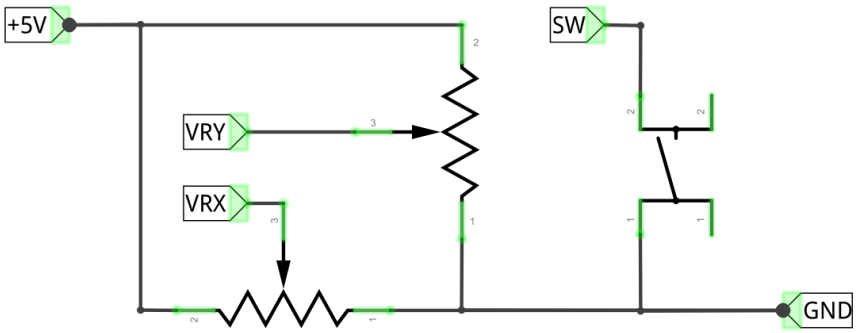
Component knowledge

Joystick

A Joystick is a kind of input sensor used with your fingers. You should be familiar with this concept already as they are widely used in gamepads and remote controls. It can receive input on two axes (Y and or X) at the same time (usually used to control direction on a two dimensional plane). And it also has a third direction capability by **pressing down (Z axis/direction)**.



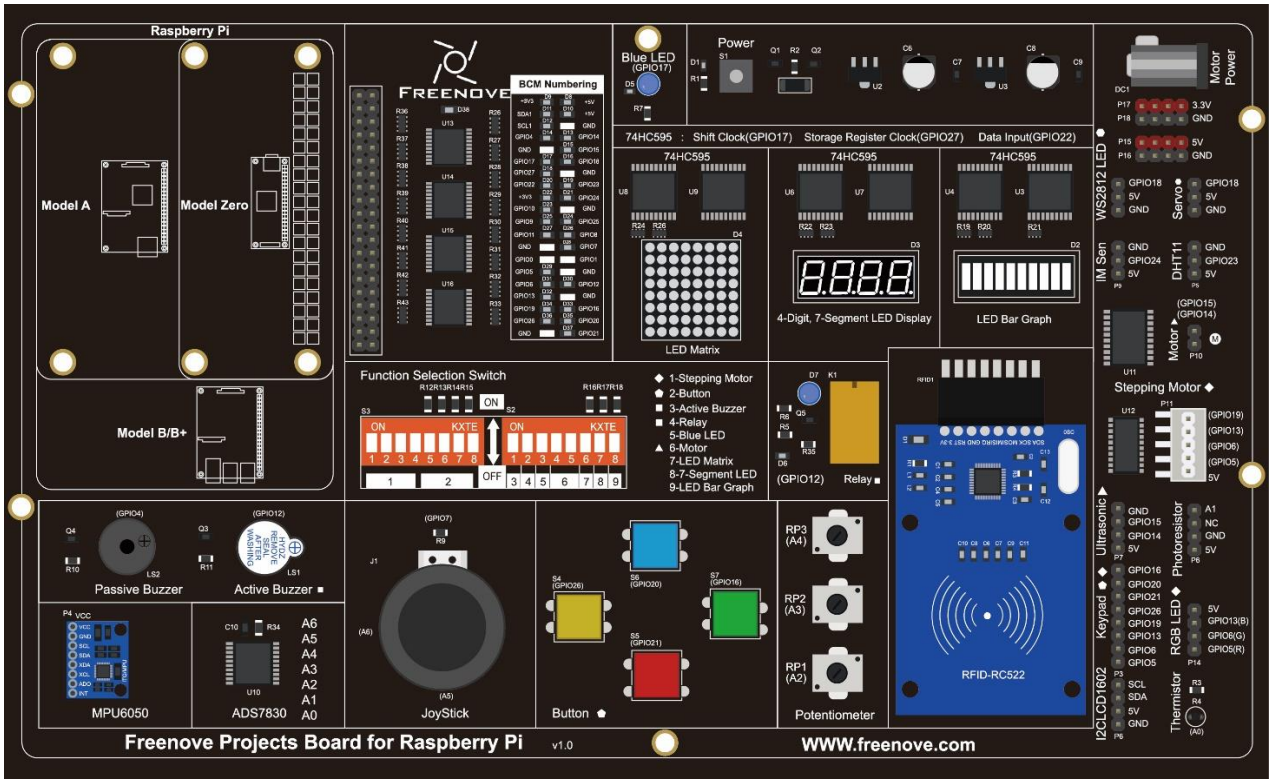
This is accomplished by incorporating two rotary potentiometers inside the Joystick Module at 90 degrees of each other, placed in such a manner as to detect shifts in two directions simultaneously and with a Push Button Switch in the “vertical” axis, which can detect when a User presses on the Joystick.



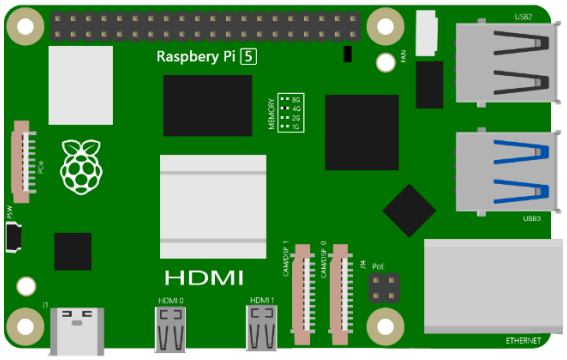
When the Joystick data is read, there are some differences between the axes: data of X and Y axes is analog, which needs to use the ADC. The data of the Z axis is digital, so you can directly use the GPIO to read this data or you have the option to use the ADC to read this.

Component List

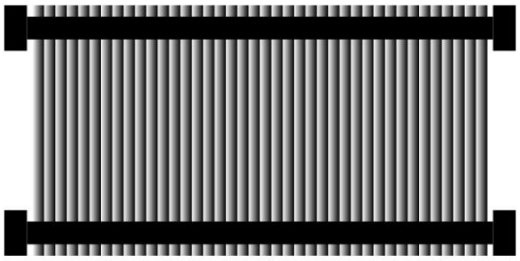
Freenove Projects Board for Raspberry Pi



Raspberry Pi

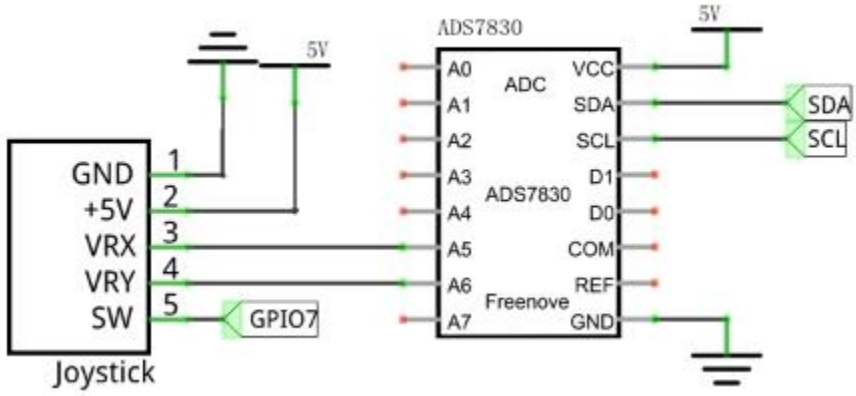


GPIO Ribbon Cable

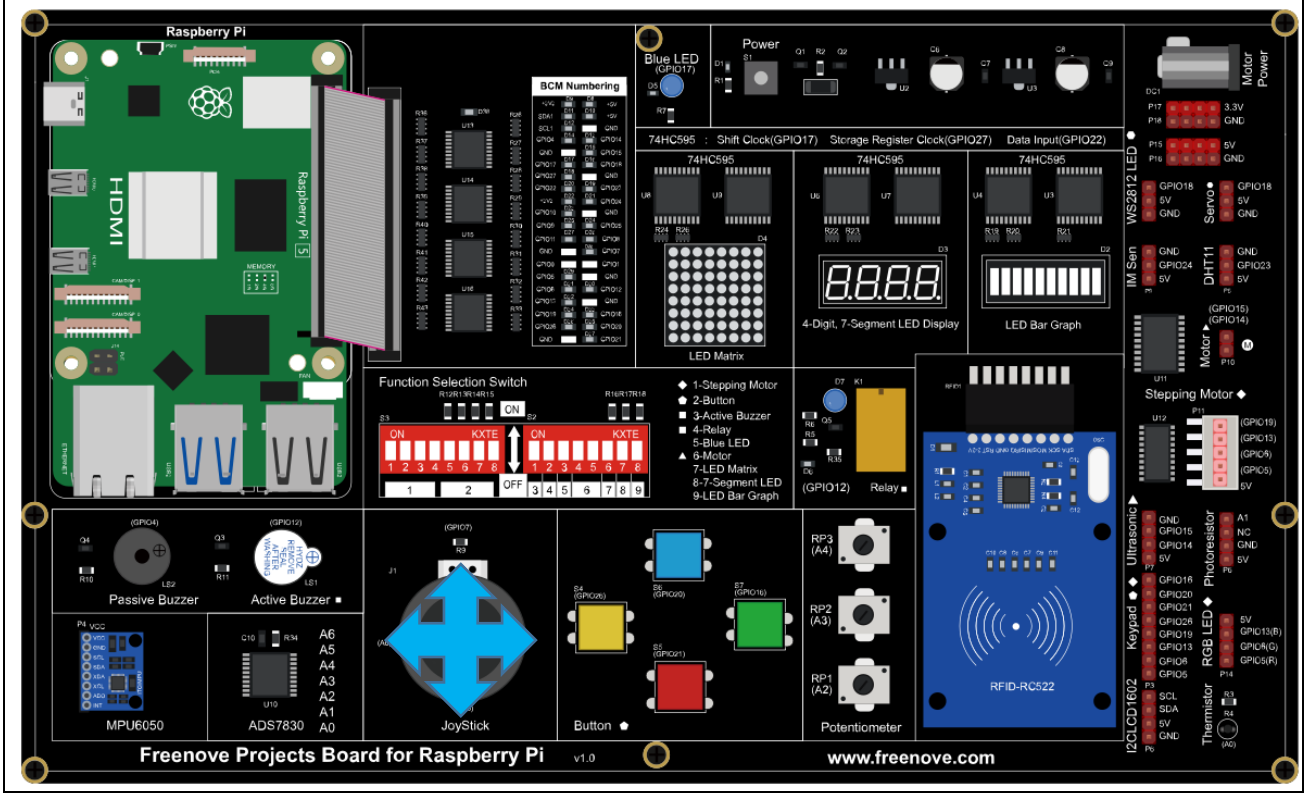


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project's code, we will read the ADC values of X and Y axes of the Joystick, and read digital quality of the Z axis, then display these out in Terminal.

C Code 10.1 Joystick

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 10_Joystick directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/10_Joystick
```

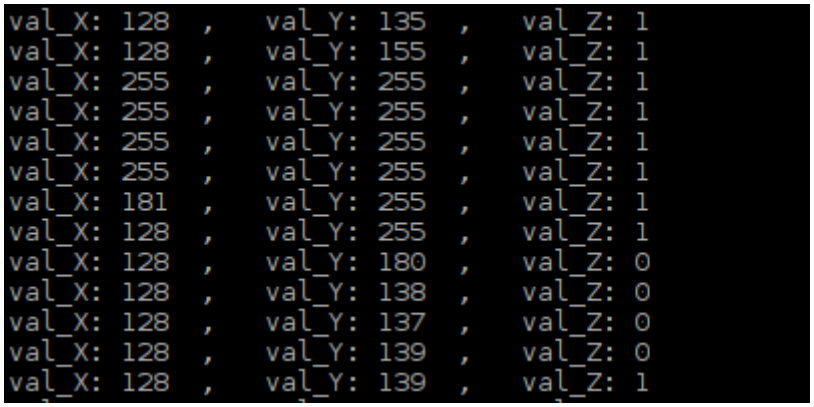
2. Use following command to compile "Joystick.cpp" and generate executable file "Joystick".

```
g++ Joystick.cpp -o Joystick -lwiringPi -lADCDevice
```

3. Then run the generated file "Joystick".

```
./Joystick
```

After the program is executed, the terminal window will display the data of 3 axes X, Y and Z. Shifting (moving) the Joystick or pressing it down will make the data change.



The flowing is the code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <softPwm.h>
#include <ADCDevice.hpp>

#define Z_Pin 7     //define pin for axis Z

ADCDevice *adc;     // Define an ADC Device class object

int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");
```

```
13
14      if(adc->detectI2C(0x48)){    // Detect the ads7830
15          delete adc;               // Free previously pointed memory
16          adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
17      }
18      else{
19          printf("No correct I2C address found, \n"
20          "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
21          "Program Exit. \n");
22          return -1;
23      }
24      wiringPiSetupGpio();
25      pinMode(Z_Pin,INPUT);       //set Z_Pin as input pin and pull-up mode
26      pullUpDnControl(Z_Pin,PUD_UP);
27      while(1){
28          int val_Z = digitalRead(Z_Pin);  //read digital value of axis Z
29          int val_Y = adc->analogRead(5);      //read analog value of axis X and Y
30          int val_X = adc->analogRead(6);
31          printf("val_X: %d  ,\tval_Y: %d  ,\tval_Z: %d \n",val_X,val_Y,val_Z);
32          delay(100);
33      }
34      return 0;
35  }
```

In the code, configure Z_Pin to pull-up input mode. In the while loop of the main function, use **analogRead** () to read the value of axes X and Y and use **digitalRead** () to read the value of axis Z, then display them.

```
    while(1){
        int val_Z = digitalRead(Z_Pin);  //read digital value of axis Z
        int val_Y = adc->analogRead(5);      //read analog value of axis X and Y
        int val_X = adc->analogRead(6);
        printf("val_X: %d  ,\tval_Y: %d  ,\tval_Z: %d \n",val_X,val_Y,val_Z);
        delay(100);
    }
```

Python Code 10.1 Joystick

If you haven't configured I2C, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

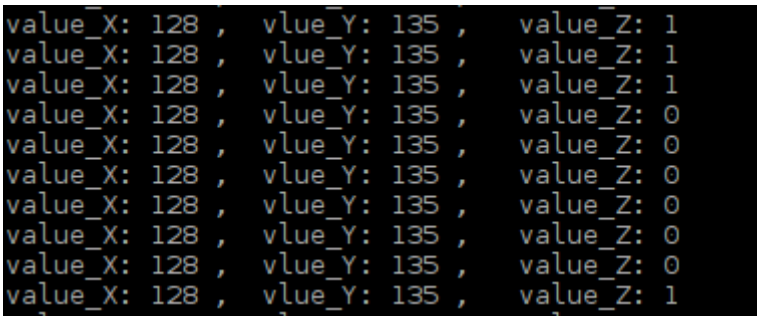
1. Use cd command to enter 10_Joystick directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/10_Joystick
```

2. Use Python command to execute Python code "Joystick.py".

```
python Joystick.py
```

After the program is executed, the Terminal window will display the data of 3 axes X, Y and Z. Shifting (moving) the joystick or pressing it down will make the data change.



The following is the program code:

```
from gpiozero import Button
import time
from ADCDevice import *

Z_Pin = 7               # define Z_Pin
button = Button(Z_Pin) # define Button pin according to BCM Numbering
adc = ADCDevice(0x48)  # Define an ADCDevice class object

def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = ADS7830(0x48)
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        val_Z = not button.value      # read digital value of axis Z
        val_Y = adc.analogRead(6)     # read analog value of axis X and Y
        val_X = adc.analogRead(5)
```

```
24          print ('value_X: %d ,\tvlue_Y: %d ,\tvalue_Z: %d'%(val_X,val_Y,val_Z))
25          time.sleep(0.01)
26
27  def destroy():
28      adc.close()
29      button.close()
30
31  if __name__ == '__main__':
32      print ('Program is starting ... ') # Program entrance
33      try:
34          setup()
35          loop()
36      except KeyboardInterrupt:  # Press ctrl-c to end the program.
37          print("Ending program")
38      finally:
39          destroy()
```

Chapter 11 Motor & Driver

In this chapter, we will learn about DC Motors and DC Motor Drivers and how to control the speed and direction of a DC Motor.

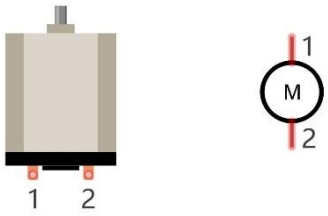
Project 11.1 Control a DC Motor with a Potentiometer

In this project, a potentiometer will be used to control a DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reached the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint, the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction.

Component knowledge

DC Motor

DC Motor is a device that converts electrical energy into mechanical energy. DC Motors consist of two major parts, a Stator and the Rotor. The stationary part of a DC Motor is the Stator and the part that Rotates is the Rotor. The Stator is usually part of the outer case of motor (if it is simply a pair of permanent magnets), and it has terminals to connect to the power if it is made up of electromagnet coils. Most Hobby DC Motors only use Permanent Magnets for the Stator Field. The Rotor is usually the shaft of motor with 3 or more electromagnets connected to a commutator where the brushes (via the terminals 1 & 2 below) supply electrical power, which can drive other mechanical devices. The diagram below shows a small DC Motor with two terminal pins.

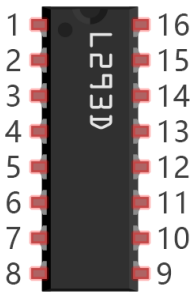


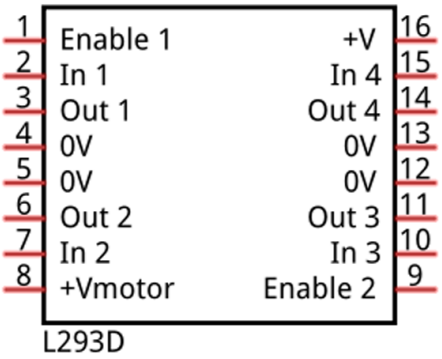
When a DC Motor is connected to a power supply, it will rotate in one direction. If you reverse the polarity of the power supply, the DC Motor will rotate in opposite direction. This is important to note.



L293D

L293D is an IC Chip (Integrated Circuit Chip) with a 4-channel motor drive. You can drive a Unidirectional DC Motor with 4 ports or a Bi-Directional DC Motor with 2 ports or a Stepper Motor (Stepper Motors are covered later in this Tutorial).





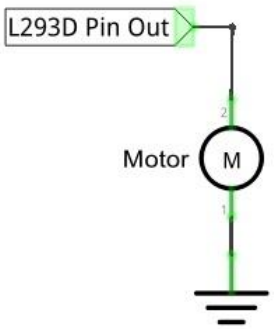
Port description of L293D module is as follows:

Pin name	Pin number	Description
In x	2, 7, 10, 15	Channel x digital signal input pin
Out x	3, 6, 11, 14	Channel x output pin, input high or low level according to In x pin, gets connected to +Vmotor or 0V
Enable1	1	Channel 1 and Channel 2 enable pin, high level enable
Enable2	9	Channel 3 and Channel 4 enable pin, high level enable
0V	4, 5, 12, 13	Power Cathode (GND)
+V	16	Positive Electrode (VCC) of power supply, supply voltage 4.5~36V
+Vmotor	8	Positive Electrode of load power supply, provide power supply for the Out pin x, the supply voltage is +V~36V

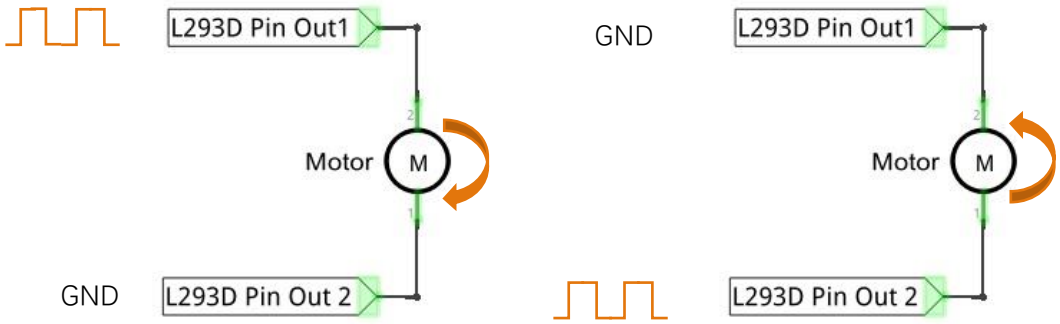
For more details, please see the datasheet for this IC Chip.

When using the L293D to drive a DC Motor, there are usually two connection options.

The following connection option uses one channel of the L239D, which can control motor speed through the PWM, However the motor then can only rotate in one direction.



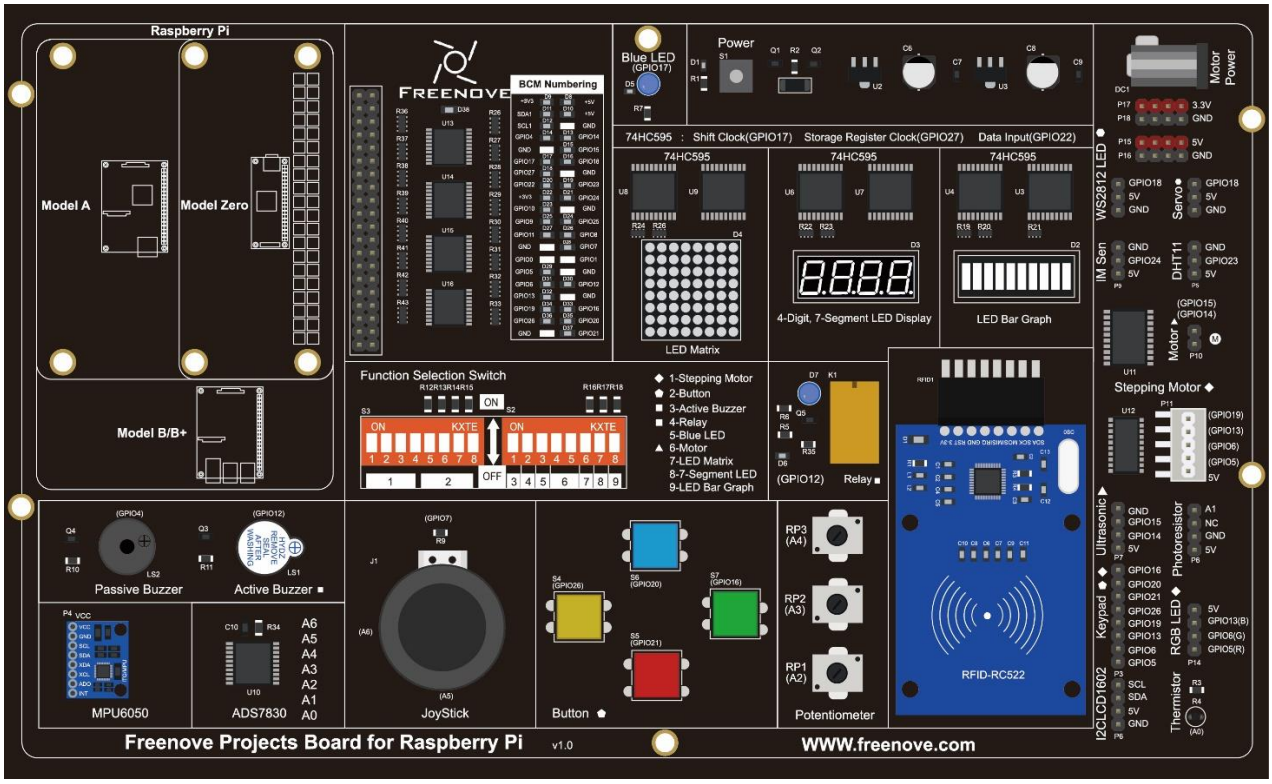
The following connection uses two channels of the L239D: one channel outputs the PWM wave, and the other channel connects to GND. Therefore, you can control the speed of the motor. When these two channel signals are exchanged, it not only controls the speed of motor, but also can control the direction of the motor.



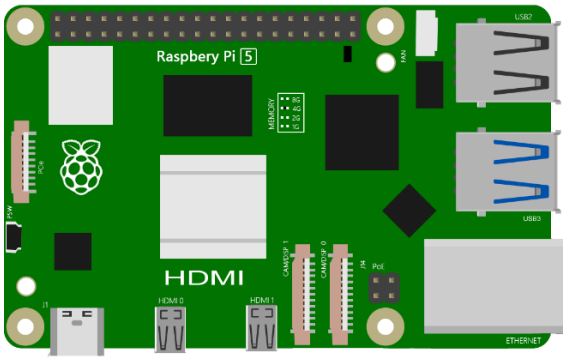
In practical use, the motor is usually connected to channel 1 and by outputting different levels to in1 and in2 to control the rotational direction of the motor, and output to the PWM wave to Enable1 port to control the motor's rotational speed. If the motor is connected to channel 3 and 4 by outputting different levels to in3 and in4 to control the motor's rotation direction, and output to the PWM wave to Enable2 pin to control the motor's rotational speed.

Component List

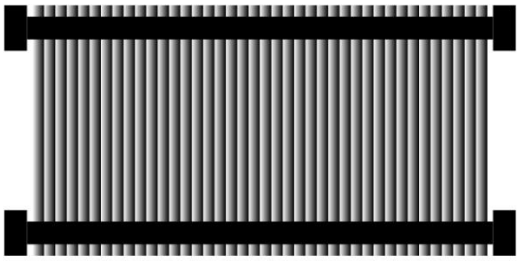
Freenove Projects Board for Raspberry Pi



Raspberry Pi



GPIO Ribbon Cable



Jumper Wire

Motor

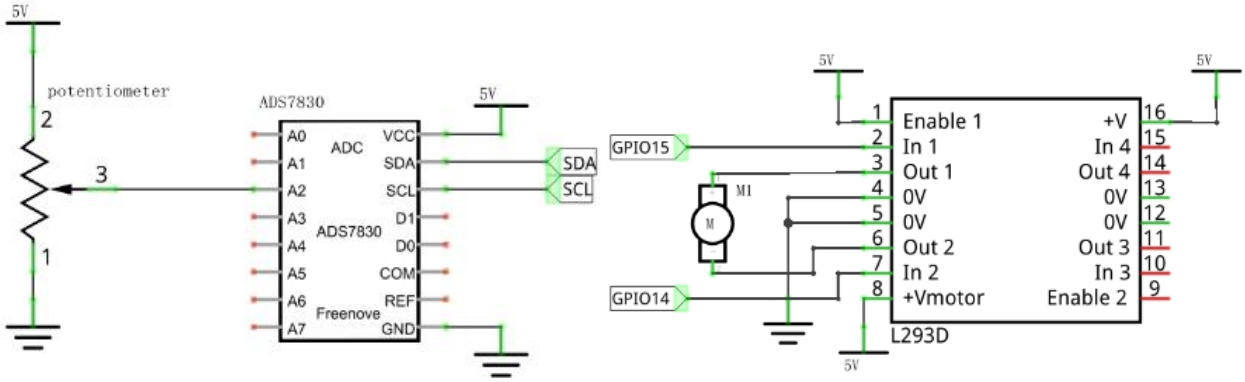


9V Battery (you provide) & 9V Battery Cable

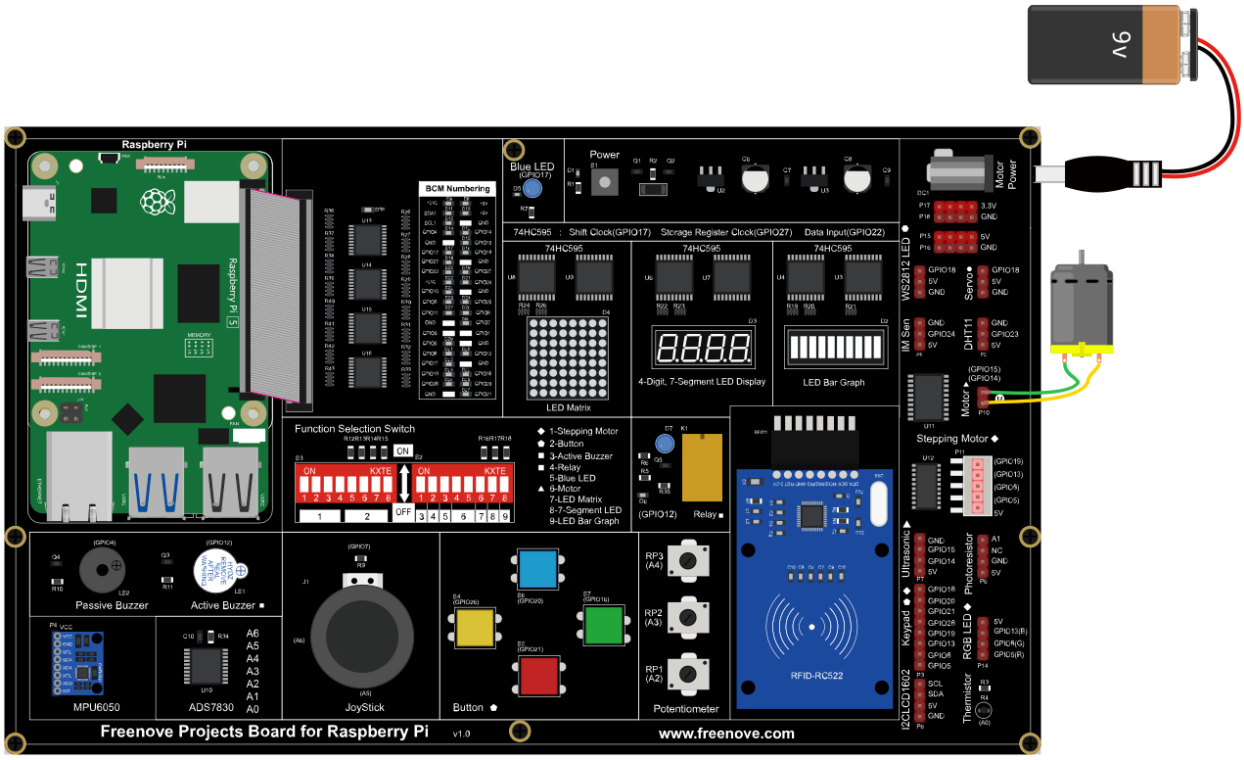


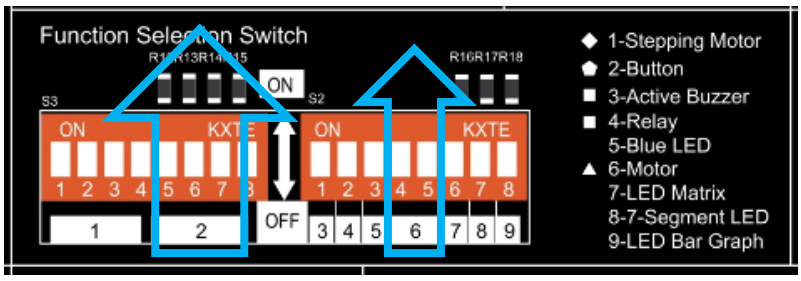
Circuit

Schematic diagram



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

In code for this project, first read the ADC value and then control the rotation direction and speed of the DC Motor according to the value of the ADC.

C Code 11.1 Motor

If you haven't configured I2C, please refer to Chapter 7. If you have done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 11_Motor directory of the C code.

```
cd ~/Freenove_Kit/Code/C_Code/11_Motor
```

2. Use the following command to compile "Motor.cpp" and generate the executable file "Motor".

```
g++ Motor.cpp -o Motor -lwiringPi -lADCDevice
```

3. Then run the generated file "Motor".

```
./Motor
```

After the program runs, you can use the Potentiometer to control the DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reaches the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint, the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction. You will also see the ADC value of the potentiometer displayed in the Terminal with the motor direction and the PWM duty cycle used to control the DC Motor's speed.

```
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
The PWM duty cycle is 66%
ADC value : 212
turn Forward...
```

The following is the code:

```
1  #include <wiringPi.h>
2  #include <stdio.h>
3  #include <softPwm.h>
4  #include <math.h>
5  #include <stdlib.h>
```

```
#include <ADCDevice.hpp>

#define motorPin1    15        //define the pin connected to L293D
#define motorPin2    14

ADCDevice *adc;  // Define an ADC Device class object

//Map function: map the value from a range to another range.
long map(long value,long fromLow,long fromHigh,long toLow,long toHigh){
    return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow;
}
//motor function: determine the direction and speed of the motor according to the ADC
void motor(int ADC){
    int value = ADC -128;
    if(value>0){
        softPwmWrite(motorPin1,map(abs(value),0,128,0,100));
        softPwmWrite(motorPin2,0);
        printf("turn Forward...\n");
    }
    else if (value<0){
        softPwmWrite(motorPin1,0);
        softPwmWrite(motorPin2,map(-value,0,128,0,100));
        printf("turn Back...\n");
    }
    else {
        digitalWrite(motorPin1,LOW);
        digitalWrite(motorPin2,LOW);
        printf("Motor Stop...\n");
    }
    printf("The PWM duty cycle is %d%%\n",abs(value)*100/127);//print the PMW duty cycle
}
int main(void){
    adc = new ADCDevice();
    printf("Program is starting ... \n");

    if(adc->detectI2C(0x48)){    // Detect the ads7830
        delete adc;                 // Free previously pointed memory
        adc = new ADS7830(0x48);       // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
```

```
50      }
51      wiringPiSetupGpio();
52      pinMode(motorPin1,OUTPUT);
53      pinMode(motorPin2,OUTPUT);
54      softPwmCreate(motorPin1,0,100);//define PMW pin
55      softPwmCreate(motorPin2,0,100);//define PMW pin
56
57      while(1){
58          int value = adc->analogRead(2);  //read analog value of A0 pin
59          printf("ADC value : %d \n",value);
60          motor(value);         //make the motor rotate with speed(analog value of A0 pin)
61          delay(100);
62      }
63      return 0;
64  }
```

When ADC value is greater than 128, motorPin2 outputs low lever and motorPin1 output high level.
When ADC value is less than 128, motorPin2 outputs high lever and motorPin1 output low level.
The difference between ADC and 128 determines the duty cycle for the PWM.

```
void motor(int ADC){
    int value = ADC -128;
    if(value>0){
        softPwmWrite(motorPin1,map(abs(value),0,128,0,100));
        softPwmWrite(motorPin2,0);
        printf("turn Forward...\n");
    }
    else if (value<0){
        softPwmWrite(motorPin1,0);
        softPwmWrite(motorPin2,map(-value,0,128,0,100));
        printf("turn Back...\n");
    }
    else {
        digitalWrite(motorPin1,LOW);
        digitalWrite(motorPin2,LOW);
        printf("Motor Stop...\n");
    }
    printf("The PWM duty cycle is %d%%\n",abs(value)*100/127);//print the PMW duty cycle
}
```

Python Code 11.1 Motor

If you haven't configured I2C and installed Smbus, please refer to Chapter 7. If you've done it, please Continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

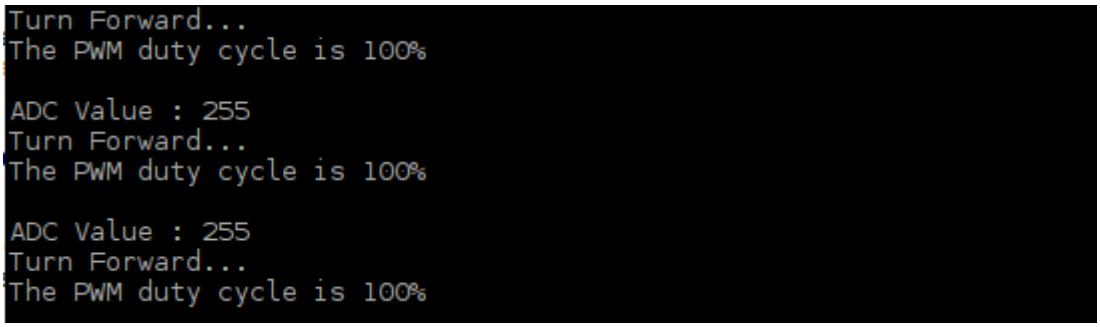
1. Use cd command to enter 11_Motor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/11_Motor
```

2. Use python command to execute the Python code "Motor.py".

```
python Motor.py
```

After the program runs, you can use the Potentiometer to control the DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reaches the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint, the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction. You will also see the ADC value of the potentiometer displayed in the Terminal with the motor direction and the PWM duty cycle used to control the DC Motor's speed.



The following is the code:

```
1  from gpiozero import PWMOutputDevice
2  import time
3  from ADCDevice import *
4
5  # define the pins connected to L293D
6  motoRPin1 = PWMOutputDevice(15,frequency=1000)
7  motoRPin2 = PWMOutputDevice(14,frequency=1000)
8  adc = ADCDevice(0x48) # Define an ADCDevice class object
9
10 def setup():
11     global adc
12     if(adc.detectI2C(0x48)): # Detect the pcf8591.
13         adc = ADS7830(0x48)
14     else:
15         print("No correct I2C address found, \n"
16         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
17         "Program Exit. \n");
18         exit(-1)
```

```
# mapNUM function: map the value from a range of mapping to another range.
def mapNUM(value,fromLow,fromHigh,toLow,toHigh):
    return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow

# motor function: determine the direction and speed of the motor according to the input ADC value input
def motor(ADC):
    value = ADC - 128
    if (value > 0):  # make motor turn forward
        motoRPin1.value=abs(value)/127          # motoRPin1 output HIHG level
        motoRPin2.value=0                       # motoRPin2 output LOW level
        print ('Turn Forward...')
    elif (value < 0): # make motor turn backward
        motoRPin1.value=0                       # motoRPin1 output LOW level
        motoRPin2.value=abs(value)/128          # motoRPin2 output HIHG level
        print ('Turn Backward...')
    else :
        motoRPin1.value=0
        motoRPin2.value=0
        print ('Motor Stop...')

def loop():
    while True:
        value = adc.analogRead(2) # read ADC value of channel 0
        print ('ADC Value : %d'%(value))
        motor(value)
        time.sleep(0.2)

def destroy():
    motoRPin1.close()
    motoRPin2.close()
    adc.close()

if __name__ == '__main__':  # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

When ADC value is greater than 128, motorPin2 outputs low lever and motorPin1 output high level.
When ADC value is less than 128, motorPin2 outputs high lever and motorPin1 output low level.
The difference between ADC and 128 determines the duty cycle for the PWM.

```
def motor(ADC):
    value = ADC - 128
    if (value > 0):  # make motor turn forward
        motoRPin1.value=abs(value)/127          # motoRPin1 output HIHG level
        motoRPin2.value=0                       # motoRPin2 output LOW level
        print ('Turn Forward...')
    elif (value < 0): # make motor turn backward
        motoRPin1.value=0                       # motoRPin1 output LOW level
        motoRPin2.value=abs(value)/128          # motoRPin2 output HIHG level
        print ('Turn Backward...')
    else :
        motoRPin1.value=0
        motoRPin2.value=0
        print ('Motor Stop...')
```

Chapter 12 Relay & LED

In this chapter, we will learn a kind of special switch module, Relay Module.

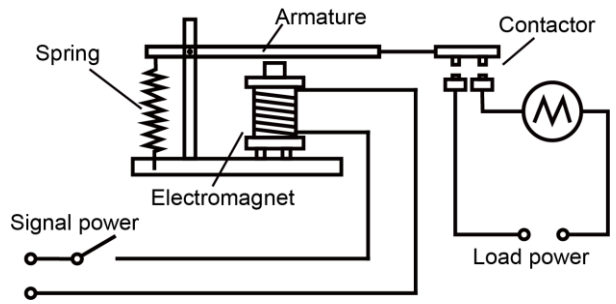
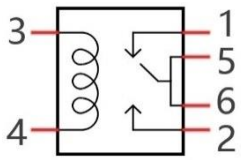
Project 12.1 Relay & LED

Component knowledge

Relay

Relays are a type of Switch that open and close circuits electromechanically or electronically. Relays control one electrical circuit by opening and closing contacts in another circuit using an electromagnet to initiate the Switch action. When the electromagnet is energized (powered), it will attract internal contacts completing a circuit, which act as a Switch. Many times Relays are used to allow a low powered circuit (and a small low amperage switch) to safely turn ON a larger more powerful circuit. They are commonly found in automobiles, especially from the ignition to the starter motor.

The following is a basic diagram of a common Relay and the image and circuit symbol diagram of the 5V relay used in this project:

Diagram	Feature:	Symbol
		

Pin 5 and pin 6 are internally connected to each other. When the coil pin3 and pin 4 are connected to a 5V power supply, pin 1 will be disconnected from pins 5 & 6 and pin 2 will be connected to pins 5 & 6. Pin 1 is called Closed End and pin 2 is called the Open End.

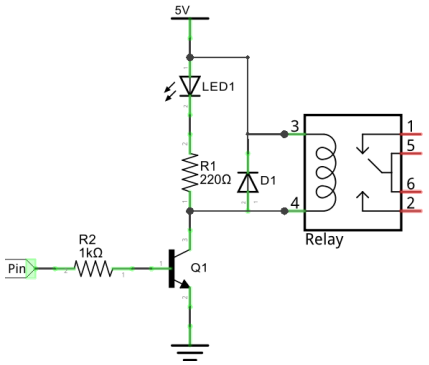
Inductor

The symbol of Inductance is “L” and the unit of inductance is the “Henry” (H). Here is an example of how this can be encountered: 1H=1000mH, 1mH=1000μH.

An Inductor is a passive device that stores energy in its Magnetic Field and returns energy to the circuit whenever required. An Inductor is formed by a Cylindrical Core with many Turns of conducting wire (usually copper wire). Inductors will hinder the changing current passing through it. When the current passing through the Inductor increases, it will attempt to hinder the increasing movement of current; and when the current passing through the inductor decreases, it will attempt to hinder the decreasing movement of current. So the current passing through an Inductor is not transient.

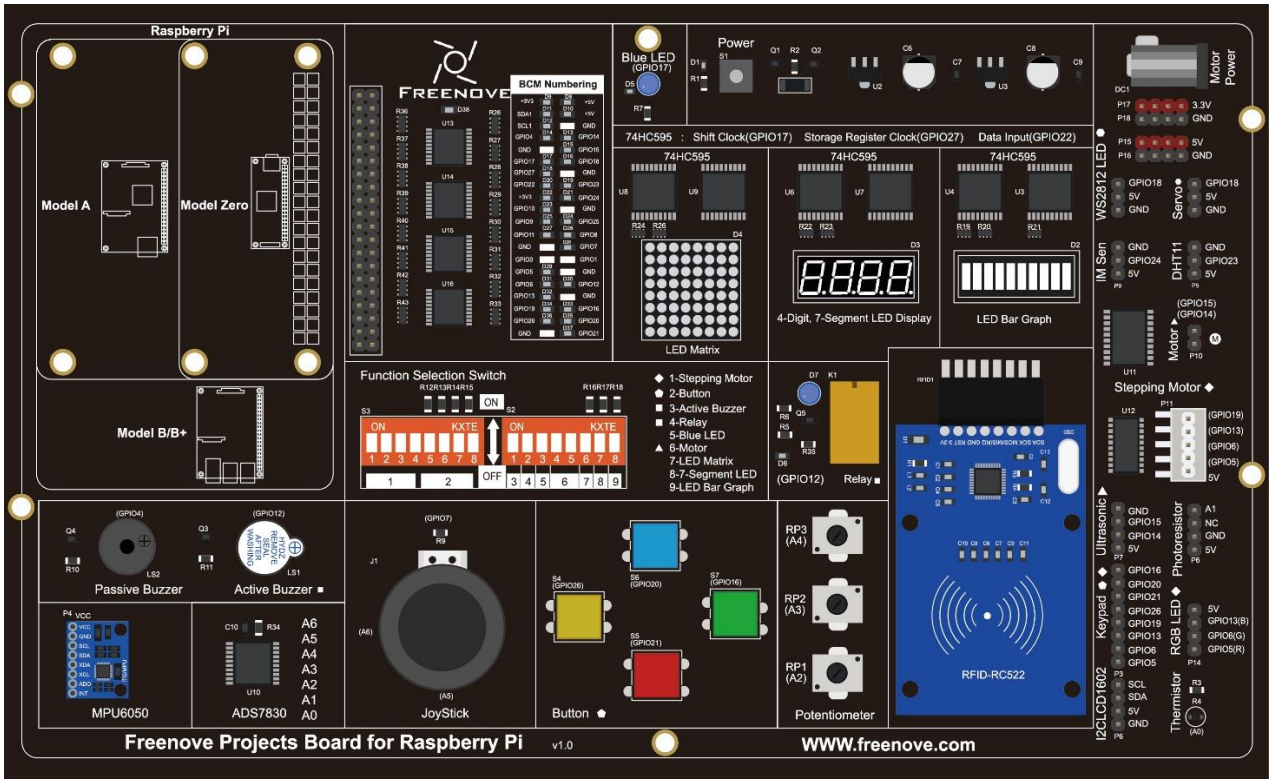


The circuit for a Relay is as follows: The coil of Relay can be equivalent to an Inductor, when a Transistor is present in this coil circuit it can disconnect the power to the relay, the current in the Relay’s coil does not stop immediately, which affects the power supply adversely. To remedy this, diodes in parallel are placed on both ends of the Relay coil pins in opposite polar direction. Having the current pass through the diodes will avoid any adverse effect on the power supply.

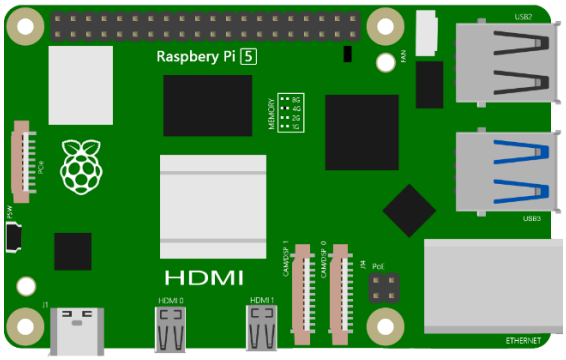


Component List

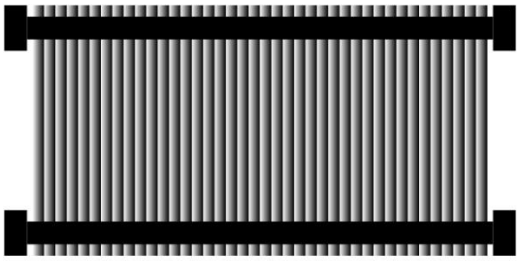
Freenove Projects Board for Raspberry Pi



Raspberry Pi

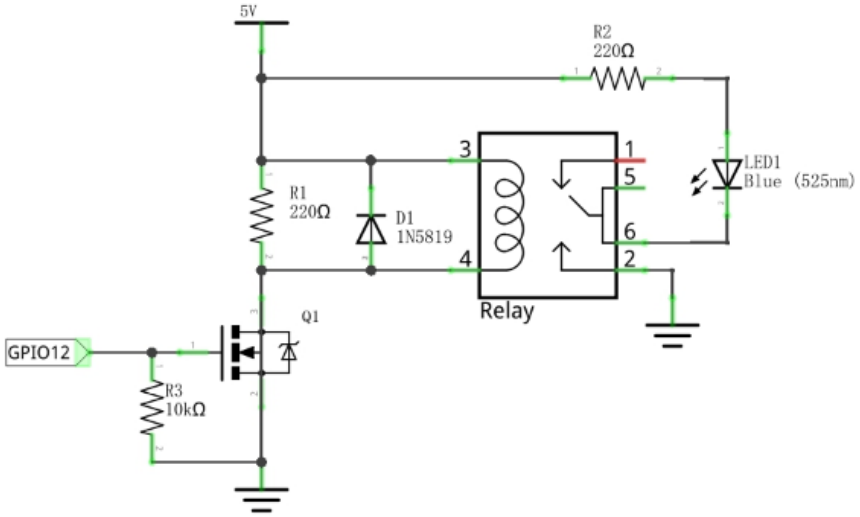


GPIO Ribbon Cable

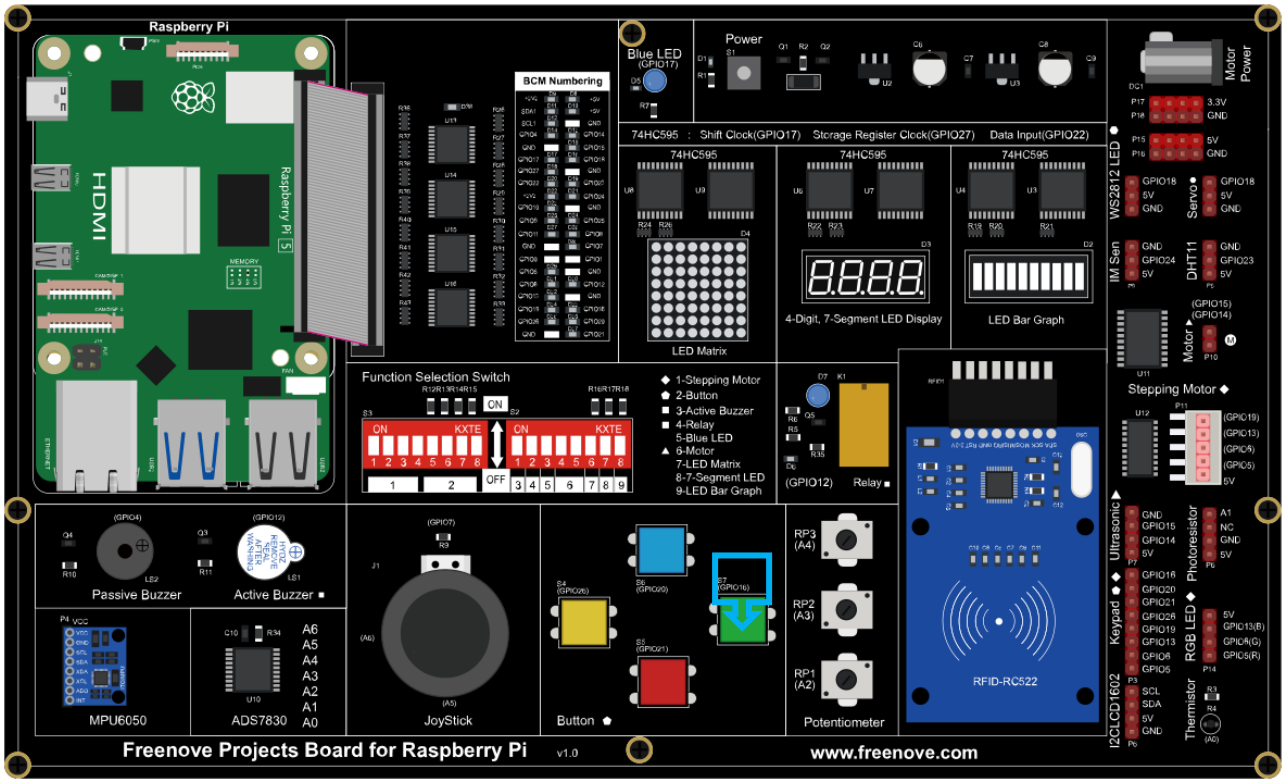


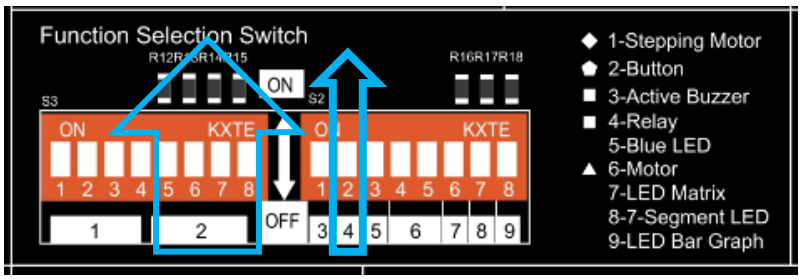
Circuit

Schematic diagram



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

C Code 12.1 Relay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 12_Relay directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/12_Relay
```

2. Use following command to compile "Relay.c" and generate executable file "Relay".

```
gcc Relay.c -o Relay -lwiringPi
```

3. Run the generated file "Relay".

```
./Relay
```

After running the program, press the button, the LED near relay will light up.

Press the button again, the LED will light OFF.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>

#define relayPin    12     //define the relayPin
#define buttonPin   16     //define the buttonPin
int relayState=LOW;        //store the State of relay
int buttonState=HIGH;      //store the State of button
int lastbuttonState=HIGH;  //store the lastState of button
long lastChangeTime;       //store the change time of button state
long captureTime=50;       //set the button state stable time
int reading;
int main(void)
{
    printf("Program is starting...\n");

    wiringPiSetupGpio();

    pinMode(relayPin, OUTPUT);
    pinMode(buttonPin, INPUT);
    pullUpDnControl(buttonPin, PUD_UP);  //pull up to high level
    while(1){
        reading = digitalRead(buttonPin); //read the current state of button
        if( reading != lastbuttonState){  //if the button state changed ,record the time
point
            lastChangeTime = millis();
        }
```

```
26          //if changing-state of the button last beyond the time we set,we considered that
27          //the current button state is an effective change rather than a buffeting
28          if(millis() - lastChangeTime > captureTime){
29              //if button state is changed, update the data.
30              if(reading != buttonState){
31                  buttonState = reading;
32                  //if the state is low, the action is pressing.
33                  if(buttonState == LOW){
34                      printf("Button is pressed!\n");
35                      relayState = !relayState;
36                      if(relayState){
37                          printf("turn on relay ...\n");
38                      }
39                      else {
40                          printf("turn off relay ...\n");
41                      }
42                  }
43                  //if the state is high, the action is releasing.
44                  else {
45                      printf("Button is released!\n");
46                  }
47              }
48          }
49          digitalWrite(relayPin,relayState);
50          lastbuttonState = reading;
51      }
52
53      return 0;
54  }
```

Python Code 12.1 Relay

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 12_Relay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/12_Relay
```

2. Use python command to execute code "Relay.py".

```
python Relay.py
```

After running the program, press the button, the LED near relay will light up.

Press the button again, the LED will light OFF.

he following is the program code:

```
from gpiozero import DigitalOutputDevice, Button
import time

relayPin =  12    # define the relayPin
buttonPin = 16    # define the buttonPin
relay = DigitalOutputDevice(relayPin)     # define LED pin according to BCM Numbering
button = Button(buttonPin) # define Button pin according to BCM Numbering
debounceTime = 50

def loop():
    relayState = 0
    lastChangeTime = round(time.time()*1000)
    buttonState = 1
    lastButtonState = 1
    reading = 1
    while True:
        reading = not button.value
        if reading != lastButtonState :
            lastChangeTime = round(time.time()*1000)
        if ((round(time.time()*1000) - lastChangeTime) > debounceTime):
            if reading != buttonState :
                buttonState = reading;
                if buttonState == 0:
                    print("Button is pressed!")
                    relayState = not relayState
                    if relayState:
                        print("Turn on relay ...")
                    else :
                        print("Turn off relay ... ")
                else :
                    print("Button is released!")
        relay.on() if (relayState==1) else relay.off()
```

```
33          lastButtonState = reading # lastButtonState store latest state
34
35  def destroy():
36      relay.close()
37      button.close()
38
39  if __name__ == '__main__':     # Program entrance
40      print ('Program is starting...')
41      try:
42          loop()
43      except KeyboardInterrupt:  # Press ctrl-c to end the program.
44          print("Ending program")
45      finally:
46          destroy()
```

Chapter 13 Servo

Previously, we learned how to control the speed and rotational direction of a DC Motor. In this chapter, we will learn about Servos which are a rotary actuator type motor that can be controlled rotate to specific angles.

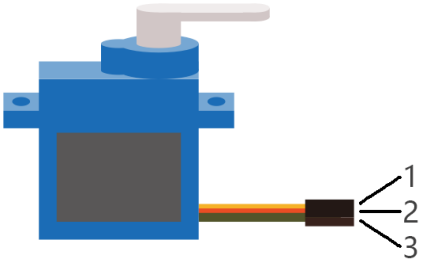
Project 13.1 Sweep

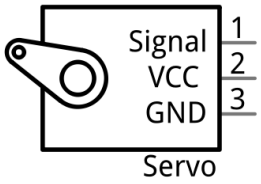
First, we need to learn how to make a Servo rotate.

Component knowledge

Servo

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their “horn”. Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.





We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The lasting time 0.5ms-2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degree linearly. Part of the corresponding values are as follows:

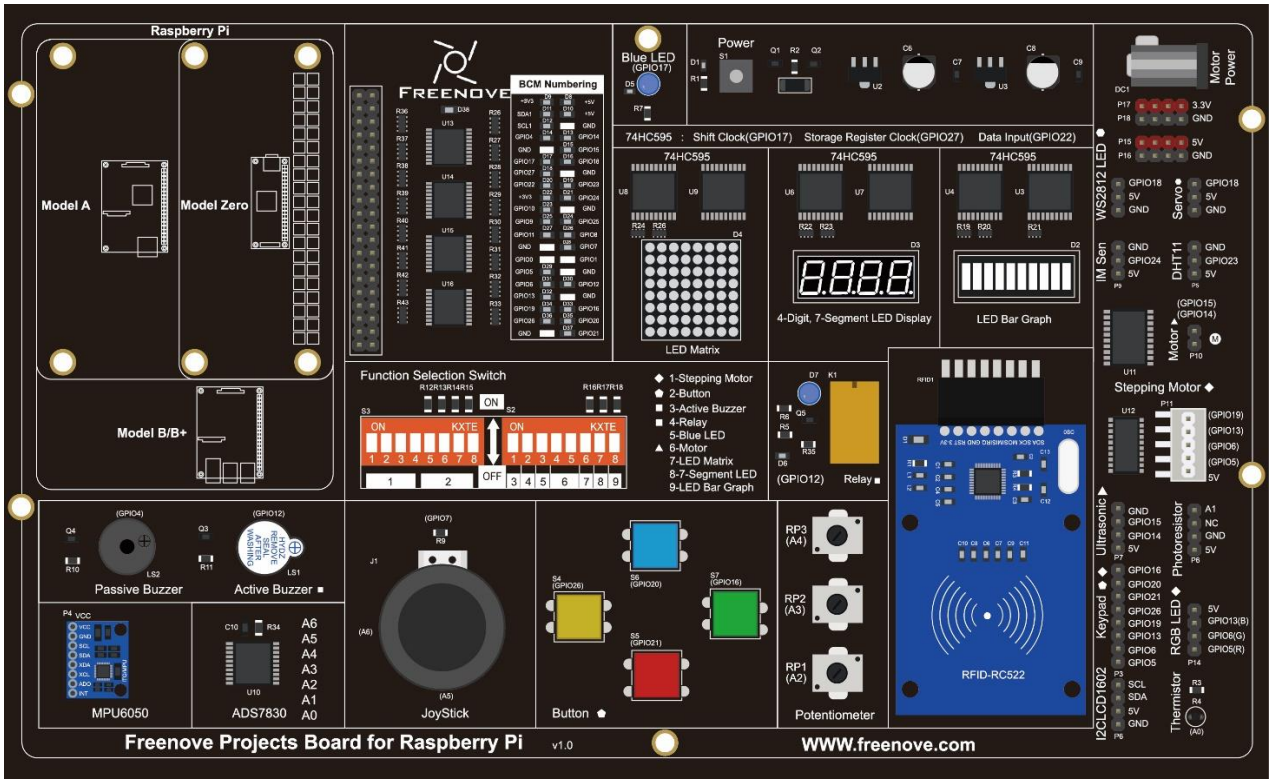
Note: the lasting time of high level corresponding to the servo angle is absolute instead of accumulating. For example, the high level time lasting for 0.5ms correspond to the 0 degree of the servo. If the high level time lasts for another 1ms, the servo rotates to 45 degrees.

High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	90 degree
2ms	135 degree
2.5ms	180 degree

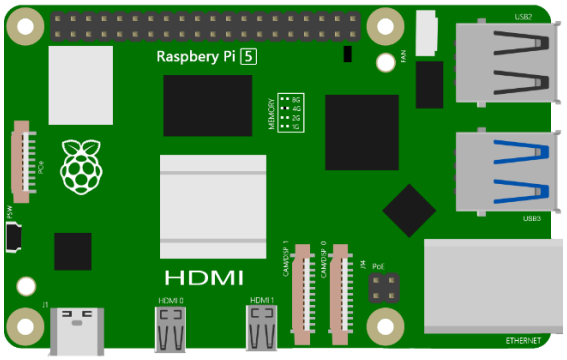
When you change the Servo signal value, the Servo will rotate to the designated angle.

Component List

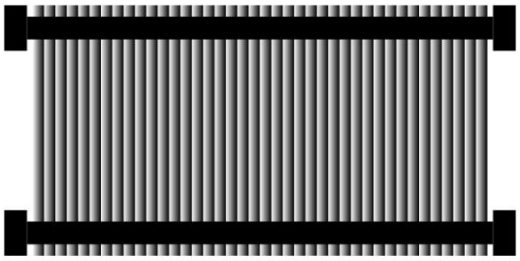
Freenove Projects Board for Raspberry Pi



Raspberry Pi

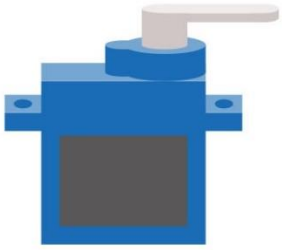


GPIO Ribbon Cable



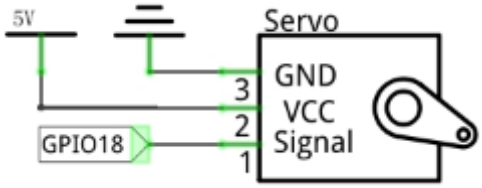
Jumper Wire

Servo

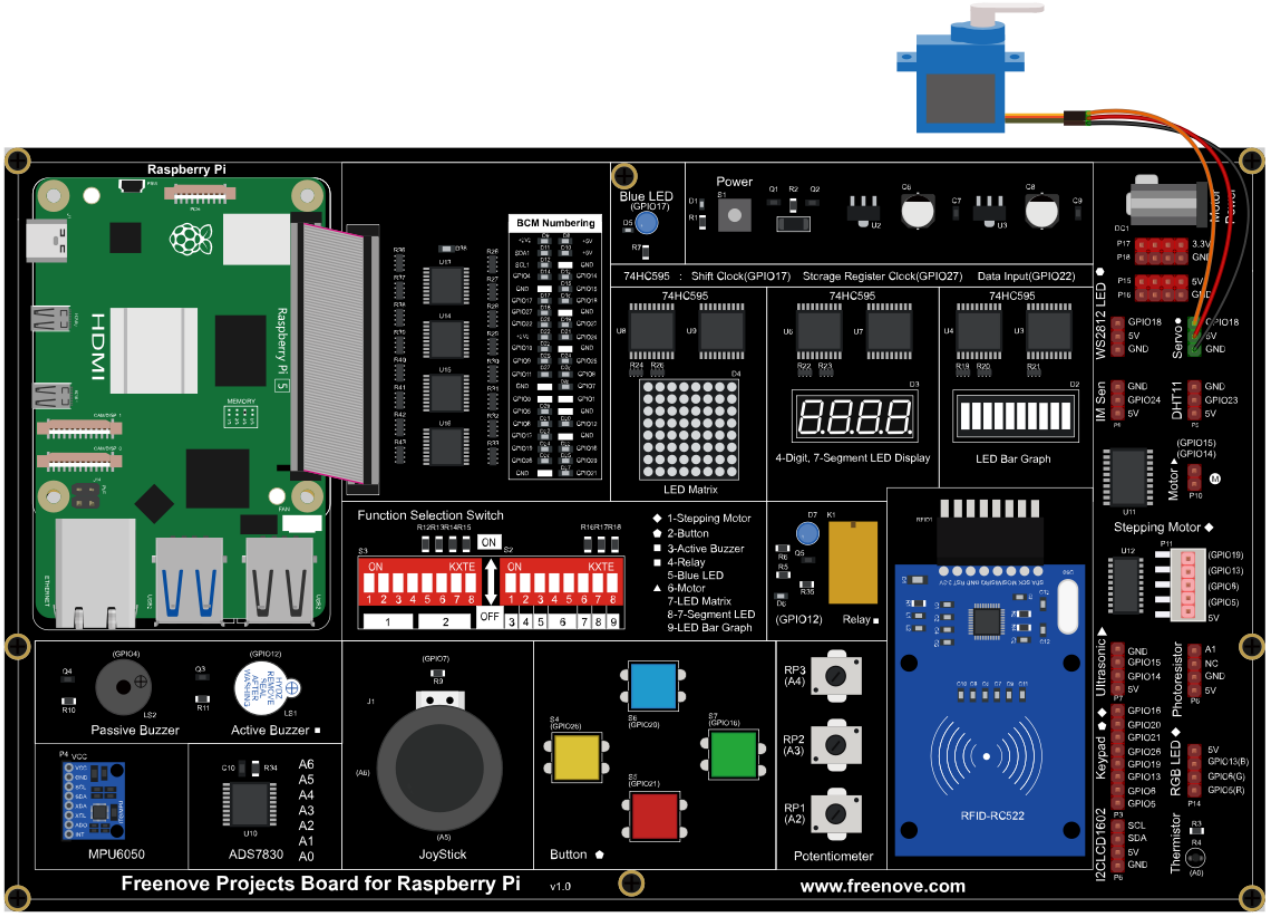


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project, we will make a Servo rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

C Code 13.1 Sweep

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 13_1_Sweep directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/13_1_Sweep
```

2. Use following command to compile "Sweep.c" and generate executable file "Sweep".

```
gcc Sweep.c -o Sweep -lwiringPi
```

3. Run the generated file "Sweep".

```
./Sweep
```

After the program runs, the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

The following is the program code:

```
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#define OFFSET_MS 3                 //Define the unit of servo pulse offset: 0.1ms
#define SERVO_MIN_MS 5+OFFSET_MS    //define the pulse duration for minimum angle of servo
#define SERVO_MAX_MS 25+OFFSET_MS   //define the pulse duration for maximum angle of servo

#define servoPin    18              //define the GPIO number connected to servo
long map(long value,long fromLow,long fromHigh,long toLow,long toHigh){
    return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow;
}
void servoInit(int pin){            //initialization function for servo PMW pin
    softPwmCreate(pin,  0, 200);
}
void servoWrite(int pin, int angle){//Specific a certain rotation angle (0-180) for the servo
    if(angle > 180)
        angle = 180;
    if(angle < 0)
        angle = 0;
    softPwmWrite(pin,map(angle,0,180,SERVO_MIN_MS,SERVO_MAX_MS));
}
void servoWriteMS(int pin, int ms){ //specific the unit for pulse(5-25ms) with specific
duration output by servo pin: 0.1ms
    if(ms > SERVO_MAX_MS)
        ms = SERVO_MAX_MS;
```

```
25      if(ms < SERVO_MIN_MS)
26          ms = SERVO_MIN_MS;
27      softPwmWrite(pin,ms);
28  }
29
30  int main(void){
31      int i;
32      printf("Program is starting ...\n");
33      servoInit(servoPin);
34      servoInit(servoPin);        //initialize PMW pin of servo
35      while(1){
36          //make servo rotate from minimum angle to maximum angle
37          for(i=SERVO_MIN_MS;i<SERVO_MAX_MS;i++){
38              servoWriteMS(servoPin,i);
39              delay(10);
40          }
41          delay(500);
42          //make servo rotate from maximum angle to minimum angle
43          for(i=SERVO_MAX_MS;i>SERVO_MIN_MS;i--){
44              servoWriteMS(servoPin,i);
45              delay(10);
46          }
47          delay(500);
48      }
49      return 0;
50  }
```

A 50 Hz pulse for a 20ms cycle is required to control the Servo. In function **softPwmCreate** (int pin, int initialValue, int pwmRange), the unit of the third parameter pwmRange is 100US, specifically 0.1ms. In order to get the PWM with a 20ms cycle, the pwmRange shoulde be set to 200. So in the subfunction of servoInit (), we create a PWM pin with a pwmRange of 200.

```
    void servoInit(int pin){        //initialization function for servo PWM pin
        softPwmCreate(pin,  0,  200);
    }
```

Since 0-180 degrees of the Servo's motion corresponds to the PWM pulse width of 0.5-2.5ms, with a PwmRange of 200 ms, we then need the function **softPwmWrite** (int pin, int value) and the scope 5-25 of the parameter values to correspond to 0-180 degrees' motion of the Servo. What's more, the number written in subfunction **servoWriteMS** () should be within the range of 5-25. However, in practice, due to the inherent error manufactured into each Servo, the pulse width will have a deviation. So we need to define a minimum and maximum pulse width and an error offset (this is essential in robotics).

```
#define OFFSET_MS 3                 //Define the unit of servo pulse offset: 0.1ms
#define SERVO_MIN_MS 5+OFFSET_MS  //define the pulse duration for minimum angle of servo
#define SERVO_MAX_MS 25+OFFSET_MS //define the pulse duration for maximum angle of servo
......
void servoWriteMS(int pin, int ms){
    if(ms > SERVO_MAX_MS)
        ms = SERVO_MAX_MS;
    if(ms < SERVO_MIN_MS)
        ms = SERVO_MIN_MS;
    softPwmWrite(pin,ms);
}
```

In subfunction **servoWrite** (), directly input an angle value (0-180 degrees), map the angle to the pulse width and then output it.

```
void servoWrite(int pin, int angle){//Specif a certain rotation angle (0-180) for the
servo
    if(angle > 180)
        angle = 180;
    if(angle < 0)
        angle = 0;
    softPwmWrite(pin,map(angle,0,180,SERVO_MIN_MS,SERVO_MAX_MS));
}
```

Finally, in the "while" loop of the main function, use two "for" loop to make servo rotate from 0 degrees to 180 degrees, and then from 180 degrees to 0 degrees.

```
    while(1){
        //make servo rotate from minimum angle to maximum angle
        for(i=SERVO_MIN_MS;i<SERVO_MAX_MS;i++){
            servoWriteMS(servoPin,i);
            delay(10);
        }
        delay(500);
        //make servo rotate from maximum angle to minimum angle
        for(i=SERVO_MAX_MS;i>SERVO_MIN_MS;i--){
            servoWriteMS(servoPin,i);
            delay(10);
        }
        delay(500);
    }
```

Python Code 13.1 Sweep

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 13_1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/13_1_Sweep
```

2. Use python command to execute code "Sweep.py".

```
python Sweep.py
```

After the program runs, the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

The following is the program code:

```
1  from gpiozero import AngularServo
2  import time
3
4  myGPIO=18
5  myCorrection=0.0
6  maxPW=(2.5+myCorrection)/1000
7  minPW=(0.5-myCorrection)/1000
8  servo =  AngularServo(myGPIO, initial_angle=0, min_angle=0, max_angle=180,
   min_pulse_width=minPW, max_pulse_width=maxPW)
9
10 def loop():
11     while True:
12         for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
13             servo.angle = angle
14             time.sleep(0.001)
15         time.sleep(0.5)
16         for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
17             servo.angle = angle
18             time.sleep(0.001)
19         time.sleep(0.5)
20
21 def destroy():
22     servo.close()
23
24 if __name__ == '__main__':     # Program entrance
25     print ('Program is starting...')
26     try:
27         loop()
28     except KeyboardInterrupt:  # Press ctrl-c to end the program.
29         print("Ending program")
30     finally:
31         destroy()
```

We use angles to control the servo, so we need to configure the parameters of the steering gear first. The time unit of the pulse is seconds.

```
servo =  AngularServo(myGPIO, initial_angle=0, min_angle=0, max_angle=180,
min_pulse_width=minPW, max_pulse_width=maxPW)
```

As 0-180 degrees of the Servo's rotation corresponds to the PWM pulse width 0.5-2.5ms within cycle 20ms. However, in practice, due to the inherent error manufactured into each Servo, the pulse width will have a deviation. So we need to define a minimum and maximum pulse width and an error offset (this is essential in robotics).

```
myCorrection=0.0
maxPW=(2.5+myCorrection)/1000
minPW=(0.5-myCorrection)/1000
```

Please note that the servo angle value should range from 0 to 180 degrees.

```
for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
    servo.angle = angle
    time.sleep(0.001)
```

Finally, in the "while" loop of main function, we need to use two separate cycles to make servo rotate from 0 degrees to 180 degrees and then from 180 degrees to 0 degrees.

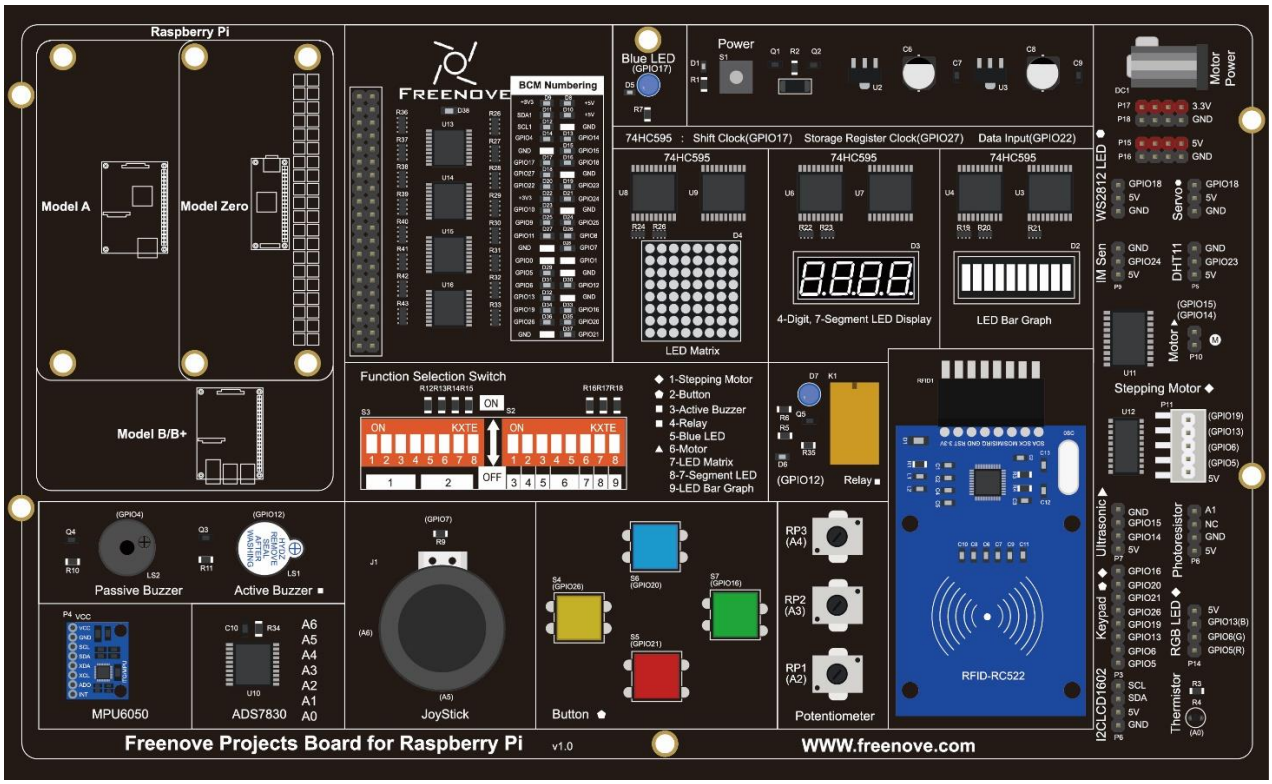
```
def loop():
    while True:
        for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
            servo.angle = angle
            time.sleep(0.001)
        time.sleep(0.5)
        for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
            servo.angle = angle
            time.sleep(0.001)
        time.sleep(0.5)
```

Project 13.2 Knob

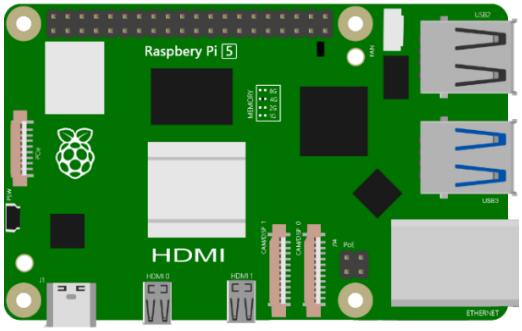
In this project, we will learn how to control the servo with a potentiometer.

Component List

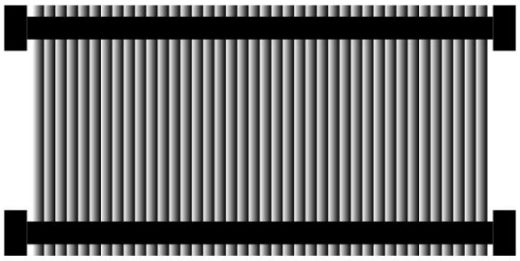
Freenove Projects Board for Raspberry Pi



Raspberry Pi

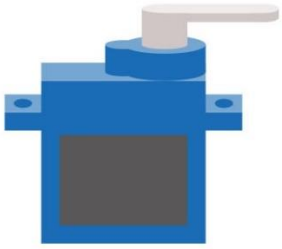


GPIO Ribbon Cable



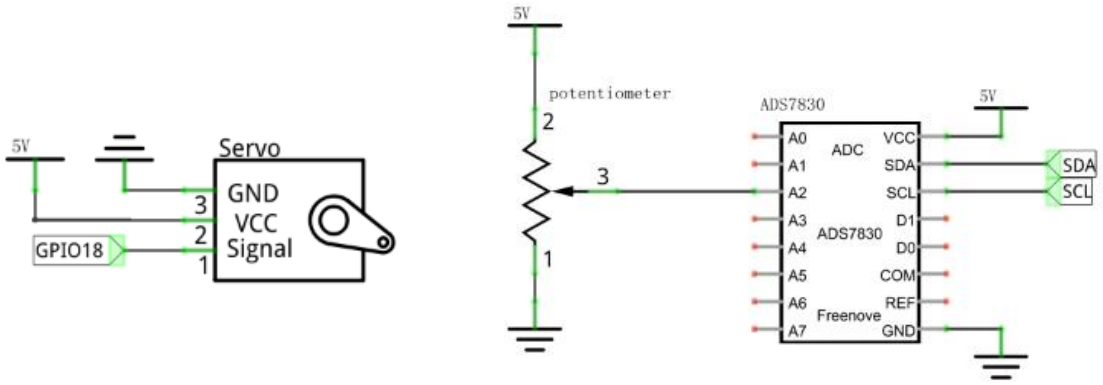
Jumper Wire

Servo

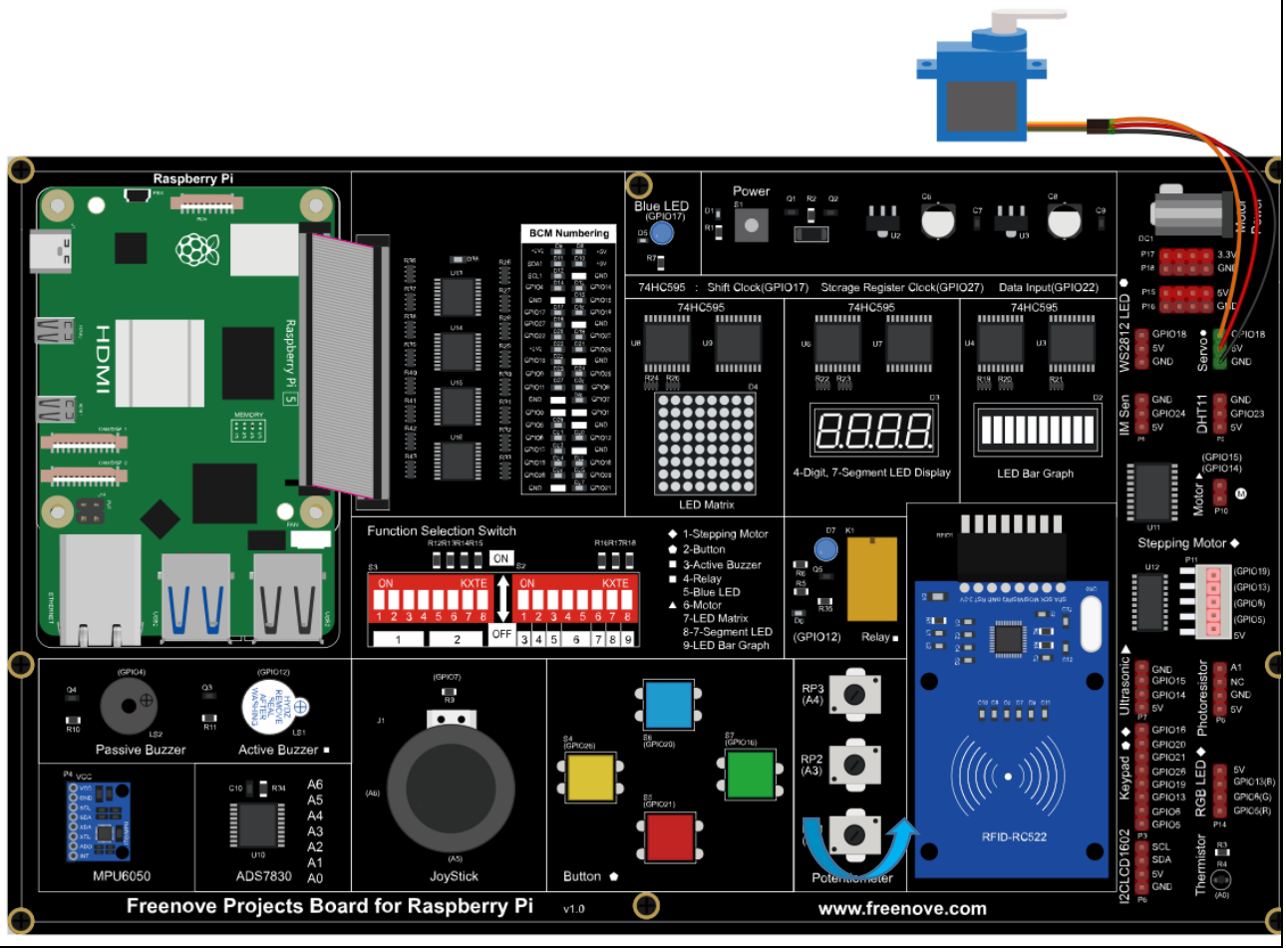


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project, we will make a Servo rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

C Code 13.2 Knob

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

4. Use cd command to enter 13_2_Knob directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/13_2_Knob
```

5. Use following command to compile " Knob.cpp" and generate executable file " Knob".

```
g++ Knob.cpp -o Knob -lwiringPi -lADCDevice
```

6. Run the generated file " Knob ".

```
./Knob
```

After running the program, you can change the angle of the servo by rotating the potentiometer.

The following is the program code:

```
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#include <ADCDevice.hpp>

#define OFFSET_MS 3                 //Define the unit of servo pulse offset: 0.1ms
#define SERVO_MIN_MS 5+OFFSET_MS    //define the pulse duration for minimum angle of servo
#define SERVO_MAX_MS 25+OFFSET_MS   //define the pulse duration for maximum angle of servo
#define servoPin  18                //define the GPIO number connected to servo
ADCDevice *adc;                     // Define an ADC Device class object

long map(long value,long fromLow,long fromHigh,long toLow,long toHigh){
    return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow;
}

void servoInit(int pin){            //initialization function for servo PMW pin
    softPwmCreate(pin,  0, 200);
}

void servoWrite(int pin, int angle){ //Specific a certain rotation angle (0-180) for the servo
    if(angle > 180)
        angle = 180;
    if(angle < 0)
        angle = 0;
    softPwmWrite(pin,map(angle,0,180,SERVO_MIN_MS,SERVO_MAX_MS));
}
```

```
//specific the unit for pulse(5-25ms) with specific duration output by servo pin: 0.1ms
void servoWriteMS(int pin, int ms){
    if(ms > SERVO_MAX_MS)
        ms = SERVO_MAX_MS;
    if(ms < SERVO_MIN_MS)
        ms = SERVO_MIN_MS;
    softPwmWrite(pin,ms);
}

int main(void)
{
    int i;
    printf("Program is starting ...\n");
    wiringPiSetupGpio();
    servoInit(servoPin);              //initialize PMW pin of servo
    adc = new ADCDevice();
    if(adc->detectI2C(0x48)){         // Detect the ads7830
        delete adc;                   // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }
    while(1){
        int adcValue = adc->analogRead(2);    //read analog value of A2 pin
        printf("ADC value : %d  \n",adcValue);
        servoWrite(servoPin,map(adcValue,0,255,0,180));
        delay(10);
    }
    return 0;
}
```

Read the ADC value of channle2, and then the servo will rotate to corresponding angle.

```
    while(1){
        int adcValue = adc->analogRead(2);    //read analog value of A2 pin
        printf("ADC value : %d  \n",adcValue);
        servoWrite(servoPin,map(adcValue,0,255,0,180));
        delay(10);
    }
```

Python Code 13.2 Knob

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

3. Use cd command to enter 13_2_Knob directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/13_2_Knob
```

4. Use python command to execute code " Knob.py".

```
python Knob.py
```

After running the program, you can change the angle of the servo by rotating the potentiometer.

The following is the program code:

```
from gpiozero import AngularServo
import time
from ADCDevice import *

adc = ADCDevice(0x48) # Define an ADCDevice class object
myGPIO=18
myCorrection=0.0
maxPW=(2.5+myCorrection)/1000
minPW=(0.5-myCorrection)/1000
servo = AngularServo(myGPIO, initial_angle=0, min_angle=0, max_angle=180,
 min_pulse_width=minPW, max_pulse_width=maxPW)

# map a value from one range to another range
def map( value, fromLow, fromHigh, toLow, toHigh):
    return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow

def setup():
    global adc
    if(adc.detectI2C(0x48)):
        adc = ADS7830(0x48)
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def servoWrite(angle):      # make the servo rotate to specific angle, 0-180
    if(angle<0):
        angle = 0
    elif(angle > 180):
        angle = 180
    servo.angle = angle # map the angle to duty cycle and output it

def loop():
```

```
34      while True:
35          value = adc.analogRead(2)     # read the ADC value of channel 2
36          servoWrite(round(value/255.0*180.0))
37          print ('ADC Value : %d'%(value))
38          time.sleep(0.1)
39
40  def destroy():
41      adc.close()
42      servo.close()
43
44  if __name__ == '__main__':     # Program entrance
45      print ('Program is starting...')
46      try:
47          setup()
48          loop()
49      except KeyboardInterrupt:  # Press ctrl-c to end the program.
50          print("Ending program")
51      finally:
52          destroy()
```

Read the ADC value of channle2, and then the servo will rotate to corresponding angle.

```
    while True:
        value = adc.analogRead(2)     # read the ADC value of channel 2
        servoWrite(round(value/255.0*180.0))
        print ('ADC Value : %d'%(value))
        time.sleep(0.1)
```

Chapter 14 Stepper Motor

Thus far, we have learned about DC Motors and Servos. A DC motor can rotate constantly in on direction but we cannot control the rotation to a specific angle. On the contrary, a Servo can rotate to a specific angle but cannot rotate constantly in one direction. In this chapter, we will learn about a Stepper Motor which is also a type of motor. A Stepper Motor can rotate constantly and also to a specific angle. Using a Stepper Motor can easily achieve higher accuracies in mechanical motion.

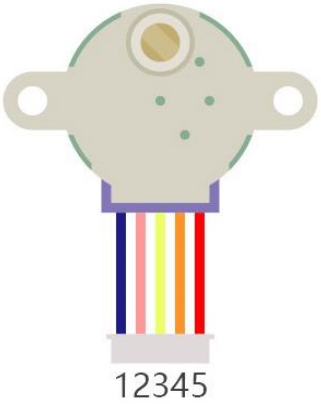
Project 14.1 Stepper Motor

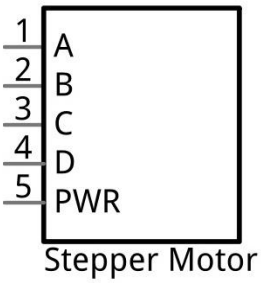
In this project, we will learn how to drive a Stepper Motor, and understand its working principle.

Component knowledge

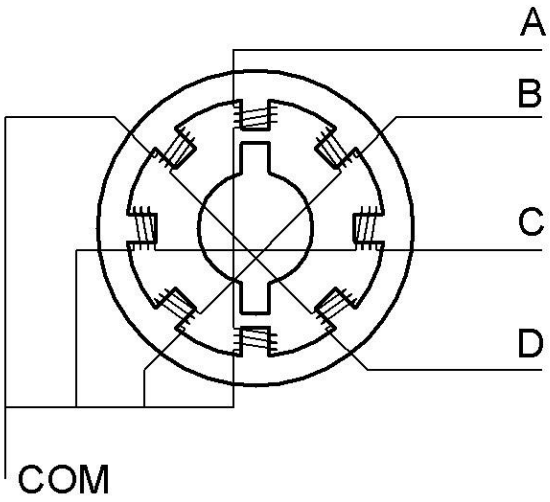
Stepper Motor

Stepper Motors are an open-loop control device, which converts an electronic pulse signal into angular displacement or linear displacement. In a non-overload condition, the speed of the motor and the location of the stops depends only on the pulse signal frequency and number of pulses and is not affected by changes in load as with a DC Motor. A small Four-Phase Deceleration Stepper Motor is shown here:



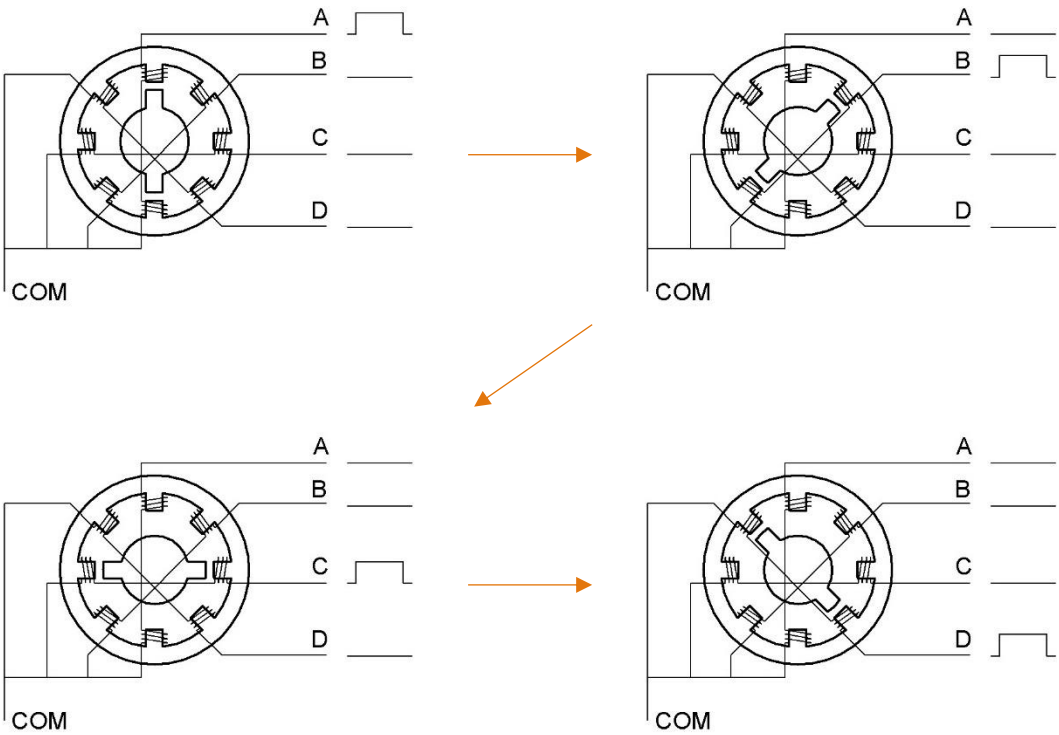


The electronic schematic diagram of a Four-Phase Stepper Motor is shown below:



The outside case or housing of the Stepper Motor is the Stator and inside the Stator is the Rotor. There is a specific number of individual coils, usually an integer multiple of the number of phases the motor has, when the Stator is powered ON, an electromagnetic field will be formed to attract a corresponding convex diagonal groove or indentation in the Rotor's surface. The Rotor is usually made of iron or a permanent magnet. Therefore, the Stepper Motor can be driven by powering the coils on the Stator in an ordered sequence (producing a series of "steps" or stepped movements).

A common driving sequence is shown here:



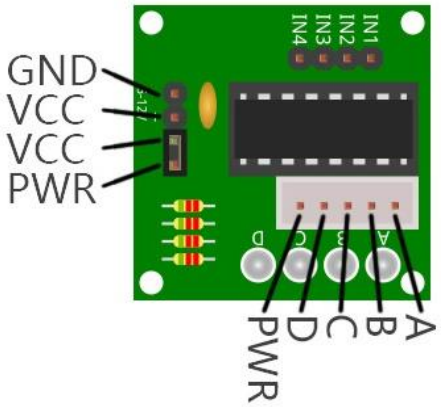
In the sequence above, the Stepper Motor rotates by a certain angle at once, which is called a “step”. By controlling the number of rotational steps, you can then control the Stepper Motor’s rotation angle. By defining the time between two steps, you can control the Stepper Motor’s rotation speed. When rotating clockwise, the order of coil powered on is: A → B → C → D → A →····· . And the rotor will rotate in accordance with this order, step by step, called four-steps, four-part. If the coils is powered ON in the reverse order, D → C → B → A → D →··· , the rotor will rotate in counter-clockwise direction.

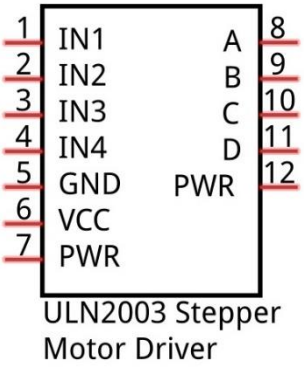
There are other methods to control Stepper Motors, such as: connect A phase, then connect A B phase, the stator will be located in the center of A B, which is called a half-step. This method can improve the stability of the Stepper Motor and reduces noise. Tise sequence of powering the coils looks like this: A → AB → B → BC → C → CD → D → DA → A →·····, the rotor will rotate in accordance to this sequence ar, a half-step at a time, called four-steps, eight-part. Conversely, if the coils are powered ON in the reverse order the Stepper Motor will rotate in the opposite direction.

The stator in the Stepper Motor we have supplied has 32 magnetic poles. Therefore, to complete one full revolution requires 32 full steps. The rotor (or output shaft) of the Stepper Motor is connected to a speed reduction set of gears and the reduction ratio is 1:64. Therefore, the final output shaft (exiting the Stepper Motor’s housing) requires 32 X 64 = 2048 steps to make one full revolution.

ULN2003 Stepper Motor driver

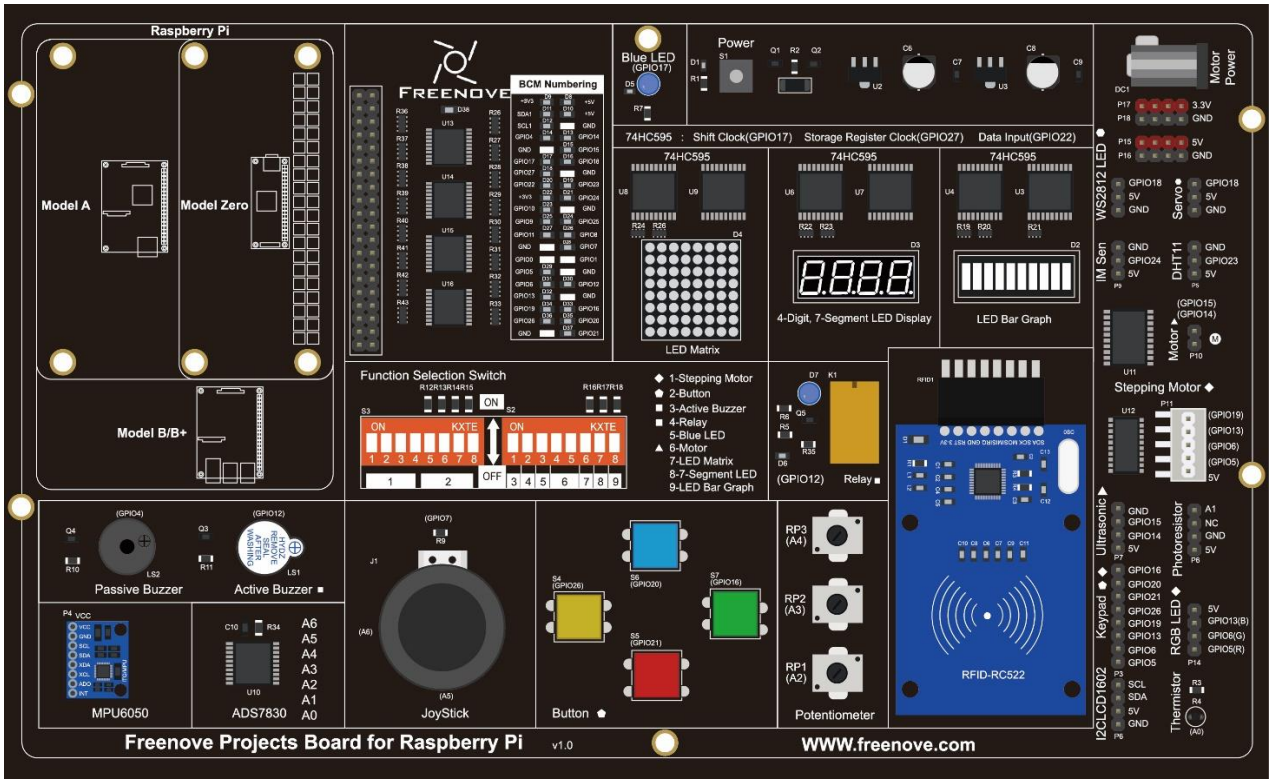
A ULN2003 Stepper Motor Driver is used to convert weak signals into more powerful control signals in order to drive the Stepper Motor. In the illustration below, the input signal IN1-IN4 corresponds to the output signal A-D, and 4 LEDs are integrated into the board to indicate the state of these signals. The PWR interface can be used as a power supply for the Stepper Motor. By default, PWR and VCC are connected.



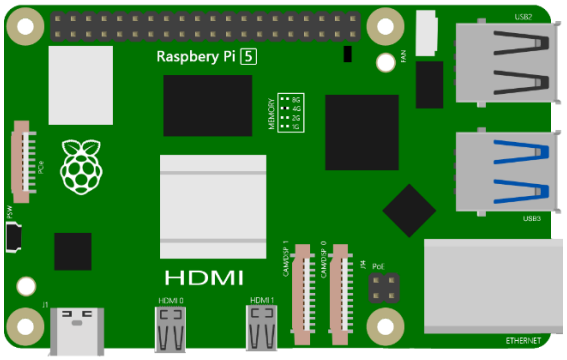


Component List

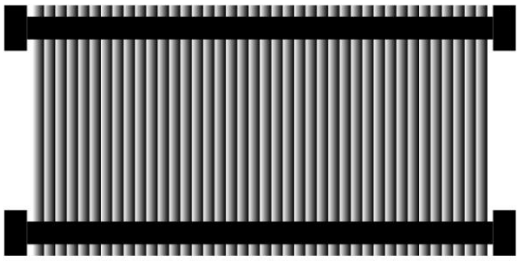
Freenove Projects Board for Raspberry Pi



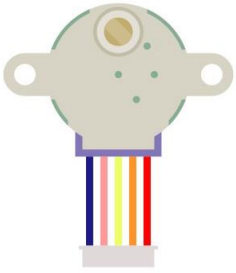
Raspberry Pi



GPIO Ribbon Cable

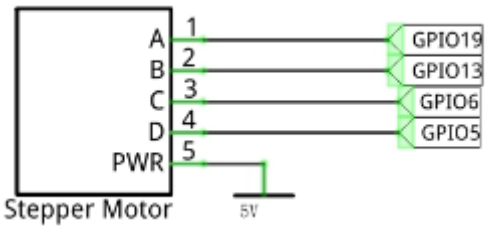


Stepper Motor

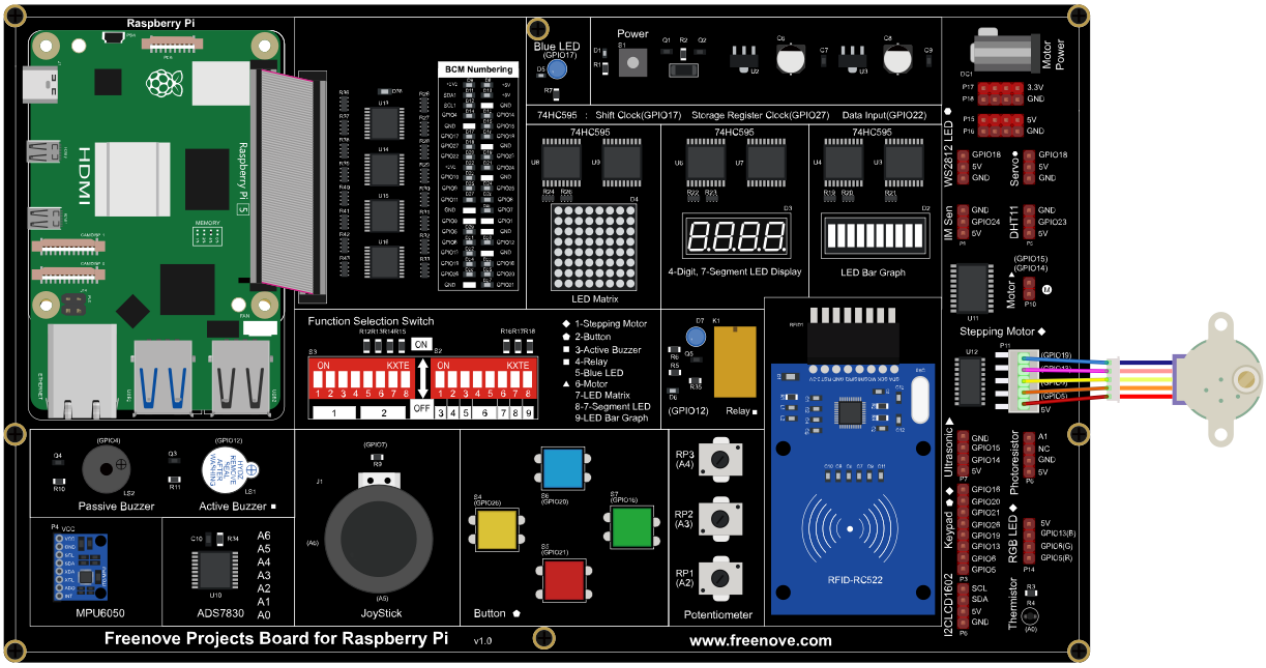


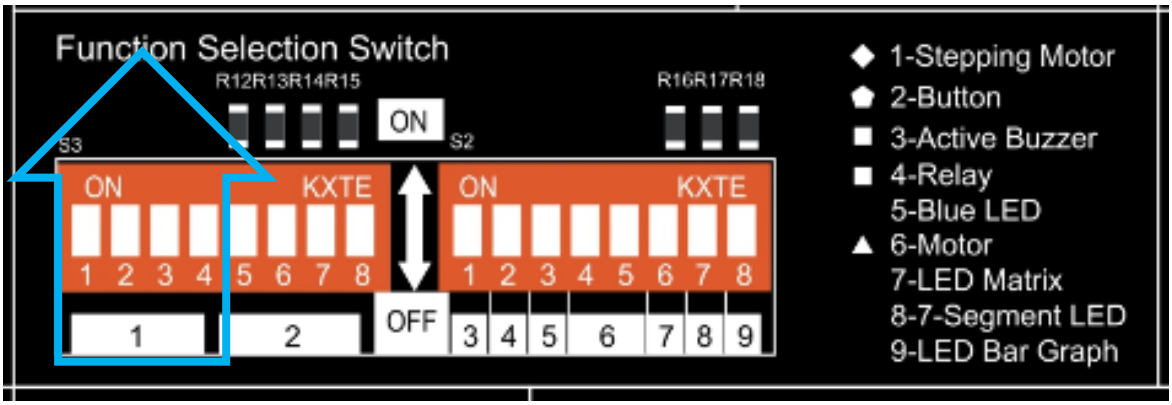
Circuit

Schematic diagram



Hardware connection.





If you have any concerns, please send an email to: support@freenove.com

Code

This code uses the four-step, four-part mode to drive the Stepper Motor in the clockwise and anticlockwise directions.

C Code 14.1 SteppingMotor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 14_SteppingMotor directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/14_SteppingMotor
```

2. Use following command to compile "SteppingMotor.c" and generate executable file "SteppingMotor".

```
gcc SteppingMotor.c -o SteppingMotor -lwiringPi
```

3. Run the generated file "SteppingMotor".

```
./SteppingMotor
```

After the program runs, the Stepper Motor will rotate 360° clockwise and then 360° anticlockwise and repeat this action in an endless loop.

The following is the program code:

```
#include <stdio.h>
#include <wiringPi.h>

const int motorPins[]={19,13,6,5}; //define pins connected to four phase ABCD of stepper motor
const int CCWStep[]={0x01,0x02,0x04,0x08};//rotating anticlockwise
const int CWStep[]={0x08,0x04,0x02,0x01}; //rotating clockwise
//as for four phase stepping motor, four steps is a cycle. the function is used to drive the stepping motor clockwise or anticlockwise to take four steps
void moveOnePeriod(int dir,int ms){
    int i=0,j=0;
    for (j=0;j<4;j++){      //cycle according to power supply order
        for (i=0;i<4;i++){  //assign to each pin, a total of 4 pins
            if(dir == 1)    //power supply order clockwise
                digitalWrite(motorPins[i],(CCWStep[j] == (1<<i)) ? HIGH : LOW);
            else            //power supply order anticlockwise
                digitalWrite(motorPins[i],(CWStep[j] == (1<<i)) ? HIGH : LOW);
            printf("motorPin %d, %d \n",motorPins[i],digitalRead(motorPins[i]));
        }
        printf("Step cycle!\n");
        //The delay can not be less than 3ms, otherwise the motor will stop.
        if(ms<3)
            ms=3;
        delay(ms);
    }
}
```

```
25  //continuous rotation function, the parameter steps specifies the rotation cycles, every four
26  steps is a cycle
27  void moveSteps(int dir, int ms, int steps){
28      int i;
29      for(i=0;i<steps;i++){
30          moveOnePeriod(dir,ms);
31      }
32  }
33  void motorStop(){   //function used to stop rotating
34      int i;
35      for(i=0;i<4;i++){
36          digitalWrite(motorPins[i],LOW);
37      }
38  }
39  int main(void){
40      int i;
41      printf("Program is starting ...\n");
42      wiringPiSetupGpio();
43      for(i=0;i<4;i++){
44          pinMode(motorPins[i],OUTPUT);
45      }
46
47      while(1){
48          moveSteps(1,3,512); //rotating 360°  clockwise, a total of 2048 steps in a circle,
    namely, 512 cycles.
49          delay(500);
50          moveSteps(0,3,512); //rotating 360°  anticlockwise
51          delay(500);
52      }
53      return 0;
54  }
```

In the code we define the four pins of the Stepper Motor and the order to supply power to the coils for a four-step rotation mode.

```
const int motorPins[]={19,13,6,5}; //define pins connected to four phase ABCD of stepper motor
const int CCWStep[]={0x01,0x02,0x04,0x08};//rotating anticlockwise
const int CWStep[]={0x08,0x04,0x02,0x01}; //rotating clockwise
```

Subfunction **moveOnePeriod** ((int dir,int ms) will drive the Stepper Motor rotating four-step clockwise or anticlockwise, four-step as a cycle. Where parameter "dir" indicates the rotation direction, if "dir" is 1, the servo will rotate clockwise, otherwise it rotates to anticlockwise. Parameter "ms" indicates the time between each two steps. The "ms" of Stepper Motor used in this project is 3ms (the shortest time period), a value of less than 3ms will exceed the limits of the Stepper Motor with a result that it does not rotate.

```
void moveOnePeriod(int dir,int ms){
    int i=0,j=0;
    for (j=0;j<4;j++){      //cycle according to power supply order
        for (i=0;i<4;i++){  //assign to each pin, a total of 4 pins
            if(dir == 1)    //power supply order clockwise
                digitalWrite(motorPins[i],(CCWStep[j] == (1<<i)) ? HIGH : LOW);
            else            //power supply order anticlockwise
                digitalWrite(motorPins[i],(CWStep[j] == (1<<i)) ? HIGH : LOW);
            printf("motorPin %d, %d \n",motorPins[i],digitalRead(motorPins[i]));
        }
        printf("Step cycle!\n");
        //The delay can not be less than 3ms, otherwise the motor will stop.
        if(ms<3)
            ms=3;
        delay(ms);
    }
}
```

Subfunction **moveSteps** (int dir, int ms, int steps) is used to specific cycle number of Stepper Motor.

```
void moveSteps(int dir, int ms, int steps){
    int i;
    for(i=0;i<steps;i++){
        moveOnePeriod(dir,ms);
    }
}
```

Subfunction **motorStop** () is used to stop the Stepper Motor.

```
void motorStop(){   //function used to stop rotating
    int i;
    for(i=0;i<4;i++){
        digitalWrite(motorPins[i],LOW);
    }
}
```

Finally, in the while loop of main function, rotate one revolution clockwise, and then one revolution anticlockwise. According to the previous material covered, the Stepper Motor rotating for one revolution requires 2048 steps, that is, 2048/4=512 cycle.

```
    while(1){
        moveSteps(1,3,512);     //rotating  360°  clockwise, a total of 2048 steps in a
circle, namely, this function(four steps) will be called 512 times.
        delay(500);
        moveSteps(0,3,512);     //rotating 360°  anticlockwise
        delay(500);
    }
```

Python Code 14.1 SteppingMotor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 14_StepperMotor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/14_StepperMotor
```

2. Use Python command to execute code "SteppingMotor.py".

```
python SteppingMotor.py
```

After the program runs, the Stepper Motor will rotate 360° clockwise and then 360° anticlockwise and repeat this action in an endless loop.

The following is the program code:

```
1  from gpiozero import OutputDevice
2  import time
3
4  motorPins = (19, 13, 6, 5) # define pins connected to four phase ABCD of stepper motor
5  motors = list(map(lambda pin: OutputDevice(pin), motorPins))
6  CCWStep = (0x01,0x02,0x04,0x08) # define power supply order for rotating anticlockwise
7  CWStep = (0x08,0x04,0x02,0x01)  # define power supply order for rotating clockwise
8
9  # as for four phase stepping motor, four steps is a cycle. the function is used to drive the
10 stepping motor clockwise or anticlockwise to take four steps
11 def moveOnePeriod(direction,ms):
12     for j in range(0,4,1):      # cycle for power supply order
13         for i in range(0,4,1):  # assign to each pin
14             if (direction == 1):# power supply order clockwise
15                 motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
16             else :              # power supply order anticlockwise
17                 motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
18         if(ms<3):       # the delay can not be less than 3ms, otherwise it will exceed speed
   limit of the motor
19             ms = 3
20         time.sleep(ms*0.001)
21
22 # continuous rotation function, the parameter steps specifies the rotation cycles, every four
   steps is a cycle
23 def moveSteps(direction, ms, steps):
24     for i in range(steps):
25         moveOnePeriod(direction, ms)
26
27 # function used to stop motor
28 def motorStop():
29     for i in range(0,4,1):
30         motors.off()
```

```
31
32  def loop():
33      while True:
34          moveSteps(0,3,512)  # rotating 360 deg clockwise, a total of 2048 steps in a circle,
    512 cycles
35          time.sleep(0.5)
36          moveSteps(1,3,512)  # rotating 360 deg anticlockwise
37          time.sleep(0.5)
38
39  def destroy():
40      for i in range(0,4,1):
41          motors[i].close()
42
43
44  if __name__ == '__main__':     # Program entrance
45      print ('Program is starting...')
46      try:
47          loop()
48      except KeyboardInterrupt:  # Press ctrl-c to end the program.
49          print("Ending program")
50      finally:
51          destroy()
```

In the code we define the four pins of the Stepper Motor and the order to supply power to the coils for a four-step rotation mode.

```
motorPins = (19, 13, 6, 5) # define pins connected to four phase ABCD of stepper motor
motors = list(map(lambda pin: OutputDevice(pin), motorPins))
CCWStep = (0x01,0x02,0x04,0x08) # define power supply order for rotating anticlockwise
CWStep = (0x08,0x04,0x02,0x01)  # define power supply order for rotating clockwise
```

Subfunction **moveOnePeriod** ((int dir, int ms) will drive the Stepper Motor rotating four-step clockwise or anticlockwise, four-step as a cycle. Where parameter "dir" indicates the rotation direction, if "dir" is 1, the servo will rotate clockwise, otherwise it rotates to anticlockwise. Parameter "ms" indicates the time between each two steps. The "ms" of Stepper Motor used in this project is 3ms (the shortest time period), a value of less than 3ms will exceed the limits of the Stepper Motor with a result that it does not rotate.

```
def moveOnePeriod(direction,ms):
    for j in range(0,4,1):      # cycle for power supply order
        for i in range(0,4,1):  # assign to each pin
            if (direction == 1):# power supply order clockwise
                motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
            else :              # power supply order anticlockwise
                motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
        if(ms<3):       # the delay can not be less than 3ms, otherwise it will exceed speed
limit of the motor
```

```
            ms = 3
        time.sleep(ms*0.001)
```

Subfunction **moveSteps** (direction, ms, steps) is used to specify the cycle number of Stepper Motor.

```
def moveSteps(direction, ms, steps):
    for i in range(steps):
        moveOnePeriod(direction, ms)
```

Subfunction **motorStop** () is used to stop the Stepper Motor.

```
def motorStop():
    for i in range(0,4,1):
        motors.off()
```

Finally, in the while loop of main function, rotate one revolution clockwise, and then one revolution anticlockwise. According to the previous material covered, the Stepper Motor rotating for one revolution requires 2048 steps, that is, 2048/4=512 cycle.

```
while True:
        moveSteps(1,3,512)  # rotating 360 deg clockwise, a total of 2048 steps in a circle,
512 cycles
        time.sleep(0.5)
        moveSteps(0,3,512)  # rotating 360 deg anticlockwise
        time.sleep(0.5)
```

Chapter 15 LEDpixel

In this chapter, we will learn Freenove 8 RGB LED Module.

Note: if your Raspberry Pi is Raspberry Pi5, skip 15.1 and 15.2 and look at 15.3 instead. Because 15.1 and 15.2 use the rpi_ws2812 library, they are not compatible with Raspberry Pi5.

Since rpi libraries do not work on bookworm systems, skip 15.1 and 15.2 if you are using that system.

15.3 Use spi to drive the color lights.

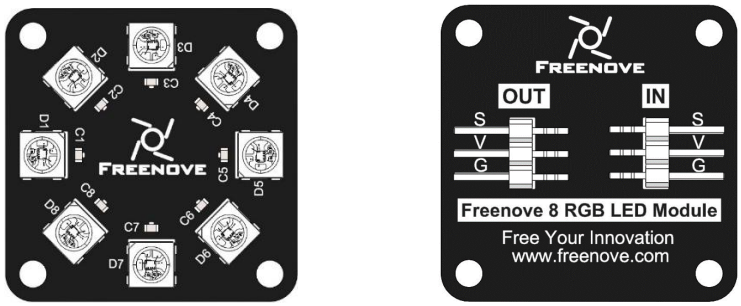
Project 15.1 LEDpixel

This project will achieve an RGB triple colored flowing water.

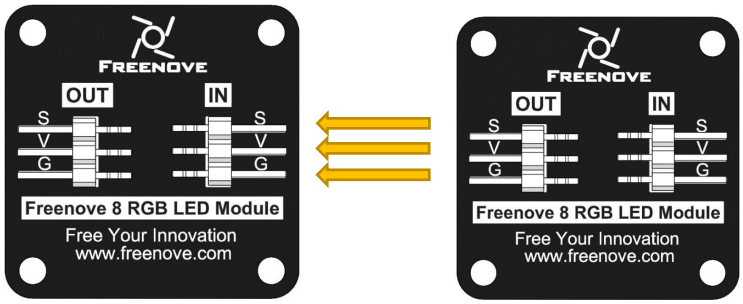
Component knowledge

Freenove 8 RGB LED Module

The Freenove 8 RGB LED Module is as below. You can use only one data pin to control the eight LEDs on the module. As shown below:



And you can also control many modules at the same time. Just connect OUT pin of one module to IN pin of another module. In such way, you can use one data pin to control 8, 16, 32 ··· LEDs.

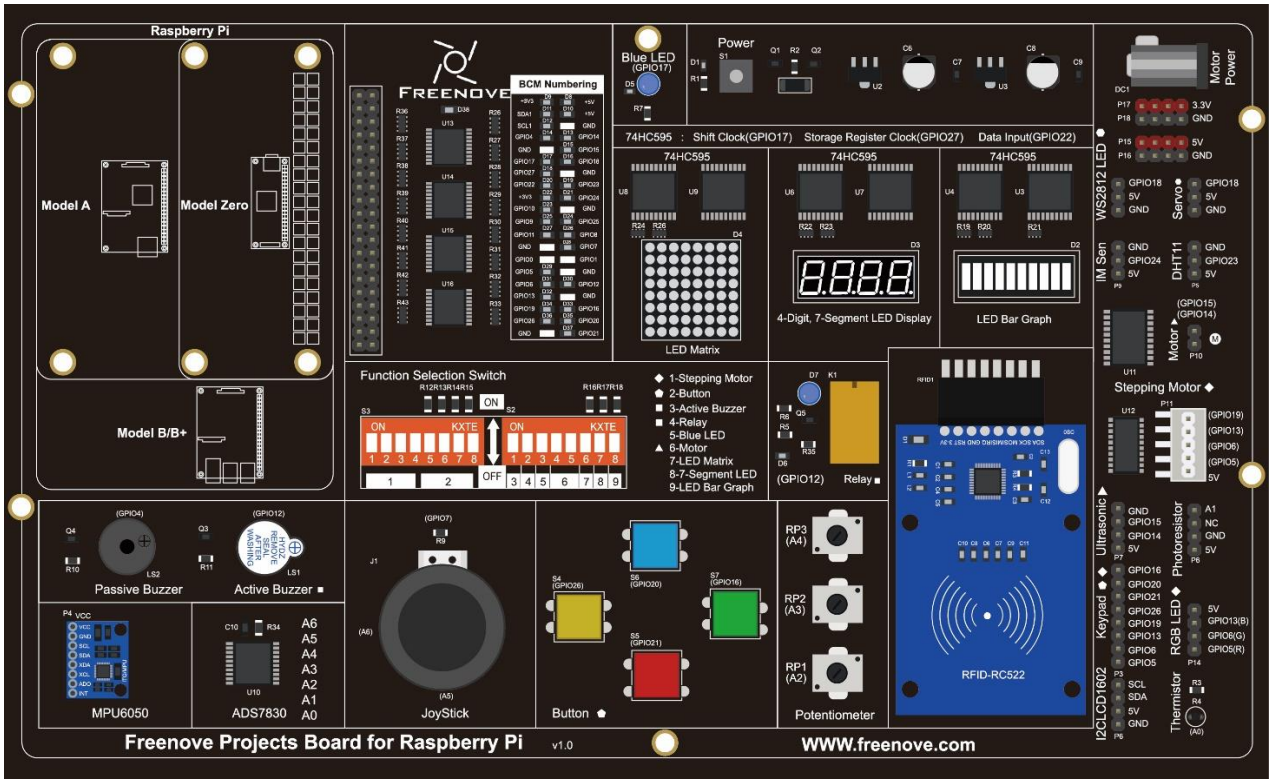


Pin description:

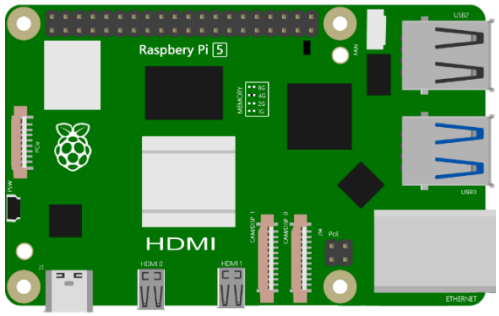
(IN)		(OUT)	
symbol	Function	symbol	Function
S	Input control signal	S	Output control signal
V	Power supply pin, +3.5V~5.5V	V	Power supply pin, +3.5V~5.5V
G	GND	G	GND

Component List

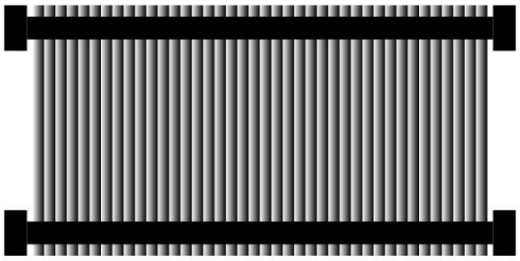
Freenove Projects Board for Raspberry Pi



Raspberry Pi

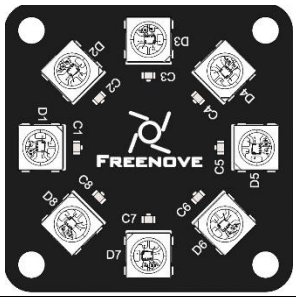


GPIO Ribbon Cable



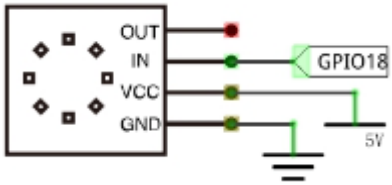
Jumper Wire

Freenove 8 RGB LED Module

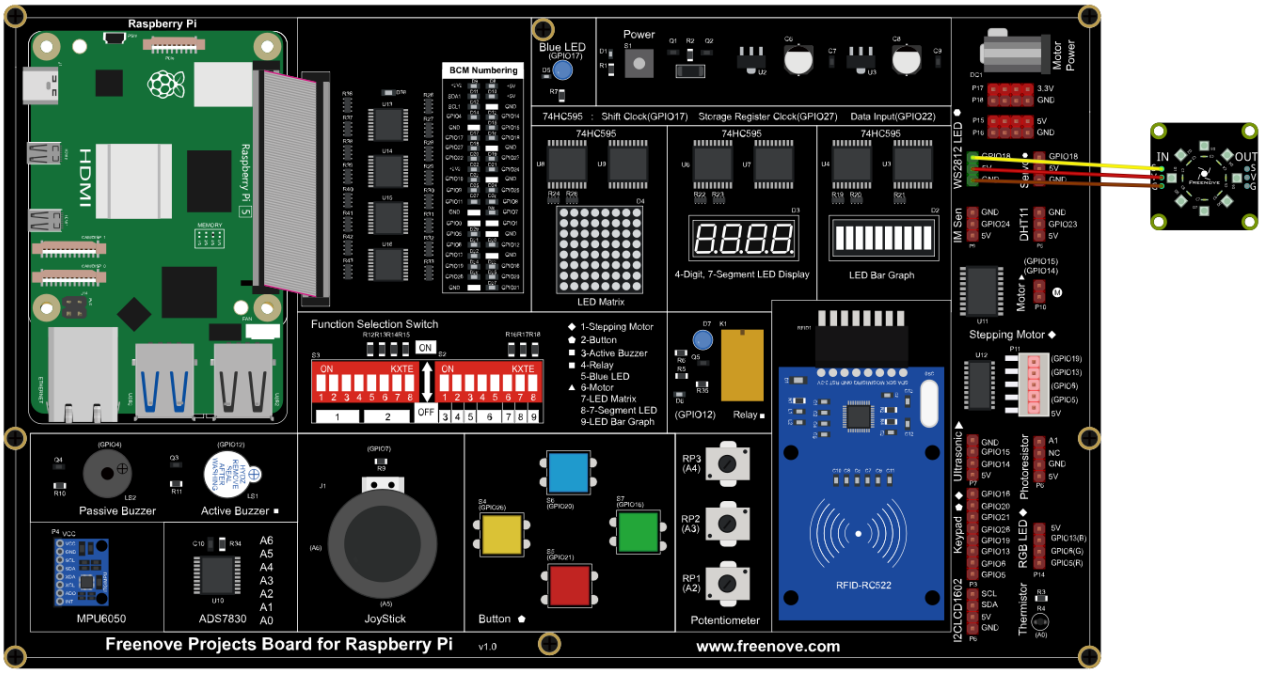


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 15.1 Ledpixel

Before running C code, please install WS281X library.

1. Enter the directory where the library locates:

```
cd ~/Freenove_Kit/Libs/C-Libs/libWS281X
```

2. Run the program

```
sudo sh ./build.sh
```

The installation is completed as shown in the figure below

```
pi@raspberrypi:~/Freenove_Kit/Code/C_Code/15_1_Ledpixel $ cd ~/Freenove_Kit/Libs
/C-Libs/libWS281X
pi@raspberrypi:~/Freenove_Kit/Libs/C-Libs/libWS281X $ sudo sh ./build.sh
build completed!
```

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

3. Use cd command to enter 15_1_Ledpixel directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/15_1_Ledpixel
```

4. Use following command to compile " Ledpixel.cpp" and generate executable file "Ledpixel".

```
sudo g++ Ledpixel.cpp -o Ledpixel -lwiringPi -lWS281X
```

5. Run the generated file " Ledpixel".

```
sudo ./Ledpixel
```

After the program runs, the LEDpixel will emit red, blue and green colors in turn like flowing water.

The following is the program code:

```
#include <wiringPi.h>
#include "Freenove_WS2812_Lib_for_Raspberry_Pi.hpp"
Freenove_WS2812 *a;
int constrain(int value,int min,int max){
    if (value>max){
        return max;
    }
    else if (value<min){
        return min;
    }
    else {
        return value;
    }
}
int main(){
    printf("Program is starting ...\n");
    int i;
    a= new Freenove_WS2812(18,8,GRB);//pin led_count type
    a->set_Led_Brightness(50);
    for(i=0;i<8;i++){
        a->set_Led_Color(i,255,0,0);
        a->show();
        delay(100);
        }
    for(i=0;i<8;i++){
        a->set_Led_Color(i,0,255,0);
        a->show();
        delay(100);
        }
    for(i=0;i<8;i++){
        a->set_Led_Color(i,0,0,255);
        a->show();
        delay(100);
        }
```

```
35      a->clear();
36      return 0;
37  }
```

Include "Freenove_WS2812_Lib_for_Raspberry_Pi.hpp"

```
    #include "Freenove_WS2812_Lib_for_Raspberry_Pi.hpp"
```

Create the object of the class and set the brightness to 50%. The eight LEDs will then light up red, green and blue in turn.

```
int main(){
    printf("Program is starting ...\n");
    int i;
    a= new Freenove_WS2812(18,8,GRB);//pin led_count type
    a->set_Led_Brightness(50);
    for(i=0;i<8;i++){
        a->set_Led_Color(i,255,0,0);
        a->show();
        delay(100);
        }
    for(i=0;i<8;i++){
        a->set_Led_Color(i,0,255,0);
        a->show();
        delay(100);
        }
    for(i=0;i<8;i++){
        a->set_Led_Color(i,0,0,255);
        a->show();
        delay(100);
        }
    a->clear();
    return 0;
}
```

Python Code 15.1 Ledpixel

Before running python code, please install WS281X library first.

1. Enter the following command to install.

```
sudo pip3 install rpi_ws281x
```

The installation is completed as shown in the figure below.



First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 15_1_Ledpixel directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15_1_Ledpixel
```

2. Use python command to execute code "Sweep.py".

```
sudo python Led.py
```

After the program runs, the LEDpixel will emit red, green and blue colors in turn like flowing water.

The following is the program code:

```
1  import time
2  from rpi_ws281x import *
3  # LED strip configuration:
4  LED_COUNT      = 8      # Number of LED pixels.
5  LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
6  LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
7  LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
8  LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
9  LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
10 LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
11 # Define functions which animate LEDs in various ways.
12 class Led:
13     def __init__(self):
14         #Control the sending order of color data
15         self.ORDER = "RGB"
16         # Create NeoPixel object with appropriate configuration.
17         self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
18 LED_BRIGHTNESS, LED_CHANNEL)
19         # Intialize the library (must be called once before other functions).
20         self.strip.begin()
21         #self.strip.setPixelColor(i, color)
22         #self.strip.show()
```

```
23
24
25  led=Led()
26  # Main program logic follows:
27  if __name__ == '__main__':
28      print ('Program is starting ... ')
29      col=[Color(255,0,0),Color(0,255,0),Color(0,0,255)]
30      try:
31          while True:
32              for c in range(3):
33                  for i in range(8):
34                      led.strip.setPixelColor(i,col[c])
35                      time.sleep(0.1)
36                      led.strip.show()
37      except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
38  executed.
39          for i in range(8):
40              led.strip.setPixelColor(i, Color(0,0,0))
41          led.strip.show()
```

Import rpi_ws281x modile. Set the number, pins and brightness of the LED.

```
from rpi_ws281x import *
# LED strip configuration:
LED_COUNT      = 8      # Number of LED pixels.
LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
```

Define LED class.

```
class Led:
    def __init__(self):
        #Control the sending order of color data
        self.ORDER = "RGB"
        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
LED_BRIGHTNESS, LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()
        #self.strip.setPixelColor(i, color)
        #self.strip.show()
```

Light up the eight LEDs in red, green and blue in turn.

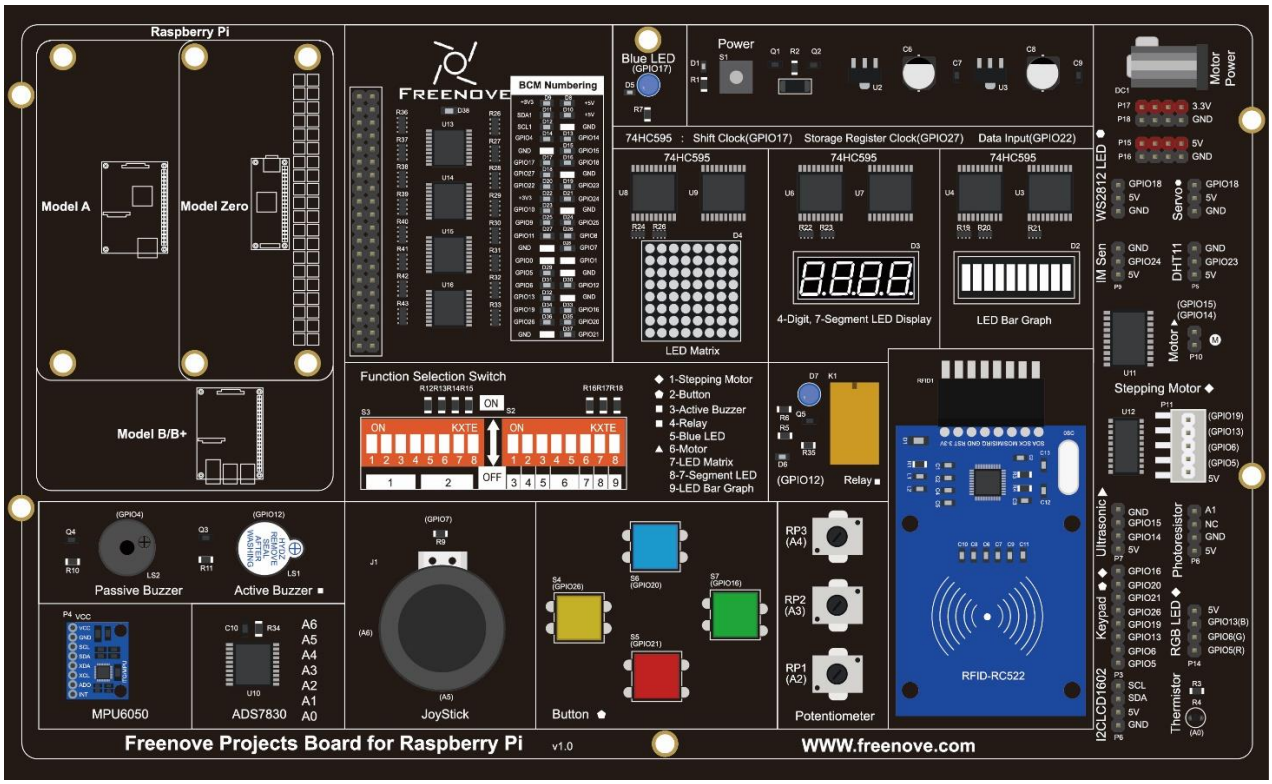
```
    col=[Color(255,0,0),Color(0,255,0),Color(0,0,255)]
    try:
        while True:
            for c in range(3):
                for i in range(8):
                    led.strip.setPixelColor(i,col[c])
                    time.sleep(0.1)
                    led.strip.show()
    except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be executed.
        for i in range(8):
            led.strip.setPixelColor(i, Color(0,0,0))
        led.strip.show()
```

Project 15.2 Rainbow Light

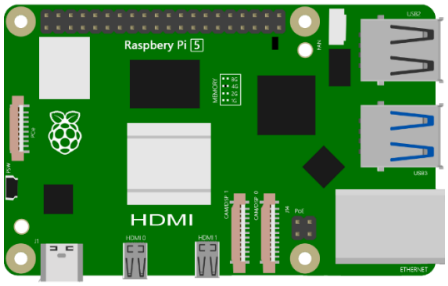
In this project, we will learn to control the LED module with a potentiometer.

Component List

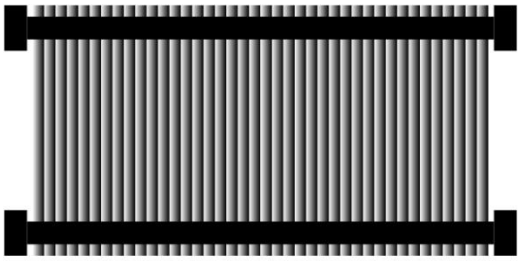
Freenove Projects Board for Raspberry Pi



Raspberry Pi



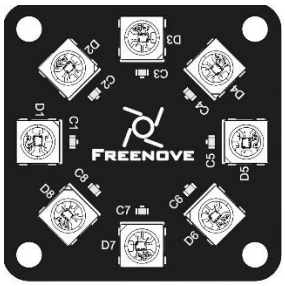
GPIO Ribbon Cable



Jumper Wire

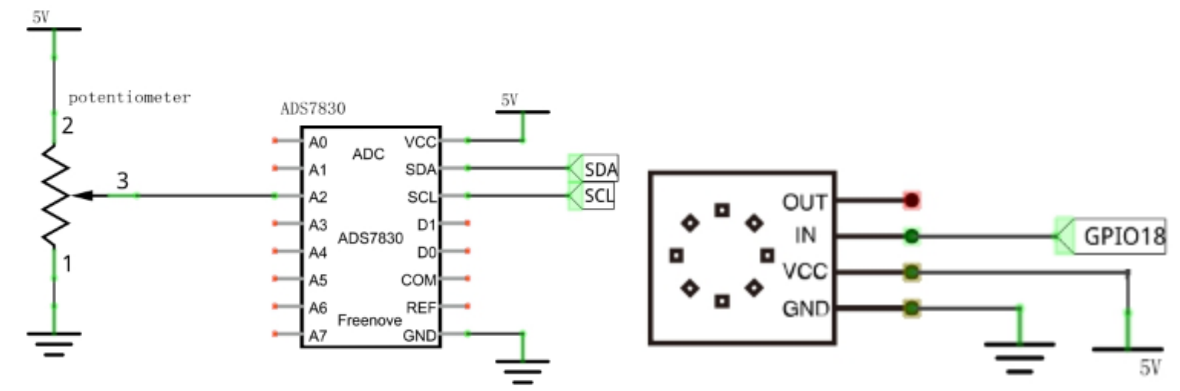


Freenove 8 RGB LED Module

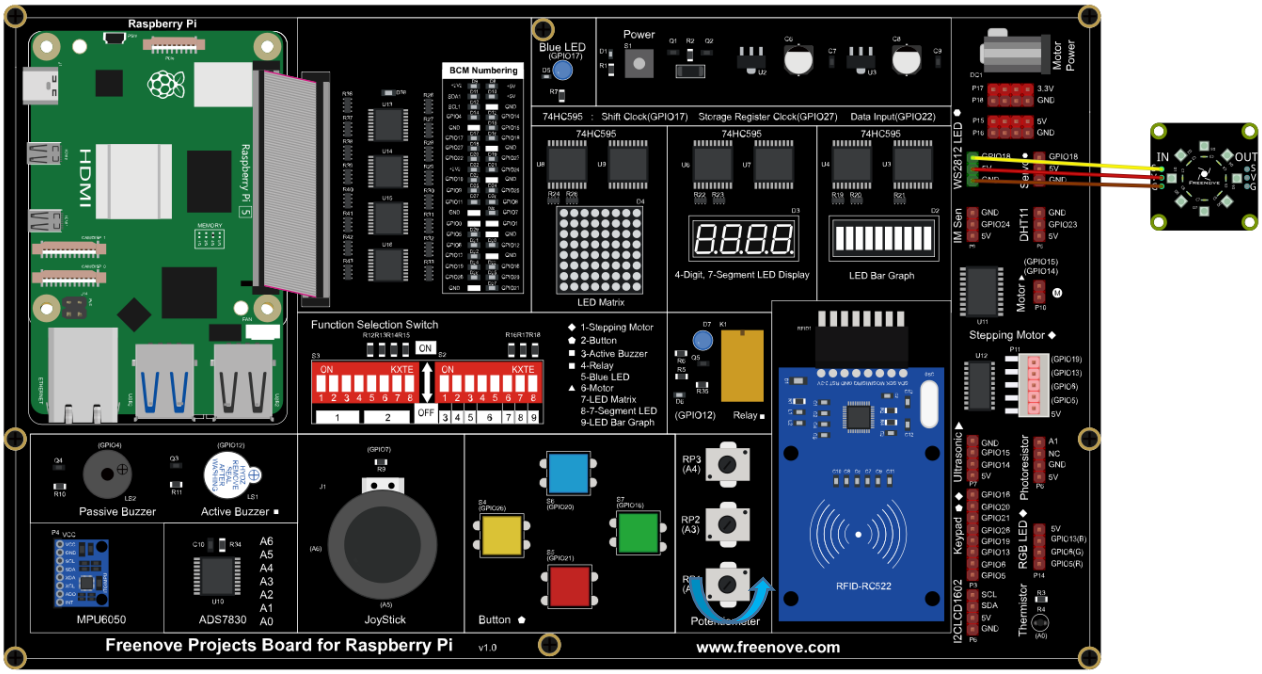


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 15.2 Rainbow Light

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

3. Use cd command to enter 15_2_RainbowLight directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/15_2_RainbowLight
```

4. Use following command to compile " RainbowLight.cpp " and generate executable file " RainbowLight ".

```
sudo g++ RainbowLight.cpp -o RainbowLight -lwiringPi -lWS281X -lADCDevice
```

5. Run the generated file " RainbowLight ".

```
sudo ./RainbowLight
```

After running the program, you can change the color of the LED module by rotating the potentiometer.

The following is the program code:

```
#include <stdio.h>
#include <wiringPi.h>
#include <ADCDevice.hpp>
#include "Freenove_WS2812_Lib_for_Raspberry_Pi.hpp"

Freenove_WS2812 *led;
ADCDevice *adc;
int red,green,blue;

void HSL_RGB(int degree){
    degree=degree/360.0*255;
    if (degree < 85){
            red = 255 - degree * 3;
            green = degree * 3;
            blue = 0;
        }
    else if (degree < 170){
            degree = degree - 85;
            red = 0;
            green = 255 - degree * 3;
            blue = degree * 3;
        }
    else{
            degree = degree - 170;
            red = degree * 3;
            green = 0;
            blue = 255 - degree * 3;
        }
```

```
}

int main(){
    printf("Program is starting ...\n");
    adc = new ADCDevice();
    int i;
    led= new Freenove_WS2812(18,8,GRB);//pin led_count type
    led->set_Led_Brightness(50);

    if(adc->detectI2C(0x48)){   // Detect the ads7830
        delete adc;               // Free previously pointed memory
        adc = new ADS7830(0x48);      // If detected, create an instance of ADS7830.
    }
    else{
        printf("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        return -1;
    }


    while(1){
        for(i=0;i<8;i++){
            int degree = (int)(adc->analogRead(2)/255.0*360+i*45);    //read analog value of
A0 pin
            if (degree > 360){
                degree=degree-360;
                }
            HSL_RGB(degree);
            led->set_Led_Color(i,red,green,blue);
            led->show();

        }
    }

    return 0;
}
```

This function converts HSL colors to RGB colors.

```
void HSL_RGB(int degree){
    degree=degree/360.0*255;
    if (degree < 85){
            red = 255 - degree * 3;
            green = degree * 3;
            blue = 0;
        }
    else if (degree < 170){
            degree = degree - 85;
            red = 0;
            green = 255 - degree * 3;
            blue = degree * 3;
        }
    else{
            degree = degree - 170;
            red = degree * 3;
            green = 0;
            blue = 255 - degree * 3;
        }
}
```

Read the ADC value of channel 2 in an infinite loop. Let the color of the eight LEDs change according to the value of the ADC.

```
while(1){
        for(i=0;i<8;i++){
            int degree = (int)(adc->analogRead(2)/255.0*360+i*45);     //read analog value of
A2 pin
            if (degree > 360){
                degree=degree-360;
                }
            HSL_RGB(degree);
            led->set_Led_Color(i,red,green,blue);
            led->show();

        }
    }
```

Python Code 15.2 Rainbow Light

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

6. Use cd command to enter 15.2 Rainbow Light directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15_2_RainbowLight
```

7. Use python command to execute code " **Led.py** ".

```
sudo python Led.py
```

After running the program, you can change the color of the LED module by rotating the potentiometer.

The following is the program code:

```
import time
from rpi_ws281x import *
from ADCDevice import *
# LED strip configuration:
LED_COUNT      = 8      # Number of LED pixels.
LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
# Define functions which animate LEDs in various ways.
class Led:
    def __init__(self):
        #Control the sending order of color data
        self.ORDER = "RGB"
        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
LED_BRIGHTNESS, LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()

        self.adc = ADCDevice(0x48) # Define an ADCDevice class object
        if(self.adc.detectI2C(0x48)):
            self.adc = ADS7830(0x48)
        else:
            print("No correct I2C address found, \n"
            "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
            "Program Exit. \n");
            exit(-1)


    def HSL_RGB(self,degree):
```

```
        degree=degree/360*255
        if degree < 85:
            red = 255 - degree * 3
            green = degree * 3
            blue = 0
        elif degree < 170:
            degree = degree - 85
            red = 0
            green = 255 - degree * 3
            blue = degree * 3
        else:
            degree = degree - 170
            red = degree * 3
            green = 0
            blue = 255 - degree * 3
        return int(red),int(green),int(blue)
led=Led()
# Main program logic follows:
if __name__ == '__main__':
    print ('Program is starting ... ')
    try:
        while True:
                for i in range(8):
                    value = round(led.adc.analogRead(2) / 255.0 * 360+i*45)     # read the ADC
value of channel 2
                    if value > 360 :
                        value = value-360
                    red,green,blue=led.HSL_RGB(value)
                    led.strip.setPixelColor(i, Color(red,green,blue))
                time.sleep(0.1)
                led.strip.show()
    except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
executed.
        led.adc.close()
        for i in range(8):
            led.strip.setPixelColor(i, Color(0,0,0))
        led.strip.show()
```

This function converts HSL colors to RGB colors.

```
    def HSL_RGB(self,degree):
        degree=degree/360*255
        if degree < 85:
            red = 255 - degree * 3
            green = degree * 3
```

```
            blue = 0
        elif degree < 170:
            degree = degree - 85
            red = 0
            green = 255 - degree * 3
            blue = degree * 3
        else:
            degree = degree - 170
            red = degree * 3
            green = 0
            blue = 255 - degree * 3
        return int(red),int(green),int(blue)
```

Read the ADC value of channel 2 in an infinite loop. Let the color of the eight LEDs change according to the value of the ADC.

```
        while True:
                for i in range(8):
                    value = round(led.adc.analogRead(2) / 255.0 * 360+i*45)     # read the ADC
value of channel 2
                    if value > 360 :
                        value = value-360
                    red,green,blue=led.HSL_RGB(value)
                    led.strip.setPixelColor(i, Color(red,green,blue))
                time.sleep(0.1)
                led.strip.show()
```

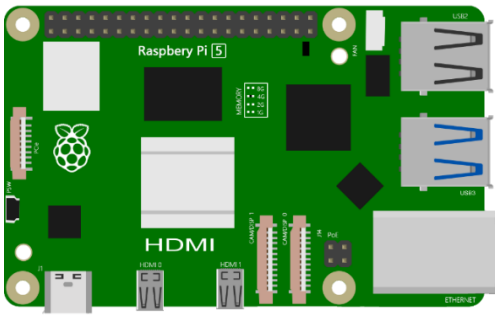
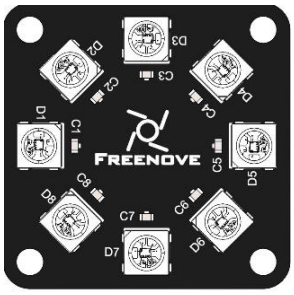
Finally, in the "while" loop of main function, we need to use two separate cycles to make servo rotate from 0 degrees to 180 degrees and then from 180 degrees to 0 degrees.

```
def loop():
    while True:
        for dc in range(0, 181, 1):   #make servo rotate from 0° to 180°
            servoWrite(dc)      # Write to servo
            time.sleep(0.001)
        time.sleep(0.5)
        for dc in range(180, -1, -1): #make servo rotate from 180° to 0°
            servoWrite(dc)
            time.sleep(0.001)
        time.sleep(0.5)
```

Project 15.3 SpiLEDpixel

This project will control ledpixel to display any color by the Raspberry Pi's SPI.
You need to turn on the spi function in this section. At the end of this chapter, turn off the spi function.

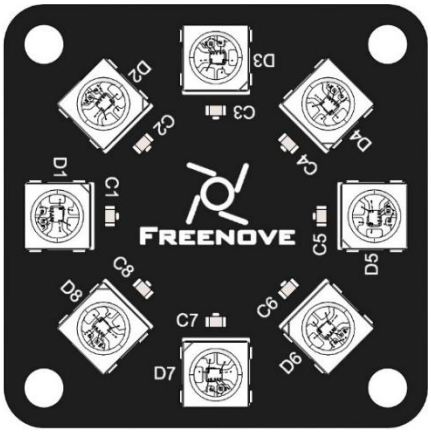
Component List

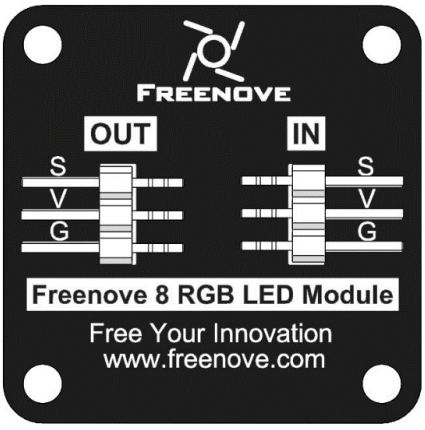
Raspbery Pi (with 40 GPIO) x1 	Jumper Wire x3 
Freenove 8 RGB LED Module x1 	

Component knowledge

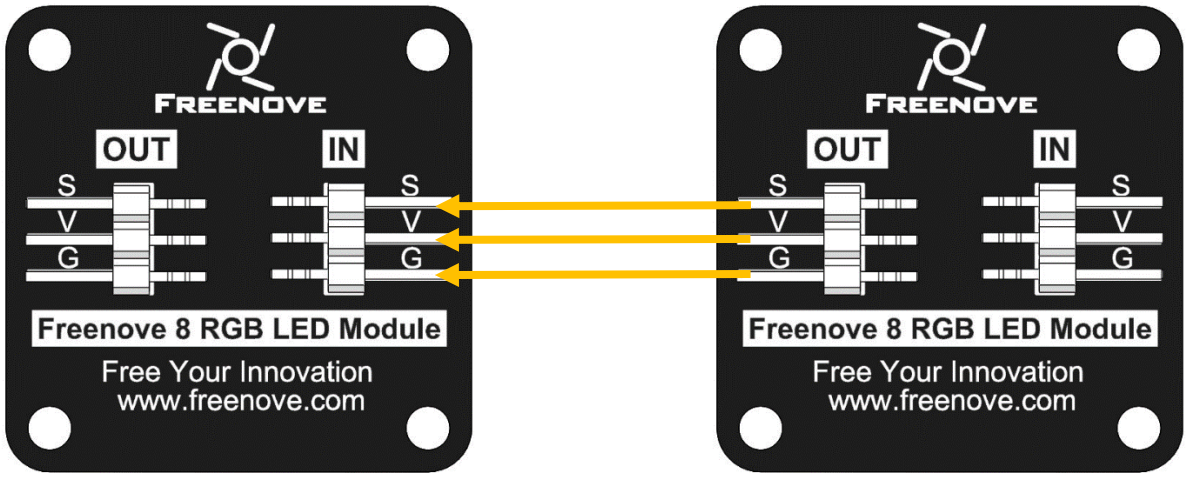
Freenove 8 RGB LED Module

The Freenove 8 RGB LED Module is as below. You can use only one data pin to control the eight LEDs on the module. As shown below:





And you can also control many modules at the same time. Just connect OUT pin of one module to IN pin of another module. In such way, you can use one data pin to control 8, 16, 32 ··· LEDs.

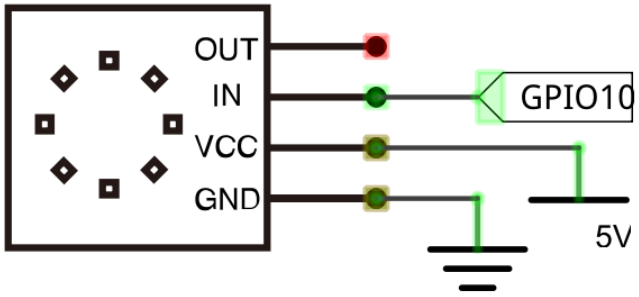


Pin description:

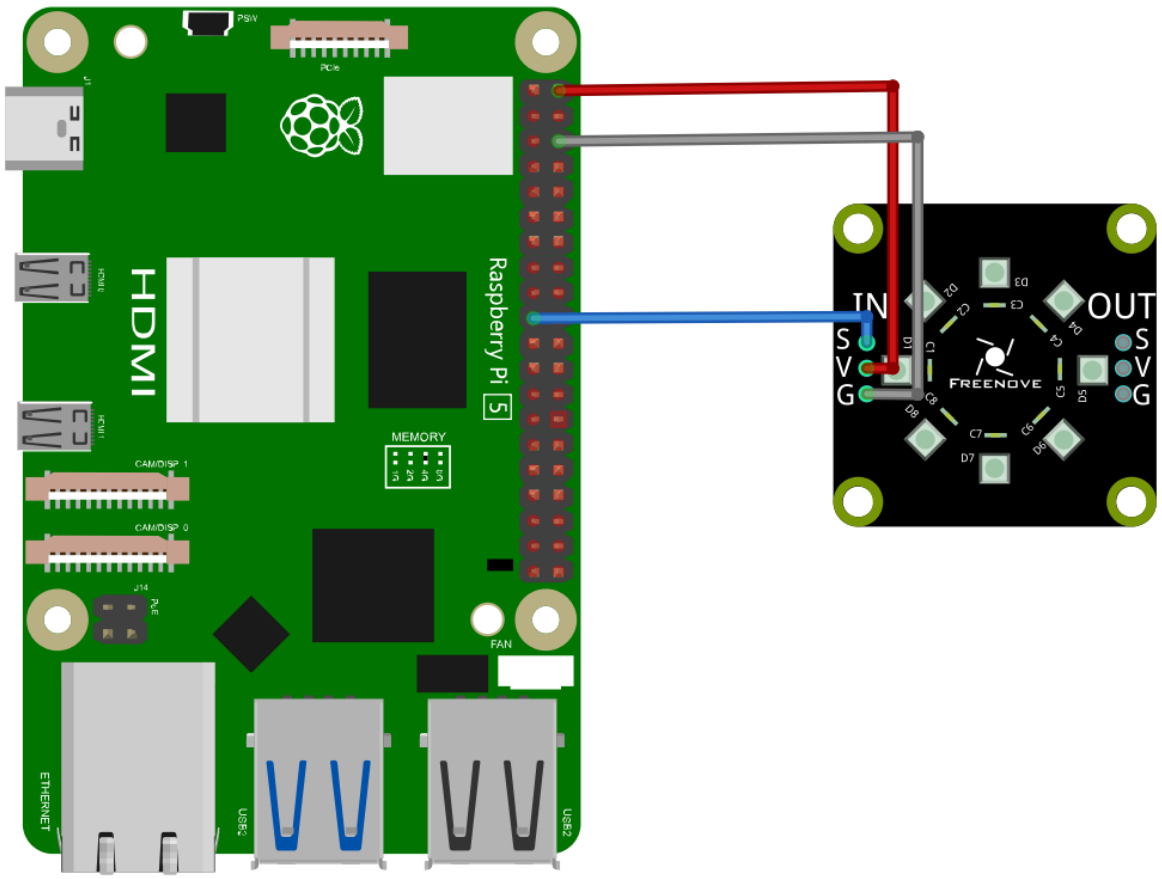
(IN)		(OUT)	
symbol	Function	symbol	Function
S	Input control signal	S	Output control signal
V	Power supply pin, +3.5V~5.5V	V	Power supply pin, +3.5V~5.5V
G	GND	G	GND

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:
support@freenove.com

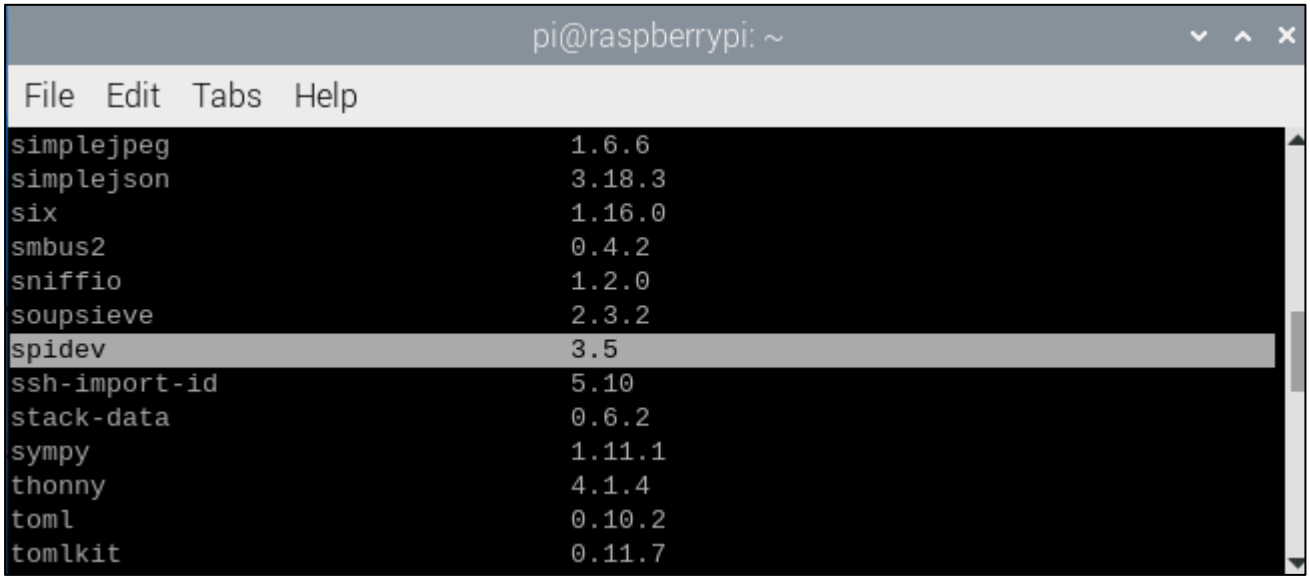


Before you run your python code, check that the spidev library exists.

Enter the following command to install.

```
pip list
```

The spidev is installed on Raspberry PI by default. As shown in the figure below.

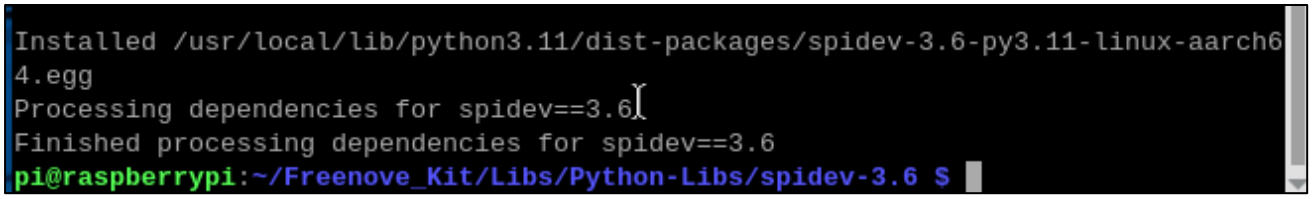


If your Raspberry PI system does not have this library, you can find **spidev-3.6.tar.gz** in **Freenove_Kit/Libs/Python-Libs**.

Enter the following instructions to install spidev.

```
cd Freenove_Kit/Libs/Python-Libs
tar –zxvf spidev-3.6.tar.gz
cd spidev-3.6
sudo python setup.py install
```

The installation is complete as shown in the following figure.



If you have any concerns, please send an email to: support@freenove.com

Additional supplement

Note that the frequency of the SPI changes as the CPU frequency self-regulates, so we need to fix the cpu frequency before we start using the code. Please refer to the following operations.

1. Open the config.txt file and prepare to edit it.

```
sudo nano /boot/firmware/config.txt
```

2, If your Raspberry PI is Raspberry PI 4 or Raspberry PI 5, please add at the bottom:

```
force_turbo=1
```

If your Raspberry PI is Raspberry PI 3, add it at the bottom:

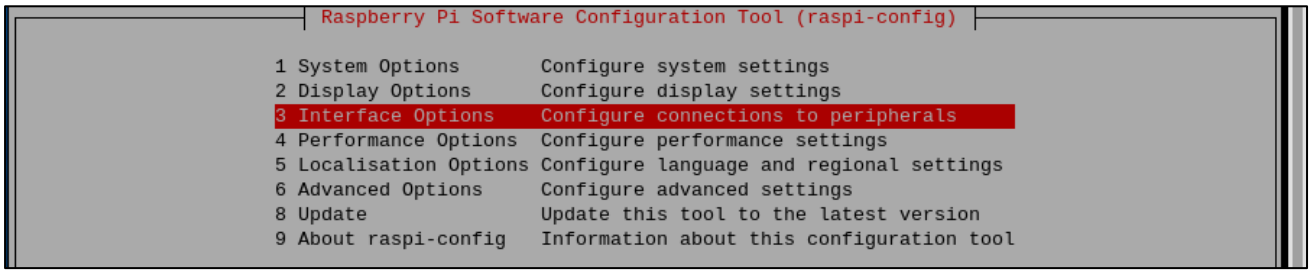
```
core_freq=250
```

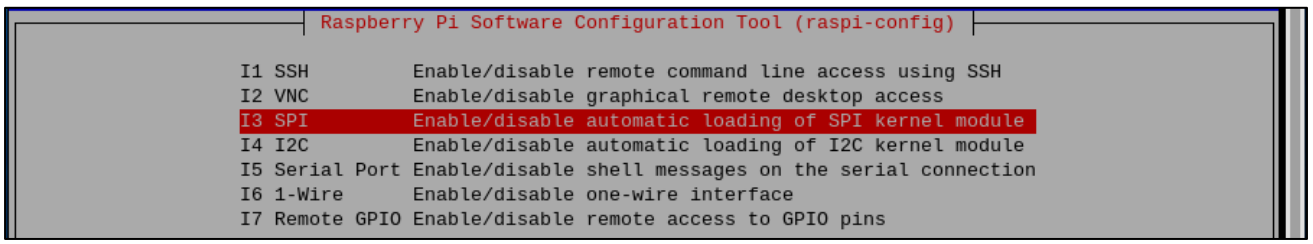
3, Save the file.

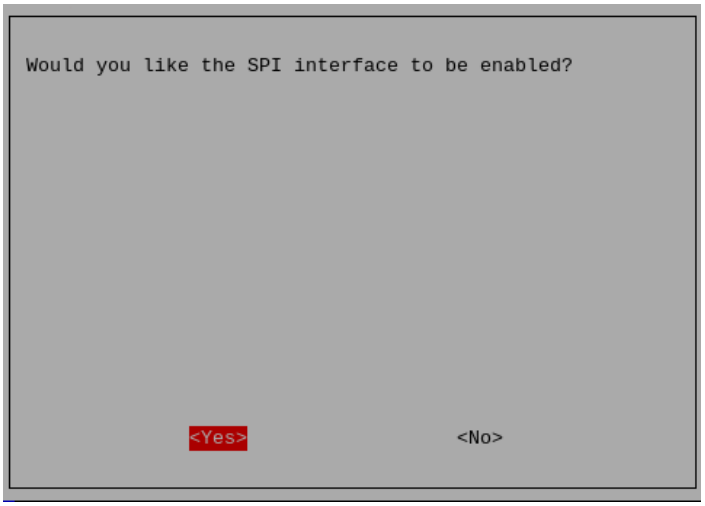
4, Turn on the spi feature of the Raspberry PI.

```
sudo raspi-config
```

5, **Select Interface Options, then SPI, and turn it on.**







6, Select Finish.

7, Reboot the Raspberry PI.

```
sudo reboot
```

Code

C Code 15.3 SpiLedpixel

Use cd command to enter 15_3_SpiLedpixel directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/15_3_SpiLedpixel
```

Use following command to compile " Ledpixel.cpp" and generate executable file "Ledpixel".

```
g++ SpiLedpixel.cpp Freenove_WS2812_SPI.cpp –o main
```

Run the generated file "main". **Please use one of the following commands to control the ledpixel.**

sudo ./main RGB
After the program runs, the led pixels will emit red, blue and green three colors in turn like water. If your Freenove 8 RGB LED module is not working, you can try additional supplements to fix it.
sudo ./main Rainbow
After the program runs, the led pixels will emit a rainbow of colors and slowly rotate.
sudo ./main Breathing
After the program runs, the led will slowly turn on and then slowly turn off.

The following is the program code:

```
#include "Freenove_WS2812_SPI.h"

//Freenove_WS2812_SPI strip = Freenove_WS2812_SPI(8, TYPE_GRB);//led_count, led_type
Freenove_WS2812_SPI strip = Freenove_WS2812_SPI();//led_count=8, led_type=TYPE_GRB

void Ctrl_C_Handler(int value){
    strip.end();
    exit(0);
}
```

```
10
11  int main(int argc, char *argv[]){
12      //Ctrl+C Event
13      signal(SIGINT, Ctrl_C_Handler);
14
15      //Init ledpixel
16      strip.begin();
17      strip.setBrightness(20);
18      strip.setLedType((LED_TYPE)2);
19
20      while(true){
21          //some tips
22          if(argc == 1){
23              printf("The use of ledpixel:\n");
24              printf("  please enter ./main RGB\n");
25              printf("  please enter ./main Rainbow\n");
26              printf("  please enter ./main Breathing\n\n");
27              exit(0);
28          }
29          else if(argc == 2 && strncmp(argv[1], "RGB", 3) == 0){
30              unsigned int i = 0, color_index = 0;
31              unsigned int
    colors[]={0xFF0000,0x00FF00,0x0000FF,0xFFFF00,0xFF00FF,0x00FFFF,0xFFFFFF};
32              for(color_index=0; color_index < sizeof(colors)/sizeof(colors[0]);
    color_index++){
33                  for(i=0;i<strip.getLedCount();i++){
34                      strip.setLedColor(i, colors[color_index]);
35                      usleep(10000);
36                  }
37                  strip.show();
38                  usleep(500000);
39              }
40          }
41          else if(argc == 2 && strncmp(argv[1], "Rainbow", 7) == 0){
42              for (int j = 0; j < 255; j += 2) {
43                  for (int i = 0; i < strip.getLedCount(); i++) {
44                      strip.setLedColorData(i, strip.Wheel((i * 256 /
    strip.getLedCount() + j) & 255));
45                  }
46                  strip.show();
47                  usleep(10000);
48              }
49          }
50          else if(argc == 2 && strncmp(argv[1], "Breathing", 9) == 0){
```

```
51              unsigned int
    colors[]={0xFF0000,0x00FF00,0x0000FF,0xFFFF00,0xFF00FF,0x00FFFF,0xFFFFFF};
52              for(unsigned int color_index=0; color_index <
    sizeof(colors)/sizeof(colors[0]); color_index++){
53                  for(unsigned int br=0;br<50;br++){
54                      strip.setBrightness(br);
55                      strip.setAllLedsColorData(colors[color_index]);
56                      strip.show();
57                      usleep(10000);
58                  }
59                  for(unsigned int br=50;br>0;br--){
60                      strip.setBrightness(br);
61                      strip.setAllLedsColorData(colors[color_index]);
62                      strip.show();
63                      usleep(10000);
64                  }
65              }
66          }
67      }
68      return 0;
69  }
```

Include "Freenove_WS2812_SPI.h".

```
    #include "Freenove_WS2812_SPI.h"
```

To create Ledpixel objects, you can use the default parameters or you can use custom parameters.

```
    //Freenove_WS2812_SPI strip = Freenove_WS2812_SPI(8, TYPE_GRB);//led_count, led_type
    Freenove_WS2812_SPI strip = Freenove_WS2812_SPI();//led_count=8, led_type=TYPE_GRB
```

Create an interrupt event if the user presses Ctrl+C to turn off the lights before exiting the program.

```
    void Ctrl_C_Handler(int value){
        strip.end();
        exit(0);
    }


        //Ctrl+C Event
        signal(SIGINT, Ctrl_C_Handler);
```

Initialize ledpixel, set the brightness value of the color light to 20, set the type of the color light to GRB type.

```
        //Init ledpixel
        strip.begin();
        strip.setBrightness(20);
        strip.setLedType((LED_TYPE)TYPE_GRB);
```

Determine the number of parameters when the executable file is invoked. If no parameters are specified, a message is displayed.

```
        //some tips
        if(argc == 1){
            printf("The use of ledpixel:\n");
            printf("  please enter ./main RGB\n");
            printf("  please enter ./main Rainbow\n");
            printf("  please enter ./main Breathing\n\n");
            exit(0);
        }
```

If the parameter is RGB, let the ledpixel display color one by one, and then switch to another color in turn, and continue to repeat the process.

```
        else if(argc == 2 && strncmp(argv[1], "RGB", 3) == 0){
            unsigned int i = 0, color_index = 0;
            unsigned int
colors[]={0xFF0000,0x00FF00,0x0000FF,0xFFFF00,0xFF00FF,0x00FFFF,0xFFFFFF};
            for(color_index=0; color_index < sizeof(colors)/sizeof(colors[0]);
color_index++){
                for(i=0;i<strip.getLedCount();i++){
                    strip.setLedColor(i, colors[color_index]);
                    usleep(10000);
                }
                strip.show();
                usleep(500000);
            }
        }
```

If the parameter is Rainbow, let the ledpixel display a rainbow of colors and turn slowly.

```
        else if(argc == 2 && strncmp(argv[1], "Rainbow", 7) == 0){
            for (int j = 0; j < 255; j += 2) {
                for (int i = 0; i < strip.getLedCount(); i++) {
                    strip.setLedColorData(i, strip.Wheel((i * 256 /
strip.getLedCount() + j) & 255));
                }
                strip.show();
                usleep(10000);
            }
        }
```

If the parameter is Breathing, let the ledpixel gradually turn on one color, slowly turn off, then switch to another color, and repeat the process.

```
        else if(argc == 2 && strncmp(argv[1], "Breathing", 9) == 0){
            unsigned int
colors[]={0xFF0000,0x00FF00,0x0000FF,0xFFFF00,0xFF00FF,0x00FFFF,0xFFFFFF};
            for(unsigned int color_index=0; color_index <
sizeof(colors)/sizeof(colors[0]); color_index++){
```

```
                for(unsigned int br=0;br<50;br++){
                    strip.setBrightness(br);
                    strip.setAllLedsColorData(colors[color_index]);
                    strip.show();
                    usleep(10000);
                }
                for(unsigned int br=50;br>0;br--){
                    strip.setBrightness(br);
                    strip.setAllLedsColorData(colors[color_index]);
                    strip.show();
                    usleep(10000);
                }
            }
        }
```

Python Code 15.3 SpiLedpixel

Use cd command to enter 15_3_Ledpixel directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15_3_Ledpixel
```

Use python command to execute code "Ledpixel.py".

```
python Ledpixel.py
```

After the program runs, ledpixel emits red, green, and blue three colors in turn. And then the colors of the rainbow.

The following is the program code:

```
1  from SPI_Ledpixel import Freenove_SPI_LedPixel
2  import time
3
4  # Main program logic follows:
5  if __name__ == '__main__':
6      # Freenove_SPI_LedPixel(led_count, led_brightness, led_transmission_sequence, spidev_bus)
7      led = Freenove_SPI_LedPixel(8, 255, 'GRB', 0)
8
9      try:
10         if led.check_spi_state() != 0:
11             led.set_led_count(8)                          # Set the number of lights.
12             led.set_led_brightness(20)                    # Set the brightness of lights.
13             color = [[255,0,0],[0,255,0],[0,0,255],[0,0,0]]   # Set the color of the lights
14             for j in range(4):
15                 for i in range(8):
16                     # Set the color of the lights, but it does not take effect
17                     led.set_led_rgb_data(i, color[j])
18                     # Send the color data and make the color data effective
19                     led.show()
20                     time.sleep(0.1)
21
```

```
22                  while True:
23                      for j in range(255):
24                          for i in range(led.led_count):
25                              #Converts values ranging from 0 to 255 to color data.
26                              led.set_led_rgb_data(i, led.wheel((round(i * 255 / led.led_count) + j)%256))
27                          led.show()
28                          time.sleep(0.002)
29              else:
30                  led.led_close()
31      except KeyboardInterrupt:
32          led.led_close()
```

Call the light library and the time library.

```
from SPI_Ledpixel import Freenove_SPI_LedPixel
import time
```

Apply for a light object, set the number of lights to 8, brightness to 255, light type to "GRB", use the mosi pin of spi0 to control the lights.

```
    # Freenove_SPI_LedPixel(led_count, led_brightness, led_transmission_sequence, spidev_bus)
    led = Freenove_SPI_LedPixel(8, 255, 'GRB', 0)
```

Check whether the SPI is configured successfully.

```
            if led.check_spi_state() != 0:
```

Reconfigure the number and brightness of ledpixel.

```
                led.set_led_count(8)                    # Set the number of lights.
                led.set_led_brightness(20)              # Set the brightness of lights.
```

Let ledpixel display red, green and blue one by one, then turn off.

```
                color = [[255,0,0],[0,255,0],[0,0,255],[0,0,0]]   # Set the color of the lights
                for j in range(4):
                    for i in range(8):
                        # Set the color of the lights, but it does not take effect
                        led.set_led_rgb_data(i, color[j])
                        # Send the color data and make the color data effective
                        led.show()
                        time.sleep(0.1)
```

Let ledpixel cycle rainbow colors.

```
                while True:
                    for j in range(255):
                        for i in range(led.led_count):
                            #Converts values ranging from 0 to 255 to color data.
                            led.set_led_rgb_data(i, led.wheel((round(i * 255 / led.led_count) + j)%256))
                        led.show()
                        time.sleep(0.002)
```

For more details, see Freenove_Kit/Code/Python_GPIOZero_Code/SPI_Ledpixel.py.

Chapter 16 74HC595 & Bar Graph LED

We have used LED Bar Graph to make a flowing water light, in which 10 GPIO ports of RPi are occupied. More GPIO ports mean that more peripherals can be connected to RPi, so GPIO resource is very precious. Can we make flowing water light with less GPIO ports? In this chapter, we will learn a component, 74HC595, which can achieve the target.

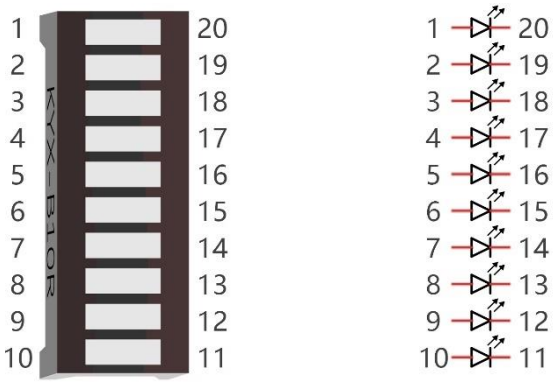
Project 16.1 Flowing Water Light

Now let us learn how to use the 74HC595 IC Chip to make a flowing water light using less GPIO.

Component knowledge

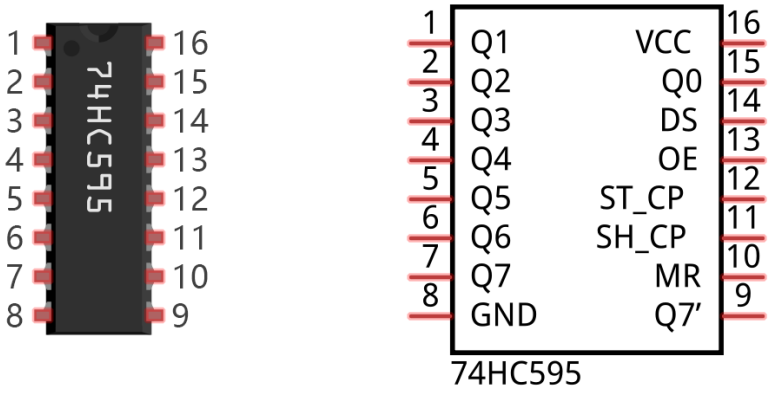
Bar Graph LED

A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.



74HC595

A 74HC595 chip is used to convert serial data into parallel data. A 74HC595 chip can convert the serial data of one byte into 8 bits, and send its corresponding level to each of the 8 ports correspondingly. With this characteristic, the 74HC595 chip can be used to expand the IO ports of a Raspberry Pi. At least 3 ports on the RPI board are required to control the 8 ports of the 74HC595 chip.



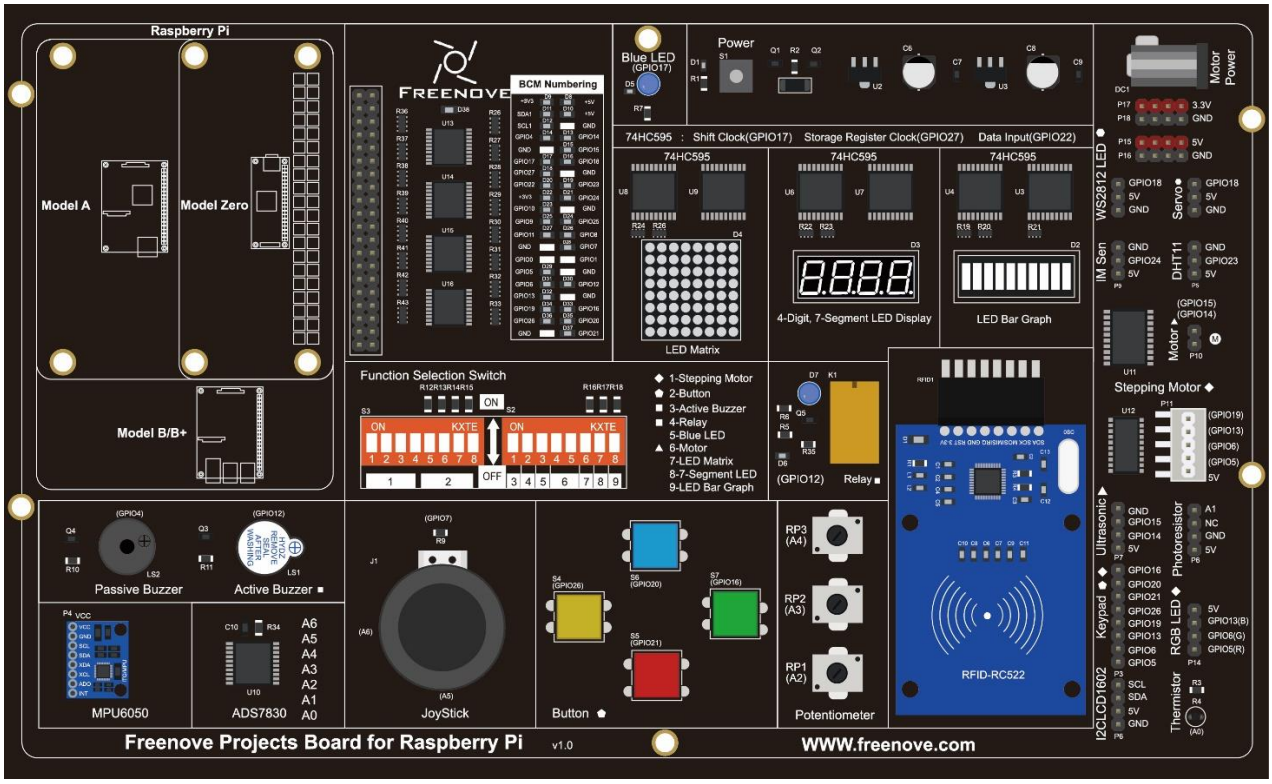
The ports of the 74HC595 chip are described as follows:

Pin name	Pin number	Description
Q0-Q7	15, 1-7	Parallel Data Output
VCC	16	The Positive Electrode of the Power Supply, the Voltage is 2~6V
GND	8	The Negative Electrode of Power Supply
DS	14	Serial Data Input
OE	13	Enable Output, When this pin is in high level, Q0-Q7 is in high resistance state When this pin is in low level, Q0-Q7 is in output mode
ST_CP	12	Parallel Update Output: when its electrical level is rising, it will update the parallel data output.
SH_CP	11	Serial Shift Clock: when its electrical level is rising, serial data input register will do a shift.
MR	10	Remove Shift Register: When this pin is in low level, the content in shift register will be cleared.
Q7'	9	Serial Data Output: it can be connected to more 74HC595 chips in series.

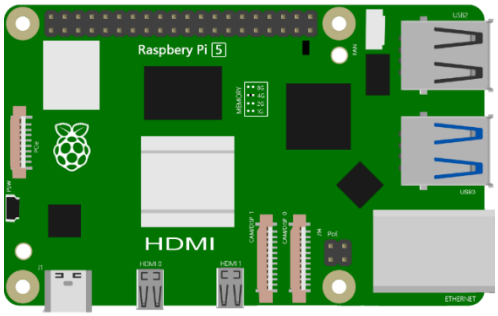
For more details, please refer to the datasheet on the 74HC595 chip.

Component List

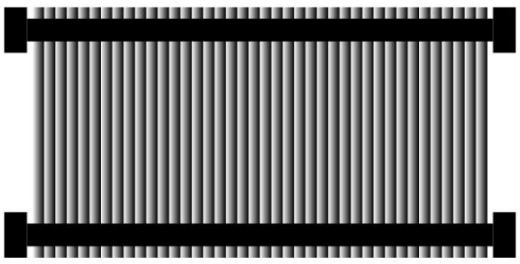
Freenove Projects Board for Raspberry Pi



Raspberry Pi



GPIO Ribbon Cable

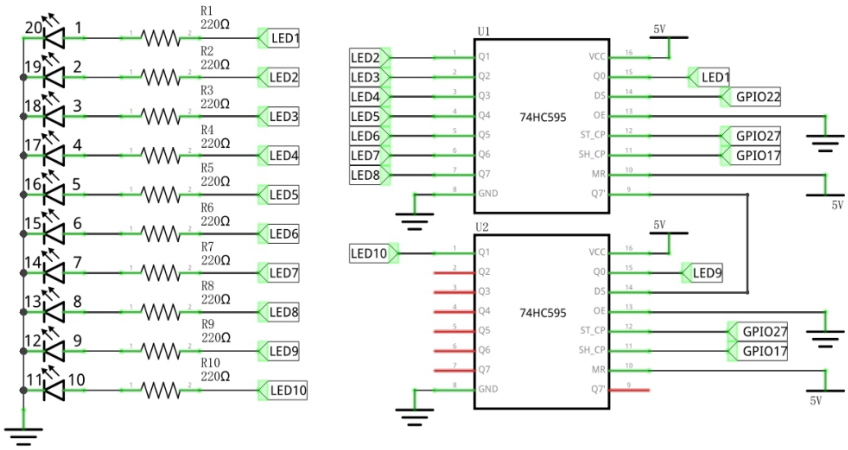


Bar Graph LED



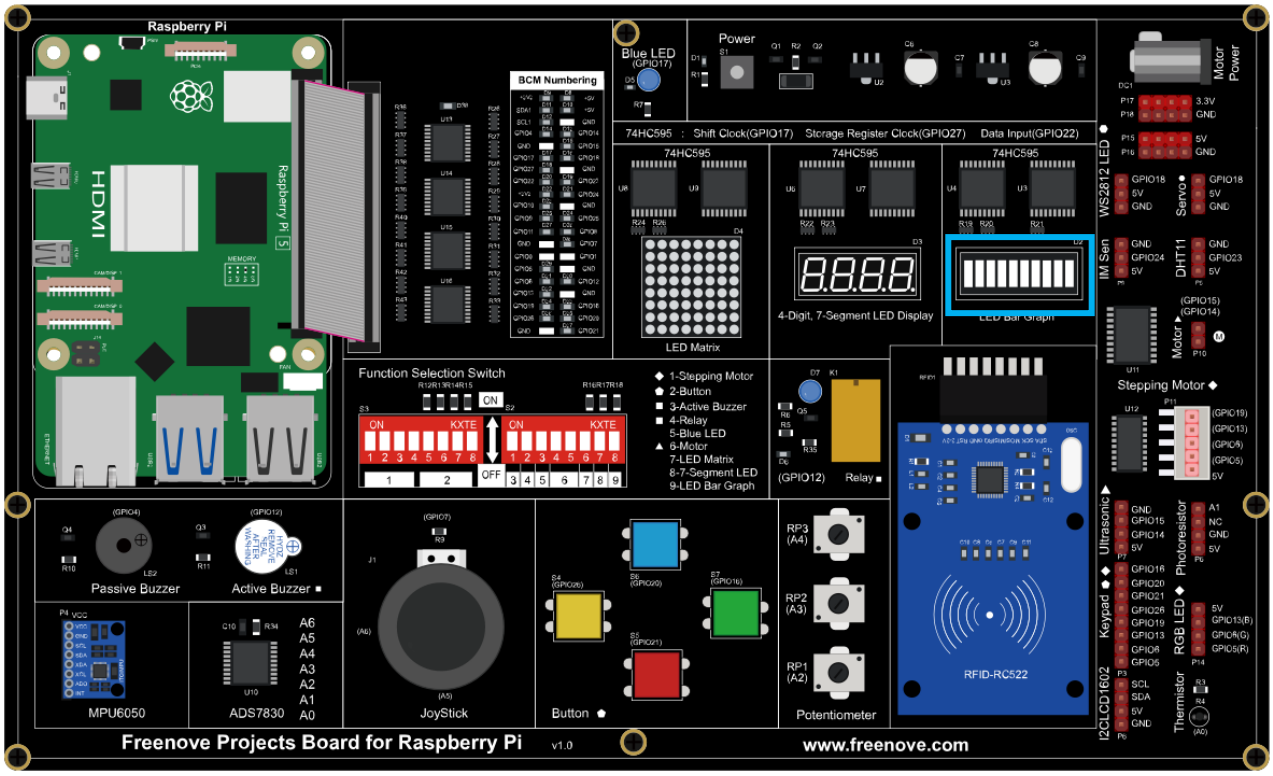
Circuit

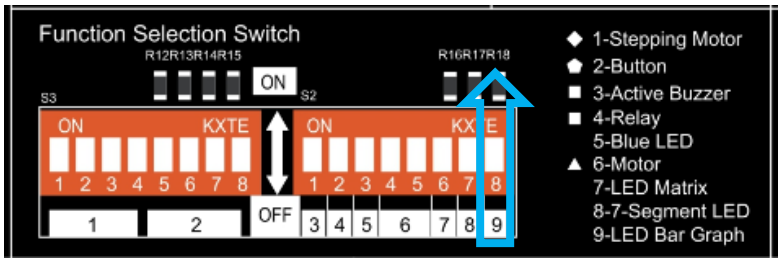
Schematic diagram



Hardware connection.

If it dosen't work, rotate the LED bar graph for 180°.





If you have any concerns, please send an email to: support@freenove.com

Code

In this project we will make a flowing water light with a 74HC595 chip to learn about its functions.

C Code 16.1 LightWater02

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 16_FlowingLight02 directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/16_FlowingLight02
```

2. Use following command to compile "FlowingLight02.c" and generate executable file "FlowingLight02".

```
gcc FlowingLight02.c -o FlowingLight02 -lwiringPi
```

3. Then run the generated file "FlowingLight02".

```
./FlowingLight02
```

After the program runs, you will see that Bar Graph LED starts with the flowing water pattern flashing from right to left and then back from left to right.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <wiringShift.h>

#define   dataPin   22   //DS Pin of 74HC595(Pin14)
#define   latchPin  27   //ST_CP Pin of 74HC595(Pin12)
#define   clockPin  17   //CH_CP Pin of 74HC595(Pin11)

void _shiftOut(int dPin,int cPin,int order,int val){
    int i;
    for(i = 0; i < 10; i++){
        digitalWrite(cPin,LOW);
        if(order == LSBFIRST){
            digitalWrite(dPin,((0x01&(val>>i)) == 0x01) ? HIGH : LOW);
            delayMicroseconds(10);
        }
        else {
            digitalWrite(dPin,((0x80&(val<<i)) == 0x80) ? HIGH : LOW);
            delayMicroseconds(10);
        }
        digitalWrite(cPin,HIGH);
        delayMicroseconds(10);
    }
}

int main(void)
```

```
{
    int i;
    unsigned long x;

    printf("Program is starting ...\n");

    wiringPiSetupGpio();

    pinMode(dataPin,OUTPUT);
    pinMode(latchPin,OUTPUT);
    pinMode(clockPin,OUTPUT);
    while(1){
        x=0x0001;
        for(i=0;i<10;i++){
            digitalWrite(latchPin,LOW);         // Output low level to latchPin
            _shiftOut(dataPin,clockPin,LSBFIRST,x);// Send serial data to 74HC595
            digitalWrite(latchPin,HIGH);   //Output high level to latchPin, and 74HC595 will
update the data to the parallel output port.
            x<<=1;      //make the variable move one bit to left once, then the bright LED
move one step to the left once.
            delay(100);
        }
        x=0x0200;
        for(i=0;i<10;i++){
            digitalWrite(latchPin,LOW);
            _shiftOut(dataPin,clockPin,LSBFIRST,x);
            digitalWrite(latchPin,HIGH);
            x>>=1;
            delay(100);
        }
    }
    return 0;
}
```

In the code, we configure three pins to control the 74HC595 chip and define a one-byte variable to control the state of the 10 LEDs (in the Bar Graph LED Module) through the 10 bits of the variable. The LEDs light ON when the corresponding bit is 1. If the variable is assigned to 0x01, that is 00000001 in binary, there will be only one LED ON.

```
x=0x0001;
```

In the “while” loop of main function, use two loops to send x to 74HC595 output pin to control the LED. In one cycle, x will shift one bit to the LEFT in one cycle, then when data of x is sent to 74HC595, the LED that is turned ON will move one bit to the LEFT once.

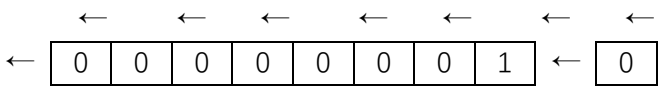
```
        for(i=0;i<10;i++){
            digitalWrite(latchPin,LOW);         // Output low level to latchPin
            _shiftOut(dataPin,clockPin,LSBFIRST,x);// Send serial data to 74HC595
            digitalWrite(latchPin,HIGH);    //Output high level to latchPin, and 74HC595 will update the data to the parallel output port.
            x<<=1;      //make the variable move one bit to left once, then the bright LED move one step to the left once.
            delay(100);
        }
```

In second cycle, the situation is the same. The difference is that x is shift from 0x80 to the RIGHT in order.

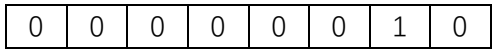
<< operator

"<<" is the left shift operator, which can make all bits of 1 byte shift by several bits to the left (high) direction and add 0 on the right (low). For example, shift binary 00000001 by 1 bit to left:

byte x = 1 << 1;

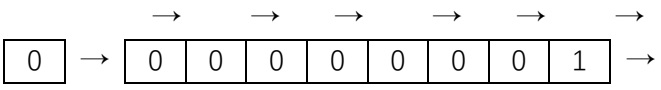


The result of x is 2（binary 00000010）.

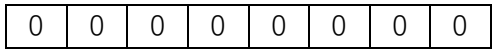


There is another similar operator" >>". For example, shift binary 00000001 by 1 bit to right:

byte x = 1 >> 1;



The result of x is 0（00000000）.



X <<= 1 is equivalent to x = x << 1 and x >>= 1 is equivalent to x = x >> 1

About shift function

void _shiftOut (uint8_t dPin, uint8_t cPin, uint8_t order, uint8_t val) ;

This is used to shift a 10-bit data value out with the data being sent out on dPin and the clock being sent out on the cPin. order is as above. Data is clocked out on the rising or falling edge - ie. dPin is set, then cPin is taken high then low - repeated for the 10 bits.

Python Code 16.1 LightWater02

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 16_FlowingLight02 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/16_FlowingLight02
```

2. Use python command to execute Python code "FlowingLight02.py".

```
python FlowingLight02.py
```

After the program runs, you will see that Bar Graph LED starts with the flowing water pattern flashing from right to left and then back from left to right.

The following is the program code:

```
from gpiozero import OutputDevice
import time
# Defines the data bit that is transmitted preferentially in the shiftOut function.
LSBFIRST = 1
MSBFIRST = 2
# define the pins for 74HC595
dataPin   = OutputDevice(22)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(17)      # CH_CP Pin of 74HC595(Pin11)

# shiftOut function, use bit serial transmission.
def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def showLedbar(order,val):
    val1 = val >> 8 & 0xff
    val2 = val & 0xff
    shiftOut(order, val1)
    shiftOut(order, val2)

def loop():
    while True:
        x=0x001
        for i in range(0,10):
            latchPin.off()# Output low level to latchPin
            showLedbar(MSBFIRST,x) # Send serial data to 74HC595
            latchPin.on()    # Output high level to latchPin, and 74HC595 will update the data
to the parallel output port.
```

```
            x<<=1 # make the variable move one bit to left once, then the bright LED move one
step to the left once.
            time.sleep(0.1)
        x=0x200
        for i in range(0,10):
            latchPin.off()
            showLedbar(MSBFIRST,x)
            latchPin.on()
            x>>=1
            time.sleep(0.1)

def destroy():
    latchPin.off()
    shiftOut(MSBFIRST, 0x00)
    shiftOut(MSBFIRST, 0x00)
    latchPin.on()
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

In the code, we define a shiftOut() function, which is used to output values with bits in order, where the dPin for the data pin, cPin for the clock and order for the priority bit flag (high or low). This function conforms to the operational modes of the 74HC595. LSBFIRST and MSBFIRST are two different flow directions.

```
def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()
```

In the loop() function, we use two loops to achieve the action goal. First, define a variable x=0x0001. When it is transferred to the output port of 74HC595, the low bit outputs high level, then an LED turns ON. Next, x is shifted one bit, when x is transferred to the output port of 74HC595 once again, the LED that turns ON will be shifted. Repeat the operation, over and over and the effect of a flowing water light will be visible. If the direction of the shift operation for x is different, the flowing direction is different.

```
def loop():
    while True:
        x=0x001
        for i in range(0,10):
            latchPin.off()# Output low level to latchPin
            showLedbar(MSBFIRST,x) # Send serial data to 74HC595
            latchPin.on()    # Output high level to latchPin, and 74HC595 will update the data to the parallel output port.
            x<<=1 # make the variable move one bit to left once, then the bright LED move one step to the left once.
            time.sleep(0.1)
        x=0x200
        for i in range(0,10):
            latchPin.off()
            showLedbar(MSBFIRST,x)
            latchPin.on()
            x>>=1
            time.sleep(0.1)
```

Chapter 17 74HC595 & 4-Digit 7-Segment Display

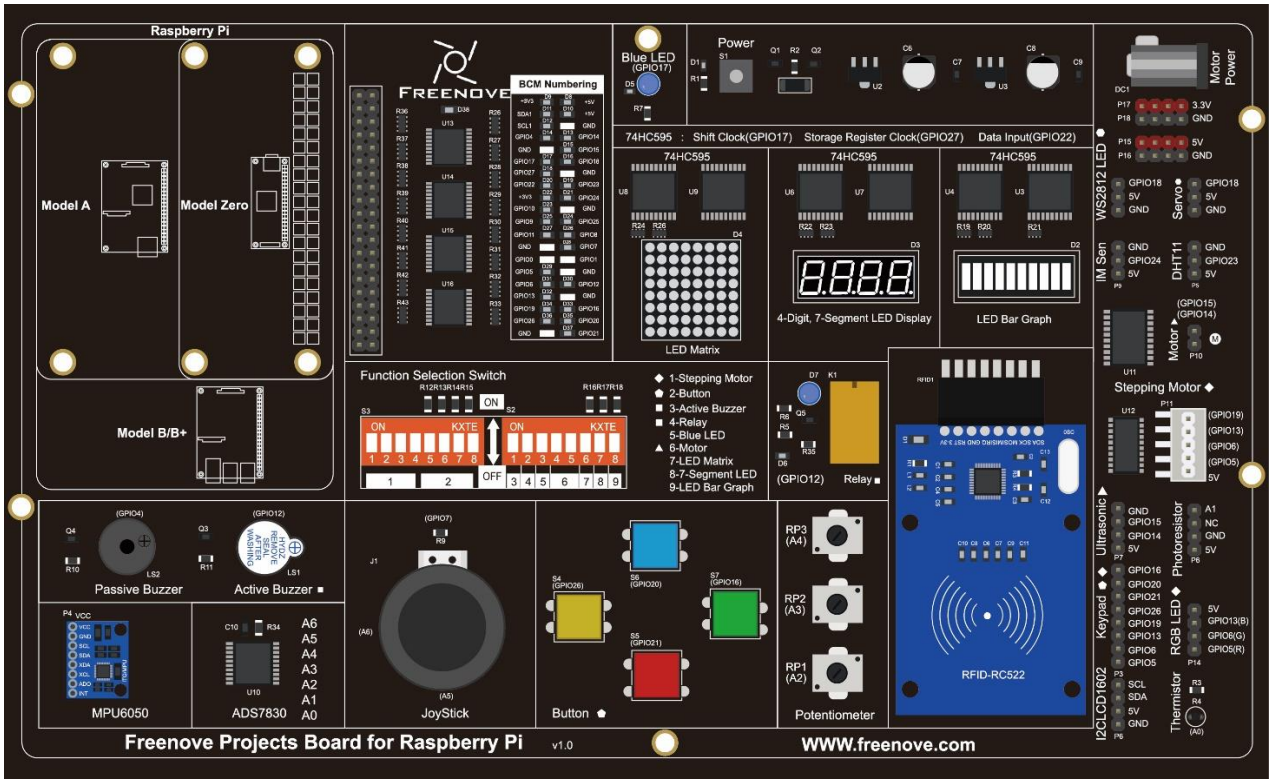
In this chapter, we will introduce the 7-Segment Display.

Project 17.1 4-Digit 7-Segment Display

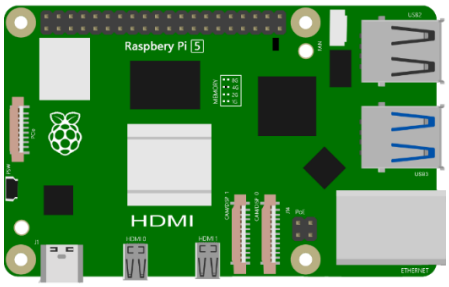
We will use a 74HC595 IC Chip to control a 4-Digit 7-Segment Display and make it display sixteen decimal characters "0" to “F".

Component List

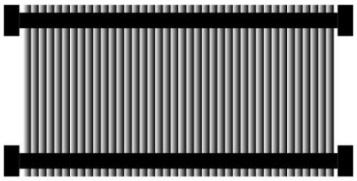
Freenove Projects Board for Raspberry Pi



Raspberry Pi



GPIO Ribbon Cable

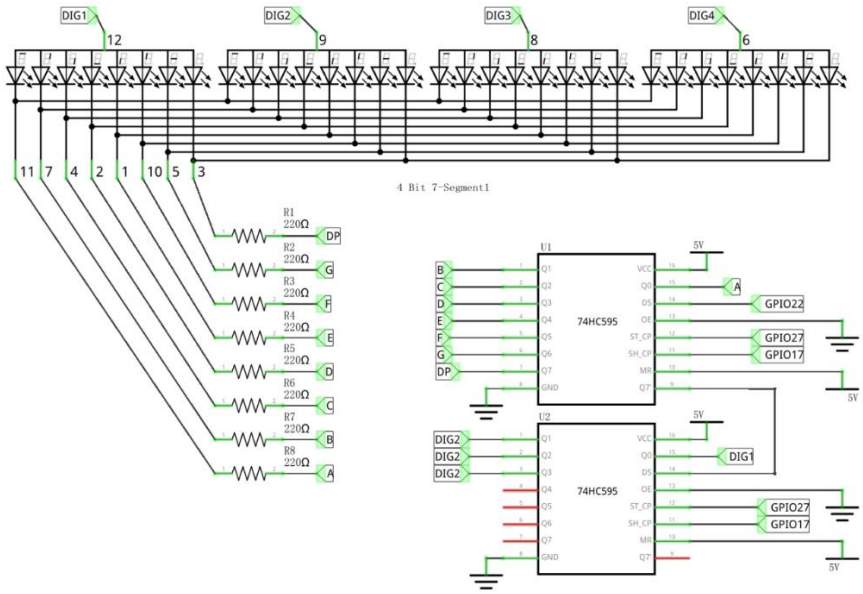


4-Digit 7-Segment Display

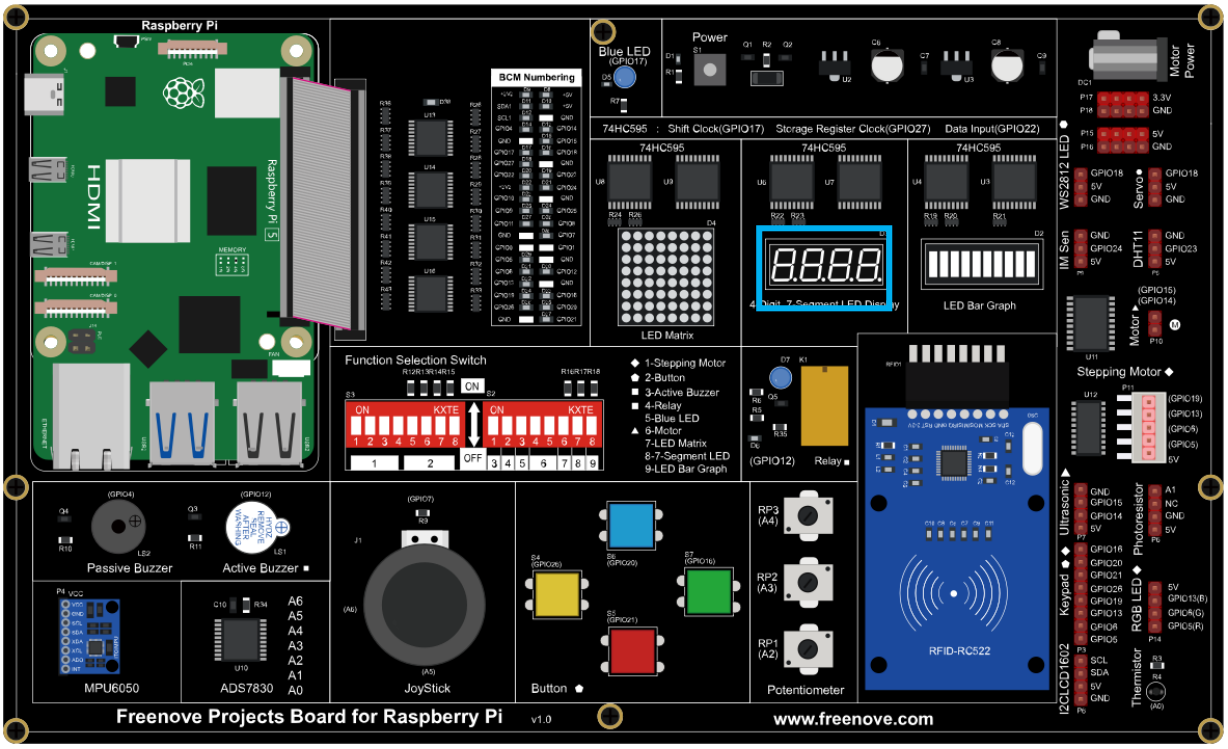


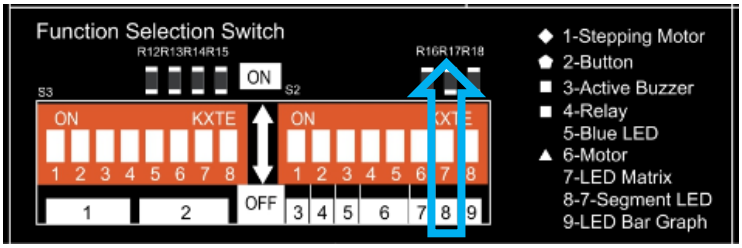
Circuit

Schematic diagram



Hardware connection.

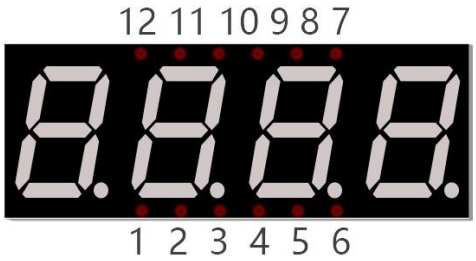




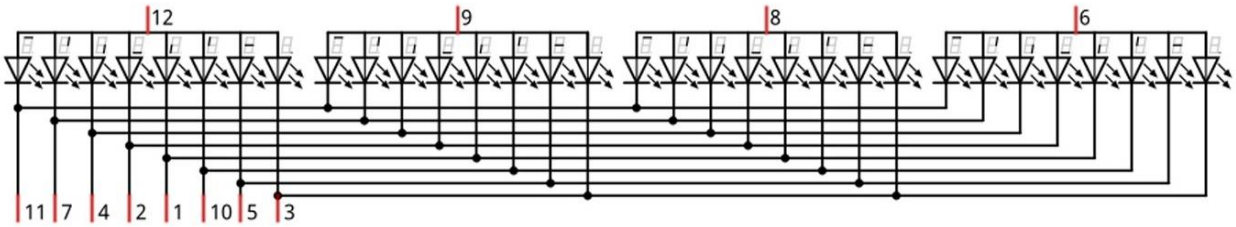
Component knowledge

4 Digit 7-Segment Display

A 4 Digit 7-segment display integrates four 7-Segment Displays into one module, therefore it can display more characters. All of the LEDs contained have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



The internal electronic circuit is shown below, and all 8 LED cathode pins of each 7-Segment Display are connected together.



Display method of 4 Digit 7-segment display is similar to 1 Digit 7-segment display. The difference between them is that the 4-Digit displays each Digit is visible in turn, one by one and not together. We need to first send high level to the common end of the first Digit Display, and send low level to the remaining three common ends, and then send content to 8 LED cathode pins of the first Digit Display. At this time, the first 7-Segment Display will show visible content and the remaining three will be OFF.

Similarly, the second, third and fourth 7-Segment Displays will show visible content in turn by scanning the display. Although the four number characters are displayed in turn separately, this process is so fast that it is unperceivable to the naked eye. This is due to the principle of optical afterglow effect and the vision persistence effect in human sight. This is how we can see all 4 number characters at the same time.

However, if each number character is displayed for a longer period, you will be able to see that the number characters are displayed separately.

Code

This code uses a 74HC595 IC Chip to control the 4-Digit 7-Segment Display. The use of the 74HC595 IC Chip is generally the same throughout this Tutorial. We need code to display the characters "0" to "F" one character at a time, and then output to display them with the 74HC595 IC Chip.

C Code 17.1 SevenSegmentDisplay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 17_1_SevenSegmentDisplay directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/17_1_SevenSegmentDisplay
```

2. Use following command to compile "SevenSegmentDisplay.c" and generate executable file "SevenSegmentDisplay".

```
gcc SevenSegmentDisplay.c -o SevenSegmentDisplay -lwiringPi
```

3. Then run the generated file "SevenSegmentDisplay".

```
./SevenSegmentDisplay
```

After the program runs, the 4-Digit 7-Segment Display starts to display the characters "0" to "F" in succession.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <wiringShift.h>

#define   dataPin   22   //DS Pin of 74HC595(Pin14)
#define   latchPin  27   //ST_CP Pin of 74HC595(Pin12)
#define   clockPin  17   //CH_CP Pin of 74HC595(Pin11)
//encoding for character 0-F of common anode SevenSegmentDisplay.
unsigned long
num[]={0xffc0,0xfff9,0xffa4,0xffb0,0xff99,0xff92,0xff82,0xfff8,0xff80,0xff90,0xff88,0xff83,0xf
fc6,0xffa1,0xff86,0xff8e};

void _shiftOut(int dPin,int cPin,int order,int val){
    int i;
    for(i = 0; i < 16; i++){
        digitalWrite(cPin,LOW);
        if(order == LSBFIRST){
            digitalWrite(dPin,((0x01&(val>>i)) == 0x01) ? HIGH : LOW);
            delayMicroseconds(10);
        }
        else {
            digitalWrite(dPin,((0x8000&(val<<i)) == 0x8000) ? HIGH : LOW);
            delayMicroseconds(10);
```

```
24              }
25              digitalWrite(cPin,HIGH);
26              delayMicroseconds(10);
27          }
28      }
29
30      int main(void)
31      {
32          int i;
33
34          printf("Program is starting ...\n");
35
36          wiringPiSetupGpio();
37
38          pinMode(dataPin,OUTPUT);
39          pinMode(latchPin,OUTPUT);
40          pinMode(clockPin,OUTPUT);
41          while(1){
42              for(i=0;i<sizeof(num);i++){
43                  digitalWrite(latchPin,LOW);
44                  _shiftOut(dataPin,clockPin,MSBFIRST,num[i]);//Output the figures and the highest
45      level is transfered preferentially.
46                  digitalWrite(latchPin,HIGH);
47                  delay(500);
48              }
49          }
50          return 0;
51      }
```

First, we need to create encoding for characters “0” to “F” in the array.

```
unsigned long
num[]={0xffc0,0xfff9,0xffa4,0xffb0,0xff99,0xff92,0xff82,0xfff8,0xff80,0xff90,0xff88,0xff83,0xffc6
,0xffa1,0xff86,0xff8e};
```

In the “for” loop of loop() function, use the 74HC595 IC Chip to output contents of array “num” successively. SevenSegmentDisplay can then correctly display the corresponding characters.

```
while(1){
            for(i=0;i<sizeof(num);i++){
                digitalWrite(latchPin,LOW);
                _shiftOut(dataPin,clockPin,MSBFIRST,num[i]);//Output the figures and the highest
level is transfered preferentially.
                digitalWrite(latchPin,HIGH);
                delay(500);
            }
        }
```

Python Code 17.1 SevenSegmentDisplay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 17_1_SevenSegmentDisplay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/17_1_SevenSegmentDisplay
```

2. Use Python command to execute Python code "SevenSegmentDisplay.py".

```
python SevenSegmentDisplay.py
```

After the program runs, the 4-Digit 7-Segment Display starts to display the characters "0" to "F" in succession.

The following is the program code:

```
from gpiozero import OutputDevice
import time

LSBFIRST = 1
MSBFIRST = 2
# define the pins for 74HC595
dataPin   = OutputDevice(22)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(17)      # CH_CP Pin of 74HC595(Pin11)
# SevenSegmentDisplay display the character "0"- "F" successively
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def showSevenSegmentDisplay(bits, values):
    shiftOut(MSBFIRST, bits)
    shiftOut(MSBFIRST, values)

def loop():
    bits = 0x0f
    while True:
        for i in range(0,len(num)):
            latchPin.off()
            showSevenSegmentDisplay(bits, num[i])  # Send serial data to 74HC595
            latchPin.on()
```

```
            time.sleep(0.5)
        for i in range(0,len(num)):
            latchPin.off()
            showSevenSegmentDisplay(bits, num[i]&0x7f)
            latchPin.on()
            time.sleep(0.5)

def destroy():
    latchPin.off()
    shiftOut(MSBFIRST, 0x00)
    shiftOut(MSBFIRST, 0x00)
    latchPin.on()
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        destroy()
```

First, we need to create encoding for characters "0" to "F" in the array.

```
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

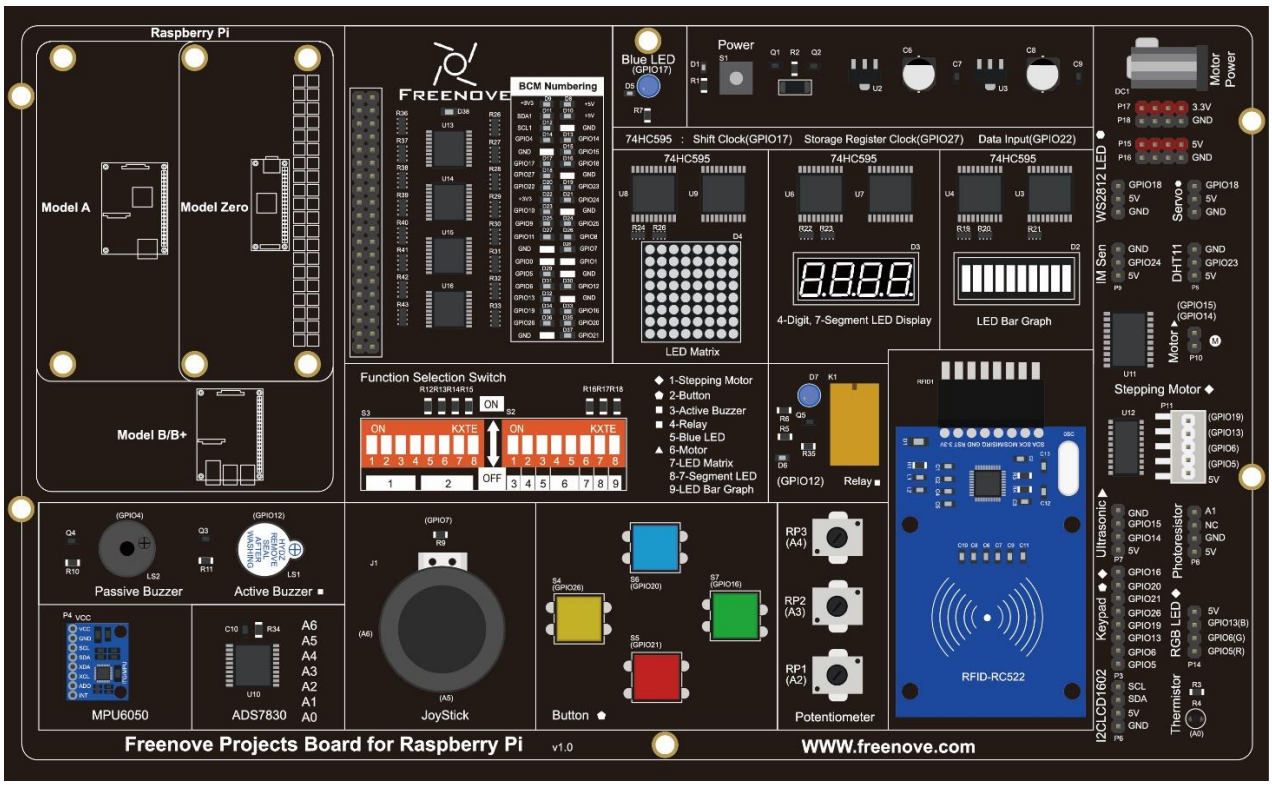
In the for loop of the loop() function, the 74HC595 IC chip is used to output the position of the ninumber in turn, and the contents of the num array. You can run the showSevenSegmentDisplay command to correctly display the corresponding characters.

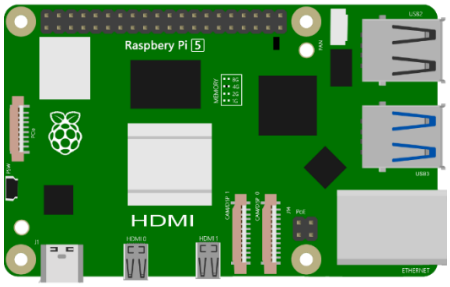
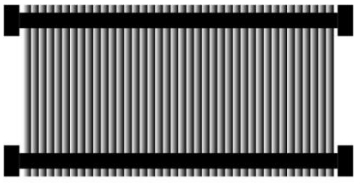
```
    bits = 0x0f
    while True:
        for i in range(0,len(num)):
            latchPin.off()
            showSevenSegmentDisplay(bits, num[i])  # Send serial data to 74HC595
            latchPin.on()
            time.sleep(0.5)
        for i in range(0,len(num)):
            latchPin.off()
            showSevenSegmentDisplay(bits, num[i]&0x7f)
            latchPin.on()
            time.sleep(0.5)
```

Project 17.2 4-Digit 7-Segment Display

Component List

Freenove Projects Board for Raspberry Pi



Raspberry Pi	GPIO Ribbon Cable	4-Digit 7-Segment Display
		

Circuit

The same as that of 17.1

If you have any concerns, please send an email to: support@freenove.com

Code

In this code, we use the 74HC595 IC Chip to control the 4-Digit 7-Segment Display, and use the dynamic scanning method to show the changing number characters.

C Code 17.2 StopWatch

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 17_2_StopWatch directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/17_2_StopWatch
```

2. Use following command to compile "StopWatch.c" and generate executable file "StopWatch".

```
gcc StopWatch.c -o StopWatch -lwiringPi
```

3. Run the generated file "SteppingMotor".

```
./StopWatch
```

After the program runs, the 4-Digit 7-Segment Display starts displaying a four-digit number dynamically, and the numeric value of this number will increase by plus 1 each second thereafter.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <wiringShift.h>
#include <signal.h>
#include <unistd.h>
#define  dataPin     22   //DS Pin of 74HC595(Pin14)
#define  latchPin    27   //ST_CP Pin of 74HC595(Pin12)
#define  clockPin    17   //CH_CP Pin of 74HC595(Pin11)
// character 0-9 code of common anode 7-segment display
unsigned char num[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};
int counter = 0;    //variable counter,the number will be displayed by 7-segment display
//Open one of the 7-segment display and close the remaining three, the parameter digit is
optional for 1,2,4,8
unsigned long selectDigit(unsigned long digit){
    if (digit==0x01){
        return (0x08<<8);
        }
    else if (digit==0x02){
        return (0x04<<8);
        }
    else if (digit==0x04){
        return (0x02<<8);
        }
    else if (digit==0x08){
        return (0x01<<8);
```

```
26          }
27      else{
28          return (0xf0<<8);
29          }
30
31  }
32  void _shiftOut(int dPin,int cPin,int order,int val){
33      int i;
34      for(i = 0; i < 16; i++){
35          digitalWrite(cPin,LOW);
36          if(order == LSBFIRST){
37              digitalWrite(dPin,((0x01&(val>>i)) == 0x01) ? HIGH : LOW);
38              delayMicroseconds(1);
39           }
40          else {//if(order == MSBFIRST){
41              digitalWrite(dPin,((0x8000&(val<<i)) == 0x8000) ? HIGH : LOW);
42              delayMicroseconds(1);
43           }
44          digitalWrite(cPin,HIGH);
45          delayMicroseconds(1);
46      }
47  }
48  void outData(unsigned long data){      //function used to output data for 74HC595
49      digitalWrite(latchPin,LOW);
50      _shiftOut(dataPin,clockPin,MSBFIRST,data);
51      digitalWrite(latchPin,HIGH);
52  }
53  void display(int dec){  //display function for 7-segment display
54      int delays = 1;
55      unsigned long  digit;
56      outData(0xffff);
57      digit=selectDigit(0x01);      //select the first, and display the single digit
58      outData(num[dec%10]|digit);
59      delay(delays);          //display duration
60
61      outData(0xffff);
62      digit=selectDigit(0x02);      //select the second, and display the tens digit
63      outData(num[dec%100/10]|digit);
64      delay(delays);
65
66      outData(0xffff);
67      digit=selectDigit(0x04);      //select the third, and display the hundreds digit
68      outData(num[dec%1000/100]|digit);
69      delay(delays);
```

```
70
71      outData(0xffff);
72      digit=selectDigit(0x08);       //select the fourth, and display the thousands digit
73      outData(num[dec%10000/1000]|digit);
74      delay(delays);
75  }
76  void timer(int  sig){       //Timer function
77      if(sig == SIGALRM){   //If the signal is SIGALRM, the value of counter plus 1, and update
78  the number displayed by 7-segment display
79          counter ++;
80          alarm(1);           //set the next timer time
81          printf("counter : %d \n",counter);
82      }
83  }
84  int main(void)
85  {
86      int i;
87
88      printf("Program is starting ...\n");
89
90      wiringPiSetupGpio();
91
92      pinMode(dataPin,OUTPUT);        //set the pin connected to74HC595 for output mode
93      pinMode(latchPin,OUTPUT);
94      pinMode(clockPin,OUTPUT);
95
96      signal(SIGALRM,timer);  //configure the timer
97      alarm(1);               //set the time of timer to 1s
98      while(1){
99          display(counter);   //display the number counter
100     }
101     return 0;
102 }
```

First, we define the pin of the 74HC595 IC Chip and the 7-Segment Display Common Anode, use character encoding and a variable "counter" to enable the counter to be visible on the 7-Segment Display.

```
#define dataPin   22    //DS Pin of 74HC595(Pin14)
#define latchPin 27    //ST_CP Pin of 74HC595(Pin12)
#define clockPin 17    //CH_CP Pin of 74HC595(Pin11)
// character 0-9 code of common anode 7-segment display
unsigned char num[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};
int counter = 0;     //variable counter,the number will be displayed by 7-segment display
```

Subfunction **selectDigit** (int digit) function is used to open one of the 7-Segment Displays while closing the other 7-Segment Displays, where the parameter digit value can be 1,2,4,8. Using "|" can open a number of a 7-Segment Display.

```
unsigned long selectDigit(unsigned long digit){
    if (digit==0x01){
        return (0x08<<8);
        }
    else if (digit==0x02){
        return (0x04<<8);
        }
    else if (digit==0x04){
        return (0x02<<8);
        }
    else if (digit==0x08){
        return (0x01<<8);
        }
    else{
        return (0xf0<<8);
        }
}
```

Subfunction **outData** (int8_t data) is used to make the 74HC595 IC Chip output a 16-bit data immediately.

```
void outData(int8_t data){      // function used to output data for 74HC595
    digitalWrite(latchPin,LOW);
    shiftOut(dataPin,clockPin,MSBFIRST,data);
    digitalWrite(latchPin,HIGH);
}
```

Subfunction **display** (int dec) is used to make a 4-Digit 7-Segment Display a 4-bit integer. First open the common end of first 7-Segment Display Digit and turn OFF the other three Digits, now it can be used as 1-Digit 7-Segment Display. The first Digit is used for displaying single digits of "dec", the second Digit is for tens, the third for hundreds and fourth for thousands respectively. Each digit will be displayed for a period by using **delay** (). The time in this code is very brief, so you will see digits all together. If the time is set long enough, you will see that every digit is displayed independently.

```
void display(int dec){  //display function for 7-segment display
    int delays = 1;
    unsigned long  digit;
    outData(0xffff);
    digit=selectDigit(0x01);      //select the first, and display the single digit
    outData(num[dec%10]|digit);
    delay(delays);          //display duration

    outData(0xffff);
```

```
    digit=selectDigit(0x02);      //select the second, and display the tens digit
    outData(num[dec%100/10]|digit);
    delay(delays);

    outData(0xffff);
    digit=selectDigit(0x04);      //select the third, and display the hundreds digit
    outData(num[dec%1000/100]|digit);
    delay(delays);

    outData(0xffff);
    digit=selectDigit(0x08);      //select the fourth, and display the thousands digit
    outData(num[dec%10000/1000]|digit);
    delay(delays);
}
```

Subfunction **timer** (int sig) is the timer function, which will set an alarm to signal. This function will be executed once at set time intervals. Accompanied by the execution, "1" will be added as the variable counter and then reset the time of timer to 1s.

```
void timer(int  sig){       //timer function
    if(sig == SIGALRM){   //If the signal is SIGALRM, the value of counter plus 1, and update the number displayed by 7-segment display
        counter ++;
        alarm(1);           //set the next timer time
    }
}
```

Finally, in the main function, configure the GPIO, and set the timer function.

```
    pinMode(dataPin,OUTPUT);        //set the pin connected to74HC595 for output mode
    pinMode(latchPin,OUTPUT);
    pinMode(clockPin,OUTPUT);

    signal(SIGALRM,timer);  //configure the timer
    alarm(1);               //set the time of timer to 1s
```

In the while loop, make the digital display variable counter value "1". The value will change in function timer (), so the content displayed by the 7-Segment Display will change accordingly.

```
    while(1){
        display(counter);   //display number counter
    }
```

Python Code 17.2 StopWatch

This code uses the four-step four-part mode to drive the Stepper Motor clockwise and reverse direction.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 17_2_StopWatch directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/17_2_StopWatch
```

2. Use python command to execute code "StopWatch.py".

```
python StopWatch.py
```

After the program runs, 4-Digit 7-segment start displaying a four-digit number dynamically, and the will plus 1 in each successive second.

The following is the program code:

```
from gpiozero import OutputDevice
import time
import threading

LSBFIRST = 1
MSBFIRST = 2
# define the pins for 74HC595
dataPin   = OutputDevice(22)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(17)      # CH_CP Pin of 74HC595(Pin11)
# SevenSegmentDisplay display the character "0"- "F" successively
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
decimalPoint = 0xff
counter = 0         # Variable counter, the number will be dislayed by 7-segment display
t = None            # define the Timer object

def setDecimalPoint(state):
    global decimalPoint
    if state==1:
        decimalPoint = 0x7f
    elif state==0:
        decimalPoint = 0xff

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def display(values, showTimes=10):
```

```
    bits = [0x01,0x02,0x04,0x08]
    bitValues=[0,0,0,0]
    bitValues[0] = values // 1000
    bitValues[1] = (values // 100) % 10
    bitValues[2] = (values // 10) % 10
    bitValues[3] = values % 10
    for j in range(showTimes):
        for i in range(0,len(bits)):
            latchPin.off()
            shiftOut(MSBFIRST, bits[i])
            shiftOut(MSBFIRST, num[bitValues[i]]&decimalPoint)
            latchPin.on()
            time.sleep(0.0002)
            latchPin.off()
            shiftOut(MSBFIRST, bits[i])
            shiftOut(MSBFIRST, 0xff)
            latchPin.on()
            time.sleep(0.00005)

def timer():
    global counter
    global t
    t = threading.Timer(1.0,timer)      # reset time of timer to 1s
    t.start()                           # Start timing
    counter+=1
    print ("counter : %d"%counter)

def loop():
    global t
    global counter
    setDecimalPoint(0)
    t = threading.Timer(1.0,timer)      # set the timer
    t.start()                           # Start timing
    while True:
        display(counter)                # display the number counter

def destroy():
    global t
    t.cancel()
    latchPin.off()
    shiftOut(MSBFIRST, 0x00)
    shiftOut(MSBFIRST, 0x00)
    latchPin.on()
    dataPin.close()
```

```
78      latchPin.close()
79      clockPin.close()
80
81  if __name__ == '__main__': # Program entrance
82      print ('Program is starting...' )
83      try:
84          loop()
85      except KeyboardInterrupt:  # Press ctrl-c to end the program.
86          print("Ending program")
87      finally:
89          destroy()
```

First, define the pin of 74HC595 and 7-segment display common end, character encoding and a variable "counter" to be displayed counter.

```
# define the pins for 74HC595
dataPin   = OutputDevice(22)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(17)      # CH_CP Pin of 74HC595(Pin11)
# SevenSegmentDisplay display the character "0"- "F" successively
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
decimalPoint = 0xff
counter = 0         # Variable counter, the number will be dislayed by 7-segment display
```

In order for the digital tube to display different numbers, it is necessary for the digital tube to open the first digital tube in a very short time, and then display the code value of the digital tube, and then close it after a delay display for a period of time. Then open the second digital tube and repeat the above operation until the loop has displayed four digital tubes. Then repeat the whole process. So the human eye can see that the nixie tube shows different numbers.

```
def display(values, showTimes=10):
    bits = [0x01, 0x02, 0x04, 0x08]
    bitValues=[0, 0, 0, 0]
    bitValues[0] = values // 1000
    bitValues[1] = (values // 100) % 10
    bitValues[2] = (values // 10) % 10
    bitValues[3] = values % 10
    for j in range(showTimes):
        for i in range(0, len(bits)):
            latchPin.off()
            shiftOut(MSBFIRST, bits[i])
            shiftOut(MSBFIRST, num[bitValues[i]]&decimalPoint)
            latchPin.on()
            time.sleep(0.0002)
            latchPin.off()
            shiftOut(MSBFIRST, bits[i])
```

```
            shiftOut(MSBFIRST, 0xff)
            latchPin.on()
            time.sleep(0.00005)
```

Subfunction **timer** () is the timer callback function. When the time is up, this function will be executed. Accompanied by the execution, the variable counter will be added 1, and then reset the time of timer to 1s. 1s later, the function will be executed again.

```
def timer():
    global counter
    global t
    t = threading.Timer(1.0,timer)      # reset time of timer to 1s
    t.start()                           # Start timing
    counter+=1
    print ("counter : %d"%counter)
```

Subfunction **setup**(), configure all input output modes for the GPIO pin used.
Finally, in loop function, make the digital tube display variable counter value in the while loop. The value will change in function **timer** (), so the content displayed by 7-segment display will change accordingly.

```
def loop():
    global t
    global counter
    t = threading.Timer(1.0,timer)      # set the timer
    t.start()                           # Start timing
    while True:
        display(counter)                # display the number counter
```

After the program runs, press "Ctrl+C", then subfunction destroy() will be executed, and GPIO resources and timers will be released in this subfunction.

```
def destroy():
    global t
    t.cancel()
    latchPin.off()
    shiftOut(MSBFIRST, 0x00)
    shiftOut(MSBFIRST, 0x00)
    latchPin.on()
    dataPin.close()
    latchPin.close()
    clockPin.close()
```

Chapter 18 74HC595 & LED Matrix

Thus far we have learned how to use the 74HC595 IC Chip to control the Bar Graph LED and the 7-Segment Display. We will now use 74HC595 IC Chips to control an LED Matrix.

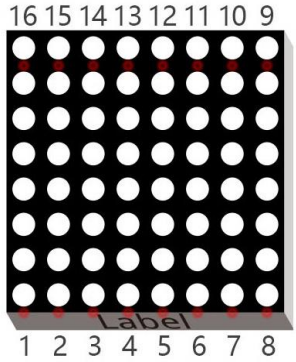
Project 18.1 LED Matrix

In this project, we will use two 74HC595 IC chips to control a monochrome (one color) (8X8) LED Matrix to make it display both simple graphics and characters.

Component knowledge

LED matrix

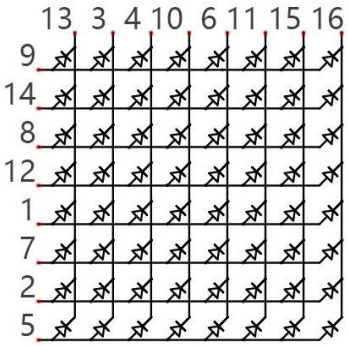
An LED Matrix is a rectangular display module that consists of a uniform grid of LEDs. The following is an 8X8 monochrome (one color) LED Matrix containing 64 LEDs (8 rows by 8 columns).



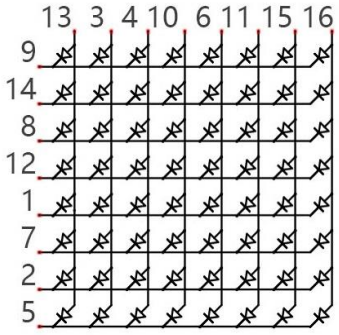
In order to facilitate the operation and reduce the number of ports required to drive this component, the Positive Poles of the LEDs in each row and Negative Poles of the LEDs in each column are respectively connected together inside the LED Matrix module, which is called a Common Anode. There is another arrangement type. Negative Poles of the LEDs in each row and the Positive Poles of the LEDs in each column are respectively connected together, which is called a Common Cathode.

The LED Matrix that we use in this project is a Common Anode LED Matrix.

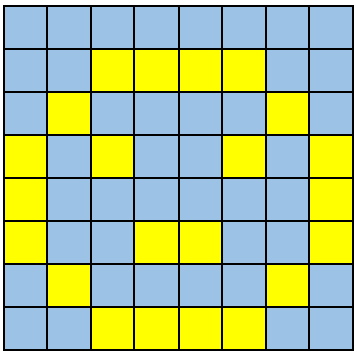
Connection mode of Common Anode



Connection mode of Common Cathode



Here is how a Common Anode LED Matrix works. First, choose 16 ports on RPI board to connect to the 16 ports of LED Matrix. Configure one port in columns for low level, which makes that column the selected port. Then configure the eight port in the row to display content in the selected column. Add a delay value and then select the next column that outputs the corresponding content. This kind of operation by column is called Scan. If you want to display the following image of a smiling face, you can display it in 8 columns, and each column is represented by one byte.



1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	1	0	1
1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

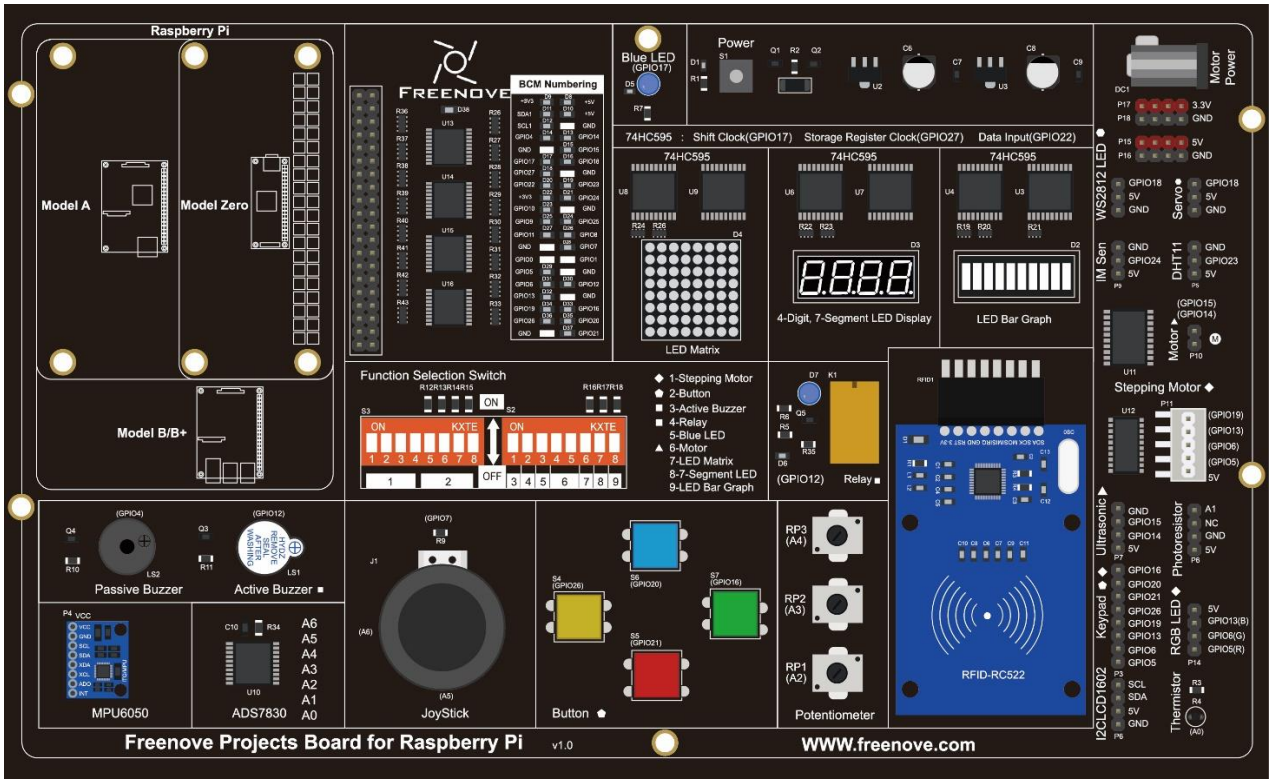
Column	Binary	Hexadecimal
1	0001 1100	0x1c
2	0010 0010	0x22
3	0101 0001	0x51
4	0100 0101	0x45
5	0100 0101	0x45
6	0101 0001	0x51
7	0010 0010	0x22
8	0001 1100	0x1c

To begin, display the first column, then turn off the first column and display the second column. (and so on) .... turn off the seventh column and display the 8th column, and then start the process over from the first column again like the control of LED Bar Graph project. The whole process will be repeated rapidly in a loop. Due to the principle of optical afterglow effect and the vision persistence effect in human sight, we will see a picture of a smiling face directly rather than individual columns of LEDs turned ON one column at a time (although in fact this is the reality we cannot perceive).

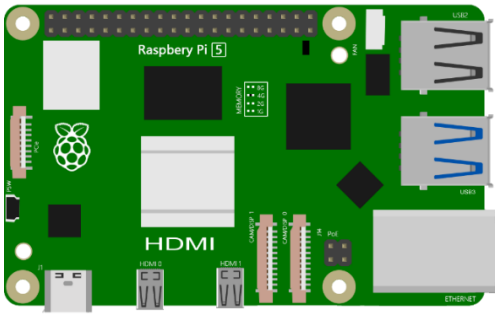
Scanning rows is another option to display on an LED Matrix (dot matrix grid). Whether scanning by row or column, 16 GPIO is required. In order to save GPIO ports of control board, two 74HC595 IC Chips are used in the circuit. Every 74HC595 IC Chip has eight parallel output ports, so two of these have a combined total of 16 ports, which is just enough for our project. The control lines and data lines of the two 74HC595 IC Chips are not all connected to the RPi, but connect to the Q7 pin of first stage 74HC595 IC Chip and to the data pin of second IC Chip. The two 74HC595 IC Chips are connected in series, which is the same as using one "74HC595 IC Chip" with 16 parallel output ports.

Component List

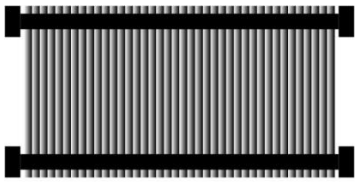
Freenove Projects Board for Raspberry Pi x1



Raspberry Pi



GPIO Ribbon Cable

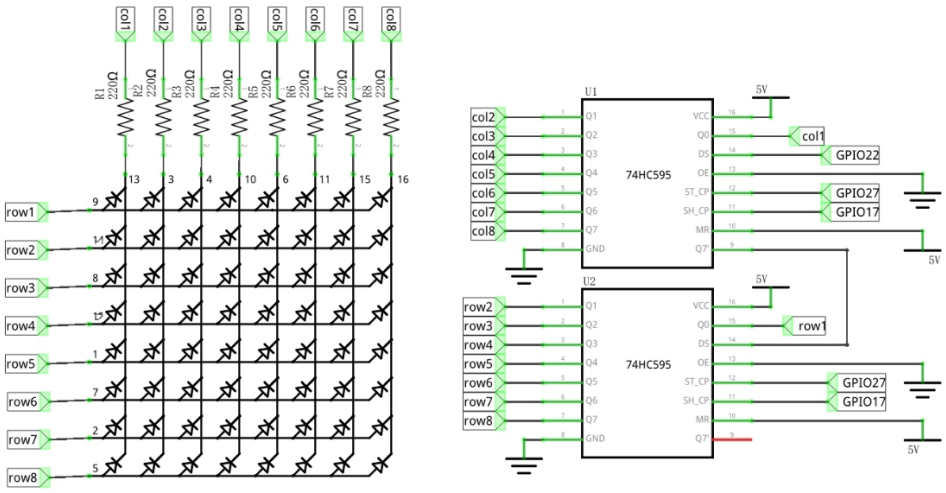


4-Digit 7-Segment Display



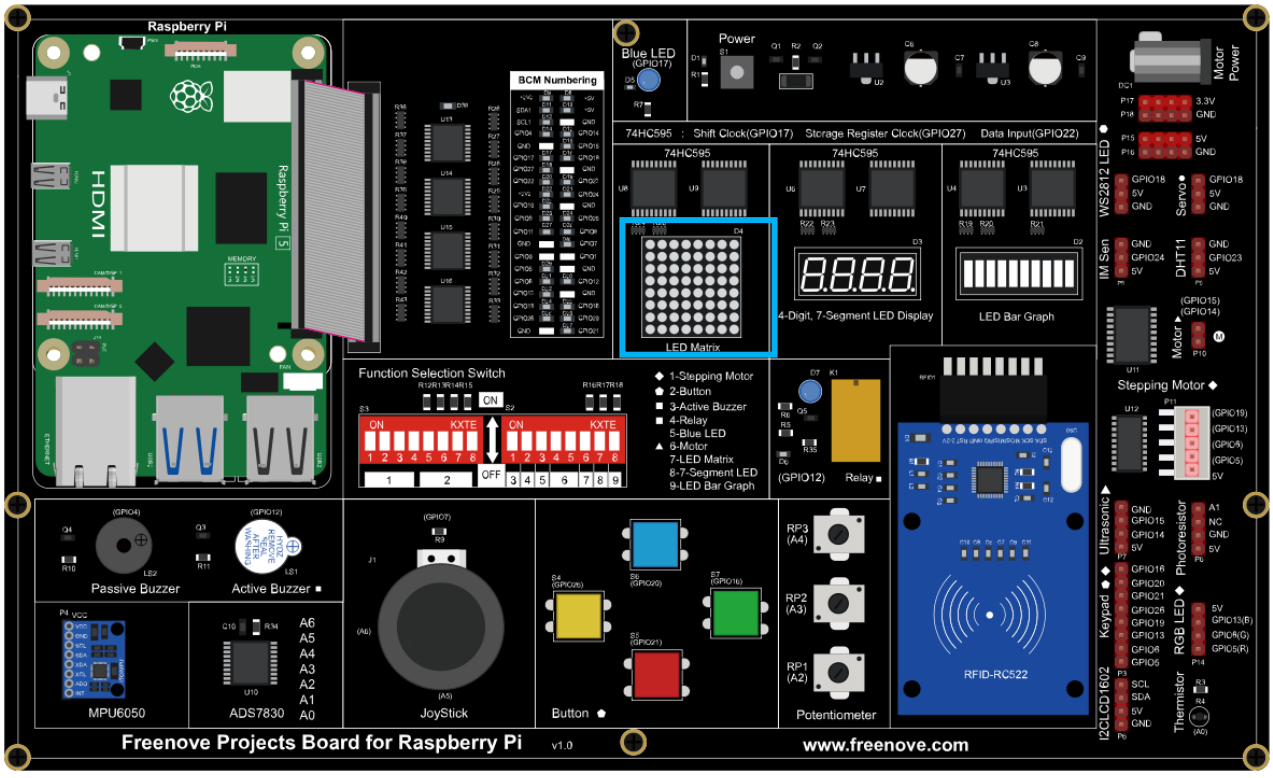
Circuit

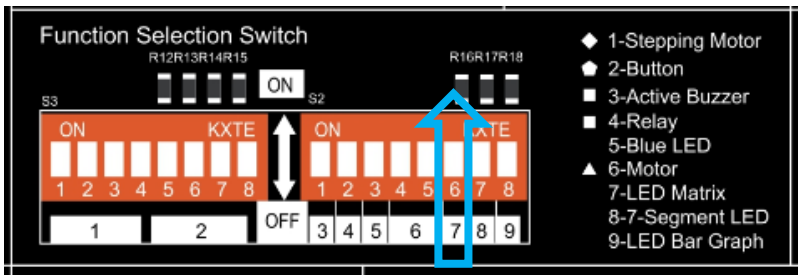
Schematic diagram



Hardware connection.

If it dosen't work, rotate the LED matrix for 180°.





Code

Two 74HC595 IC Chips are used in this project, one for controlling the LED Matrix's columns and the other for controlling the rows. According to the circuit connection, row data should be sent first, then column data. The following code will make the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

C Code 18.1 LEDMatrix

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 18_LEDMatrix directory of C language.

```
cd ~/Freenove_Kit/Code/C_Code/18_LEDMatrix
```

2. Use following command to compile "LEDMatrix.c" and generate executable file "LEDMatrix".

```
gcc LEDMatrix.c -o LEDMatrix -lwiringPi
```

3. Then run the generated file "LEDMatrix".

```
./LEDMatrix
```

After the program runs, the LED Matrix displays a smiling face, and then displays characters "0 to F" scrolling in a loop on the LED Matrix.

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <wiringShift.h>

#define   dataPin   22   //DS Pin of 74HC595(Pin14)
#define   latchPin  27   //ST_CP Pin of 74HC595(Pin12)
#define   clockPin  17   //SH_CP Pin of 74HC595(Pin11)
// data of smile face
unsigned char pic[]={0x1c,0x22,0x51,0x45,0x45,0x51,0x22,0x1c};
unsigned char data[]={  // data of "0-F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // " "
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x3E, 0x00, 0x00, // "0"
    0x00, 0x00, 0x21, 0x7F, 0x01, 0x00, 0x00, 0x00, // "1"
    0x00, 0x00, 0x23, 0x45, 0x49, 0x31, 0x00, 0x00, // "2"
    0x00, 0x00, 0x22, 0x49, 0x49, 0x36, 0x00, 0x00, // "3"
    0x00, 0x00, 0x0E, 0x32, 0x7F, 0x02, 0x00, 0x00, // "4"
    0x00, 0x00, 0x79, 0x49, 0x49, 0x46, 0x00, 0x00, // "5"
    0x00, 0x00, 0x3E, 0x49, 0x49, 0x26, 0x00, 0x00, // "6"
    0x00, 0x00, 0x60, 0x47, 0x48, 0x70, 0x00, 0x00, // "7"
    0x00, 0x00, 0x36, 0x49, 0x49, 0x36, 0x00, 0x00, // "8"
    0x00, 0x00, 0x32, 0x49, 0x49, 0x3E, 0x00, 0x00, // "9"
    0x00, 0x00, 0x3F, 0x44, 0x44, 0x3F, 0x00, 0x00, // "A"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x36, 0x00, 0x00, // "B"
```

```
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x22, 0x00, 0x00, // "C"
    0x00, 0x00, 0x7F, 0x41, 0x41, 0x3E, 0x00, 0x00, // "D"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x41, 0x00, 0x00, // "E"
    0x00, 0x00, 0x7F, 0x48, 0x48, 0x40, 0x00, 0x00, // "F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // " "
};
void _shiftOut(int dPin,int cPin,int order,int val){
    int i;
    for(i = 0; i < 8; i++){
        digitalWrite(cPin,LOW);
        if(order == LSBFIRST){
            digitalWrite(dPin,((0x01&(val>>i)) == 0x01) ? HIGH : LOW);
            delayMicroseconds(10);
        }
        else {//if(order == MSBFIRST){
            digitalWrite(dPin,((0x80&(val<<i)) == 0x80) ? HIGH : LOW);
            delayMicroseconds(10);
        }
        digitalWrite(cPin,HIGH);
        delayMicroseconds(10);
    }
}
int main(void)
{
    int i,j,k;
    unsigned char x;

    printf("Program is starting ...\n");

    wiringPiSetupGpio();

    pinMode(dataPin,OUTPUT);
    pinMode(latchPin,OUTPUT);
    pinMode(clockPin,OUTPUT);
    while(1){
        //Repeat enough times to display the smiling face a period of time
        for(j=0;j<500;j++){
            x=0x80;
            for(i=0;i<8;i++){
                digitalWrite(latchPin,LOW);
                // first shift data of line information to the first stage 74HC959
                _shiftOut(dataPin,clockPin,MSBFIRST,pic[i]);
                //then shift data of column information to the second stage 74HC959
                _shiftOut(dataPin,clockPin,MSBFIRST,~x);
```

```
                //Output data of two stage 74HC595 at the same time
                digitalWrite(latchPin,HIGH);
                x>>=1;   //display the next column
                delay(1);
            }
        }
        for(k=0;k<sizeof(data)-8;k++){  //sizeof(data) total number of "0-F" columns
            for(j=0;j<20;j++){
                x=0x80;          //Set the column information to start from the first column
                for(i=k;i<8+k;i++){
                    digitalWrite(latchPin,LOW);
                    _shiftOut(dataPin,clockPin,MSBFIRST,data[i]);
                    _shiftOut(dataPin,clockPin,MSBFIRST,~x);
                    digitalWrite(latchPin,HIGH);
                    x>>=1;
                    delay(1);
                }
            }
        }
    }
    return 0;
}
```

The first “for” loop in the “while” loop is used to display a static smile. Displaying column information from left to right, one column at a time with a total of 8 columns. This repeats 500 times to ensure sufficient display time.

```
        for(j=0;j<500;j++){
            x=0x80;
            for(i=0;i<8;i++){
                digitalWrite(latchPin,LOW);
                // first shift data of line information to the first stage 74HC959
                _shiftOut(dataPin,clockPin,MSBFIRST,pic[i]);
                //then shift data of column information to the second stage 74HC959
                _shiftOut(dataPin,clockPin,MSBFIRST,~x);
                //Output data of two stage 74HC595 at the same time
                digitalWrite(latchPin,HIGH);
                x>>=1;   //display the next column
                delay(1);
            }
        }
```

The second “for” loop is used to display scrolling characters "0 to F", for a total of 18 X 8 = 144 columns. Displaying the 0-8 column, then the 1-9 column, then the 2-10 column..... and so on…138-144 column in consecutively to achieve the scrolling effect. The display of each frame is repeated a certain number of times and the more repetitions, the longer the single frame display will be and the slower the scrolling movement.

```
        for(k=0;k<sizeof(data)-8;k++){  //sizeof(data) total number of "0-F" columns
            for(j=0;j<20;j++){
                x=0x80;          //Set the column information to start from the first column
                for(i=k;i<8+k;i++){
                    digitalWrite(latchPin,LOW);
                    _shiftOut(dataPin,clockPin,MSBFIRST,data[i]);
                    _shiftOut(dataPin,clockPin,MSBFIRST,~x);
                    digitalWrite(latchPin,HIGH);
                    x>>=1;
                    delay(1);
                }
            }
        }
```

Python Code 18.1 LEDMatrix

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 18_LEDMatrix directory of Python language.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/18_LEDMatrix
```

2. Use Python command to execute Python code “LEDMatrix.py”.

```
python LEDMatrix.py
```

After the program runs, the LED Matrix displayss a smiling face, and then displays characters "0 to F" scrolling in a loop on the LED Matrix.

The following is the program code:

```
from gpiozero import OutputDevice
import time

LSBFIRST = 1
MSBFIRST = 2
# define the pins connect to 74HC595
dataPin   = OutputDevice(22)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(17)      # CH_CP Pin of 74HC595(Pin11)
pic = [0x1c,0x22,0x51,0x45,0x45,0x51,0x22,0x1c]  # data of smiling face
data = [     # data of "0-F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x3E, 0x00, 0x00, # "0"
    0x00, 0x00, 0x21, 0x7F, 0x01, 0x00, 0x00, 0x00, # "1"
    0x00, 0x00, 0x23, 0x45, 0x49, 0x31, 0x00, 0x00, # "2"
    0x00, 0x00, 0x22, 0x49, 0x49, 0x36, 0x00, 0x00, # "3"
    0x00, 0x00, 0x0E, 0x32, 0x7F, 0x02, 0x00, 0x00, # "4"
    0x00, 0x00, 0x79, 0x49, 0x49, 0x46, 0x00, 0x00, # "5"
    0x00, 0x00, 0x3E, 0x49, 0x49, 0x26, 0x00, 0x00, # "6"
    0x00, 0x00, 0x60, 0x47, 0x48, 0x70, 0x00, 0x00, # "7"
    0x00, 0x00, 0x36, 0x49, 0x49, 0x36, 0x00, 0x00, # "8"
    0x00, 0x00, 0x32, 0x49, 0x49, 0x3E, 0x00, 0x00, # "9"
    0x00, 0x00, 0x3F, 0x44, 0x44, 0x3F, 0x00, 0x00, # "A"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x36, 0x00, 0x00, # "B"
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x22, 0x00, 0x00, # "C"
    0x00, 0x00, 0x7F, 0x41, 0x41, 0x3E, 0x00, 0x00, # "D"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x41, 0x00, 0x00, # "E"
    0x00, 0x00, 0x7F, 0x48, 0x48, 0x40, 0x00, 0x00, # "F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
]

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def loop():
    while True:
       for j in range(0,500):#Repeat enough times to display the smiling face a period of time
```

```
            x=0x80
            for i in range(0,8):
                latchPin.off()
                shiftOut(MSBFIRST,pic[i]) #first shift data of line information
                shiftOut(MSBFIRST,~x)     #then shift data of column information
                latchPin.on()             # Output data of two stage 74HC595 at the same time
                time.sleep(0.001) # display the next column
                x>>=1

        for k in range(0,len(data)-8): #len(data) total number of "0-F" columns
            for j in range(0,20):
                x=0x80      # Set the column information to start from the first column
                for i in range(k,k+8):
                    latchPin.off()
                    shiftOut(MSBFIRST,data[i])
                    shiftOut(MSBFIRST,~x)
                    latchPin.on()
                    time.sleep(0.001)
                    x>>=1

def destroy():
    latchPin.off()
    shiftOut(MSBFIRST,0x00)
    shiftOut(MSBFIRST,0x00)
    latchPin.on()           # Output data of two stage 74HC595 at the same time
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__':  # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:   # Press Ctrl+C to end the program.
        print("Ending program")
    finally:
        destroy()
```

The first “for” loop in the “while” loop is used to display a static smile. Displaying column information from left to right, one column at a time with a total of 8 columns. This repeats 500 times to ensure sufficient display time.

```
        for j in range(0,500):#Repeat enough times to display the smiling face a period of time
            x=0x80
            for i in range(0,8):
                latchPin.off()
                shiftOut(MSBFIRST,pic[i]) #first shift data of line information
                shiftOut(MSBFIRST,~x)     #then shift data of column information
                latchPin.on()             # Output data of two stage 74HC595 at the same time
                time.sleep(0.001) # display the next column
                x>>=1
```

The second “for” loop is used to display scrolling characters "0 to F", for a total of 18 X 8 = 144 columns. Displaying the 0-8 column, then the 1-9 column, then the 2-10 column..... and so on…138-144 column in consecutively to achieve the scrolling effect. The display of each frame is repeated a certain number of times and the more repetitions, the longer the single frame display will be and the slower the scrolling movement.

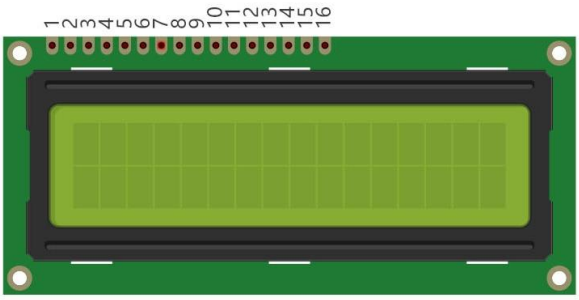
```
        for k in range(0,len(data)-8): #len(data) total number of "0-F" columns
            for j in range(0,20):
                x=0x80      # Set the column information to start from the first column
                for i in range(k,k+8):
                    latchPin.off()
                    shiftOut(MSBFIRST,data[i])
                    shiftOut(MSBFIRST,~x)
                    latchPin.on()
                    time.sleep(0.001)
                    x>>=1
```

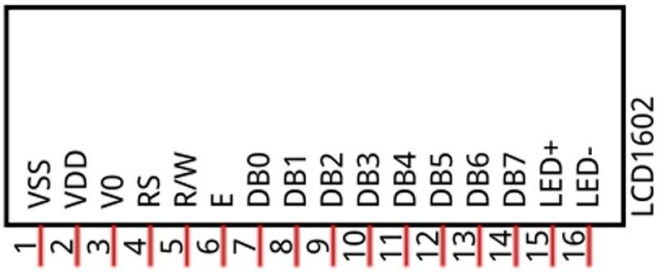
Chapter 19 LCD1602

In this chapter, we will learn about the LCD1602 Display Screen,

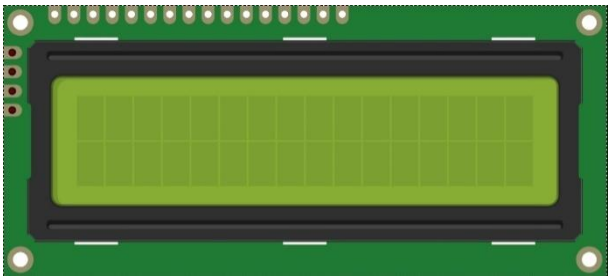
Project 19.1 I2C LCD1602

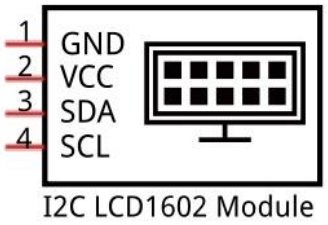
There are LCD1602 display screen and the I2C LCD. We will introduce both of them in this chapter. But what we use in this project is an I2C LCD1602 display screen. The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram





I2C LCD1602 Display Screen integrates a I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to only use 4 lines to operate the LCD1602.

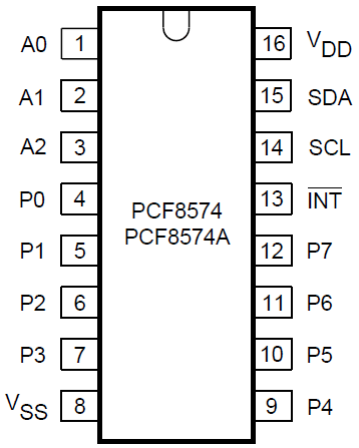




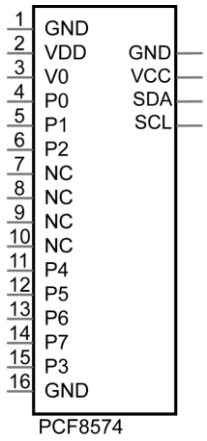
The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F). You can also view the RPI bus on your I2C device address through command "i2cdetect -y 1" (refer to the "configuration I2C" section below).

Below is the PCF8574 chip pin diagram and its module pin diagram:

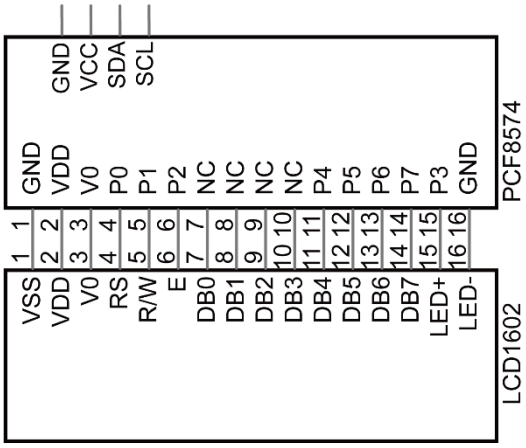
PCF8574 chip pin diagram:



PCF8574 module pin diagram



PCF8574 module pins and LCD1602 pins correspond to each other and are connected to each other:

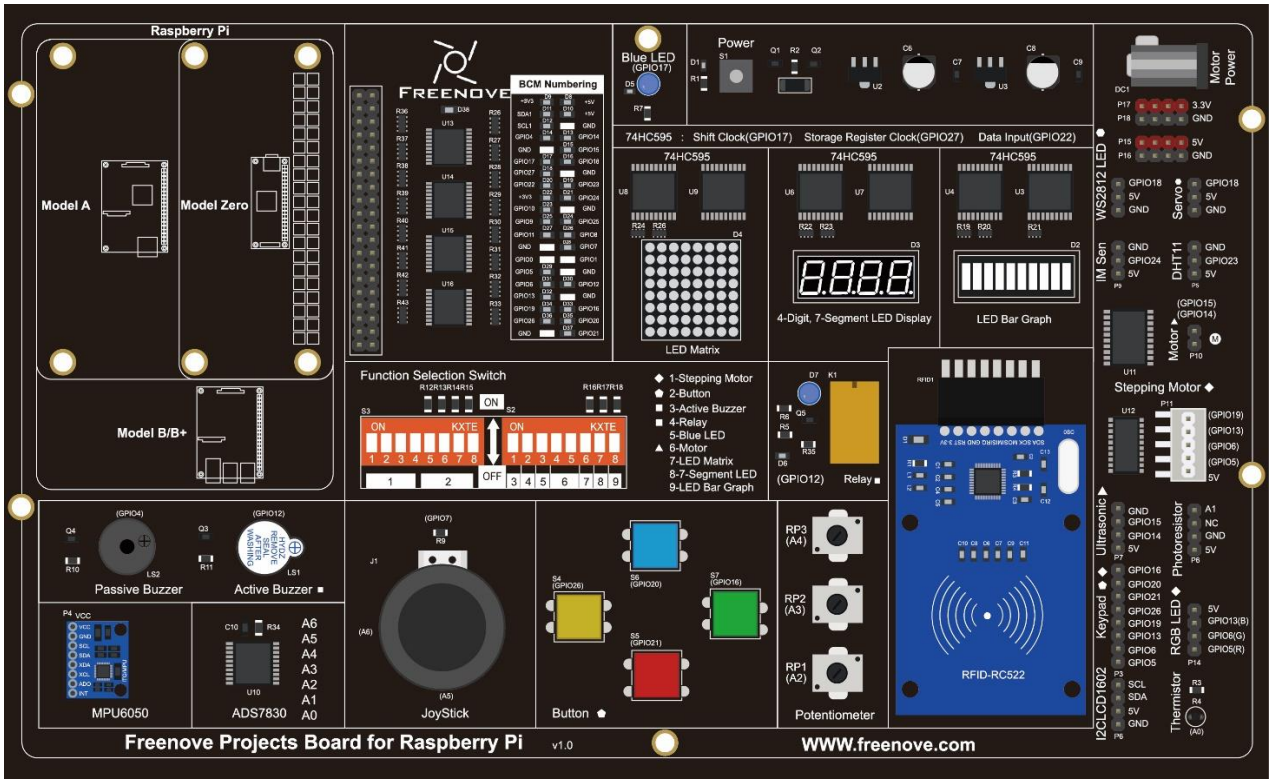


Because of this, as stated earlier, we only need 4 pins to control the 16 pins of the LCD1602 Display Screen through the I2C interface.

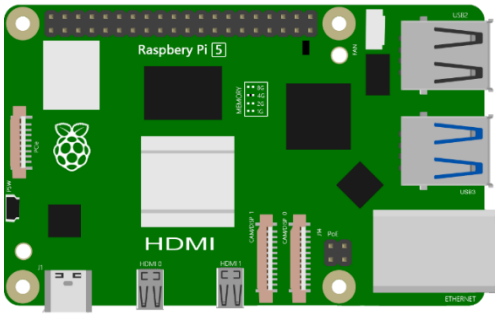
In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

Component List

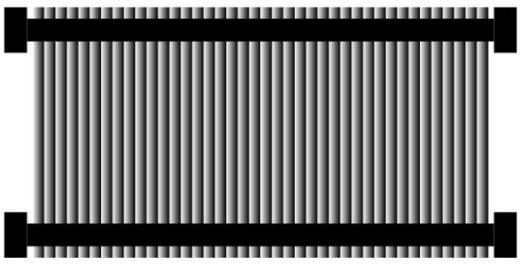
Freenove Projects Board for Raspberry Pi



Raspberry Pi



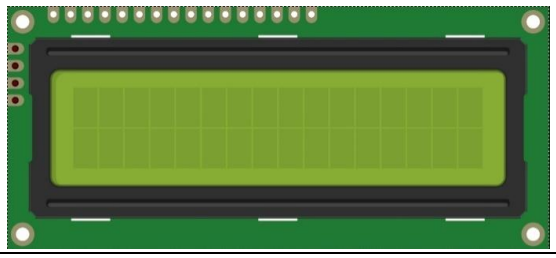
GPIO Ribbon Cable



Jumper Wire



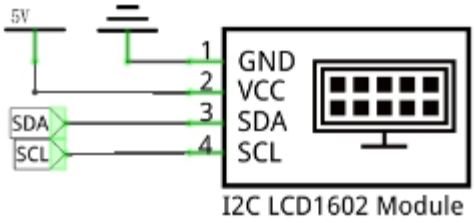
I2C LCD1602 Module



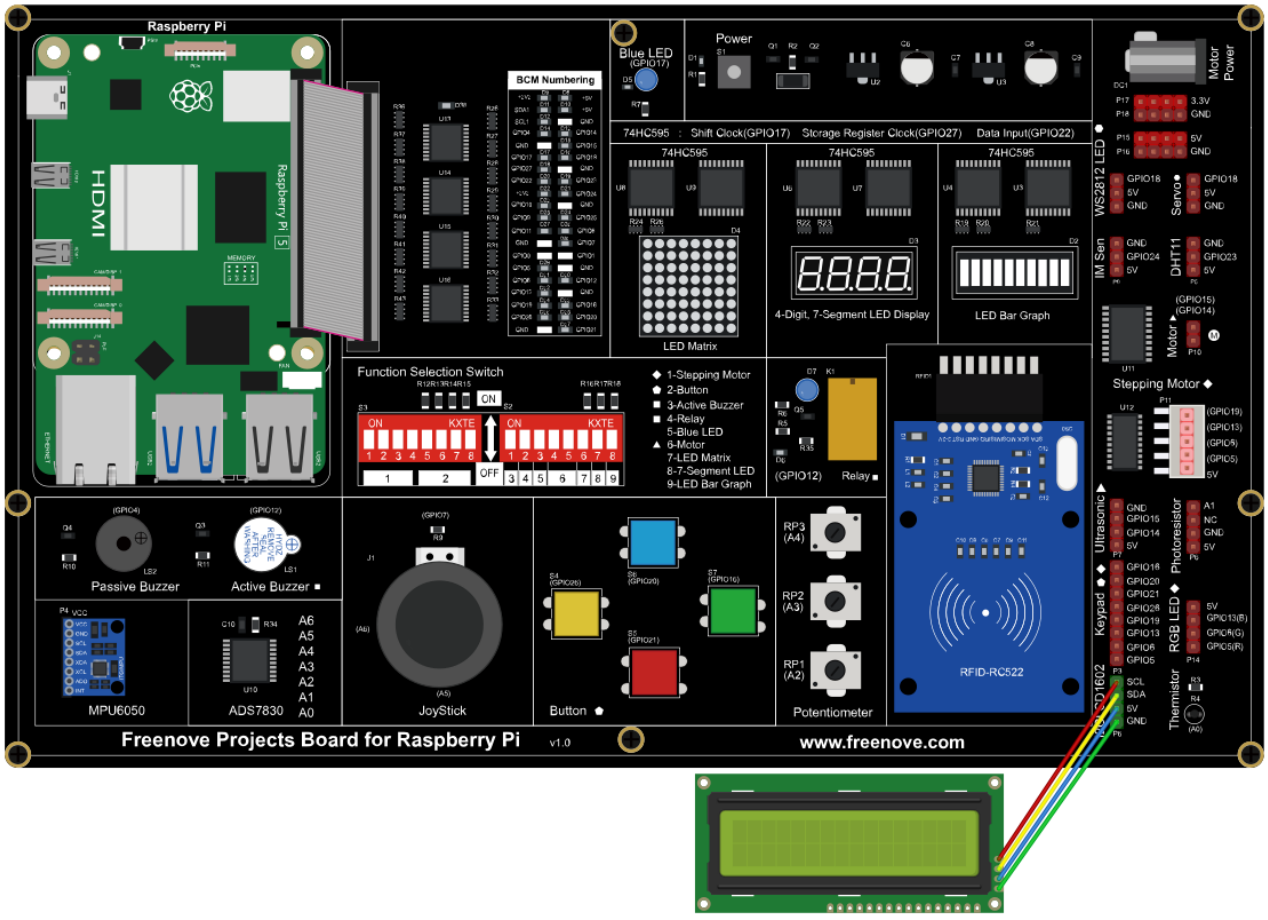
Circuit

Note that the power supply for I2C LCD1602 in this circuit is 5V.

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

This code will have your RPi's CPU temperature and System Time Displayed on the LCD1602.

C Code 19.1 I2CLCD1602

If you haven't configured I2C and install Smbus, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 19_I2CLCD1602 directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/19_I2CLCD1602
```

2. Use following command to compile "I2CLCD1602.c" and generate executable file "I2CLCD1602".

```
gcc I2CLCD1602.c -o I2CLCD1602 -lwiringPi -lwiringPiDev
```

3. Then run the generated file "I2CLCD1602".

```
./I2CLCD1602
```

After the program runs, the LCD1602 Screen will display your RPi's CPU Temperature and System Time.

NOTE: After the program runs, if you nothing displays or the display is not clear, you can try to rotate the white knob on back of LCD1602 slowly, which adjusts the contrast, until the screen can display the Time and Temperature clearly.



The following is the program code:

```
#include <stdlib.h>
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiI2C.h>
#include <pcf8574.h>
#include <lcd.h>
#include <time.h>

int pcf8574_address = 0x27;        // PCF8574T:0x27, PCF8574AT:0x3F
#define BASE 64         // BASE any number above 64
//Define the output pins of the PCF8574, which are directly connected to the LCD1602 pin.
#define RS      BASE+0
#define RW      BASE+1
#define EN      BASE+2
```

```
#define LED     BASE+3
#define D4      BASE+4
#define D5      BASE+5
#define D6      BASE+6
#define D7      BASE+7

int lcdhd;// used to handle LCD
void printCPUTemperature(){// sub function used to print CPU temperature
    FILE *fp;
    char str_temp[15];
    float CPU_temp;
    // CPU temperature data is stored in this directory.
    fp=fopen("/sys/class/thermal/thermal_zone0/temp","r");
    fgets(str_temp,15,fp);      // read file temp
    CPU_temp = atof(str_temp)/1000.0;   // convert to Celsius degrees
    printf("CPU's temperature : %.2f \n",CPU_temp);
    lcdPosition(lcdhd,0,0);     // set the LCD cursor position to (0,0)
    lcdPrintf(lcdhd,"CPU:%.2fC",CPU_temp);// Display CPU temperature on LCD
    fclose(fp);
}
void printDataTime(){//used to print system time
    time_t rawtime;
    struct tm *timeinfo;
    time(&rawtime);// get system time
    timeinfo = localtime(&rawtime);//convert to local time
    printf("%s \n",asctime(timeinfo));
    lcdPosition(lcdhd,0,1);// set the LCD cursor position to (0,1)

lcdPrintf(lcdhd,"Time:%02d:%02d:%02d",timeinfo->tm_hour,timeinfo->tm_min,timeinfo->tm_sec);
//Display system time on LCD
}
int detectI2C(int addr){
    int _fd = wiringPiI2CSetup (addr);
    if (_fd < 0){
        printf("Error address : 0x%x \n",addr);
        return 0 ;
    }
    else{
        if(wiringPiI2CWrite(_fd,0) < 0){
            printf("Not found device in address 0x%x \n",addr);
            return 0;
        }
        else{
            printf("Found device in address 0x%x \n",addr);
```

```
59              return 1 ;
60          }
61      }
62  }
63  int main(void){
64      int i;
65
66      printf("Program is starting ...\n");
67
68      wiringPiSetup();
69      if(detectI2C(0x27)){
70          pcf8574_address = 0x27;
71      }else if(detectI2C(0x3F)){
72          pcf8574_address = 0x3F;
73      }else{
74          printf("No correct I2C address found, \n"
75          "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
76          "Program Exit. \n");
77          return -1;
78      }
79      pcf8574Setup(BASE,pcf8574_address);//initialize PCF8574
80      for(i=0;i<8;i++){
81          pinMode(BASE+i,OUTPUT);      //set PCF8574 port to output mode
82      }
83      digitalWrite(LED,HIGH);      //turn on LCD backlight
84      digitalWrite(RW,LOW);        //allow writing to LCD
85      lcdhd = lcdInit(2,16,4,RS,EN,D4,D5,D6,D7,0,0,0,0);// initialize LCD and return  “handle”
86  used to handle LCD
87      if(lcdhd == -1){
88          printf("lcdInit failed !");
89          return 1;
90      }
91      while(1){
92          printCPUTemperature();//print CPU temperature
93          printDataTime();         // print system time
94          delay(1000);
95      }
        return 0;
    }
```

From the code, we can see that the PCF8591 and the PCF8574 have many similarities in using the I2C interface to expand the GPIO RPI.
First, define the I2C address of the PCF8574 and the Extension of the GPIO pin, which is connected to the GPIO pin of the LCD1602. LCD1602 has two different i2c addresses. Set 0x27 as default.

```
int pcf8574_address = 0x27;        // PCF8574T:0x27, PCF8574AT:0x3F
#define BASE 64         // BASE any number above 64
//Define the output pins of the PCF8574, which are directly connected to the LCD1602 pin.
#define RS      BASE+0
#define RW      BASE+1
#define EN      BASE+2
#define LED     BASE+3
#define D4      BASE+4
#define D5      BASE+5
#define D6      BASE+6
#define D7      BASE+7
```

Then, in main function, initialize the PCF8574, set all the pins to output mode, and turn ON the LCD1602 backlight (without the backlight the Display is difficult to read).

```
    pcf8574Setup(BASE, pcf8574_address);// initialize PCF8574
    for(i=0;i<8;i++){
        pinMode(BASE+i, OUTPUT);      // set PCF8574 port to output mode
    }
    digitalWrite(LED, HIGH);      // turn on LCD backlight
```

Then use lcdInit() to initialize LCD1602 and set the RW pin of LCD1602 to 0 (can be written) according to requirements of this function. The return value of the function called "Handle" is used to handle LCD1602".

```
        lcdhd = lcdInit(2, 16, 4, RS, EN, D4, D5, D6, D7, 0, 0, 0, 0);// initialize LCD and return
“handle” used to handle LCD
```

Details about lcdInit():

int lcdInit (int rows, int cols, int bits, int rs, int strb,
int d0, int d1, int d2, int d3, int d4, int d5, int d6, int d7) ;

This is the main initialization function and must be executd first before you use any other LCD functions. **Rows** and **cols** are the rows and columns of the Display (e.g. 2, 16 or 4, 20). **Bits** is the number of how wide the number of bits is on the interface (4 or 8). The **rs** and **strb** represent the pin numbers of the Display's RS pin and Strobe (E) pin. The parameters **d0** to **d7** are the pin numbers of the 8 data pins connected from the RPi to the display. Only the first 4 are used if you are running the display in 4-bit mode.
The return value is the 'handle' to be used for all subsequent calls to the lcd library when dealing with that LCD, or -1 to indicate a fault (usually incorrect parameter)
For more details about LCD Library, please refer to: https://projects.drogon.net/raspberry-pi/wiringpi/lcd-library/

In the next “while”, two subfunctions are called to display the RPi’s CPU Temperature and the SystemTime. First look at subfunction printCPUTemperature(). The CPU temperature data is stored in the "/sys/class/thermal/thermal_zone0/temp" file. We need to read the contents of this file, which converts it to temperature value stored in variable CPU_temp and uses lcdPrintf() to display it on LCD.

```
void printCPUTemperature(){//subfunction used to print CPU temperature
    FILE *fp;
    char str_temp[15];
    float CPU_temp;
    // CPU temperature data is stored in this directory.
    fp=fopen("/sys/class/thermal/thermal_zone0/temp","r");
    fgets(str_temp,15,fp);       // read file temp
    CPU_temp = atof(str_temp)/1000.0;   // convert to Celsius degrees
    printf("CPU's temperature : %.2f \n",CPU_temp);
    lcdPosition(lcdhd,0,0);      // set the LCD cursor position to (0,0)
    lcdPrintf(lcdhd,"CPU:%.2fC",CPU_temp);// Display CPU temperature on LCD
    fclose(fp);
}
```

Details about lcdPosition() and lcdPrintf():

lcdPosition (int handle, int x, int y);
Set the position of the cursor for subsequent text entry.
lcdPutchar (int handle, uint8_t data) **lcdPuts (int handle, char *string)** **lcdPrintf (int handle, char *message, …)**
These output a single ASCII character, a string or a formatted string using the usual print formatting commands to display individual characters (it is how you are able to see characters on your computer monitor).

Next is subfunction printDataTime() used to display System Time. First, it gets the Standard Time and stores it into variable Rawtime, and then converts it to the Local Time and stores it into timeinfo, and finally displays the Time information on the LCD1602 Display.

```
void printDataTime(){//used to print system time
    time_t rawtime;
    struct tm *timeinfo;
    time(&rawtime);// get system time
    timeinfo = localtime(&rawtime);// convert to local time
    printf("%s \n",asctime(timeinfo));
    lcdPosition(lcdhd,0,1);// set the LCD cursor position to (0,1)
    lcdPrintf(lcdhd,"Time:%d:%d:%d",timeinfo->tm_hour,timeinfo->tm_min,timeinfo->tm_sec);
//Display system time on LCD
}
```

Python Code 19.1 I2CLCD1602

If you haven't configured I2C and install Smbus, please refer to Chapter 7. If you've done it, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 19_I2CLCD1602 directory of Python code.

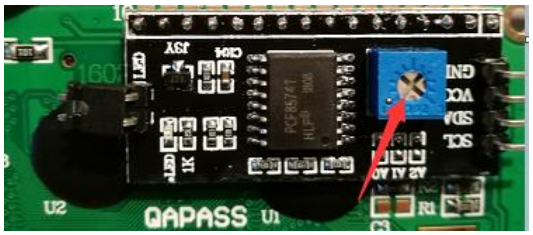
```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/19_I2CLCD1602
```

2. Use Python command to execute Python code "I2CLCD1602.py".

```
python I2CLCD1602.py
```

After the program runs, the LCD1602 Screen will display your RPi's CPU Temperature and System Time.

NOTE: After the program runs, if nothing displays or the display is not clear, try rotating the white knob on back of LCD1602 slowly, which adjusts the contrast, until the screen can display the Time and Temperature clearly.



The following is the program code:

```
from time import sleep
from datetime import datetime
from LCD1602 import CharLCD1602

lcd1602 = CharLCD1602()
def get_cpu_temp():  # get CPU temperature from file "/sys/class/thermal/thermal_zone0/temp"
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C '

def get_time_now():     # get system time
    return datetime.now().strftime('    %H:%M:%S')

def loop():
    lcd1602.init_lcd()
    count = 0
    while(True):
        lcd1602.clear()
        lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
        lcd1602.write(0, 1, get_time_now() )   # display the time
        sleep(1)

def destroy():
    lcd1602.clear()
```

```
26 if __name__ == '__main__':
27     print ('Program is starting ... ')
28     try:
29         loop()
30     except KeyboardInterrupt:
31         destroy()
```

In this section, we introduce the LCD1602 module for displaying LCD1602 and the datatime module for getting the time.

```
from time import sleep
from datetime import datetime
from LCD1602 import CharLCD1602
```

In the loop function, lCD1602.init_lcd () will initialize LCD1602 for later use.
get_cpu_temp() is used to obtain the system temperature value, get_time_now() is used to obtain the current time, and then lcd1602.write() function is called to display the system temperature and the current time.

```
def loop():
    lcd1602.init_lcd()
    count = 0
    while(True):
        lcd1602.clear()
        lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
        lcd1602.write(0, 1, get_time_now() )   # display the time
        sleep(1)
```

CPU temperature is stored in file "/sys/class/thermal/thermal_zone0/temp". Open the file and read content of the file, and then convert it to Celsius degrees and return. Subfunction used to get CPU temperature is shown below:

```
def get_cpu_temp():  # get CPU temperature from file "/sys/class/thermal/thermal_zone0/temp"
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C '
```

Subfunction used to get time:

```
def get_time_now():     # get system time
    return datetime.now().strftime('     %H:%M:%S')
```

Module LCD1602.CharLCD1602()

This module provides the basic operation method of LCD1602, including class CharLCD1602. Some member functions are described as follows:

def init_lcd(self,addr=None, bl=1): Initialize the lcd1602 device and turn on the backlight.

def clear(self): clear the screen

def write(self, x, y, str): x represents the column, y represents the row, and str represents the character displayed.

def display_num(self, x, y, num): x represents the column, y represents the row, and num represents the number displayed.

Chapter 20 Hygrothermograph DHT11

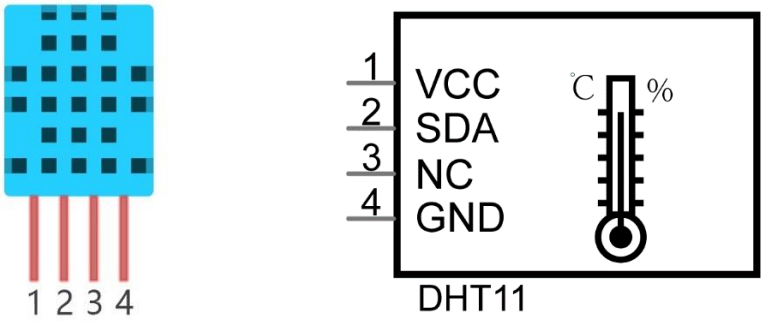
In this chapter, we will learn about a commonly used sensor called a Hygrothermograph DHT11.

Project 20.1 Hygrothermograph

Hygrothermograph is an important tool in our lives to give us data on the temperature and humidity in our environment. In this project, we will use the RPi to read Temperature and Humidity data of the DHT11 Module.

Component knowledge

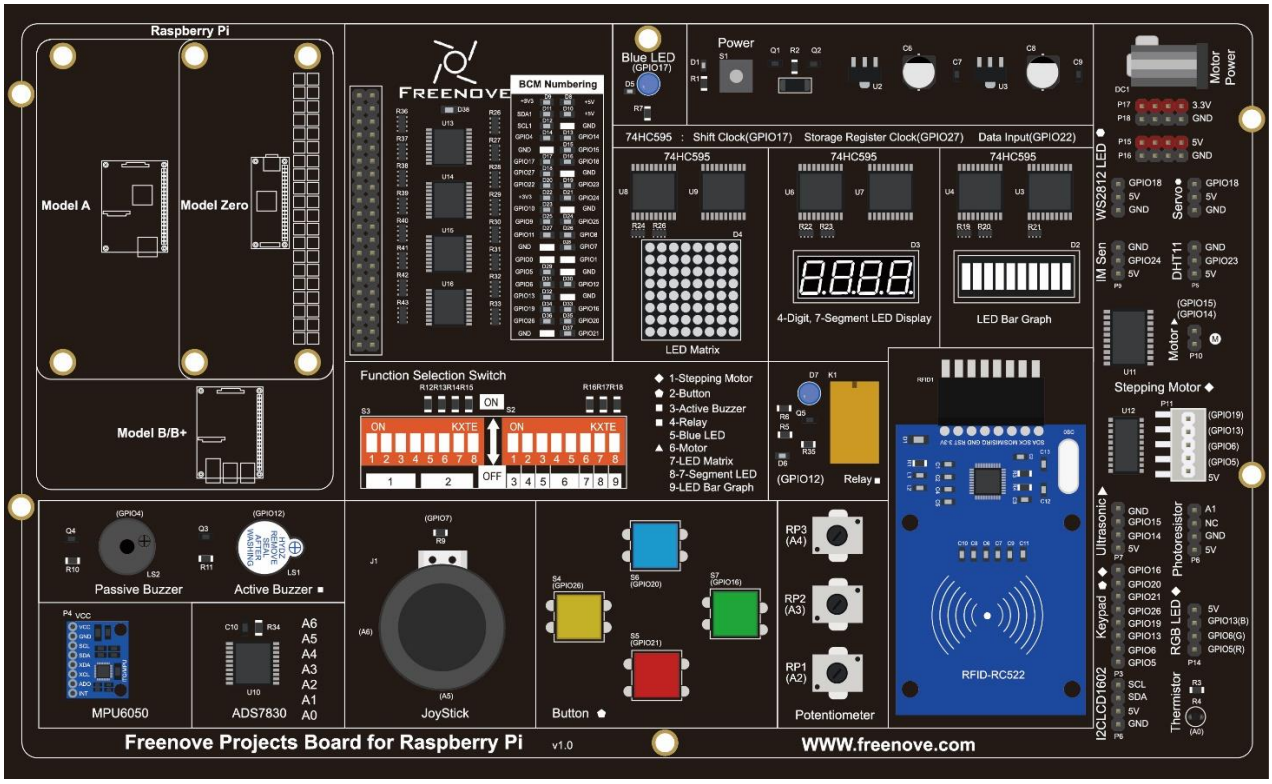
The Temperature & Humidity Sensor DHT11 is a compound temperature & humidity sensor, and the output digital signal has been calibrated by its manufacturer.



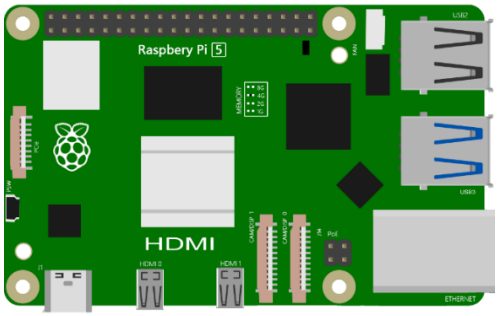
After being powered up, it will initialize in 1 second. Its operating voltage is within the range of 3.3V-5.5V. The SDA pin is a data pin, which is used to communicate with other devices.
The NC pin (Not Connected Pin) is a type of pin found on various integrated circuit packages. Such pin has no functional purpose to the outside circuit (but may have an unknown functionality during manufacture and test). It **should not be connected** to any of the circuit connections.

Component List

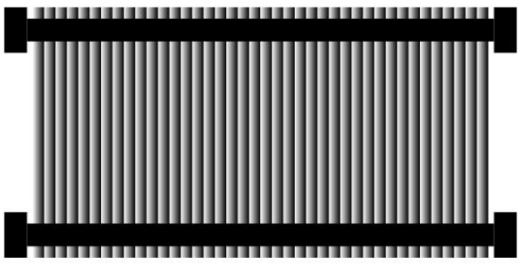
Freenove Projects Board for Raspberry Pi



Raspberry Pi



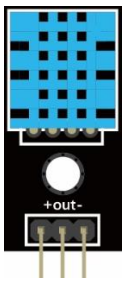
GPIO Ribbon Cable



Jumper Wire

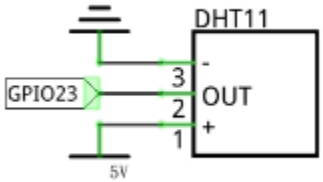


DHT11 Module

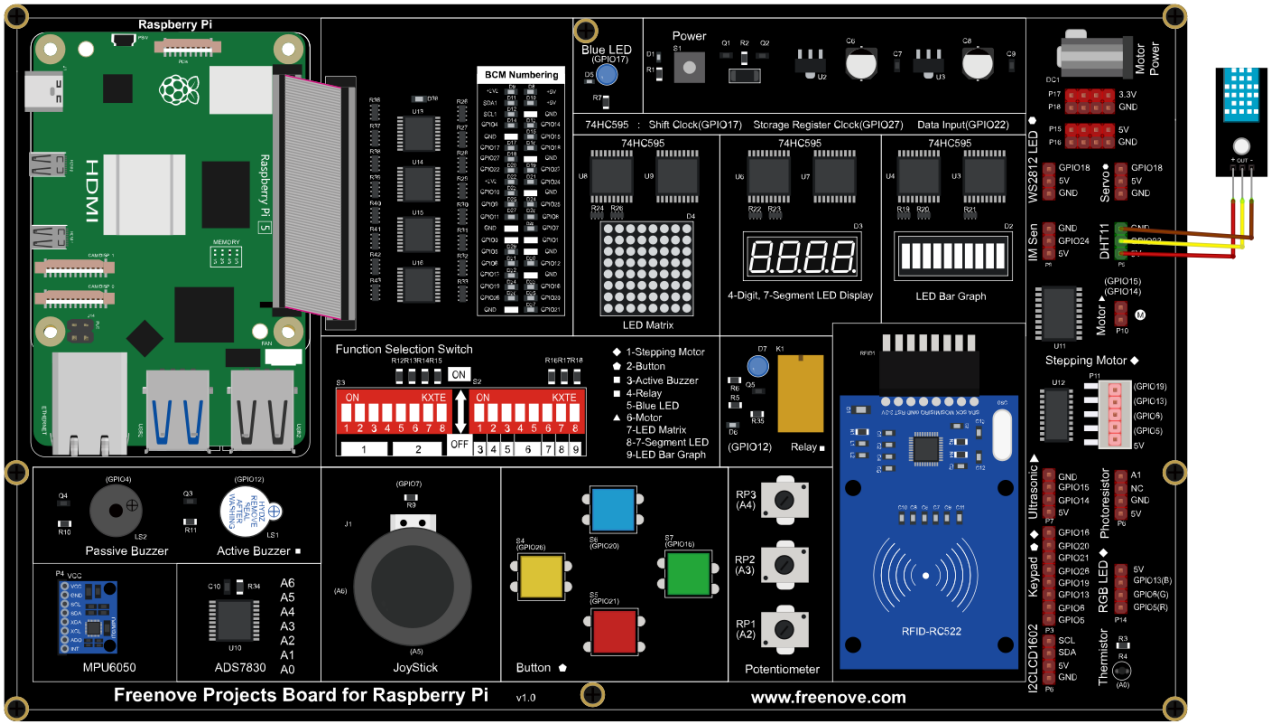


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

The code is used to read the temperature and humidity data of DHT11, and display them.

C Code 20.1 DHT11

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 20_DHT11 directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/20_DHT11
```

2. The code used in this project contains a custom header file. Use the following command to compile the code DHT11.cpp and DHT.cpp and generate executable file DHT11. The custom header file will be compiled at the same time.

```
gcc DHT.cpp DHT11.cpp -o DHT11 -lwiringPi
```

3. Run the generated file "DHT11".

```
./DHT11
```

After the program runs, the Terminal window will display the current total number of read times, the read state, as well as temperature and humidity values as is shown below:

```
Measurement counts : 1
DHT11,OK!
Humidity is 50.00 %,     Temperature is 27.50 *C

Measurement counts : 2
DHT11,OK!
Humidity is 53.00 %,     Temperature is 27.50 *C

Measurement counts : 3
DHT11,OK!
Humidity is 54.00 %,     Temperature is 27.50 *C

Measurement counts : 4
DHT11,OK!
Humidity is 54.00 %,     Temperature is 27.50 *C
```

The following is the program code:

```
1  #include <wiringPi.h>
2  #include <stdio.h>
3  #include <stdint.h>
4  #include "DHT.hpp"
5
6  #define DHT11_Pin  23  //define the pin of sensor
7
8  int main(){
9      DHT dht;           //create a DHT class object
10     int chk,sumCnt;    //chk:read the return value of sensor; sumCnt:times of reading sensor
11
12     printf("Program is starting ...\n");
13
```

```
14      wiringPiSetupGpio();
15
16      while(1){
17          chk = dht.readDHT11(DHT11_Pin); //read DHT11 and get a return value. Then determine
18  whether data read is normal according to the return value.
19          sumCnt++;       //counting number of reading times
20          printf("The sumCnt is : %d \n",sumCnt);
21          switch(chk){
22              case DHTLIB_OK:     //if the return value is DHTLIB_OK, the data is normal.
23                  printf("DHT11,OK! \n");
24                  break;
25              case DHTLIB_ERROR_CHECKSUM:     //data check has errors
26                  printf("DHTLIB_ERROR_CHECKSUM! \n");
27                  break;
28              case DHTLIB_ERROR_TIMEOUT:      //reading DHT times out
29                  printf("DHTLIB_ERROR_TIMEOUT! \n");
30                  break;
31              case DHTLIB_INVALID_VALUE:      //other errors
32                  printf("DHTLIB_INVALID_VALUE! \n");
33                  break;
34          }
35          printf("Humidity is %.2f %%, \t Temperature is %.2f
36  *C\n\n",dht.humidity,dht.temperature);
37          delay(3000);
38      }
39      return 1;
40  }
```

In this project code, we use a custom library file "DHT.hpp". It is located in the same directory with the program files "DHT11.cpp" and "DHT.cpp", and methods for reading DHT sensor are provided in the library file. By using this library, we can easily read the DHT Sensor. First, we create a DHT class object in the code.

```
    DHT dht;
```

In the "while" loop, the value of DHT11 is read every 3 seconds through the dht.readdht11 () function.

```
while(1){
        chk = dht.readDHT11(DHT11_Pin); //read DHT11 and get a return value. Then determine
whether data read is normal according to the return value.
        sumCnt++;       //counting number of reading times
        printf("The sumCnt is : %d \n",sumCnt);
        switch(chk){
            case DHTLIB_OK:     //if the return value is DHTLIB_OK, the data is normal.
                printf("DHT11,OK! \n");
                break;
            case DHTLIB_ERROR_CHECKSUM:     //data check has errors
                printf("DHTLIB_ERROR_CHECKSUM! \n");
                break;
```

```
            case DHTLIB_ERROR_TIMEOUT:      //reading DHT times out
                printf("DHTLIB_ERROR_TIMEOUT! \n");
                break;
            case DHTLIB_INVALID_VALUE:      //other errors
                printf("DHTLIB_INVALID_VALUE! \n");
                break;
        }
        printf("Humidity is %.2f %%, \t Temperature is %.2f
*C\n\n",dht.humidity,dht.temperature);
        delay(3000);
    }
```

Finally display the results:

```
printf("Humidity is %.2f %%, \t Temperature is %.2f
*C\n\n",dht.humidity,dht.temperature);
```

Library file "DHT.hpp" contains a DHT class and this public member function int **readDHT11** (int pin) is used to read sensor DHT11 and store the temperature and humidity data read to member variables double humidity and temperature. The implementation method of the function is included in the file "DHT.cpp".

```
1  #define _DHT_H_
2
3  #include <wiringPi.h>
4  #include <stdio.h>
5  #include <stdint.h>
6
7  ////read return flag of sensor
8  #define DHTLIB_OK               0
9  #define DHTLIB_ERROR_CHECKSUM   -1
10 #define DHTLIB_ERROR_TIMEOUT    -2
11 #define DHTLIB_INVALID_VALUE    -999
12
13 #define DHTLIB_DHT11_WAKEUP     18
14 #define DHTLIB_DHT_WAKEUP       1
15
16 #define DHTLIB_TIMEOUT          100
17
18 class DHT{
19     public:
20         double humidity,temperature;    //use to store temperature and humidity data read
21         int readDHT11(int pin);     //read DHT11
22     private:
23         uint8_t bits[5];    //Buffer to receiver data
24         int readSensor(int pin,int wakeupDelay);    //
25
26 };
```

Python Code 20.1 DHT11

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

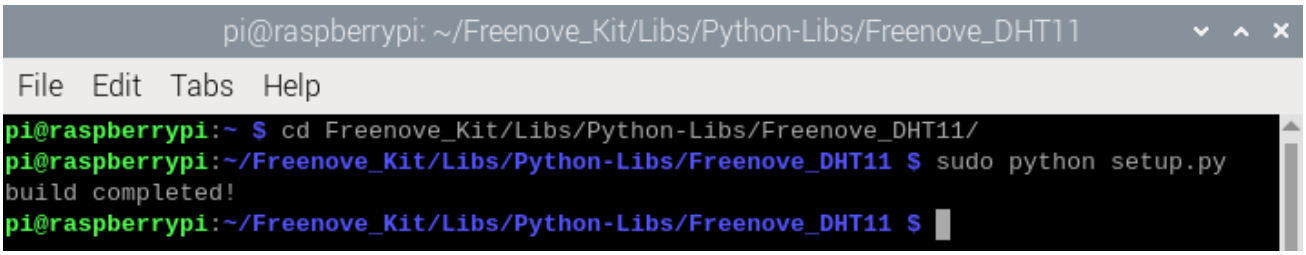
1. Use cd command to enter folder of the ADC Device library.

```
cd ~/Freenove_Kit/Libs/Python-Libs/Freenove_DHT11
```

2. Execute command below to install the library.

```
sudo python setup.py
```

A successful installation, without error prompts, is shown below:



Next, we will execute the code for this project.

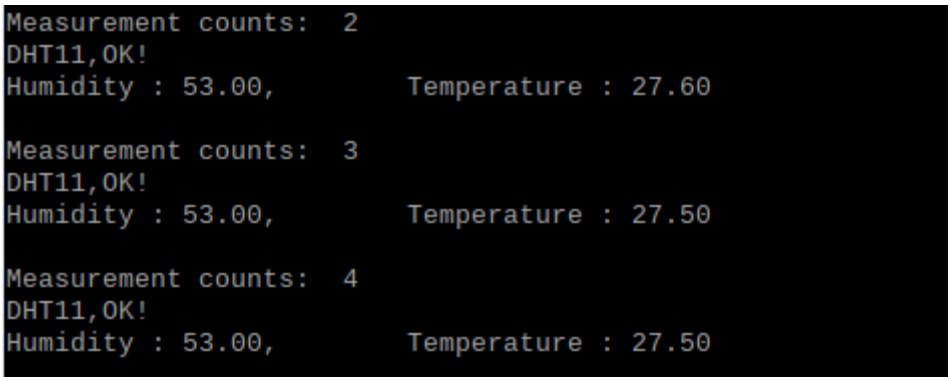
1. Use cd command to enter 20_DHT11 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/20_DHT11
```

2. Use Python command to execute code "DHT11.py".

```
python DHT11.py
```

After the program runs, the Terminal window will display the current total number of read times, the read state, as well as temperature and humidity values as is shown below:



The following is the program code:

```
1  import time
2  from Freenove_DHT import DHT
3
4  DHTPin = 23     #define the pin of DHT11
5
6  def loop():
7      dht = DHT(DHTPin)
8      time.sleep(1)
9      counts = 0      # Measurement counts
10     while(True):
11         counts += 1
```

```
        print("Measurement counts: ", counts)
        for i in range(0,15):
            chk = dht.readDHT11() #read DHT11 and get a return value. Then determine whether
data read is normal according to the return value.
            if (chk == 0):        #read DHT11 and get a return value. Then determine whether
data read is normal according to the return value.
                print("DHT11,OK!")
                break
            time.sleep(0.1)

        print("Humidity : %.2f, \t Temperature : %.2f \n"%(dht.getHumidity(),
dht.getTemperature()))
        time.sleep(2)

if __name__ == '__main__':
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:
        pass
        exit()
```

In this project code, we use a module "**Freenove_DHT**.py", which provides the method of reading the DHT Sensor. It is located in the same directory with program files "**DHT11**.py". By using this library, we can easily read the DHT Sensor. First, we create a DHT class object in the code.

```
    dht = DHT.DHT(DHTPin)    #create a DHT class object
```

Then in the "while" loop, use chk = dht.**readDHT11** (DHT11Pin) to read the DHT11, and determine whether the data read is normal according to the return value "chk".

```
    while(True):
        t=time.time()
        counts += 1
        print("Measurement counts: ", counts)
        for i in range(0,15):
            chk = dht.readDHT11()      #read DHT11 and get a return value. Then determine
whether data read is normal according to the return value.
            if (chk is dht.DHTLIB_OK): #read DHT11 and get a return value. Then determine
whether data read is normal according to the return value.
                print("DHT11,OK!")
                break
            time.sleep(0.1)
        print("Humidity : %.2f, \t Temperature : %.2f \n"%(dht.getHumidity(),
dht.getTemperature()))
        time.sleep(2)
```

Module "**Freenove_DHT.py**" contains a DHT class. The class function of the def **readDHT11** (pin) is used to read the DHT11 Sensor and store the temperature and humidity data read to member variables humidity and temperature.

Freenove_DHT Module
This is a Python module for reading the temperature and humidity data of the DHT Sensor. Partial functions and variables are described as follows: **getHumidity()**: store humidity data read from sensor **getTemperature()**: store temperature data read from sensor **readDHT11()**: read the temperature and humidity of sensor DHT11, and return values used to determine whether the data is normal.

Chapter 21 Matrix Keypad

Earlier we learned about a single Push Button Switch. In this chapter, we will learn about Matrix Keyboards, which integrates a number of Push Button Switches as Keys for the purposes of Input.

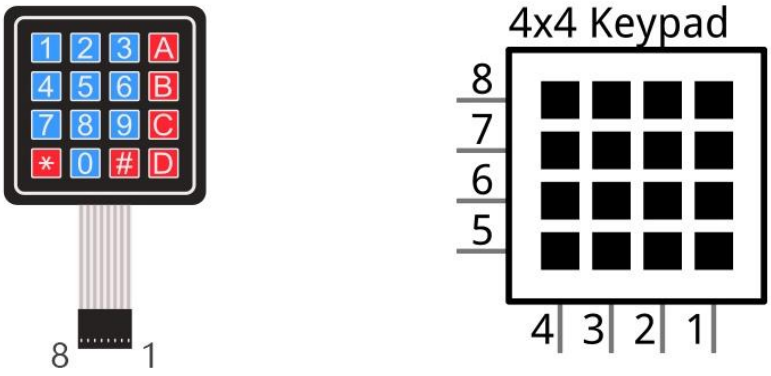
Project 21 Matrix Keypad

In this project, we will attempt to get every key code on the Matrix Keypad to work.

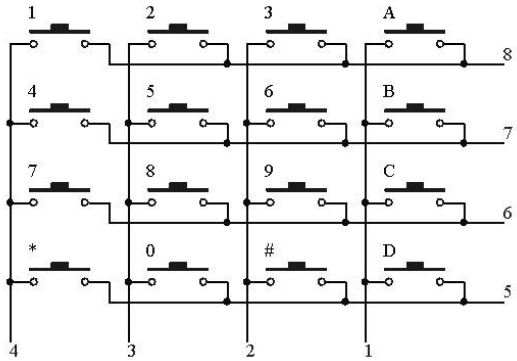
Component knowledge

4x4 Matrix Keypad

A Keypad Matrix is a device that integrates a number of keys in one package. As is shown below, a 4x4 Keypad Matrix integrates 16 keys (think of this as 16 Push Button Switches in one module):



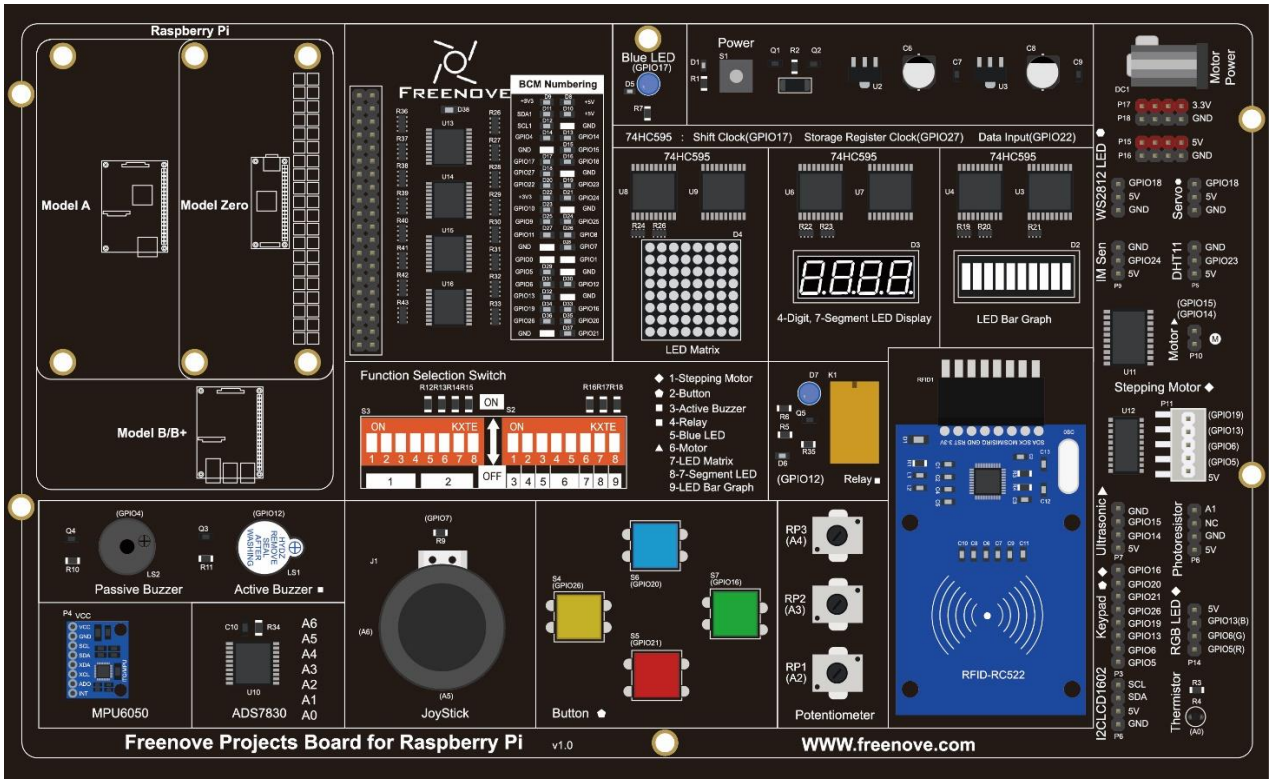
Similar to the integration of an LED Matrix, the 4x4 Keypad Matrix has each row of keys connected with one pin and this is the same for the columns. Such efficient connections reduce the number of processor ports required. The internal circuit of the Keypad Matrix is shown below.



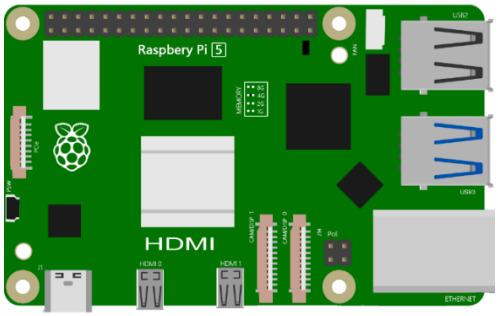
The method of usage is similar to the Matrix LED, by using a row or column scanning method to detect the state of each key's position by column and row. Take column scanning method as an example, send low level to the first 1 column (Pin1), detect level state of rows 5, 6, 7, 8 to judge whether the keys A, B, C, D are pressed. Then send low level to columns 2, 3, 4 in turn to detect whether other keys are pressed. Therefore, you can get the state of all of the keys.

Component List

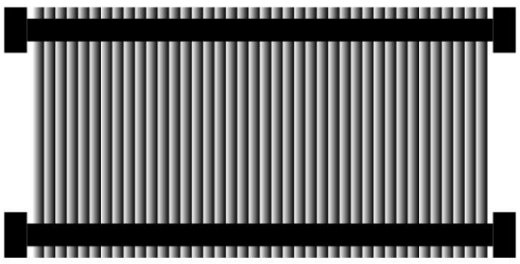
Freenove Projects Board for Raspberry Pi



Raspberry Pi



GPIO Ribbon Cable

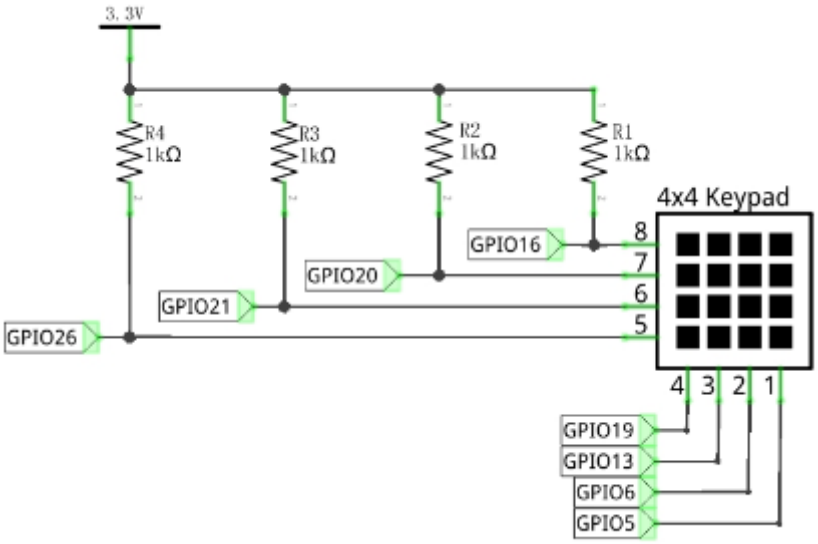


4x4 Matrix Keypad

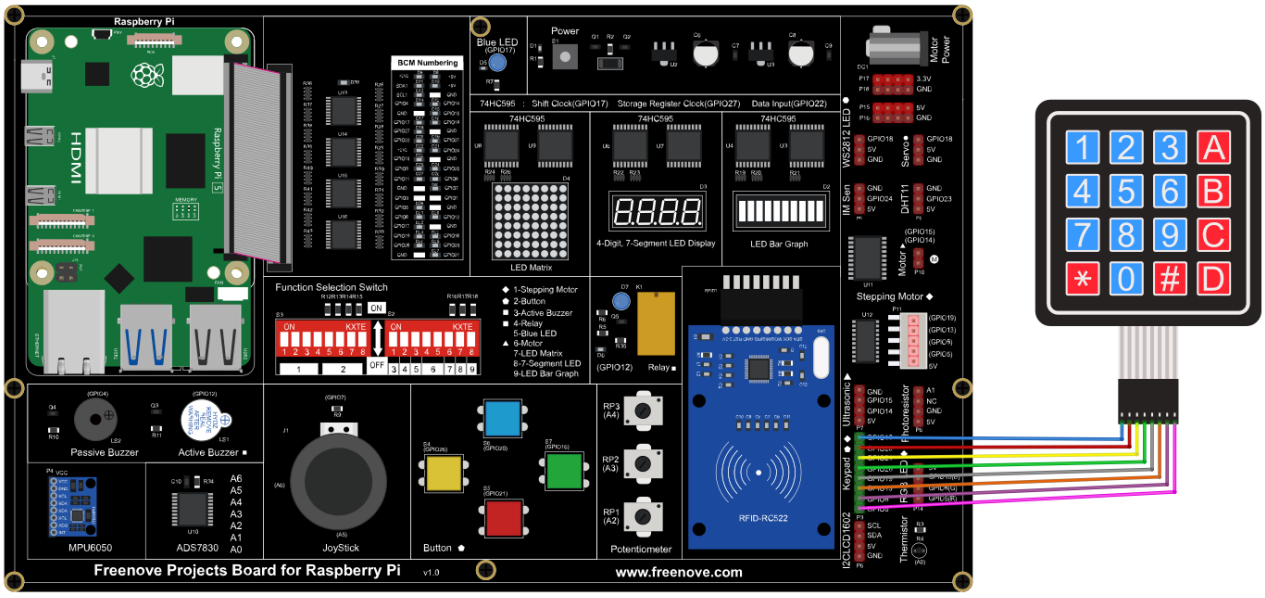


Circuit

Schematic diagram



Hardware connection.



If you have any concerns, please send an email to: support@freenove.com

Code

This code is used to obtain all key codes of the 4x4 Matrix Keypad, when one of the keys is pressed, the key code will be displayed in the terminal window.

C Code 21.1 MatrixKeypad

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 21_MatrixKeypad directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/21_MatrixKeypad
```

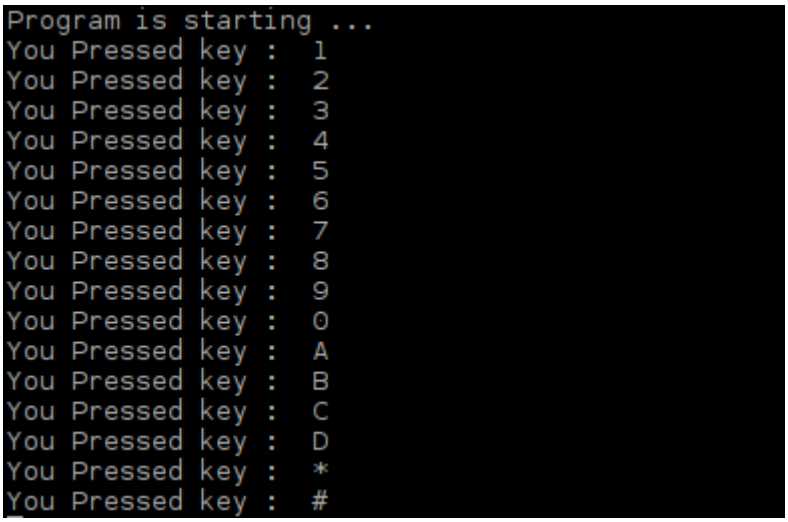
2. Code of this project contains a custom header file. Use the following command to compile the code MatrixKeypad.cpp, Keypad.cpp and Key.cpp generate executable file MatrixKeypad. The custom header file will be compiled at the same time.

```
gcc MatrixKeypad.cpp Keypad.cpp Key.cpp -o MatrixKeypad -lwiringPi
```

3. Run the generated file "MatrixKeypad".

```
./MatrixKeypad
```

After the program runs, pressing any key on the MatrixKeypad, will display the corresponding key code on the Terminal. As is shown below:



The following is the program code:

```
#include "Keypad.hpp"
#include <stdio.h>
const byte ROWS = 4; //four rows
const byte COLS = 4; //four columns
char keys[ROWS][COLS] = {  //key code
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
```

```
byte rowPins[ROWS] = {16,20,21,26 }; //define the row pins for the keypad
byte colPins[COLS] = {19,13,  6,  5};  //define the column pins for the keypad
//create Keypad object
Keypad keypad = Keypad( makeKeymap(keys), rowPins, colPins, ROWS, COLS );

int main(){
    printf("Program is starting ... \n");

    wiringPiSetupGpio();

     char key = 0;
     keypad.setDebounceTime(50);
    while(1){
        key = keypad.getKey();  //get the state of keys
        if (key){       //if a key is pressed, print out its key code
            printf("You Pressed key :  %c \n",key);
        }
    }
    return 1;
}
```

In this project code, we use two custom library file "**Keypad**.hpp" and "**Key**.hpp". They are located in the same directory with program files "**MatrixKeypad**.cpp", "**Keypad**.cpp" and "**Key**.cpp". The Library Keypad is "transplanted" from the Arduino Library Keypad. This library file provides a method to read the Matrix Keyboard's input. By using this library, we can easily read the pressed keys of the Matrix Keyboard.
First, we define the information of the Matrix Keyboard used in this project: the number of rows and columns, code designation of each key and GPIO pin connected to each column and row. It is necessary to include the header file "**Keypad**.hpp".

```
#include "Keypad.hpp"
#include <stdio.h>
const byte ROWS = 4; //four rows
const byte COLS = 4; //four columns
char keys[ROWS][COLS] = {  //key code
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {16,20,21,26 }; //define the row pins for the keypad
byte colPins[COLS] = {19,13,  6,  5};  //define the column pins for the keypad
```

Then, based on the above information, initiates a Keypad class object to operate the Matrix Keyboard.

```
Keypad keypad = Keypad( makeKeymap(keys), rowPins, colPins, ROWS, COLS );
```

Set the debounce time to 50ms, and this value can be set based on the actual characteristics of the keyboard's flexibly, with a default time of 10ms.

```
keypad.setDebounceTime(50);
```

In the "while" loop, use the function key= keypad.**getKey** () to read the keyboard constantly. If there is a key pressed, its key code will be stored in the variable "key", then be displayed.

```
    while(1){
        key = keypad.getKey();  //get the state of keys
        if (key){       // if a key is pressed, print out its key code
            printf("You Pressed key :  %c \n",key);
        }
    }
```

The Keypad Library used for the RPi is transplanted from the Arduino Keypad Library. And the source files can be obtained by visiting http://playground.arduino.cc/Code/Keypad. As for transplanted function library, the function and method of all classes, functions, variables, etc. are the same as the original library. Partial contents of the Keypad library are described below:

class Keypad

Keypad(char *userKeymap, byte *row, byte *col, byte numRows, byte numCols);

Constructor, the parameters are: key code of keyboard, row pin, column pin, the number of rows, the number of columns.

char getKey();

Get the key code of the pressed key. If no key is pressed, the return value is NULL.

void setDebounceTime(uint);

Set the debounce time. And the default time is 10ms.

void setHoldTime(uint);

Set the time when the key holds stable state after pressed.

bool isPressed(char keyChar);

Judge whether the key with code "keyChar" is pressed.

char waitForKey();

Wait for a key to be pressed, and return key code of the pressed key.

KeyState getState();

Get state of the keys.

bool keyStateChanged();

Judge whether there is a change of key state, then return True or False.

For More information about Keypad, please visit: http://playground.arduino.cc/Code/Keypad or through the opening file "Keypad.hpp".

Python Code 21.1 MatrixKeypad

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 21_MatrixKeypad directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/21_MatrixKeypad
```

2. Use Python command to execute code "MatrixKeypad.py".

```
python MatrixKeypad.py
```

After the program runs, pressing any key on the MatrixKeypad, will display the corresponding key code on the Terminal. As is shown below:

```
Program is starting ...
You Pressed Key : 1
You Pressed Key : 2
You Pressed Key : 3
You Pressed Key : 4
You Pressed Key : 5
You Pressed Key : 6
You Pressed Key : 7
You Pressed Key : 8
You Pressed Key : 9
You Pressed Key : *
You Pressed Key : 0
You Pressed Key : #
You Pressed Key : A
You Pressed Key : B
You Pressed Key : C
You Pressed Key : D
```

The following is the program code:

```
import Keypad   #import module Keypad
ROWS = 4        # number of rows of the Keypad
COLS = 4        #number of columns of the Keypad
keys =  [   '1','2','3','A',     #key code
            '4','5','6','B',
            '7','8','9','C',
            '*','0','#','D'      ]
rowsPins = [16, 20, 21, 26]   #connect to the row pinouts of the keypad
colsPins = [19, 13, 6, 5]     #connect to the column pinouts of the keypad
def loop():
    keypad = Keypad.Keypad(keys,rowsPins,colsPins,ROWS,COLS)     #creat Keypad object
    keypad.setDebounceTime(50)       #set the debounce time
    while(True):
        key = keypad.getKey()        #obtain the state of keys
        if(key != keypad.NULL):      #if there is key pressed, print its key code.
            print ("You Pressed Key : %c "%(key))

if __name__ == '__main__':     #Program start from here
    print ("Program is starting ... ")
    try:
        loop()
    except KeyboardInterrupt:  #When 'Ctrl+C' is pressed, exit the program.
        print("Ending program")
```

Import Keypad. Define row and column. Define key value variable. Define row pins and column pins.

```
import Keypad        #import module Keypad
ROWS = 4         # number of rows of the Keypad
COLS = 4         #number of columns of the Keypad
keys =  [   '1','2','3','A',     #key code
            '4','5','6','B',
            '7','8','9','C',
            '*','0','#','D'      ]
rowsPins = [16, 20, 21, 26]   #connect to the row pinouts of the keypad
colsPins = [19, 13, 6, 5]     #connect to the column pinouts of the keypad
```

Then, based on the above information, initiates a Keypad class object to operate the Matrix Keyboard.

```
keypad = Keypad.Keypad(keys,rowsPins,colsPins,ROWS,COLS)     #creat Keypad object
```

Set the debounce time to 100ms, and this value can be set based on the actual characteristics of the keyboard's flexibly, with a default time of 10ms.

```
keypad.setDebounceTime(100)       #set the debounce time
```

In the "while" loop, use the function key= keypad.**getKey** () to read the keyboard constantly. If there is a key pressed, its key code will be stored in the variable "key", and then be displayed.

```
    while(True):
        key = keypad.getKey()        #obtain the state of keys
        if(key != keypad.NULL):      #if there is key pressed, print its key code.
            print ("You Pressed Key : %c "%(key))
```

class Keypad

def __init__(self,usrKeyMap,row_Pins,col_Pins,num_Rows,num_Cols):

Constructed function, the parameters are: key code of keyboard, row pin, column pin, the number of rows, the number of columns.

def getKey(self):

Get a pressed key. If no key is pressed, the return value is keypad NULL.

def setDebounceTime(self,ms):

Set the debounce time. And the default time is 10ms.

def setHoldTime(self,ms):

Set the time when the key holds stable state after pressed.

def isPressed(keyChar):

Judge whether the key with code "keyChar" is pressed.

def waitForKey():

Wait for a key to be pressed, and return key code of the pressed key.

def getState():

Get state of the keys.

def keyStateChanged():

Judge whether there is a change of key state, then return True or False.

Chapter 22 Infrared Motion Sensor

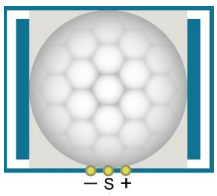
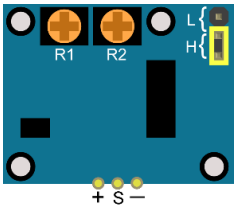
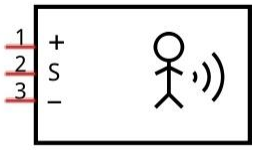
In this chapter, we will learn a widely used sensor, Infrared Motion Sensor.

Project 22.1 PIR Infrared Motion Detector with LED Indicator

In this project, we will make a Motion Detector, with the human body infrared pyroelectric sensors. When someone is in close proximity to the Motion Detector, it will automatically light up and when there is no one close by, it will be out.
This Infrared Motion Sensor can detect the infrared spectrum (heat signatures) emitted by living humans and animals.

Component Knowledge

The following is the diagram of the Infrared Motion Sensor（HC SR-501）a PIR Sensor:

<table>
<tr>
<td>Top
 
</td>
<td>Bottom
 
</td>
<td>Schematic
 

Infrared Motion Sensor(HC SR501)</td>
</tr>
</table>

Description:

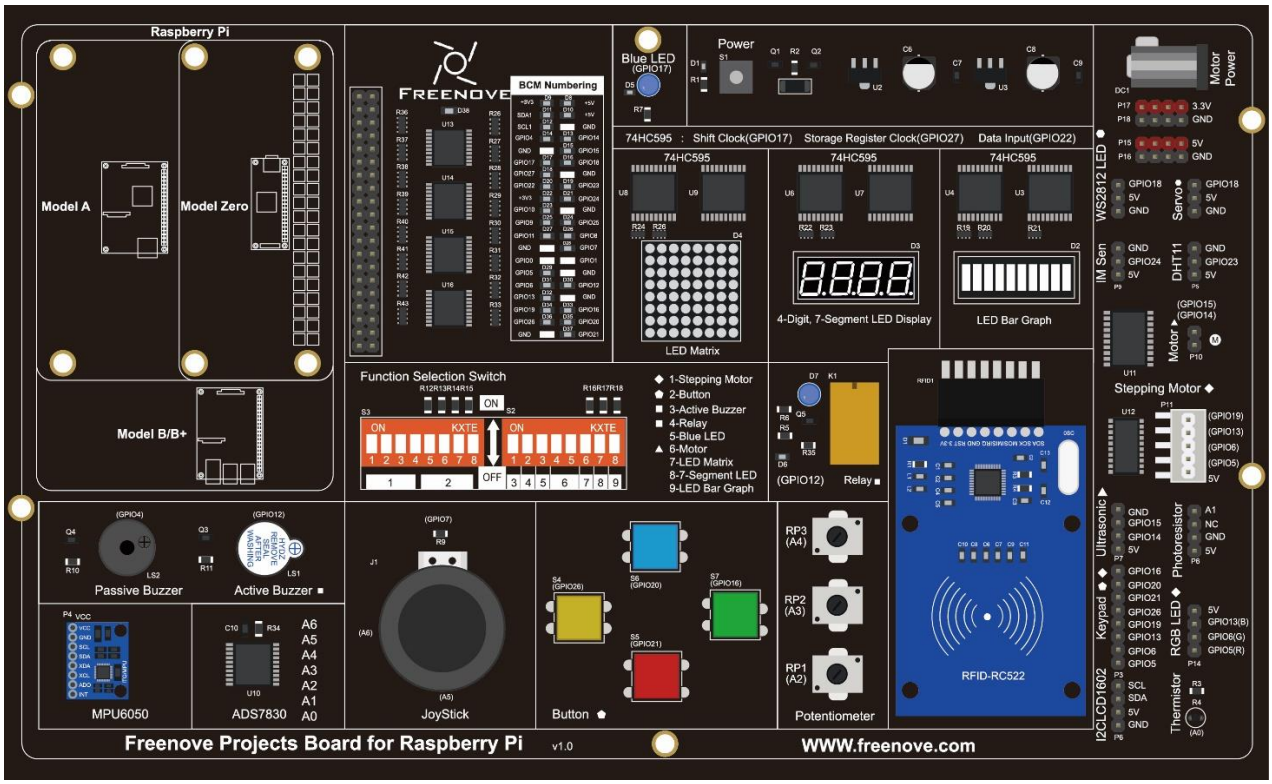
1. Working voltage: 5v-20v(DC)　Static current: 65uA.
2. Automatic Trigger. When a living body enters into the active area of sensor, the module will output high level (3.3V). When the body leaves the sensor's active detection area, it will output high level lasting for time period T, then output low level(0V). Delay time T can be adjusted by the potentiometer R1.
3. According to the position of Fresnel lenses dome, you can choose non-repeatable trigger modes or repeatable modes.
   L: non-repeatable trigger mode. The module outputs high level after sensing a body, then when the delay time is over, the module will output low level. During high level time, the sensor no longer actively senses bodies.
   H: repeatable trigger mode. The distinction from the L mode is that it can sense a body until that body leaves during the period of high level output. After this, it starts to time and output low level after delaying T time.
4. Induction block time: the induction will stay in block condition and does not induce external signal at lesser time intervals (less than delay time) after outputting high level or low level
5. Initialization time: the module needs about 1 minute to initialize after being powered ON. During this period, it will alternately output high or low level.
6. One characteristic of this sensor is when a body moves close to or moves away from the sensor's dome edge, the sensor will work at high sensitively. When a body moves close to or moves away from the sensor's dome in a vertical direction (perpendicular to the dome), the sensor cannot detect well (please

take note of this deficiency). Actually this makes sense when you consider that this sensor is usually placed on a celling as part of a security product. Note: The Sensing Range (distance before a body is detected) is adjusted by the potentiometer.

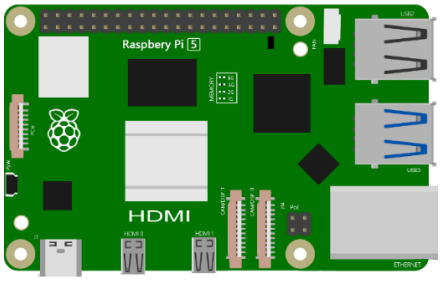
We can regard this sensor as a simple inductive switch when in use.

Component List

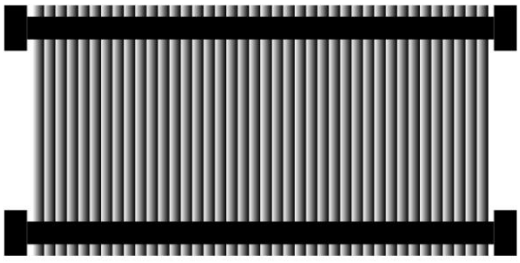
Freenove Projects Board for Raspberry Pi



Raspberry Pi



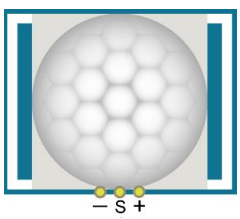
GPIO Ribbon Cable



Jumper Wire

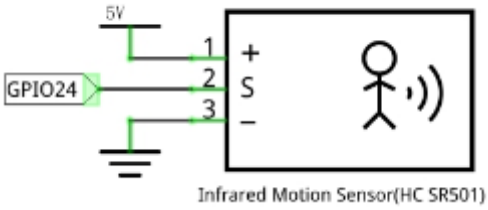


HC SR501

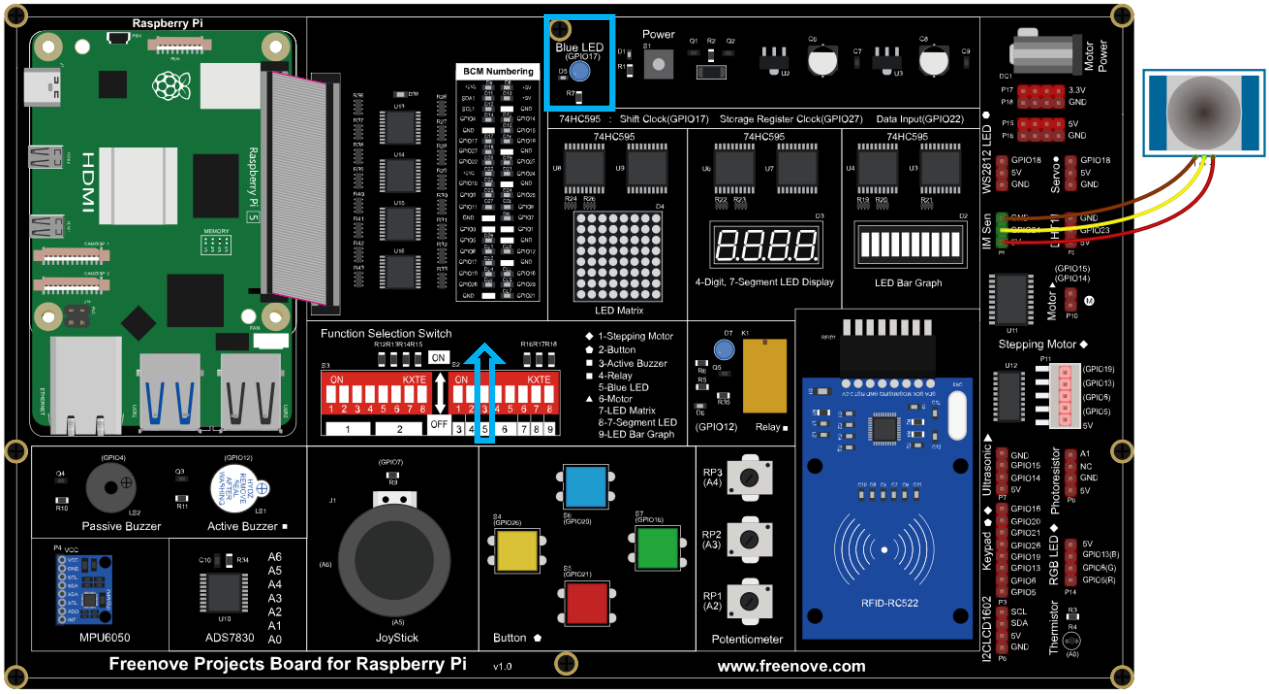


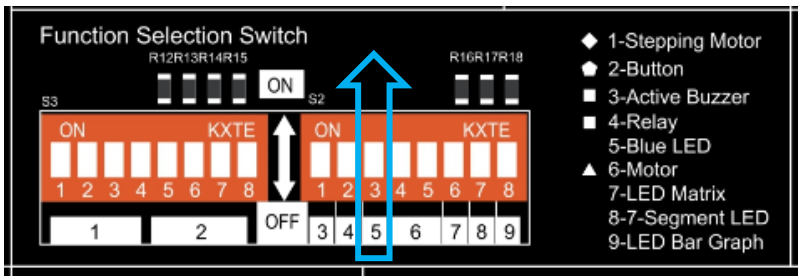
Circuit

Schematic diagram

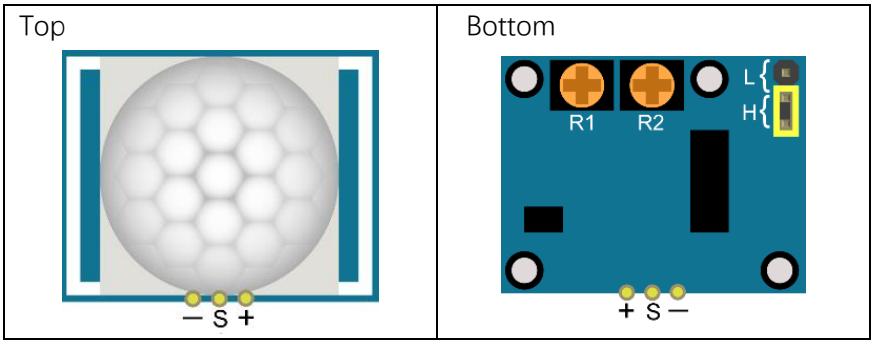


Hardware connection.





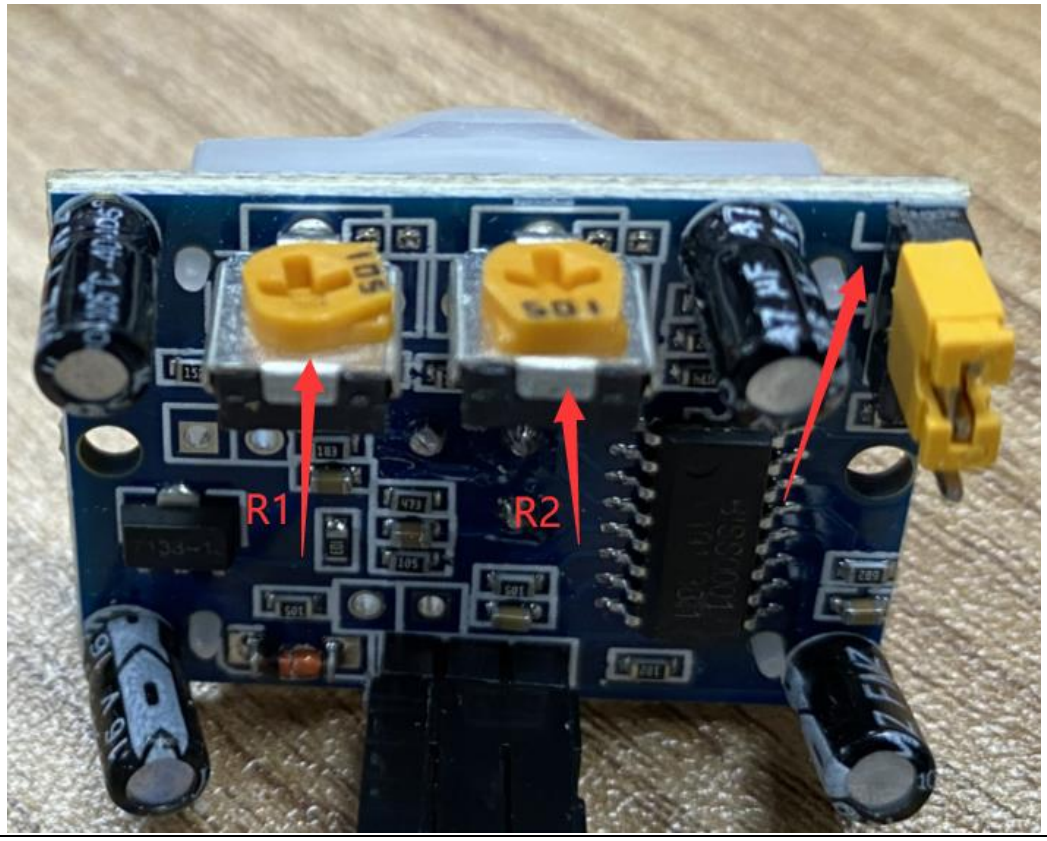
How to use this sensor?



Description:

1. You can choose non-repeatable trigger modes or repeatable modes.
   L: non-repeatable trigger mode. The module outputs high level after sensing a body, then when the delay time is over, the module will output low level. During high level time, the sensor no longer actively senses bodies.
   H: repeatable trigger mode. The distinction from the L mode is that it can sense a body until that body leaves. After this, it starts to time and output low level after delaying T time.
2. R1 is used to adjust HIGH level lasting time when sensor detects human motion, 1.2s-320s.
3. R2 is used to adjust the maxmum distance the sensor can detect, 3~5m.

Here we connect L and adjust R1 and R2 like below to do this project.
Put you hand close and away from the sensor slowly. Obsever the LED in previous circuit.
It need some time between two detections.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project, we will use the Infrared Motion Sensor to trigger an LED, essentially making the Infrared Motion sensor act as a Motion Switch. Therefore, the code is very similar to the earlier project "Push Button Switch and LED". The difference is that, when Infrared Motion Sensor detects change, it will output high level; when button is pressed, it will output low level. When the sensor output high level, the LED turns ON, or it will turn OFF.

C Code 22.1 SenseLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 22_1_InfraredSensor directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/22_1_InfraredSensor
```

2. Use following command to compile "SenseLED.c" and generate executable file "SenseLED".

```
gcc SenseLED.c -o SenseLED -lwiringPi
```

3. Run the generated file "SenseLED".

```
./SenseLED
```

After the program runs, **wait 1 minute for initialization**. Then move away from or move closer to the Infrared Motion Sensor and observe whether the LED turns ON or OFF. The Terminal window will continuously display the state of LED. As is shown below:



The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>

#define ledPin    17    //define the ledPin
#define sensorPin 24    //define the sensorPin

int main(void)
{
    printf("Program is starting ... \n");

    wiringPiSetupGpio();

    pinMode(ledPin, OUTPUT);
    pinMode(sensorPin, INPUT);

```

```
16      while(1){
17
18          if(digitalRead(sensorPin) == HIGH){ //if read value of sensor is HIGH level
19              digitalWrite(ledPin, HIGH);   //make led on
20              printf("led turned on >>> \n");
21          }
22          else {
23              digitalWrite(ledPin, LOW);   //make led off
24              printf("led turned off <<< \n");
25          }
26      }
27
28      return 0;
29  }
```

Python Code 22.1 SenseLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 22_InfraredSensor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/22_InfraredSensor
```

2. Use Python command to execute code "SenseLED.py".

```
python SenseLED.py
```

After the program runs, **wait 1 minute for initialization**. Then move away from or move closer to the Infrared Motion Sensor and observe whether the LED turns ON or OFF. The Terminal window will continuously display the state of LED. As is shown below:



The following is the program code:

```
from gpiozero import LED, Button

led = LED(17)
button = Button(24)

def loop():
    key_state = 0
    while True:
        if button.is_pressed==True and key_state == 0:  # if button is pressed
            led.on()        # turn on led
            key_state = 1
            print("Button is pressed, led turned on >>>")
        elif button.is_pressed==False and key_state == 1:
            led.off() # turn off led
            key_state = 0
            print("Button is released, led turned off <<<")

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
    finally:
        led.close()
        button.close()
```

Chapter 23 Ultrasonic Ranging

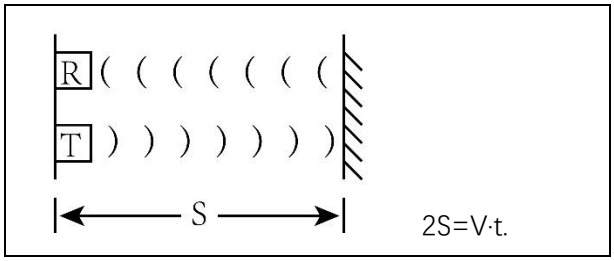
In this chapter, we learn a module which use ultrasonic to measure distance, HC SR04.

Project 23.1 Ultrasonic Ranging

In this project, we use ultrasonic ranging module to measure distance, and print out the data in the terminal.

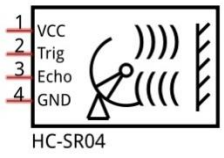
Component Knowledge

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will be reflected when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (delta) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about v=340m/s, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: s=vt/2.



The HC-SR04 Ultrasonic Ranging Module integrates both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the HC SR04 Ultrasonic Ranging Module are shown below:





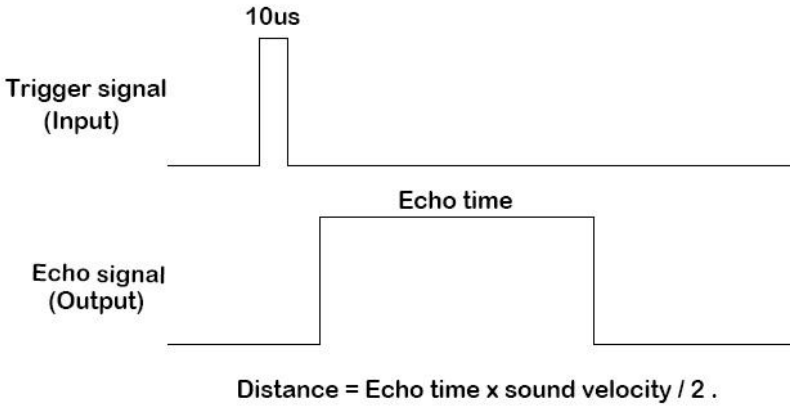
Pin description:

VCC	power supply pin
Trig	trigger pin
Echo	Echo pin
GND	GND

Technical specs:

Working voltage: 5V
Working current: 12mA
Minimum measured distance: 2cm
Maximum measured distance: 200cm

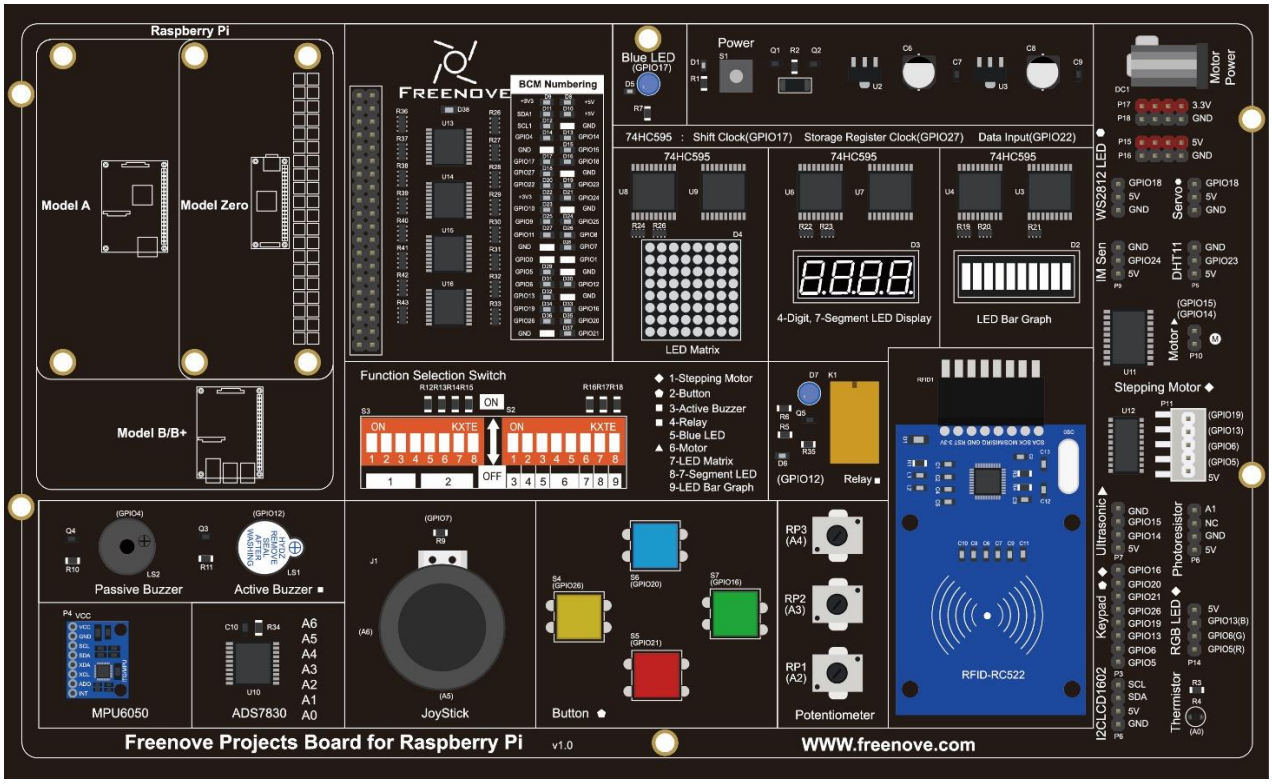
Instructions for Use: output a high-level pulse in Trig pin lasting for least 10uS, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, s=vt/2. This is done constantly.



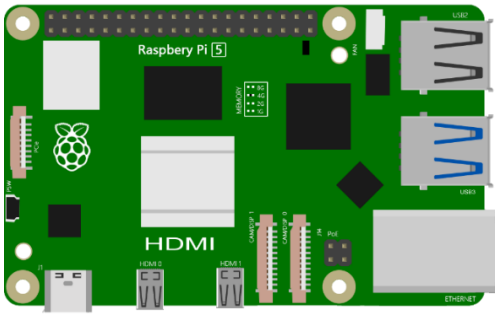
Distance = Echo time x sound velocity / 2 .

Component List

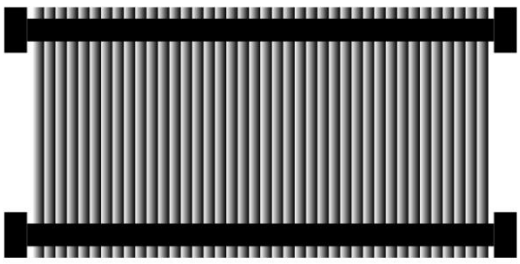
Freenove Projects Board for Raspberry Pi



Raspberry Pi



GPIO Ribbon Cable



Jumper Wire

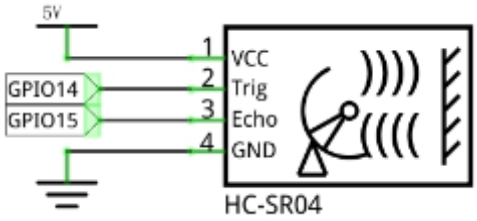


HC SR04



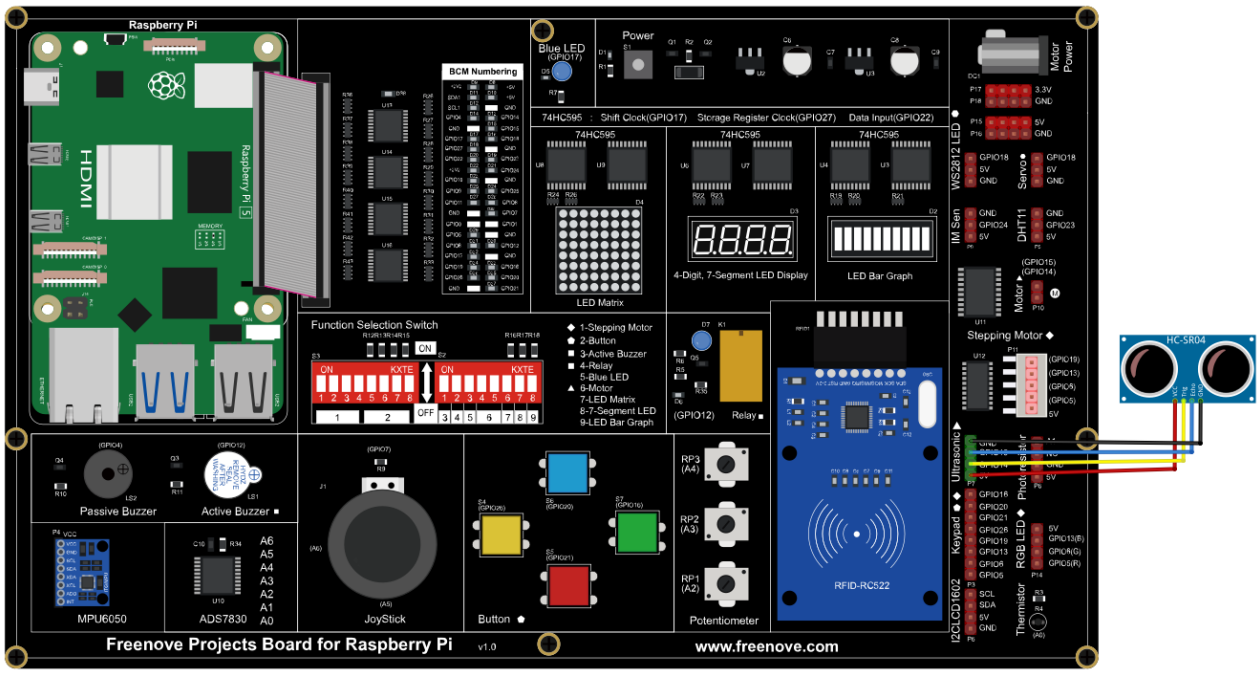
Circuit

Schematic diagram



Hardware connection.

After running the program, hold an object in front of the sensor and change their distance.



If you have any concerns, please send an email to: support@freenove.com

Code

C Code 23.1 UltrasonicRanging

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 23_UltrasonicRanging directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/23_UltrasonicRanging
```

2. Use following command to compile "UltrasonicRanging.c" and generate executable file "UltrasonicRanging".

```
gcc UltrasonicRanging.c -o UltrasonicRanging -lwiringPi
```

3. Then run the generated file "UltrasonicRanging".

```
./UltrasonicRanging
```

After the program runs, aim the Ultrasonic Ranging Module's detectors ("eyes") perpendicular to the surface of an object (try using your hand). The distance between the ultrasonic module and the object will be displayed in the terminal. As is shown below:

```
The distance is : 198.82 cm
The distance is : 198.37 cm
The distance is : 198.37 cm
The distance is : 199.63 cm
The distance is : 197.52 cm
The distance is : 198.39 cm
The distance is : 198.41 cm
```

The following is the program code:

```
#include <wiringPi.h>
#include <stdio.h>
#include <sys/time.h>

#define trigPin 14
#define echoPin 15
#define MAX_DISTANCE 220        // define the maximum measured distance
#define timeOut MAX_DISTANCE*60 // calculate timeout according to the maximum measured
distance
//function pulseIn: obtain pulse time of a pin
int pulseIn(int pin, int level, int timeout);
float getSonar(){   //get the measurement result of ultrasonic module with unit: cm
    long pingTime;
    float distance;
    digitalWrite(trigPin,HIGH); //send 10us high level to trigPin
    delayMicroseconds(10);
    digitalWrite(trigPin,LOW);
    pingTime = pulseIn(echoPin,HIGH,timeOut);    //read plus time of echoPin
    distance = (float)pingTime*340.0/2.0/10000.0;//calculate distance with sound speed 340m/s
    return distance;
```

```
}

int main(){
    printf("Program is starting ... \n");

    wiringPiSetupGpio();

    float distance = 0;
    pinMode(trigPin,OUTPUT);
    pinMode(echoPin,INPUT);
    while(1){
        distance = getSonar();
        printf("The distance is : %.2f cm\n",distance);
        delay(1000);
    }
    return 1;
}

int pulseIn(int pin, int level, int timeout)
{
   struct timeval tn, t0, t1;
   long micros;
   gettimeofday(&t0, NULL);
   micros = 0;
   while (digitalRead(pin) != level)
   {
      gettimeofday(&tn, NULL);
      if (tn.tv_sec > t0.tv_sec) micros = 1000000L; else micros = 0;
      micros += (tn.tv_usec - t0.tv_usec);
      if (micros > timeout) return 0;
   }
   gettimeofday(&t1, NULL);
   while (digitalRead(pin) == level)
   {
      gettimeofday(&tn, NULL);
      if (tn.tv_sec > t0.tv_sec) micros = 1000000L; else micros = 0;
      micros = micros + (tn.tv_usec - t0.tv_usec);
      if (micros > timeout) return 0;
   }
   if (tn.tv_sec > t1.tv_sec) micros = 1000000L; else micros = 0;
   micros = micros + (tn.tv_usec - t1.tv_usec);
   return micros;
}
```

First, define the pins and the maximum measurement distance.

```
#define trigPin 14
#define echoPin 15
#define MAX_DISTANCE 220        // define the maximum measured distance
```

If the module does not return high level, we cannot wait for this forever, so we need to calculate the time period for the maximum distance, that is, time Out. **timeOut= 2*MAX_DISTANCE/100/340*1000000**. The result of the constant part in this formula is approximately 58.8.

```
#define timeOut MAX_DISTANCE*60 // calculate timeout according to the maximum measured
distance
```

Subfunction **getSonar** () function is used to start the Ultrasonic Module to begin measurements and return the measured distance in cm units. In this function, first let trigPin send 10us high level to start the Ultrasonic Module. Then use **pulseIn** () to read the Ultrasonic Module and return the duration time of high level. Finally, the measured distance according to the time is calculated.

```
float getSonar(){   //get the measurement result of ultrasonic module with unit: cm
    long pingTime;
    float distance;
    digitalWrite(trigPin,HIGH); //send 10us high level to trigPin
    delayMicroseconds(10);
    digitalWrite(trigPin,LOW);
    pingTime = pulseIn(echoPin,HIGH,timeOut);    //read plus time of echoPin
    distance = (float)pingTime*340.0/2.0/10000.0;//calculate distance with sound speed 340m/s
    return distance;
}
```

Lastly, in the while loop of main function, get the measurement distance and display it continually.

```
while(1){
        distance = getSonar();
        printf("The distance is : %.2f cm\n",distance);
        delay(1000);
    }
```

About function **pulseIn**():

int pulseIn(int pin, int level, int timeout);

Return the length of the pulse (in microseconds) or 0 if no pulse is completed before the timeout (unsigned long).

Python Code 23.1 UltrasonicRanging

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 23_UltrasonicRanging directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/23_UltrasonicRanging
```

2. Use Python command to execute code "UltrasonicRanging.py".

```
python UltrasonicRanging.py
```

After the program runs, aim the Ultrasonic Ranging Module's detectors ("eyes") perpendicular to the surface of an object (try using your hand). The distance between the ultrasonic module and the object will be displayed in the terminal. As is shown below:

```
The distance is : 198.75 cm
The distance is : 199.22 cm
The distance is : 198.42 cm
The distance is : 198.74 cm
The distance is : 198.37 cm
The distance is : 198.47 cm
The distance is : 198.41 cm
```

The following is the program code:

```
1  from gpiozero import DistanceSensor
2  from time import sleep
3  import warnings
4  warnings.filterwarnings("ignore")
5
6  trigPin = 14
7  echoPin = 15
8  sensor = DistanceSensor(echo=echoPin, trigger=trigPin ,max_distance=3)
9
10 def loop():
11     while True:
12         print('Distance: ', sensor.distance * 100,'cm')
13         sleep(1)
14
15 if __name__ == '__main__':     # Program entrance
16     print ('Program is starting...')
17     try:
18         loop()
19     except KeyboardInterrupt:  # Press ctrl-c to end the program.
20         print("Ending program")
21     finally:
22         sensor.close()
```

Importing libraries and modules, gpiozero library integrates a module for ultrasonic sensors **DistanceSensor**, we can use it to obtain ultrasonic sensor data. Use the warning module to mask some unimportant warnings.

```
from gpiozero import DistanceSensor
from time import sleep
import warnings
warnings.filterwarnings("ignore")
```

Set the pin of the ultrasonic data and initialize it. Note that max_distance is measured in meters.

```
trigPin = 14
echoPin = 15
sensor = DistanceSensor(echo=echoPin, trigger=trigPin, max_distance=3)
```

The ultrasonic sensor data is obtained using Sensor.distance and converted to a value in centimeters.

```
def loop():
    while True:
        print('Distance: ', sensor.distance * 100,'cm')
        sleep(1)
```

Release the GPIO resource.

```
sensor.close()
```

Chapter 24 Attitude Sensor MPU6050

In this chapter, we will learn about a MPU6050 Attitude sensor, which integrates an Accelerometer and Gyroscope.

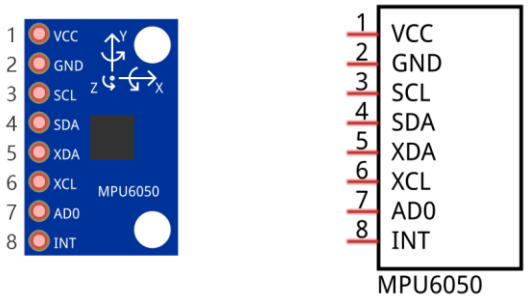
Project 24.1 Read an MPU6050 Sensor Module

In this project, we will read Acceleration and Gyroscope Data of the MPU6050 Sensor.

Component knowledge

MPU6050

MPU6050 Sensor Module is a complete 6-axis Motion Tracking Device. It combines a 3-axis Gyroscope, a 3-axis Accelerometer and a DMP (Digital Motion Processor) all in a small package. The settings of the Accelerometer and Gyroscope of MPU6050 can be changed. A precision wide range digital temperature sensor is also integrated to compensate data readings for changes in temperature, and temperature values can also be read. The MPU6050 Module follows the I2C communication protocol and the default address is 0x68.



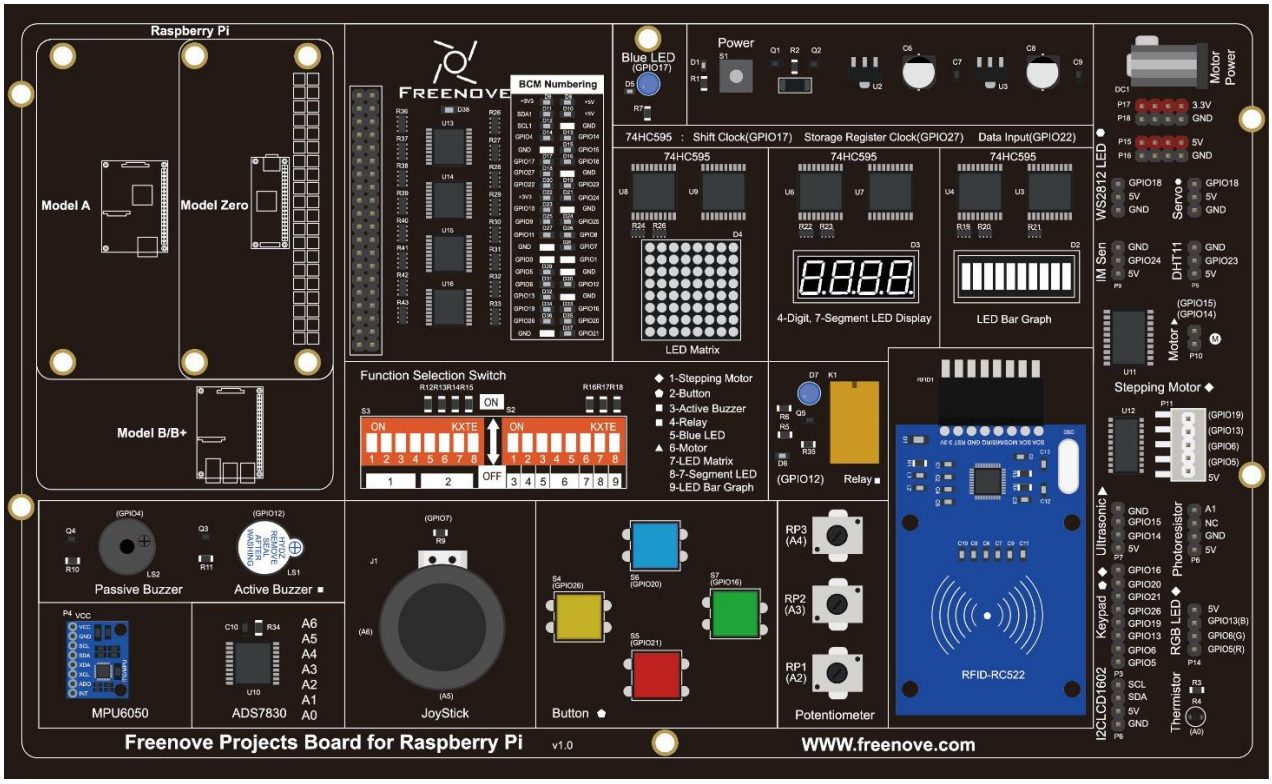
The port description of the MPU6050 Module is as follows:

Pin name	Pin number	Description
VCC	1	Positive pole of power supply with voltage 5V
GND	2	Negative pole of power supply
SCL	3	I2C communication clock pin
SDA	4	I2C communication data pin
XDA	5	I2C host data pin which can be connected to other devices.
XCL	6	I2C host clock pin which can be connected to other devices.
AD0	7	I2C address bit control pin. Low level: the device address is 0x68 High level: the device address is 0x69
INT	8	Output interrupt pin

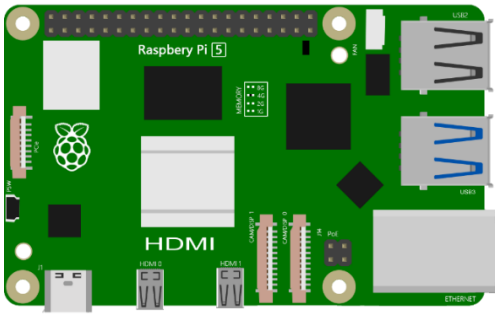
For more detail, please refer to the MPU6050 datasheet.

Component List

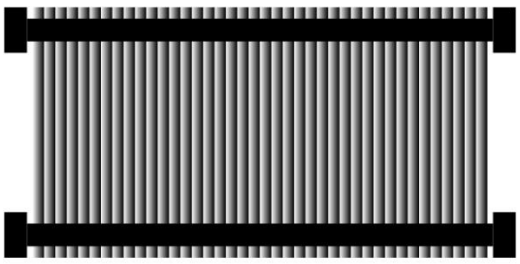
Freenove Projects Board for Raspberry Pi



Raspberry Pi

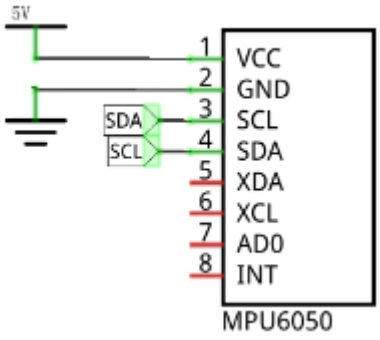


GPIO Ribbon Cable



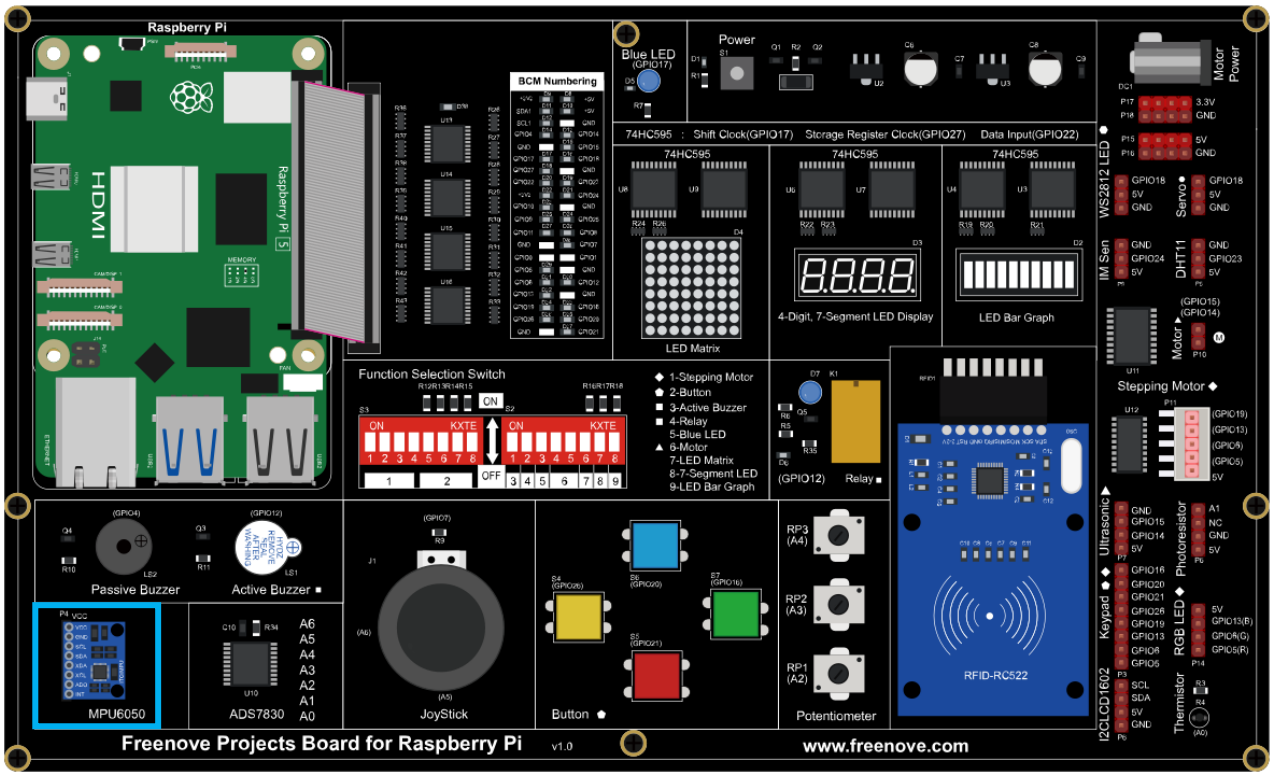
Circuit

Schematic diagram



Hardware connection.

After running the program, hold the board and turn it over to observe the changes in the running results.



If you have any concerns, please send an email to: support@freenove.com

Code

In this project, we will read the acceleration data and gyroscope data of MPU6050, and print them out.

C Code 24.1 MPU6050RAW

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 24_MPU6050 directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/24_MPU6050
```

2. Use following command to compile "MPU6050RAW.c", "MPU6050.cpp" and "I2Cdev.cpp", and generate executable file "MPU6050RAW".

```
gcc MPU6050RAW.cpp MPU6050.cpp I2Cdev.cpp -o MPU6050RAW
```

3. Then run the generated file "MPU6050RAW".

```
./MPU6050RAW
```

After the program runs, the Terminal will display active accelerometer and gyroscope data of the MPU6050, as well as the conversion to gravity acceleration and angular velocity as units of data. As shown in the following figure:

```
a/g:   1360    120  15840     -320   -193   -114
a/g: 0.08 g 0.01 g 0.97 g   -2.44 d/s -1.47 d/s -0.87 d/s
a/g:   1108    -88  15476     -354   -252   -115
a/g: 0.07 g -0.01 g 0.94 g   -2.70 d/s -1.92 d/s -0.88 d/s
a/g:   1344   -264  15764     -396   -236   -121
a/g: 0.08 g -0.02 g 0.96 g   -3.02 d/s -1.80 d/s -0.92 d/s
a/g:   1440   -180  15720     -375   -162   -114
a/g: 0.09 g -0.01 g 0.96 g   -2.86 d/s -1.24 d/s -0.87 d/s
a/g:   1436     56  16608     -400   -154   -136
a/g: 0.09 g 0.00 g 1.01 g   -3.05 d/s -1.18 d/s -1.04 d/s
a/g:   1008    144  14940     -345   -142   -129
a/g: 0.06 g 0.01 g 0.91 g   -2.63 d/s -1.08 d/s -0.98 d/s
```

The following is the program code:

```
1  #include <stdio.h>
2  #include <stdint.h>
3  #include <unistd.h>
4  #include "I2Cdev.h"
5  #include "MPU6050.h"
6
7  MPU6050 accelgyro;      //creat MPU6050 class object
8
9  int16_t ax, ay, az;     //store acceleration data
10 int16_t gx, gy, gz;     //store gyroscope data
11
12 void setup() {
13     // initialize device
14     printf("Initializing I2C devices...\n");
```

```
    accelgyro.initialize();     //initialize MPU6050

    // verify connection
    printf("Testing device connections...\n");
    printf(accelgyro.testConnection() ? "MPU6050 connection successful\n" : "MPU6050
connection failed\n");
}

void loop() {
    // read accel/gyro values of MPU6050
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    // display accel/gyro x/y/z values
    printf("a/g: %6hd %6hd %6hd   %6hd %6hd %6hd\n",ax,ay,az,gx,gy,gz);
    printf("a/g: %.2f g %.2f g %.2f g   %.2f d/s %.2f d/s %.2f d/s
\n",(float)ax/16384,(float)ay/16384,(float)az/16384,
        (float)gx/131,(float)gy/131,(float)gz/131);
}

int main()
{
    setup();
    while(1){
        loop();
    }
    return 0;
}
```

Two library files "**MPU6050**.h" and "**I2Cdev**.h" are used in the code and will be compiled with others. Class MPU6050 is used to operate the MPU6050 Sensor. When used, first it initiates an object.

```
MPU6050 accelgyro;      //creat MPU6050 class object
```

In the setup function, the MPU6050 is initialized and the result of the initialization will be tested.

```
void setup() {
    // initialize device
    printf("Initializing I2C devices...\n");
    accelgyro.initialize();     //initialize MPU6050

    // verify connection
    printf("Testing device connections...\n");
    printf(accelgyro.testConnection() ? "MPU6050 connection successful\n" : "MPU6050
connection failed\n");
}
```

In the loop function, read the original data of MPU6050, display them and then convert the original data into the corresponding acceleration and angular velocity values, then display the converted data out.

```
void loop() {
    // read accel/gyro values of MPU6050
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    // display accel/gyro x/y/z values
    printf("a/g: %6hd %6hd %6hd   %6hd %6hd %6hd\n",ax,ay,az,gx,gy,gz);
    printf("a/g: %.2f g %.2f g %.2f g   %.2f d/s %.2f d/s %.2f d/s
\n",(float)ax/16384,(float)ay/16384,(float)az/16384,
        (float)gx/131,(float)gy/131,(float)gz/131);
}
```

Finally, the main functions, called setup function and loop function respectively.

```
int main()
{
    setup();
    while(1){
        loop();
    }
    return 0;
}
```

About class MPU6050:

Class MPU6050

This is a class library used to operate the MPU6050, which can directly read and set the MPU6050. Here are its functions:

MPU6050()/MPU6050(uint8_t address):

Constructor. The parameter is I2C address, and the default I2C address is 0x68.

void initialize();

Initialization function, used to wake up MPU6050. Range of accelerometer is ±2g and range of gyroscope is ±250 degrees/sec.

void getMotion6(int16_t* ax, int16_t* ay, int16_t* az, int16_t* gx, int16_t* gy, int16_t* gz);

Get the original data of accelerometer and gyroscope.

int16_t getTemperature();

Get the original temperature data of MPU6050.

Python Code 24.1 MPU6050RAW

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 24_MPU6050 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/24_MPU6050
```

2. Use Python command to execute code "MPU6050RAW.py".

```
python MPU6050RAW.py
```

After the program runs, the Terminal will display active accelerometer and gyroscope data of the MPU6050, as well as the conversion to gravity acceleration and angular velocity as units of data. As shown in the following figure:

```
a/g:1326        -160    16548   -48     -25     -16
a/g:0.08 g      -0.01 g 1.01 g  -0.37 d/s        -0.19 d/s       -0.12 d/s
a/g:1174        -116    15972   -44     -25     -17
a/g:0.07 g      -0.01 g 0.97 g  -0.34 d/s        -0.19 d/s       -0.13 d/s
a/g:1134        -130    16066   -45     -21     -17
a/g:0.07 g      -0.01 g 0.98 g  -0.34 d/s        -0.16 d/s       -0.13 d/s
a/g:1234        -76     15976   -45     -30     -16
a/g:0.08 g      -0.00 g 0.98 g  -0.34 d/s        -0.23 d/s       -0.12 d/s
a/g:996 -88     15748   -45     -22     -16
a/g:0.06 g      -0.01 g 0.96 g  -0.34 d/s        -0.17 d/s       -0.12 d/s
a/g:1196        -174    16182   -46     -25     -15
a/g:0.07 g      -0.01 g 0.99 g  -0.35 d/s        -0.19 d/s       -0.11 d/s
```

The following is the program code:

```
1  import MPU6050
2  import time
3
4  mpu = MPU6050.MPU6050()      # instantiate a MPU6050 class object
5  accel = [0]*3                # define an arry to store accelerometer data
6  gyro = [0]*3                 # define an arry to store gyroscope data
7  def setup():
8      mpu.dmp_initialize()     # initialize MPU6050
9
10 def loop():
11     while(True):
12         accel = mpu.get_acceleration()      # get accelerometer data
13         gyro = mpu.get_rotation()           # get gyroscope data
14         print("a/g:%d\t%d\t%d\t%d\t%d\t%d
15 "%(accel[0],accel[1],accel[2],gyro[0],gyro[1],gyro[2]))
16         print("a/g:%.2f g\t%.2f g\t%.2f g\t%.2f d/s\t%.2f d/s\t%.2f
17 d/s"%(accel[0]/16384.0,accel[1]/16384.0,
18             accel[2]/16384.0,gyro[0]/131.0,gyro[1]/131.0,gyro[2]/131.0))
19         time.sleep(0.1)
20
21 if __name__ == '__main__':     # Program entrance
22     print("Program is starting ... ")
```

```
23      setup()
24      try:
25          loop()
26      except KeyboardInterrupt:  # Press ctrl-c to end the program.
27          pass
```

A module "**MPU6050**.py" is used in the code. The module includes a class used to operate MPU6050. When using it, first initiate an object.

```
mpu = MPU6050.MPU6050()      # instantiate a MPU6050 class object
```

In the setup function, the MPU6050 is initialized.

```
def setup():
    mpu.dmp_initialize()
```

In the loop function, read the original data of MPU6050, display them and then convert the original data into the corresponding acceleration and angular velocity values, then display the converted data out.

```
def loop():
    while(True):
        accel = mpu.get_acceleration()       #get accelerometer data
        gyro = mpu.get_rotation()            #get gyroscope data
        print("a/g:%d\t%d\t%d\t%d\t%d\t%d
"%(accel[0],accel[1],accel[2],gyro[0],gyro[1],gyro[2]))
        print("a/g:%.2f g\t%.2f g\t%.2f g\t%.2f d/s\t%.2f d/s\t%.2f
d/s"%(accel[0]/16384.0,accel[1]/16384.0,
            accel[2]/16384.0,gyro[0]/131.0,gyro[1]/131.0,gyro[2]/131.0))
        time.sleep(0.1)
```

Class MPU6050

This is a class library used to operate MPU6050, which can directly read and set MPU6050. Here are some member functions:

```
def __init__(self, a_bus=1, a_address=C.MPU6050_DEFAULT_ADDRESS,
                   a_xAOff=None, a_yAOff=None, a_zAOff=None, a_xGOff=None,
                   a_yGOff=None, a_zGOff=None, a_debug=False):
```

Constructor

```
def dmp_initialize(self):
```

Initialization function, used to wake up MPU6050. Range of accelerometer is ±2g and range of gyroscope is ±250 degrees/sec.

```
def get_acceleration(self):   &   def get_rotation(self):
```

Get the original data of accelerometer and gyroscope.

For details of more relevant member functions, please refer to MPU6050.py in the code folder.

Chapter 25 RFID

In this chapter, we will learn how to use RFID.

Project 25.1 RFID

In this project, we will use RC522 RFID card reader to read and write the M1-S50 card.

Component Knowledge

RFID

RFID（Radio Frequency Identification）is a form of wireless communication technology. A complete RFID system is generally composed of a transponder and a reader. Generally, the transponder may be known as a tag, and each tag has a unique code, which is attached to an object to identify the target object. The reader is a device that reads (or writes) information in the tag.

Products derived from RFID technology can be divided into three categories: passive RFID products, active RFID products and semi active RFID products, among which, Passive RFID products are the earliest, the most mature and most widely used products in the market. It can be seen everywhere in our daily life such as, the bus card, dining card, bank card, hotel access cards, etc., and all of them are classified as close-range contact recognition. The main operating frequency of Passive RFID products are: 125KHZ (low frequency), 13.56MHZ (high frequency), 433MHZ (ultrahigh frequency), 915MHZ (ultrahigh frequency). Active and semi active RFID products work at higher frequencies.

The RFID module we use is a passive RFID product with the operating frequency of 13.56MHz.

MFRC522

The MFRC522 is a highly integrated reader/writer IC for contactless communication at 13.56MHz.

The MFRC522's internal transmitter is able to drive a reader/writer antenna designed to communicate with ISO/IEC 14443 A/MIFARE cards and transponders without additional active circuitry. The receiver module provides a robust and efficient implementation for demodulating and decoding signals from ISO/IEC 14443 A/MIFARE compatible cards and transponders. The digital module manages the complete ISO/IEC 14443A framing and error detection (parity and CRC) functionality

This RFID Module uses MFRC522 as the control chip, and SPI (Peripheral Interface Serial) as the reserved interface.

Technical specs:

Operating Voltage	13-26mA(DC)\3.3V
Idle current	10-13mA(DC)\3.3V
Sleep current in the	<80uA
Peak current	<30mA
Operating frequency	13.56MHz
Supported card type	Mifare1 S50、Mifare1 S70、Mifare Ultralight、Mifare Pro、Mifare Desfire
Size	40mmX60mm

Operation temperature	20-80 degrees(Celsius)
Storage temperature	40-85 degrees (Celsius)
Operation humidity	5%-95%(Relative humidity)

Mifare1 S50 Card

Mifare S50 is often called Mifare Standard with the capacity of 1K bytes. And each card has a 4-bytes global unique identifier number (USN/UID), which can be rewritten 100 thousand times and read infinite times. Its storage period can last for 10 years.

The Mifare S50 capacity (1K byte) is divided into 16 sectors (Sector0-Sector15). Each sector contains 4 data block (Block0-Block3. 64 blocks of 16 sectors will be numbered according absolute address, from 0 to 63).

And each block contains 16 bytes (Byte0-Byte15), 64*16=1024. As is shown in the following table:

Sector No.	Block No.	Storage area	Block type	Absolute block No.
sector 0	block 0	vendor code	vendor block	0
	block 1		data block	1
	block 2		data block	2
	block 3	Password A-access control-password B	control block	3
sector 1	block 0		data block	4
	block 1		data block	5
	block 2		data block	6
	block 3	Password A-access control-password B	control block	7
.....	.....	.....	.....	
sector 15	block 0		data block	60
	block 1		data block	61
	block 2		data block	62
	block 3	Password A-access control-password B	control block	63

Each sector has a set of independent password and access control put in its last block, that is, Block 3, which is also known as sector trailer. Sector 0, block 0 (namely absolute address 0) of S50 is used to store the card serial number and vendor code, which has been solidified and can't be changed. Except the manufacturer and the control block, the rest of the cards are data blocks, which can be used to store data. Data block can be used for two kinds of applications:

(1) used as general data storage and can be operated for reading and writing data.

(2) used as data value, and can be operated for initializing, adding, subtracting and reading the value.

The sector trailer block in each sector is the control block, including a 6-byte password A, a 4-byte access control and a 6-byte password B. For example, the control block of a brand new card is as follows:

A0 A1 A2 A3 A4 A5	FF 07 80 69	B0 B1 B2 B3 B4 B5
password A	access control	password B

The default password of a brand new card is generally 0A1A2A3A4A5 for password A and B0B1B2B3B4B5 for password B, or both the password A and password B are 6 FF. Access control is used to set the access conditions for each block (including the control block itself) in a sector.

Blocks of S50 are divided into data blocks and control blocks. There are four operations, "read", "write", "add value", "subtract value (including transmission and storage)" for data blocks, and there are two operations, "read" and "write" for control blocks.

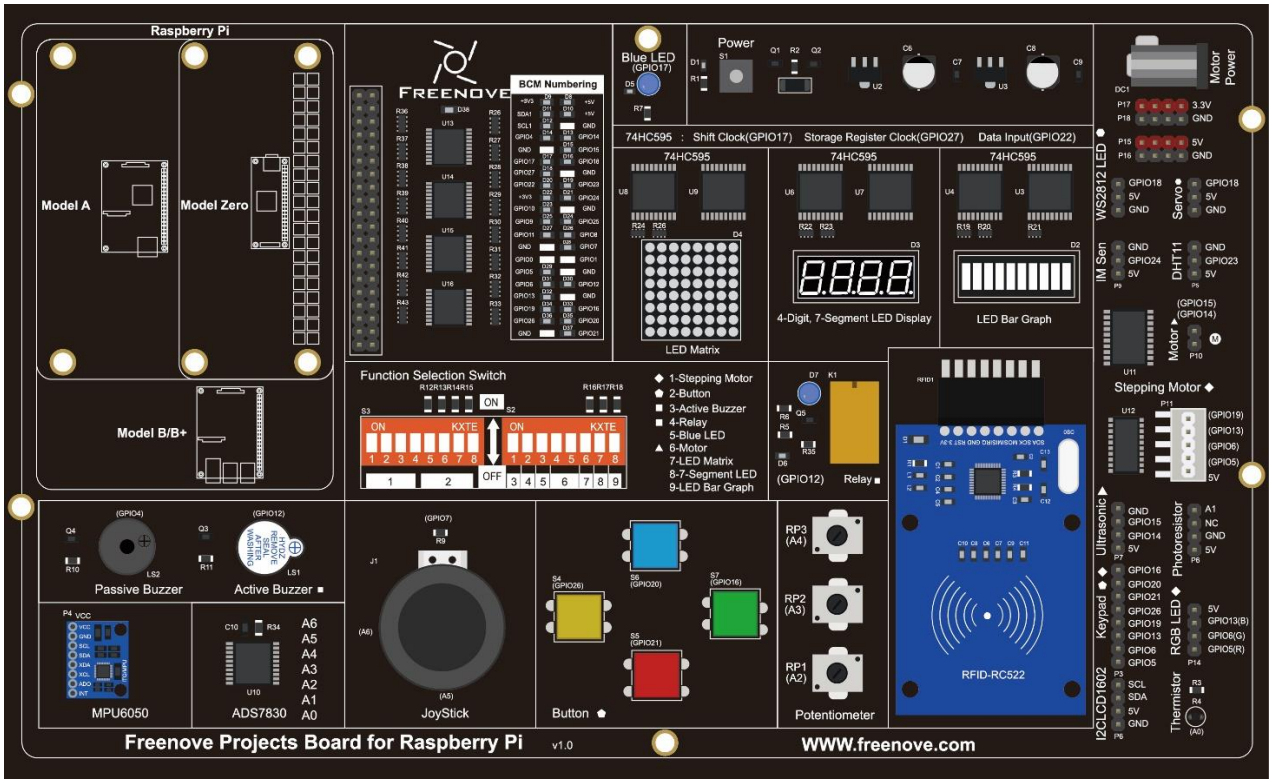
For more details about how to set data blocks and control blocks, please refer to Datasheet.

By default, after verifying password A or password B, we can do reading or writing operation to data blocks. And after verifying password A, we can do reading or writing operation to control blocks. But password A can never be read, so if you choose to verify password A but forget the password A, the block will never be able to read again. **It is highly recommended that beginners should not try to change the contents of control blocks.**

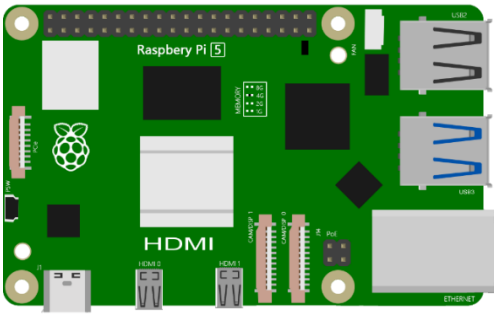
For Mifare1 S50 card equipped in Freenove RFID Kit, the default password A and B are both FFFFFFFFFFFF.

Component List

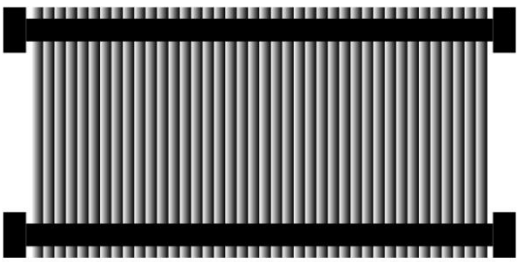
Freenove Projects Board for Raspberry Pi



Raspberry Pi



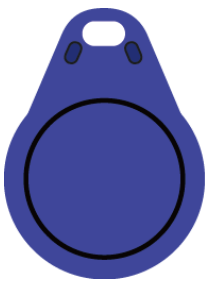
GPIO Ribbon Cable



Mifare1 S50 Standard card

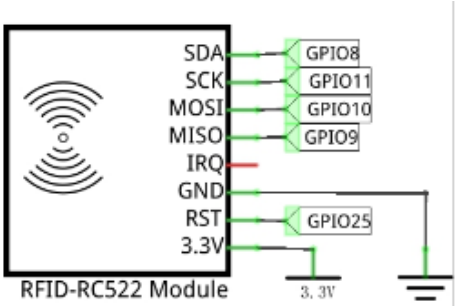


Mifare1 S50 Non-standard card



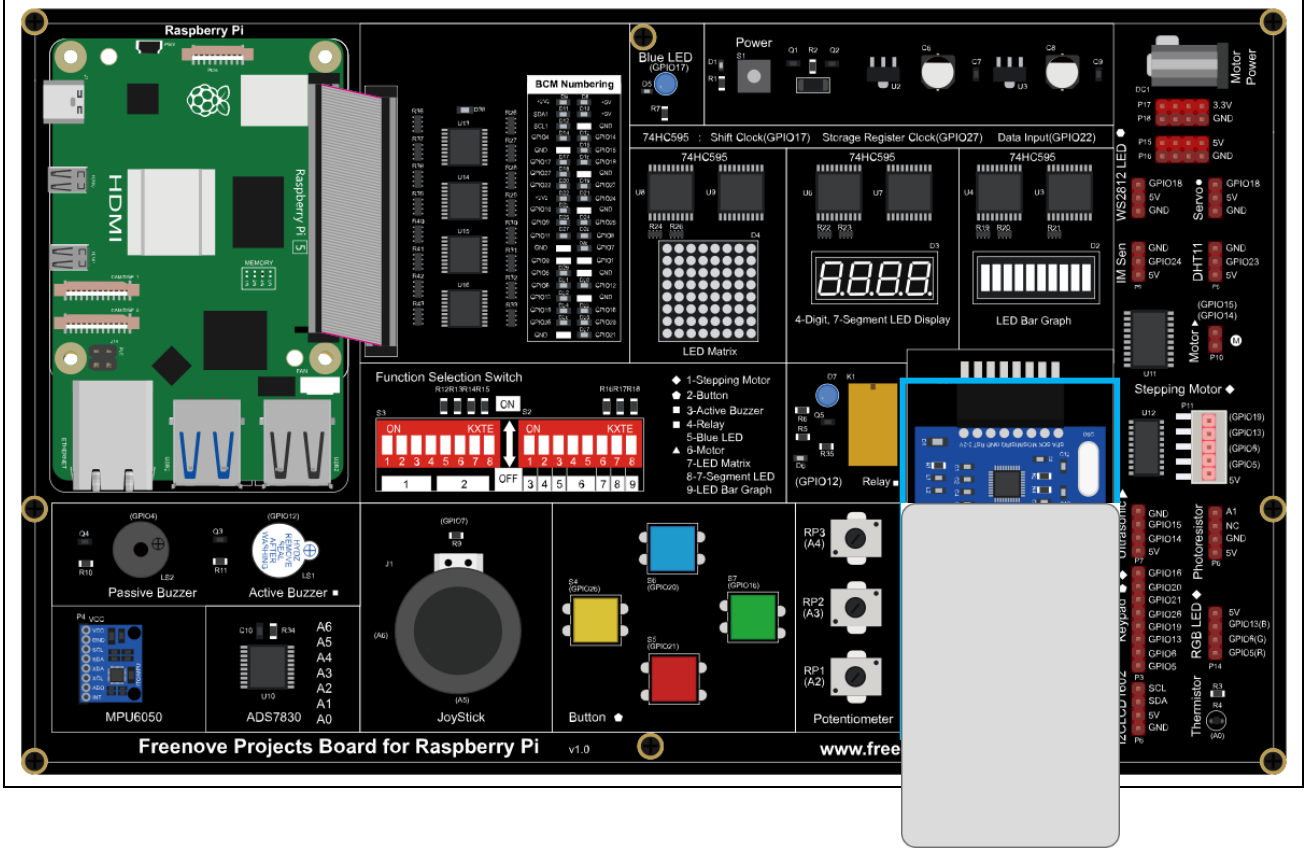
Circuit

Schematic diagram



Hardware connection.

Put RFID card down on here. When the program is running, don't move it away.



If you have any concerns, please send an email to: support@freenove.com

Configure SPI

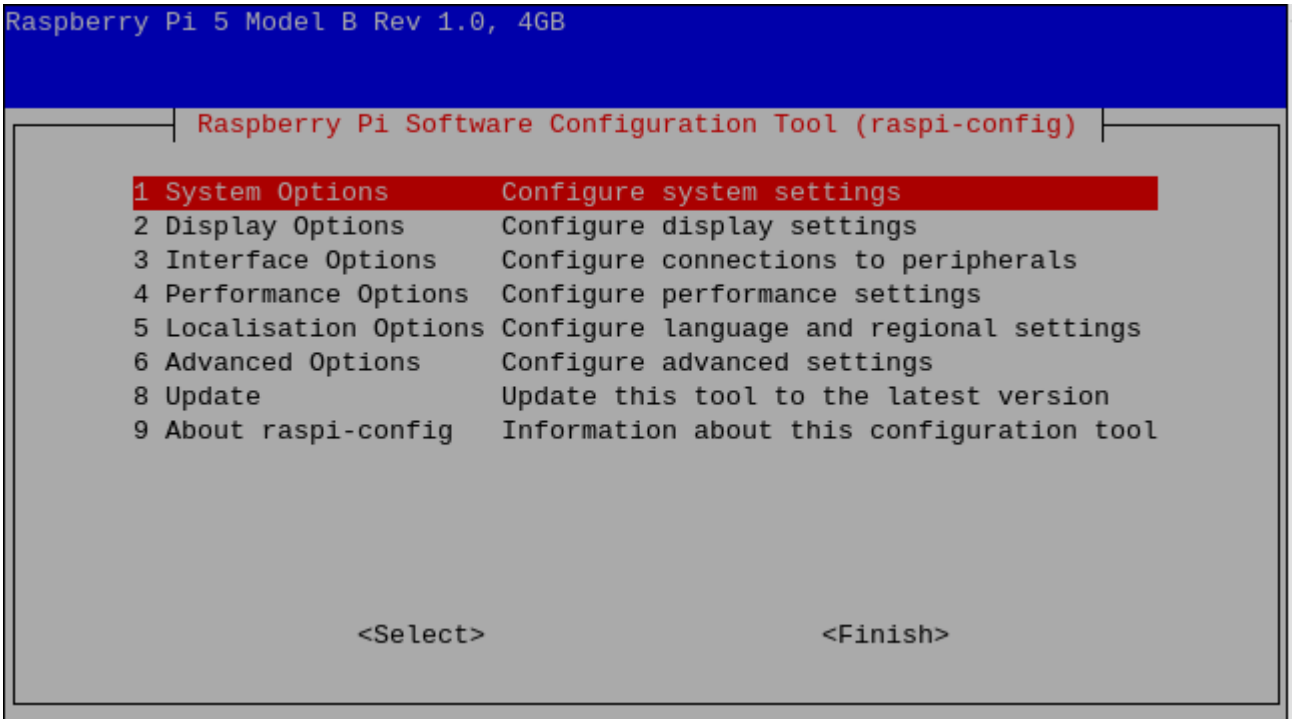
Enable SPI

The SPI interface of raspberry pi is closed by default. You need to open it manually. You can enable the SPI interface in the following way.

Type the following command in the terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose “3 Interfacing Options”→“I4 SPI”→“Yes”→“Finish” in order and then restart your RPi. Then the SPI module is started.

Type the following command to check whether the module SPI is loaded successfully:

```
ls /dev/sp*
```

The following result indicates that the module SPI has been loaded successfully:

```
pi@raspberrypi:~ $ ls /dev/sp*
/dev/spidev0.0  /dev/spidev0.1
```

Code

The project code uses human-computer interaction command line mode to read and write the M1-S50 card.

C Code 25.1 RFID

First observe the running result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 25_RFID directory of C code.

```
cd ~/Freenove_Kit/Code/C_Code/25_RFID
```

2. Use the following command to compile and generate executable file "RFID".

```
sh ./build.sh
```

3. Then run the generated file "RFID".

```
./RFID
```

After the program runs, the following contents will be displayed in the terminal:

```
pi@raspberrypi:~/Freenove_Kit/Code/C_Code/24.1.1_RFID $ sudo sh ./build.sh
Build finished!
pi@raspberrypi:~/Freenove_Kit/Code/C_Code/24.1.1_RFID $ sudo ./RFID
Try to open device /dev/spidev0.0
Device opened
Device Number:3
SPI mode [OK]
SPI word bits[OK]
SPI max speed[OK]
User Space RC522 Application
RC522>
```

Here, type the command “quit” to exit the program.

Type command "scan", and then the program begins to detect whether there is a card close to the sensing area of MFRC522 reader. Place a M1-S50 card in the sensing area. The following results indicate that the M1-S50 card has been detected, the UID of which is E6CF5C8EFB (HEX).

```
RC522>scan
Scanning
.....
.............Card detected    0xE6 0xCF 0x5C 0x8E, Check Sum = 0xFB
Card Selected, Type:PICC_TYPE_MIFARE_1K
RC522>E6CF5C8E>
```

When the Card is placed in the sensing area, you can read and write the card with the following command.

```
Usage:
        read <blockstart>
        dump
        halt
        clean <blockaddr>
        write <blockaddr> <data>
```

In the command read<blockstart>, the parameter blockstart is the address of the data block, and the range is 0-63. This command is used to display all the data from blockstart address to the end of the sector. For example, sector 0 contains data block 0,1,2,3. Using the command “read 0” can display all contents of data block 0,1,2,3. Using the command “read 1” can display all contents of data block 1,2,3. As is shown below:

```
RC522>E6CF5C8E>read 0
read
Auth Block (0x00) with key 0xFF 0xFF 0xFF 0xFF 0xFF ...OK
Read block address 0x00 ....OK read 144 bits
Read block address 0x01 ....OK read 144 bits
Read block address 0x02 ....OK read 144 bits
Read block address 0x03 ....OK read 144 bits
     0: e6 cf 5c 8e fb 08 04 00 62 63 64 65 66 67 68 69 : ..\.....bcdefghi
    16: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    32: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    48: 00 00 00 00 00 00 ff 07 80 69 ff ff ff ff ff ff : .........i......
RC522>E6CF5C8E>read 1
read
Auth Block (0x01) with key 0xFF 0xFF 0xFF 0xFF 0xFF ...OK
Read block address 0x01 ....OK read 144 bits
Read block address 0x02 ....OK read 144 bits
Read block address 0x03 ....OK read 144 bits
     0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    16: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    32: 00 00 00 00 00 00 ff 07 80 69 ff ff ff ff ff ff : .........i......
RC522>E6CF5C8E>
```

Command “dump” is used to display the content of all data blocks in all sectors.
Command <address> <data> is used to write “data" to data block with address “address”, where the address range is 0-63 and the data length is 0-16. For example, if you want to write the string "Freenove" to the data block with address “1”, you can type the following command.

```
write 1 Freenove
```

```
RC522>E6CF5C8E>write 1 Freenove
write
Auth Block (0x01) with key 0xFF 0xFF 0xFF 0xFF 0xFF ...OK
Try to write block 1 with 8 byte data...OK
```

Read the contents of this sector and check the data just written.

```
read 0
```

The following results indicate that the string "Freenove" has been written successfully into the data block 1.

```
Read block address 0x03 ....OK read 144 bits
     0: e6 cf 5c 8e fb 08 04 00 62 63 64 65 66 67 68 69 : ..\.....bcdefghi
    16: 46 72 65 65 6e 6f 76 65 00 00 00 00 00 00 00 00 : Freenove........
    32: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    48: 00 00 00 00 00 00 ff 07 80 69 ff ff ff ff ff ff : .........i......
```

Command “clean <address>” is used to remove the contents of the data block with address "address". For example, if you want to clear the contents of the data block 1 that has just been written, you can type the following command.

```
clean 1
```

```
RC522>E6CF5C8E>clean 1
clean
Auth Block (0x01) with key 0xFF 0xFF 0xFF 0xFF 0xFF ...OK
Try to clean block 1...OK
```

Read the contents of data blocks in this sector again to check whether the data is erased. The following results indicate that the contents of data block 1 have been erased.

```
RC522>E6CF5C8E>read 0
read
Auth Block (0x00) with key 0xFF 0xFF 0xFF 0xFF 0xFF ...OK
Read block address 0x00 ....OK read 144 bits
Read block address 0x01 ....OK read 144 bits
Read block address 0x02 ....OK read 144 bits
Read block address 0x03 ....OK read 144 bits
     0: e6 cf 5c 8e fb 08 04 00 62 63 64 65 66 67 68 69 : ..\.....bcdefghi
    16: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    32: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 : ................
    48: 00 00 00 00 00 00 ff 07 80 69 ff ff ff ff ff ff : .........i......
```

Command “halt” is used to quit the selection state of the card.

```
RC522>E6CF5C8E>halt
halt
Halt
RC522>
```

The following is the program code:

```
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <string.h>
#include <getopt.h>
#include <stdlib.h>
#include "mfrc522.h"
#define DISP_COMMANDLINE() printf("RC522>")

int scan_loop(uint8_t *CardID);
int tag_select(uint8_t *CardID);

int main(int argc, char **argv) {
    MFRC522_Status_t ret;
    //Recognized card ID
    uint8_t CardID[5] = { 0x00, };
    uint8_t tagType[16] = {0x00,};
    static char command_buffer[1024];

    ret = MFRC522_Init('B');
    if (ret < 0) {
        printf("Failed to initialize.\r\nProgram exit.\r\n");
        exit(-1);
    }

    printf("User Space RC522 Application\r\n");

    while (1) {
```

```
29          /*Main Loop Start*/
30          DISP_COMMANDLINE();
31
32          scanf("%s", command_buffer);
33          if (strcmp(command_buffer, "scan") == 0) {
34              puts("Scanning ... ");
35              while (1) {
36                  ret = MFRC522_Request(PICC_REQIDL, tagType);
37                  if (ret == MI_OK) {
38                      printf("Card detected!\r\n");
39                      ret = MFRC522_Anticoll(CardID);
40                      if(ret == MI_OK){
41                          ret = tag_select(CardID);
42                          if (ret == MI_OK) {
43                              ret = scan_loop(CardID);
44                              if (ret < 0) {
45                                  printf("Card error...\r\n");
46                                  break;
47                              } else if (ret == 1) {
48                                  puts("Halt...\r\n");
49                                  break;
50                              }
51                          }
52                      }
53                      else{
54                          printf("Get Card ID failed!\r\n");
55                      }
56                  }
57                  MFRC522_Halt();
58              }
59              MFRC522_Halt();
60              MFRC522_Init('B');
61          } else if (strcmp(command_buffer, "quit") == 0
62                  || strcmp(command_buffer, "exit") == 0) {
63              return 0;
64          } else {
65              puts("Unknown command");
66              puts("scan:scan card and dump");
67              puts("quit:exit program");
68          }
69          /*Main Loop End*/
70      }
71  }
72  int scan_loop(uint8_t *CardID) {
```

```
73
74      while (1) {
75
76          char input[32];
77          int block_start;
78          DISP_COMMANDLINE();
79          printf("%02X%02X%02X%02X>", CardID[0], CardID[1], CardID[2], CardID[3]);
80          scanf("%s", input);
81          puts((char*)input);
82          if (strcmp(input, "halt") == 0) {
83              MFRC522_Halt();
84              return 1;
85          } else if (strcmp(input, "dump") == 0) {
86              if (MFRC522_Debug_CardDump(CardID) < 0)
87                  return -1;
88          } else if (strcmp(input, "read") == 0) {
89              scanf("%d", &block_start);
90              if (MFRC522_Debug_DumpSector(CardID, block_start) < 0) {
91                  return -1;
92              }
93          } else if(strcmp(input, "clean") == 0){
94              char c;
95              scanf("%d", &block_start);
96              while ((c = getchar()) != '\n' && c != EOF)
97                  ;
98              if (MFRC522_Debug_Clean(CardID, block_start)) {
99                  return -1;
100             }
101
102         } else if (strcmp(input, "write") == 0) {
103             char write_buffer[256];
104             size_t len = 0;
105             scanf("%d", &block_start);
106             scanf("%s",write_buffer);
107             if (len >= 0) {
108                 if (MFRC522_Debug_Write(CardID, block_start, write_buffer,
109                         strlen(write_buffer)) < 0) {
110                     return -1;
111                 }
112             }
113         } else {
114
115             printf(
116                     "Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n"
```

```
                        "\tclean <blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n");
            //return 0;
        }
    }
    return 0;

}
int tag_select(uint8_t *CardID) {
    int ret_int;
    printf(
            "Card UID: 0x%02X 0x%02X 0x%02X 0x%02X, Check Sum = 0x%02X\r\n",
            CardID[0], CardID[1], CardID[2], CardID[3], CardID[4]);
    ret_int = MFRC522_SelectTag(CardID);
    if (ret_int == 0) {
        printf("Card Select Failed\r\n");
        return -1;
    } else {
        printf("Card Selected, Type:%s\r\n",
                MFRC522_TypeToString(MFRC522_ParseType(ret_int)));
    }
    ret_int = 0;
    return ret_int;
}
```

In the code, first initialize the MFRC522. If the initialization fails, the program will exit.

```
    ret = MFRC522_Init('B');
    if (ret < 0) {
        printf("Failed to initialize.\r\nProgram exit.\r\n");
        exit(-1);
    }
```

In the main function, wait for the command input. If command "scan" is received, the function will begin to detect whether there is a card close to the sensing area. If a card is detected, the card will be selected and card UID will be acquired. Then enter the function scan_loop (). If command "quit" or "exit" is received, the program will exit.

```
        scanf("%s", command_buffer);
        if (strcmp(command_buffer, "scan") == 0) {
            puts("Scanning ... ");
            while (1) {
                ret = MFRC522_Request(PICC_REQIDL, tagType);
                if (ret == MI_OK) {
                    printf("Card detected!\r\n");
                    ret = MFRC522_Anticoll(CardID);
                    if(ret == MI_OK){
```

```
                        ret = tag_select(CardID);
                        if (ret == MI_OK) {
                            ret = scan_loop(CardID);
                            if (ret < 0) {
                                printf("Card error...\r\n");
                                break;
                            } else if (ret == 1) {
                                puts("Halt...\r\n");
                                break;
                            }
                        }
                    }
                    else{
                        printf("Get Card ID failed!\r\n");
                    }
                }
                MFRC522_Halt();
            }
            MFRC522_Halt();
            MFRC522_Init('B');
        } else if (strcmp(command_buffer, "quit") == 0
                || strcmp(command_buffer, "exit") == 0) {
            return 0;
        } else {
            puts("Unknown command");
            puts("scan:scan card and dump");
            puts("quit:exit program");
        }
        /*Main Loop End*/
```

The function scan_loop() will detect command read, write, clean, halt, dump and do the corresponding processing to each command. The functions of each command and the method have been introduced before.

```
int scan_loop(uint8_t *CardID) {

    while (1) {

        char input[32];
        int block_start;
        DISP_COMMANDLINE();
        printf("%02X%02X%02X%02X>", CardID[0], CardID[1], CardID[2], CardID[3]);
        scanf("%s", input);
        puts((char*)input);
        if (strcmp(input, "halt") == 0) {
```

```
            MFRC522_Halt();
            return 1;
        } else if (strcmp(input, "dump") == 0) {
            if (MFRC522_Debug_CardDump(CardID) < 0)
                return -1;
        } else if (strcmp(input, "read") == 0) {
            scanf("%d", &block_start);
            if (MFRC522_Debug_DumpSector(CardID, block_start) < 0) {
                return -1;
            }
        } else if(strcmp(input, "clean") == 0){
            char c;
            scanf("%d", &block_start);
            while ((c = getchar()) != '\n' && c != EOF)
                ;
            if (MFRC522_Debug_Clean(CardID, block_start)) {
                return -1;
            }

        } else if (strcmp(input, "write") == 0) {
            char write_buffer[256];
            size_t len = 0;
            scanf("%d", &block_start);
            scanf("%s",write_buffer);
            if (len >= 0) {
                if (MFRC522_Debug_Write(CardID, block_start, write_buffer,
                        strlen(write_buffer)) < 0) {
                    return -1;
                }
            }
        } else {

            printf(
                    "Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n"
                        "\tclean <blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n");
            //return 0;
        }
    }
    return 0;

}
```

Python Code 25.1 RFID

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/25_RFID/Python3
```

1. Use python command to execute code "RFID.py".

```
python RFID.py
```

After the program runs, the following contents will be displayed in the terminal:

```
pi@raspberrypi:~ $ cd ~/Freenove_Kit/Code/Python_Code/24.1.1_RFID/Python3
pi@raspberrypi:~/Freenove_Kit/Code/Python_Code/24.1.1_RFID/Python3 $ python RFID.py
Program is starting ...
Press Ctrl-C to exit.
RC522>
```

Here, type the command “quit” to exit the program.

Type command "scan", then the program begins to detect whether there is a card close to the sensing area of MFRC522 reader. Place a M1-S50 card in the sensing area. The following results indicate that the M1-S50 card has been detected, the UID of which is E6CF5C8EFB (HEX).

```
RC522> scan
scan
Scanning ...
Card detected
Card UID: ['0xe6', '0xcf', '0x5c', '0x8e', '0xfb']
Size: 8
RC522> E6CF5C8EFB>
```

When the Card is placed in the sensing area, you can read and write the card with the following command.

```
Usage:
        read <blockstart>
        dump
        halt
        clean <blockaddr>
        write <blockaddr> <data>
```

In the command read<blockstart>, the parameter blockstart is the address of the data block, and the range is 0-63. As is shown below:

In the command read<blockstart>, the parameter blockstart is the address of the data block, and the range is 0-63. This command is used to read the data of data block with address “blockstart”. For example, using command “read 0” can display the content of data block 0. Using the command “read 1” can display the content of data block 1. As is shown below:

```
RC522> E6CF5C8EFB> read 0
['read', '0']
Sector 0 :  e6 cf 5c 8e fb 8 4 0 62 63 64 65 66 67 68 69  | ??bcdefghi
RC522> E6CF5C8EFB> read 1
['read', '1']
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
RC522> E6CF5C8EFB>
```

Command “dump” is used to display the content of all data blocks in all sectors.

Command <address> <data> is used to write “data" to data block with address “address”, where the address range is 0-63 and the data length is 0-16. In the process of writing data to the data block, both the

contents of data block before written and after written will be displayed. For example, if you want to write the string "Freenove" to the data block with address “1”, you can type the following command.

```
write 1 Freenove
```

```
RC522> E6CF5C8EFB> write 1 Freenove
['write', '1', 'Freenove']
Before writing , The data in block 1  is:
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
4 backdata &0x0F == 0x0A 10
Data written
After written , The data in block 1  is:
Sector 1 :  46 72 65 65 6e 6f 76 65 0 0 0 0 0 0 0 0  | Freenove
```

Command “clean <address>” is used remove the contents of the data block with address "address". For example, if you want to clear the contents of the data block 1 that has just been written, you can type the following command.

```
clean 1
```

```
RC522> E6CF5C8EFB> clean 1
['clean', '1']
Before cleaning , The data in block 1  is:
Sector 1 :  46 72 65 65 6e 6f 76 65 0 0 0 0 0 0 0 0  | Freenove
4 backdata &0x0F == 0x0A 10
Data written
After cleaned , The data in block 1  is:
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
```

Command “halt” is used to quit the selection state of the card.

```
RC522> E6CF5C8EFB> halt
['halt']
RC522>
```

The following is the program code :

```
from gpiozero import OutputDevice
import MFRC522
import sys
import os

# Create an object of the class MFRC522
mfrc = MFRC522.MFRC522()


def dis_ConmandLine():
    print ("RC522>",end="")
def dis_CardID(cardID):
    print ("%2X%2X%2X%2X%2X>"%(cardID[0],cardID[1],cardID[2],cardID[3],cardID[4]),end="")
def setup():
    print ("Program is starting ... "  )
    print ("Press Ctrl-C to exit.")
    pass

def loop():
    global mfrc3s
```

```
    while(True):
        dis_ConmandLine()
        inCmd = input()
        print (inCmd)
        if (inCmd == "scan"):
            print ("Scanning ... ")
            mfrc = MFRC522.MFRC522()
            isScan = True
            while isScan:
                # Scan for cards
                 (status,TagType) = mfrc.MFRC522_Request(mfrc.PICC_REQIDL)
                # If a card is found
                if status == mfrc.MI_OK:
                    print ("Card detected")
                # Get the UID of the card
                 (status,uid) = mfrc.MFRC522_Anticoll()
                # If we have the UID, continue
                if status == mfrc.MI_OK:
                    print ("Card UID: "+ str(map(hex,uid)))
                    # Select the scanned tag
                    if mfrc.MFRC522_SelectTag(uid) == 0:
                        print ("MFRC522_SelectTag Failed!")
                    if cmdloop(uid) < 1 :
                        isScan = False

        elif inCmd == "quit":
            destroy()
            exit(0)
        else :
            print ("\tUnknown command\n"+"\tscan:scan card and dump\n"+"\tquit:exit
program\n")

def cmdloop(cardID):
    pass
    while(True):
        dis_ConmandLine()
        dis_CardID(cardID)
        inCmd = input()
        cmd = inCmd.split(" ")
        print (cmd)
        if(cmd[0] == "read"):
            blockAddr = int(cmd[1])
            if((blockAddr<0) or (blockAddr>63)):
                print ("Invalid Address!")
```

```
64              # This is the default key for authentication
65              key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
66              # Authenticate
67              status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
68              # Check if authenticated
69              if status == mfrc.MI_OK:
70                  mfrc.MFRC522_Readstr(blockAddr)
71              else:
72                  print ("Authentication error")
73                  return 0
74
75          elif cmd[0] == "dump":
76              # This is the default key for authentication
77              key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
78              mfrc.MFRC522_Dump_Str(key,cardID)
79
80          elif cmd[0] == "write":
81              blockAddr = int(cmd[1])
82              if((blockAddr<0) or (blockAddr>63)):
83                  print ("Invalid Address!")
84              data = [0]*16
85              if(len(cmd)<2):
86                  data = [0]*16
87              else:
88                  data = cmd[2][0:17]
89                  data = map(ord,data)
90                  data = list(data)
91                  lenData = len(list(data))
92              if lenData<16:
93                      data+=[0]*(16-lenData)
94              # This is the default key for authentication
95              key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
96              # Authenticate
97              status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
98              # Check if authenticated
99              if status == mfrc.MI_OK:
100                 print ("Before writing , The data in block %d  is: "%(blockAddr))
101                 mfrc.MFRC522_Readstr(blockAddr)
102                 mfrc.MFRC522_Write(blockAddr, data)
103                 print ("After written , The data in block %d  is: "%(blockAddr))
104                 mfrc.MFRC522_Readstr(blockAddr)
105             else:
106                 print ("Authentication error")
107                 return 0
```

```
108
109         elif cmd[0] == "clean":
110             blockAddr = int(cmd[1])
111             if((blockAddr<0) or (blockAddr>63)):
112                 print ("Invalid Address!")
113             data = [0]*16
114             # This is the default key for authentication
115             key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
116             # Authenticate
117             status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
118             # Check if authenticated
119             if status == mfrc.MI_OK:
120                 print ("Before cleaning , The data in block %d  is: "%(blockAddr))
121                 mfrc.MFRC522_Readstr(blockAddr)
122                 mfrc.MFRC522_Write(blockAddr, data)
123                 print ("After cleaned , The data in block %d  is: "%(blockAddr))
                    mfrc.MFRC522_Readstr(blockAddr)
124             else:
125                 print ("Authentication error")
126                 return 0
127         elif cmd[0] == "halt":
128             return 0
129         else :
130             print ("Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n" "\tclean
    <blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n")
131
132 def destroy():
133     print("Ending program")
134
135 if __name__ == "__main__":
136     setup()
137     try:
138         loop()
139     except KeyboardInterrupt:  # Ctrl+C captured, exit
140         destroy()
```

In the code, first create an MFRC522 class object.

```
    mfrc = MFRC522.MFRC522()
```

In the function loop, wait for the command input. If command "scan" is received, the function will begin to detect whether there is a card close to the sensing area. If a card is detected, the card will be selected and card UID will be acquired. Then enter the function scan_loop (). If command "quit" or "exit" is received, the program will exit.

```
        if (inCmd == "scan"):
            print "Scanning ... "
            isScan = True
```

```
            while isScan:
                ...........
                    if cmdloop(uid) < 1 :
                        isScan = False
        elif inCmd == "quit":
            destroy()
            exit(0)
        else :
            print "\tUnknown command\n"+"\tscan:scan card and dump\n"+"\tquit:exit program\n"
```

The function cmdloop() will detect command read, write, clean, halt, dump and do the corresponding processing to each command. The functions of each command and the method have been introduced before.

```
def cmdloop(cardID):
    pass
    while(True):
        dis_ConmandLine()
        dis_CardID(cardID)
        inCmd = raw_input()
        cmd = inCmd.split(" ")
        print cmd
        if(cmd[0] == "read"):
            ...........
        elif cmd[0] == "dump":
            ...........
        elif cmd[0] == "write":
            ...........
        elif cmd[0] == "clean":
            ...........
        elif cmd[0] == "halt":
            return 0
        else :
            print "Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n" "\tclean
<blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n"
```

The file "MFRC522.py" contains the associated operation method for the MFRC522. You can open the file to view all the definitions and functions.

What's Next?

THANK YOU for participating in this learning experience! If you have completed all of the projects successfully you can consider yourself a Raspberry Pi Master.

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us:
support@freenove.com
We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you are interested in processing, you can study the Processing.pdf in the unzipped folder.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.
http://www.freenove.com/

Thank you again for choosing Freenove products.